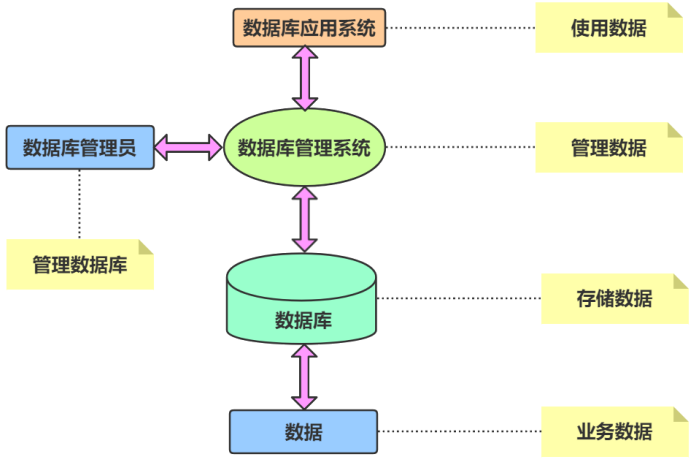
数据库基础知识

先谈发音

MySQL如何发音？在国内MySQL发音有很多种，Oracle官方文档说他们念作

My sequal['si:kwəl]。





数据库基本概念

1. **数据**

   数据（Data）是指对客观事物进行描述并可以鉴别的符号，这些符号是可识别的、抽象的。它不仅指狭义上的数字，而是有多种表现形式：字母、文字、文本、图形、音频、视频等。

2. **数据库**

   数据库是数据管理的有效技术，是由一批数据构成的有序集合，这些数据被存放在结构化的数据表里。数据表之间相互关联，反映客观事物间的本质联系。

3. **数据库管理系统**

   数据库管理系统（Database Management System，DBMS）是用来定义和管理数据的软件。

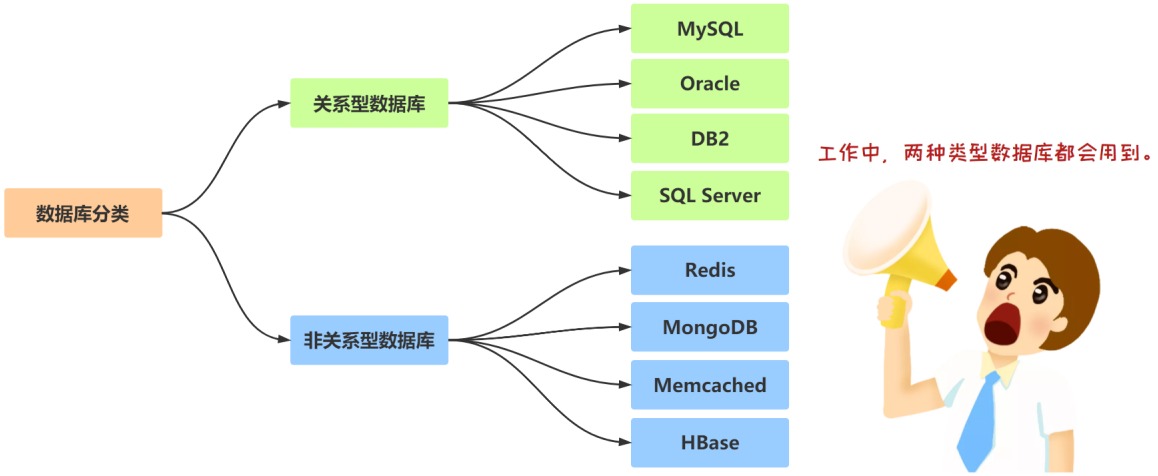
4. **数据库应用程序**

   数据库应用程序（Database Application System，DBAS）是在数据库管理系统基础上，使用数据库管理系统的语法，开发的直接面对最终用户的应用程序。

5. **数据库管理员**

   数据库管理员（Database Administrator，DBA）是指对数据库管理系统进行操作的人员，其主要负责数据库的运营和维护。

数据库分类



关系型数据库

关系型数据库最典型的**数据结构是表**，由**二维表**及其**之间的联系**所组成的一个数据组
织。可以采用结构化查询语言（SQL）对数据库进行操作。

优点：

1. 易于维护：都是使用表结构，格式一致；
2. 使用方便：SQL语言通用，可用于复杂查询；
3. 复杂操作：支持SQL，可用于一个表以及多个表之间非常复杂的查询。

缺点：

1. 读写性能比较差，尤其是海量数据的高效率读写；

2. 固定的表结构，灵活度稍欠；
3. 高并发读写需求，传统关系型数据库来说，硬盘I/O是一个很大的瓶颈。

非关系型数据库

非关系型数据库也称之为NoSQL数据库，是一种数据结构化存储方法的集合，可以是文档或者键值对等。

优点：

1. 格式灵活：存储数据的格式可以是key,value形式、文档形式、图片形式等等，文档形式、图片形式等等，使用灵活，应用场景广泛，而关系型数据库则只支持基础类型。
2. 速度快：nosql可以使用硬盘或者随机存储器作为载体，而关系型数据库只能使用硬盘；
3. 高扩展性；
4. 成本低：nosql数据库部署简单，基本都是开源软件。

缺点：

1. 不提供sql支持，学习和使用成本较高；
2. 无事务处理；
3. 数据结构相对复杂，复杂查询方面稍欠。

实时效果反馈

1.关系型数据库模型是将复杂的数据结构用来表示。

A 文件

B 一维结构

C 二维表结构

D 自定义方式

2.数据库管理系统是用来定义和的软件。

A 管理应用程序

B 管理密码

C 管理数据

D 管理管理员

答案

1=>C　2=>C

MySQL基础知识



MySQL简介

MySQL 是一个关系型数据库管理系统， 由瑞典 MySQL AB 公司开发， 目前属于 Oracle 公司。MySQL 是一种关系型数据库管理系统，关系型数据库将数据保存在不同的表 中，而不是将所有数据放在一个大仓库内，这样就增加了速度并提高了灵活性。

MySQL特点

MySQL 是开源的。

MySQL 支持大型系统的数据库。可以处理拥有上千万条记录的大型数据库。 MySQL 使用标准的 SQL 数据语言形式。

MySQL 可以运行于多个系统上，并且支持多种语言。这些编程语言包括 C 、C++、 Python 、Java 、Perl 、PHP 等。

MySQL 存储数据量较大，32 位系统表文件最大可支持 4GB ， 64 位系统支持最大的 表文件为 8TB。

MySQL 是可以定制的，采用了 GPL 协议，你可以修改源码来开发自己的 MySQL 系 统。

MySQL分类

1. MySQL分为社区版
   社区版是完全开源免费的，社区版也支持多种数据类型和标准的SQL查询语言，能够对数据进行各种查询、增加、删除、修改等操作，所以一般情况下社区版就可以满足开发需求了。
2. 企业版
   企业版是收费的。即使在开发中需要用到一些付费的附加功能，价格相对于昂贵的 Oracle、DB2等也是有很大优势的。对数据库可靠性要求比较高的企业可以选择企业版。

实时效果反馈

1.MySQL它是一个的关系型数据库管理系统

A 闭源

B 开源

C 半开源

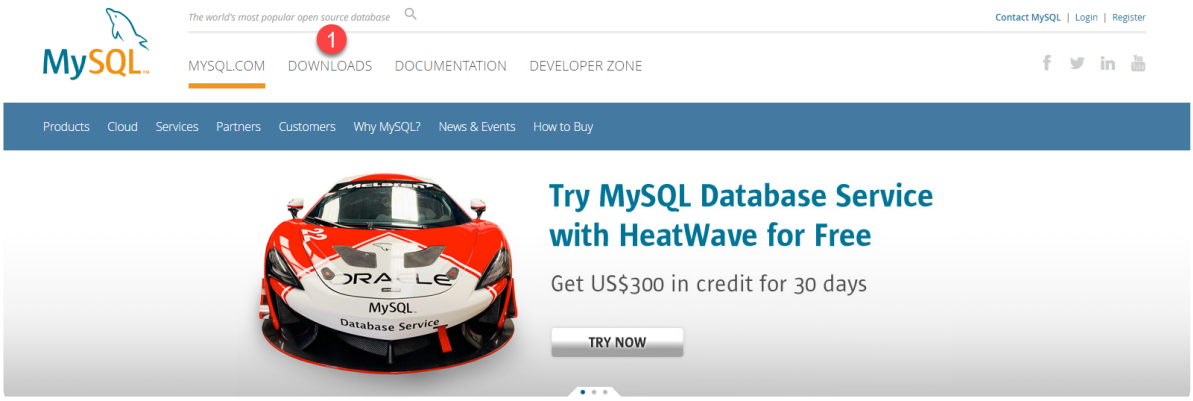
D 完全收费

答案

1=>B

下载MySQL

MySQL官网地址：mysql.com



MySQL Enterprise Edition

MySQL Enterprise Edition includes the most comprehensive set of advanced features, management tools and technical support for MySQL.

Learn More »
Customer Download »
Trial Download »

MySQL Cluster CGE

MySQL Cluster is a real-time open source transactional database designed for fast, always-on access to data under high throughput conditions.

- MySQL Cluster
- MySQL Cluster Manager
- Plus, everything in MySQL Enterprise Edition

Learn More »
Customer Download » (Select Patches & Updates Tab, Product Search)
Trial Download »

MySQL Community (GPL) Downloads » 1

MySQL Community Downloads

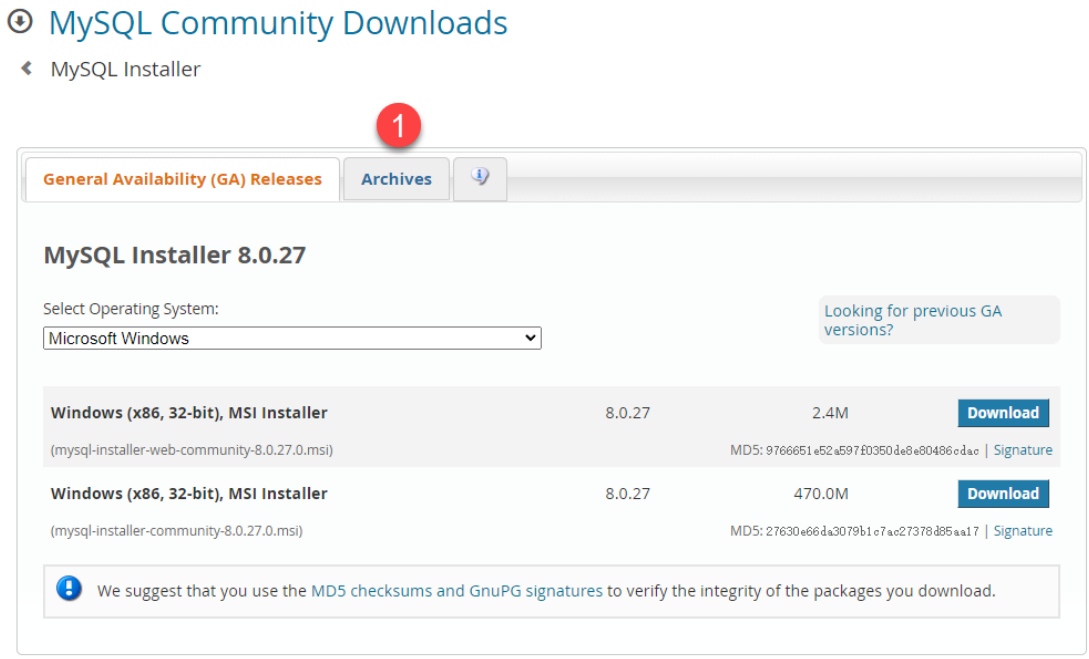
- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository

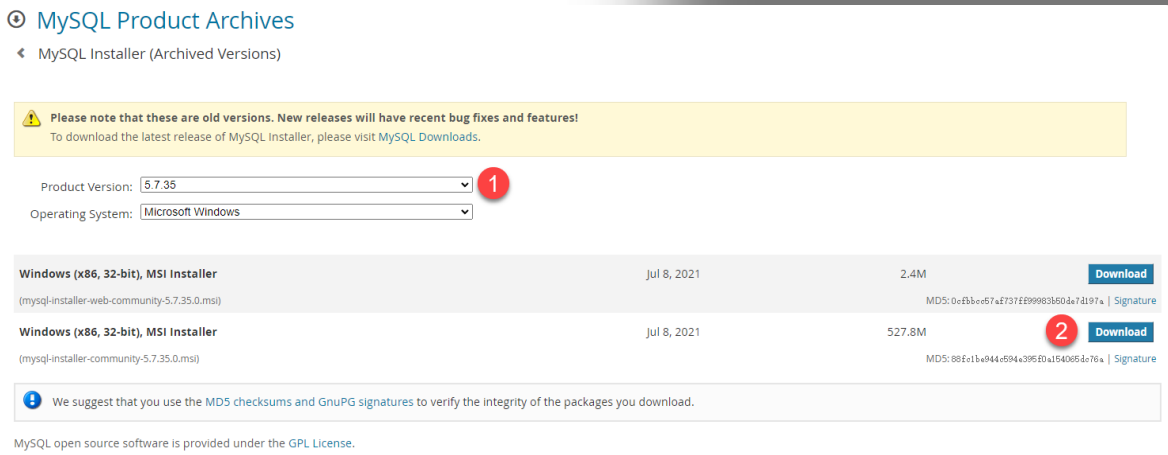
- MySQL Community Server
- MySQL Cluster
- MySQL Router
- MySQL Shell
- MySQL Workbench

- MySQL Installer for Windows 1
- MySQL for Visual Studio

- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP

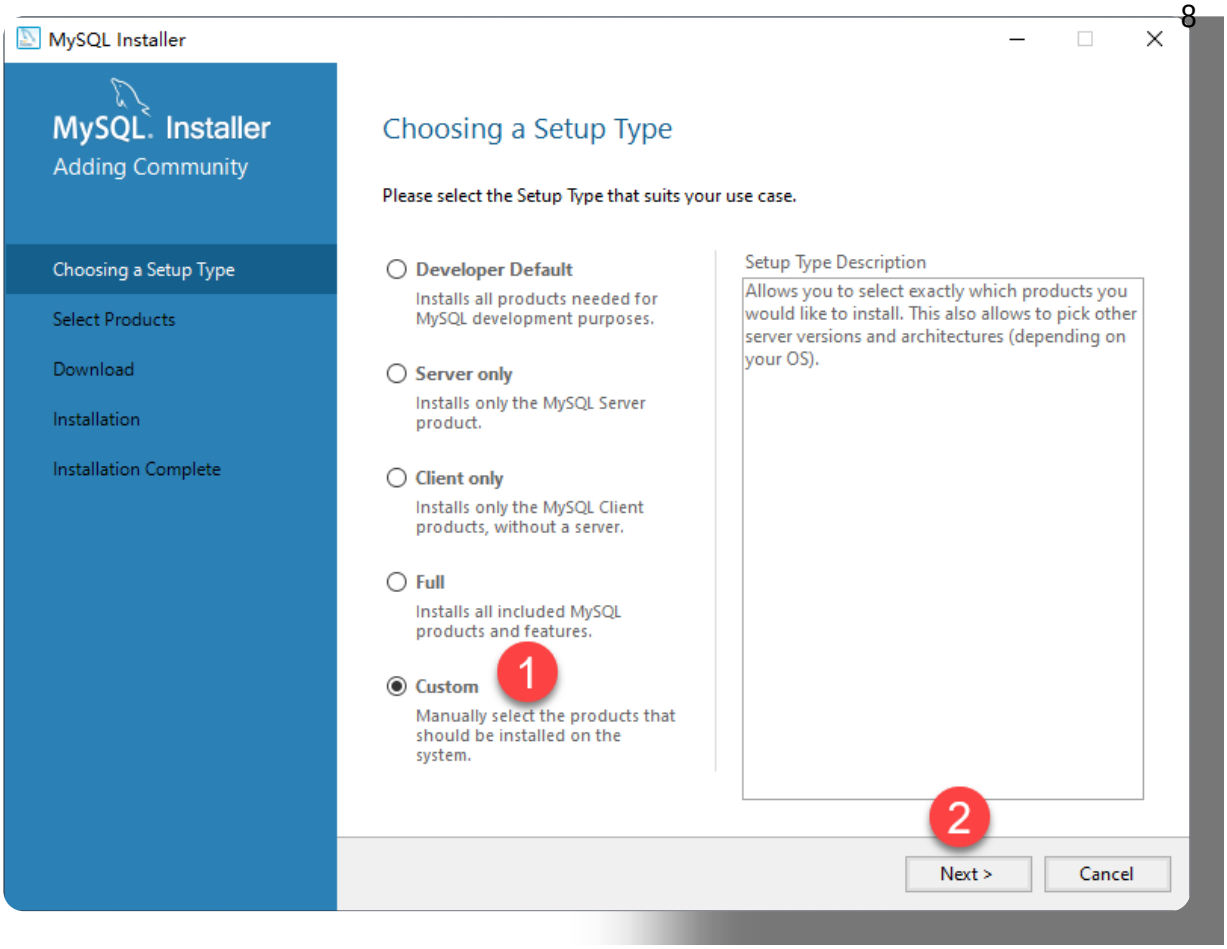
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

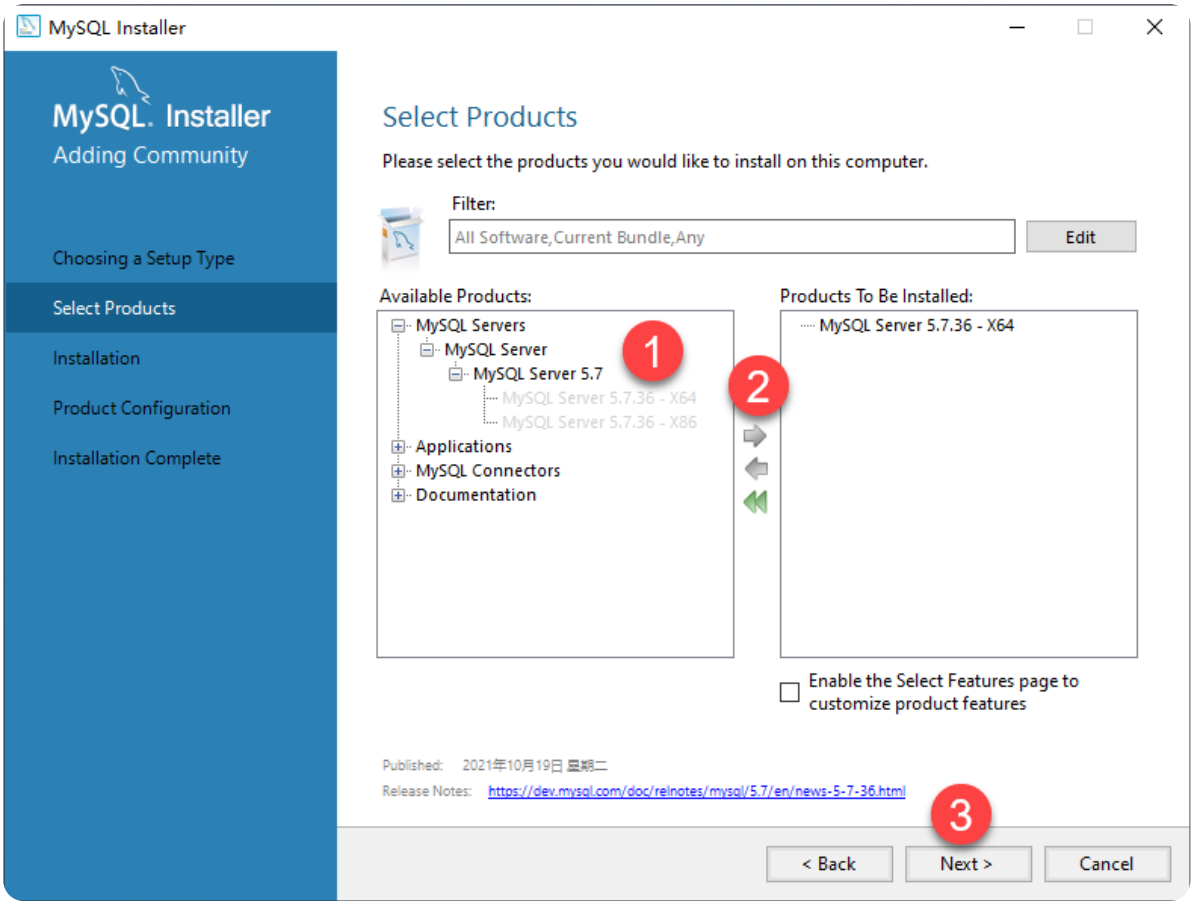


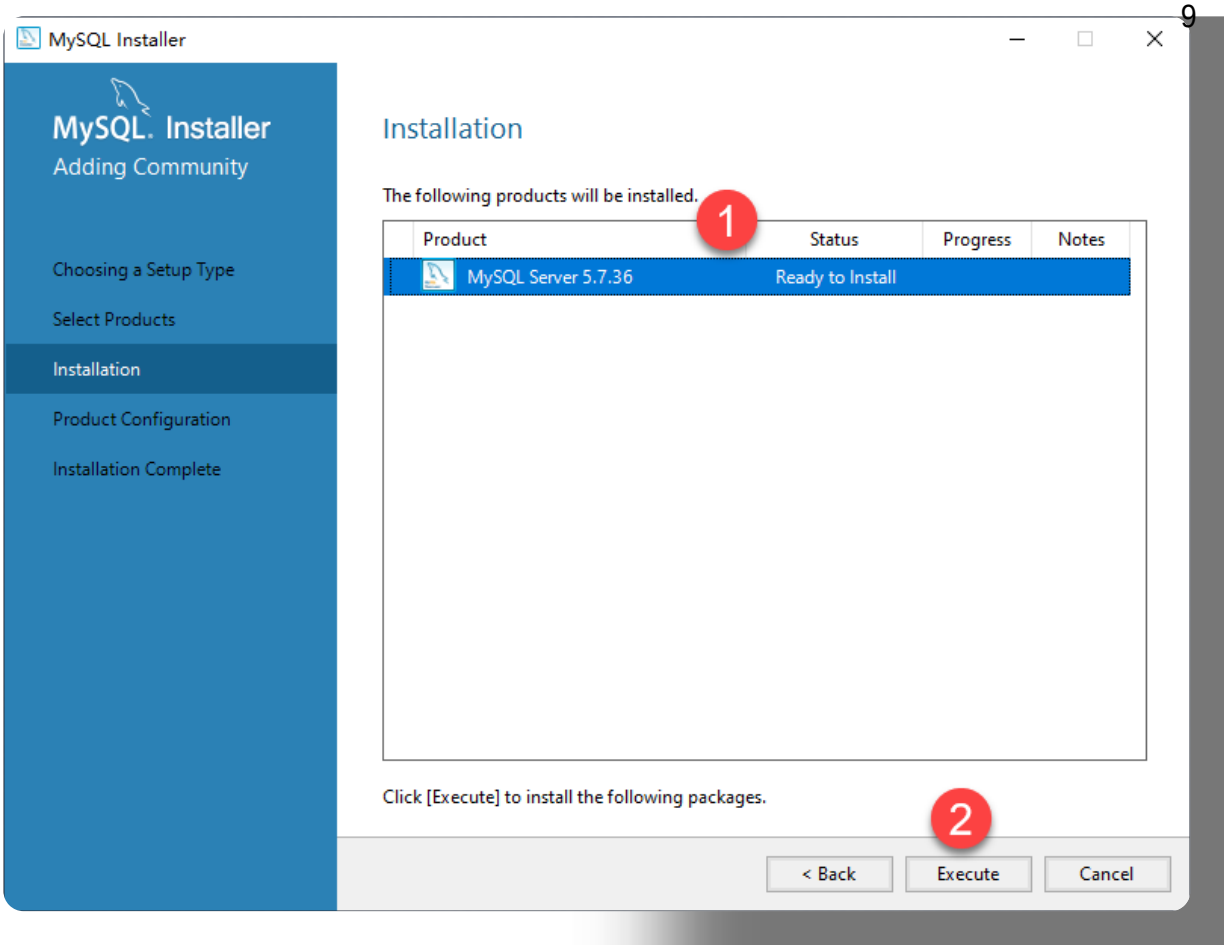


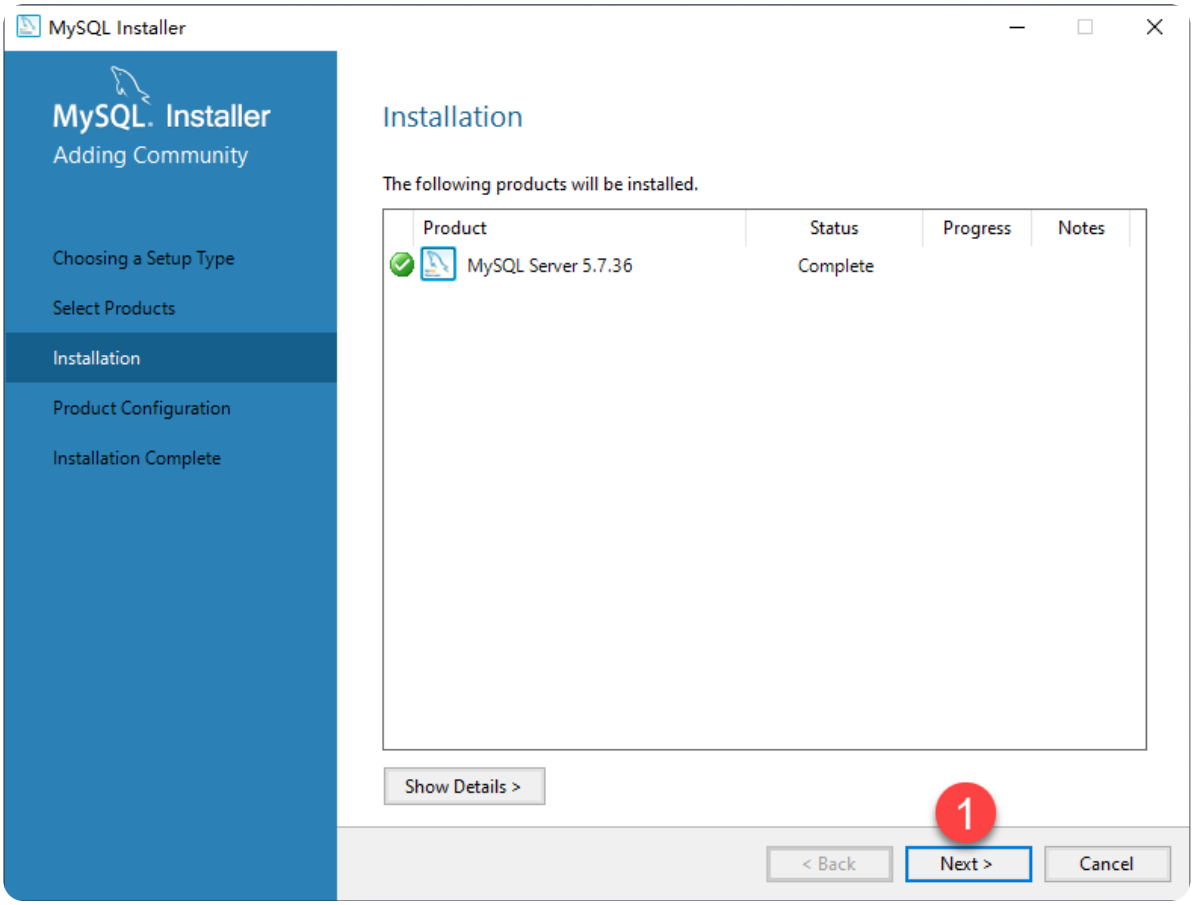
MySQL的安装与卸载

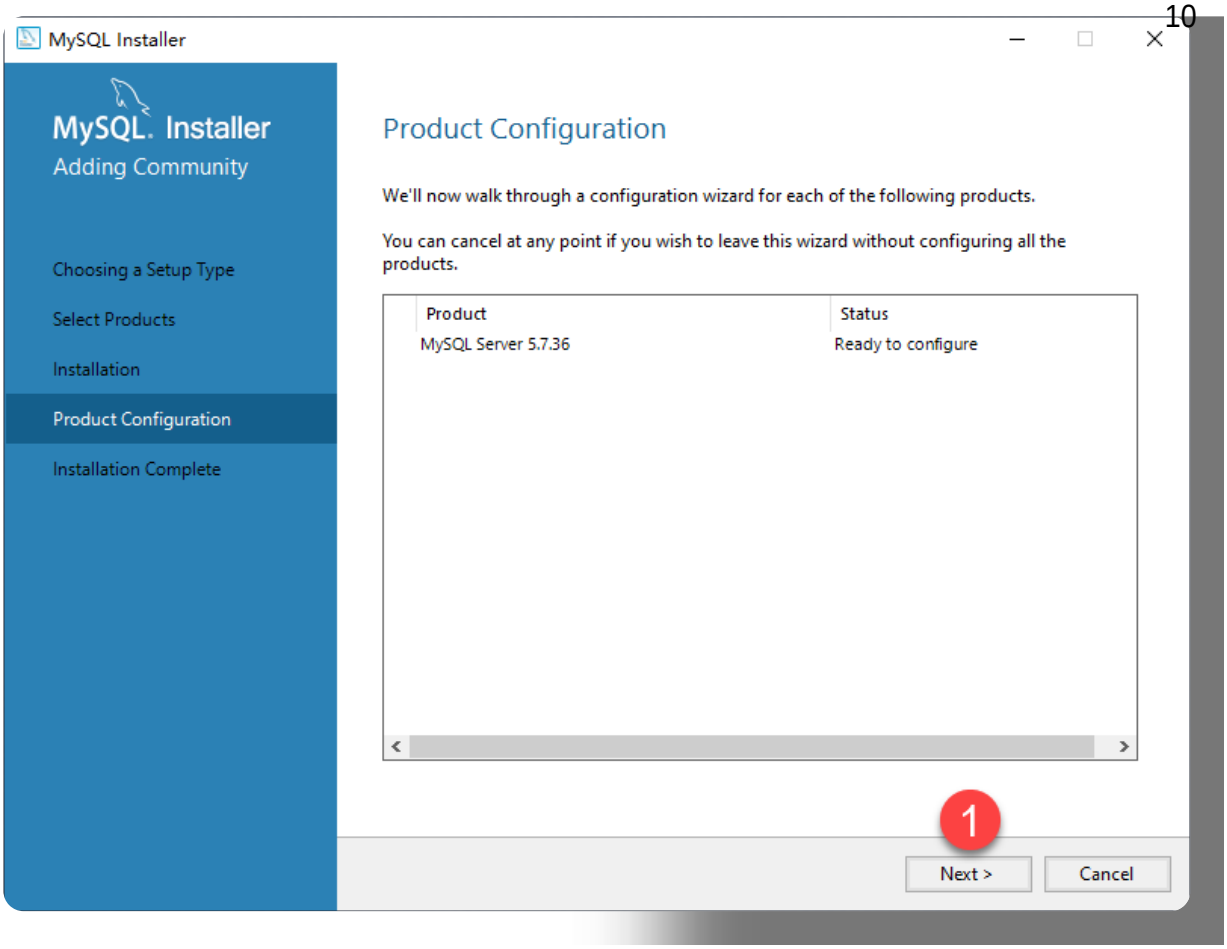
MySQL安装

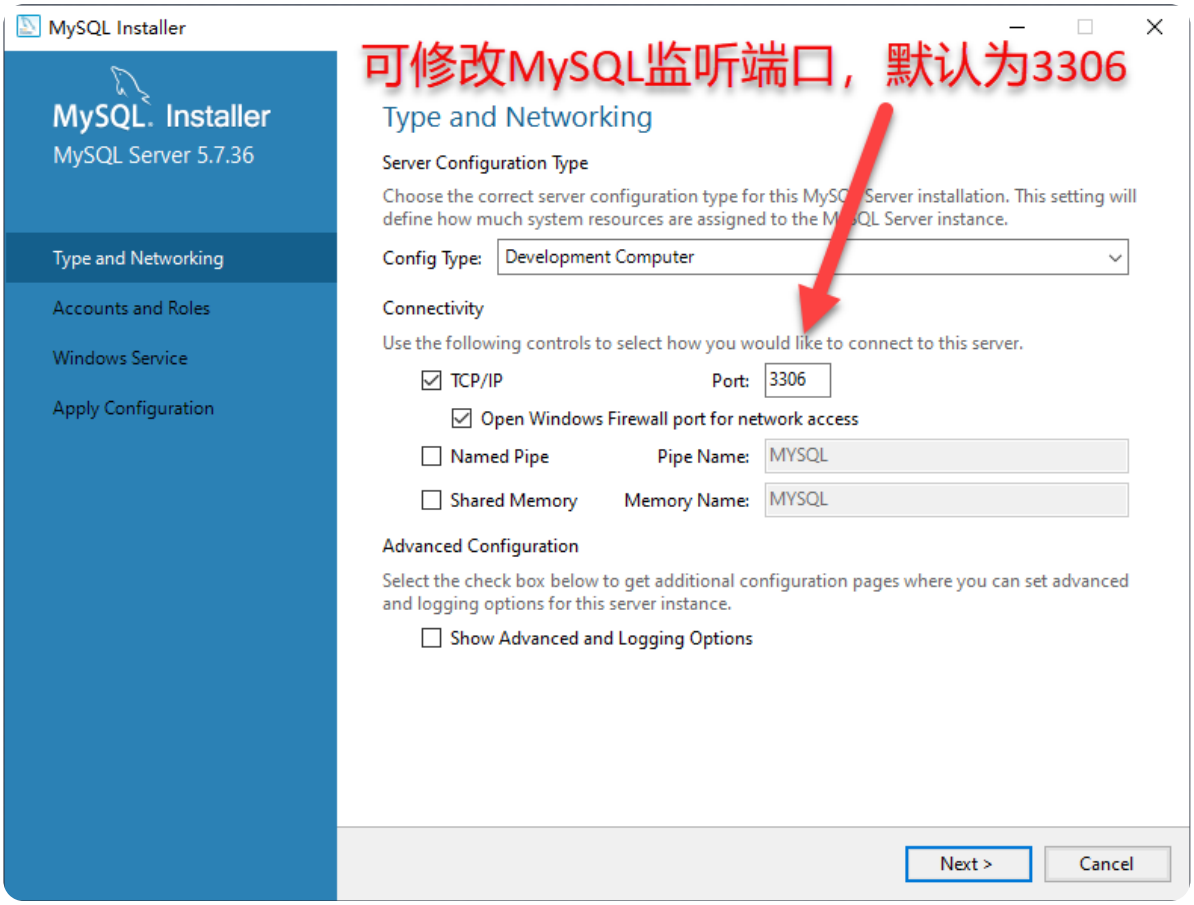

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Download
Installation
Installation Complete
Choosing a Setup Type
Please select the Setup Type that suits your use case.
Developer Default
Installs all products needed for MySQL development purposes.
Server only
Installs only the MySQL Server product.
Client only
Installs only the MySQL Client products, without a server.
Full
Installs all included MySQL products and features.
Custom
Manually select the products that should be installed on the system.
1
Setup Type Description
Allows you to select exactly which products you would like to install. This also allows to pick other server versions and architectures (depending on your OS).
2
Next >
Cancel

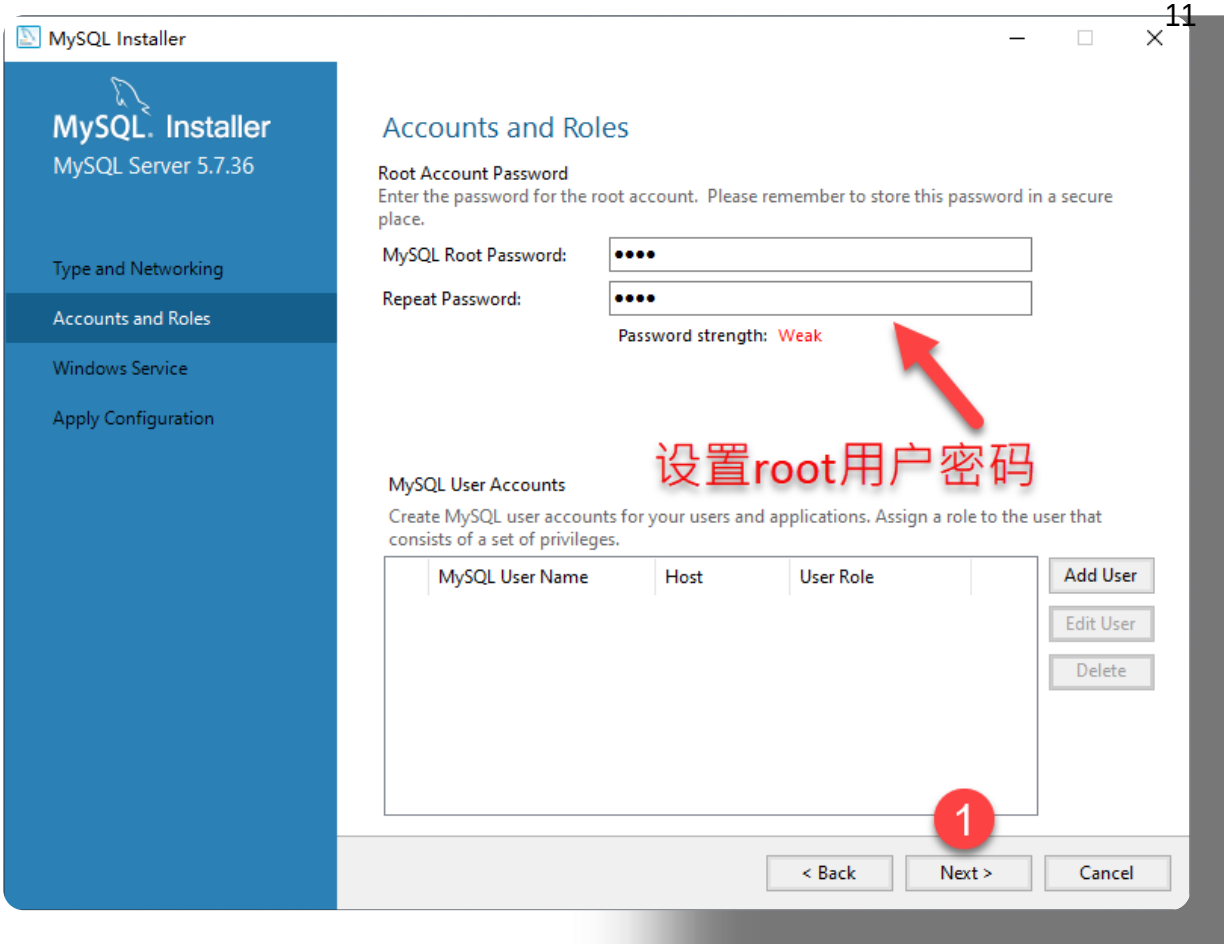

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Select Products
Please select the products you would like to install on this computer.
Filter:
All Software,Current Bundle,Any
Edit
Available Products:
MySQL Servers
MySQL Server
MySQL Server 5.7
MySQL Server 5.7.36 - X64
MySQL Server 5.7.36 - X86
Applications
MySQL Connectors
Documentation
1
2
Products To Be Installed:
MySQL Server 5.7.36 - X64
Enable the Select Features page to customize product features
Published: 2021年10月19日 星期二
Release Notes: https://dev.mysql.com/doc/relnotes/mysql/5.7/en/news-5-7-36.html
3
< Back
Next >
Cancel

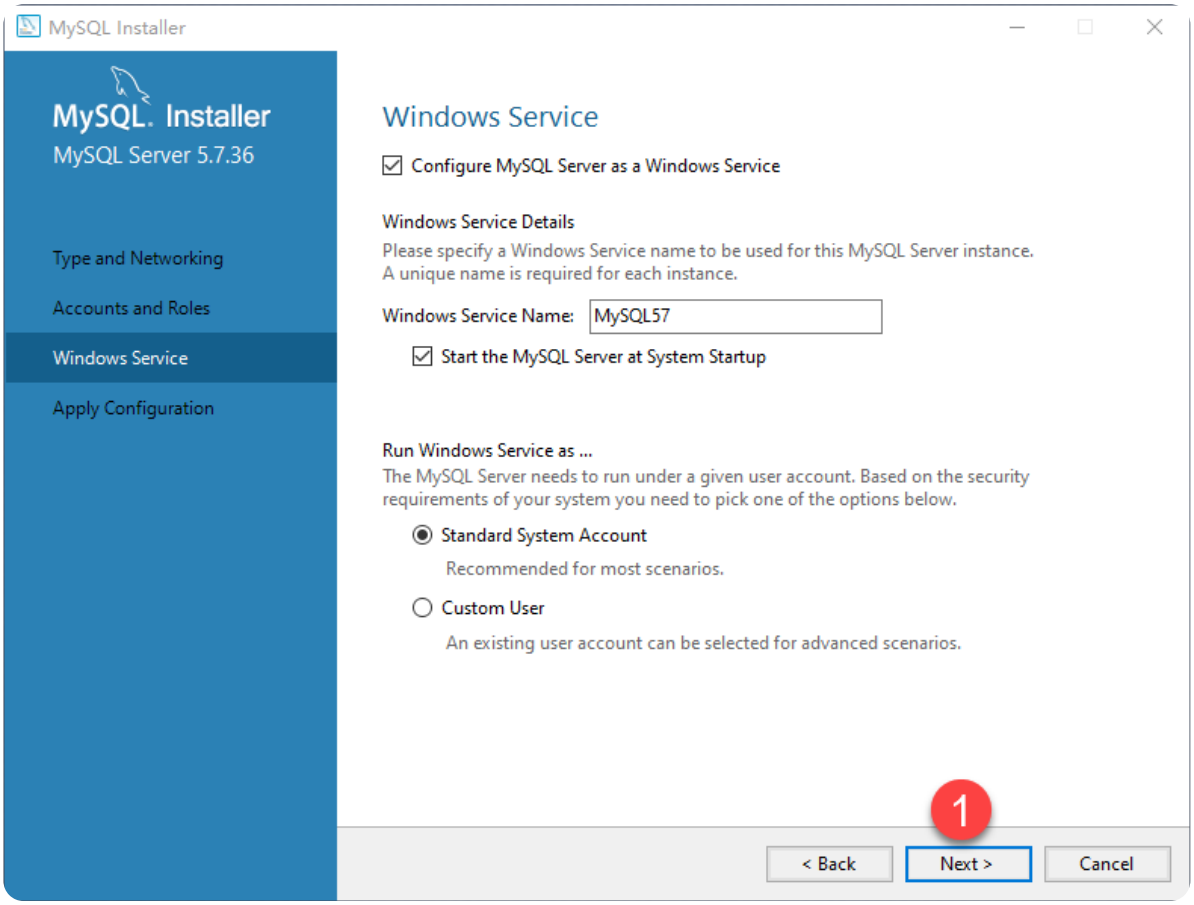

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Installation
The following products will be installed.
1
Product
Status
Progress
Notes
MySQL Server 5.7.36
Ready to Install
Click [Execute] to install the following packages.
2
< Back
Execute
Cancel

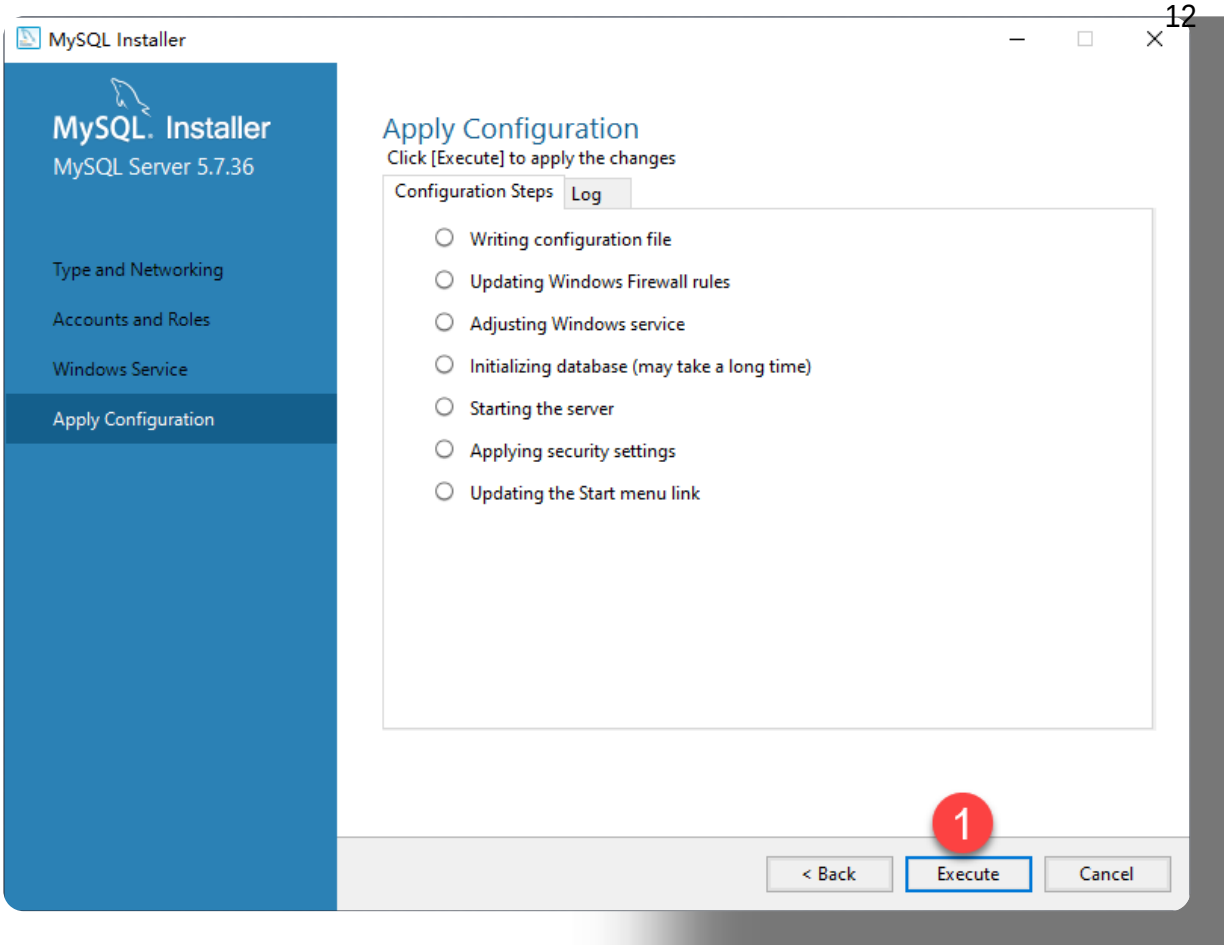

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Installation
The following products will be installed.
Product
Status
Progress
Notes
MySQL Server 5.7.36
Complete
Show Details >
1
< Back
Next >
Cancel

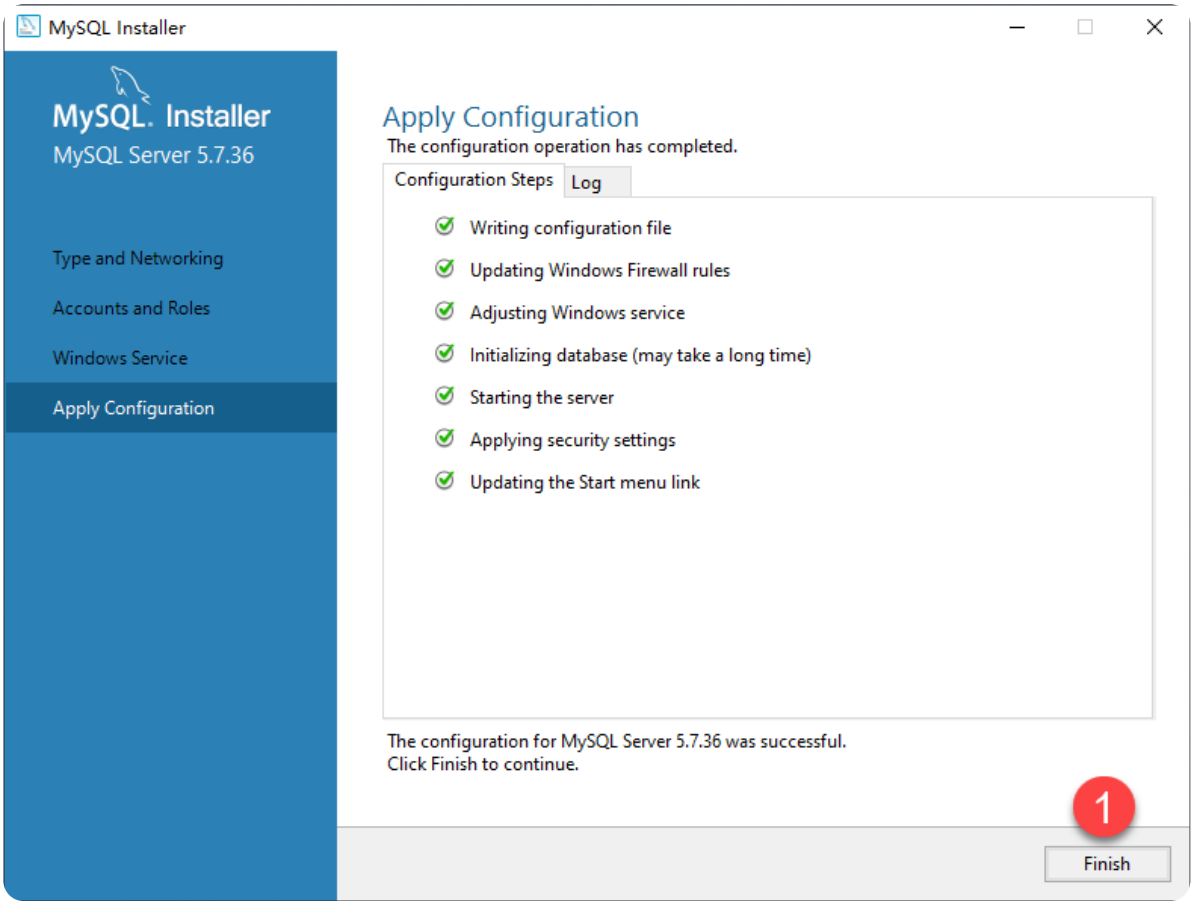

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Product Configuration
We'll now walk through a configuration wizard for each of the following products.
You can cancel at any point if you wish to leave this wizard without configuring all the products.
Product
Status
MySQL Server 5.7.36
Ready to configure
1
Next >
Cancel

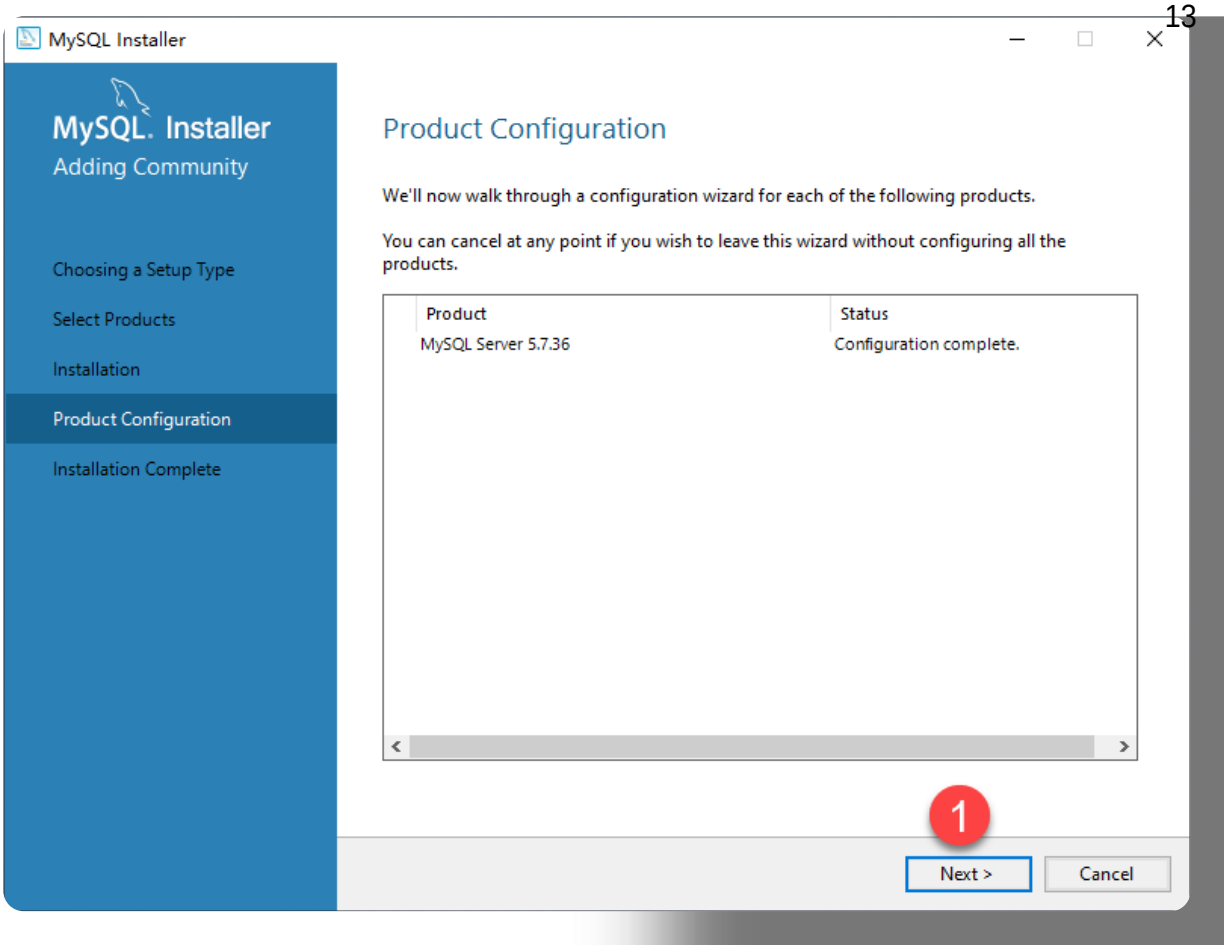

MySQL Installer
可修改MySQL监听端口，默认为3306
MySQL. Installer
MySQL Server 5.7.36
Type and Networking
Accounts and Roles
Windows Service
Apply Configuration
Type and Networking
Server Configuration Type
Choose the correct server configuration type for this MySQL Server installation. This setting will define how much system resources are assigned to the MySQL Server instance.
Config Type: Development Computer
Connectivity
Use the following controls to select how you would like to connect to this server.
TCP/IP
Port: 3306
Open Windows Firewall port for network access
Named Pipe
Pipe Name: MYSQL
Shared Memory
Memory Name: MYSQL
Advanced Configuration
Select the check box below to get additional configuration pages where you can set advanced and logging options for this server instance.
Show Advanced and Logging Options
Next >
Cancel

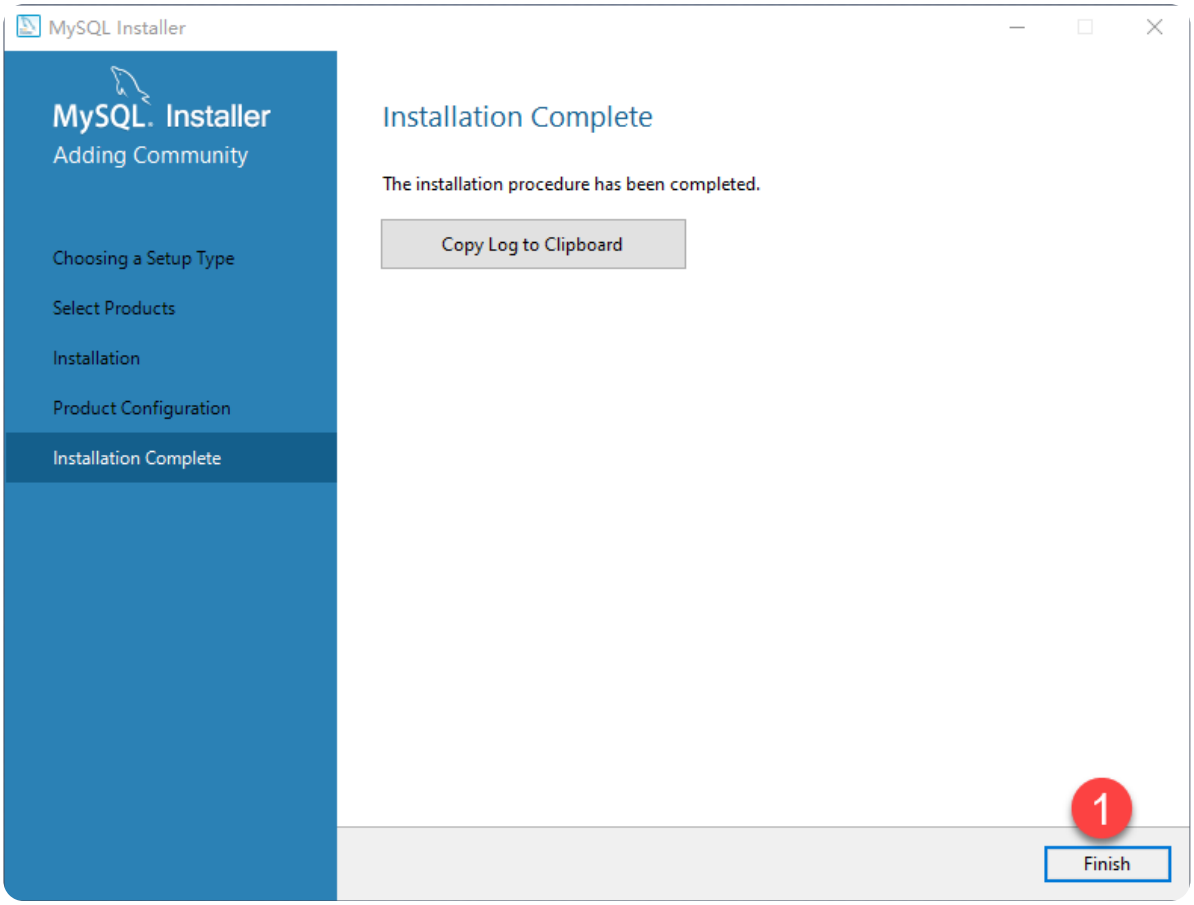

MySQL Installer
MySQL. Installer
MySQL Server 5.7.36
Type and Networking
Accounts and Roles
Windows Service
Apply Configuration
Accounts and Roles
Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.
MySQL Root Password:
Repeat Password:
Password strength: Weak
设置root用户密码
MySQL User Accounts
Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.
MySQL User Name
Host
User Role
Add User
Edit User
Delete
1
< Back
Next >
Cancel


MySQL Installer
MySQL. Installer
MySQL Server 5.7.36
Type and Networking
Accounts and Roles
Windows Service
Apply Configuration
Windows Service
Configure MySQL Server as a Windows Service
Windows Service Details
Please specify a Windows Service name to be used for this MySQL Server instance.
A unique name is required for each instance.
Windows Service Name:
MySQL57
Start the MySQL Server at System Startup
Run Windows Service as ...
The MySQL Server needs to run under a given user account. Based on the security requirements of your system you need to pick one of the options below.
Standard System Account
Recommended for most scenarios.
Custom User
An existing user account can be selected for advanced scenarios.
1
< Back
Next >
Cancel

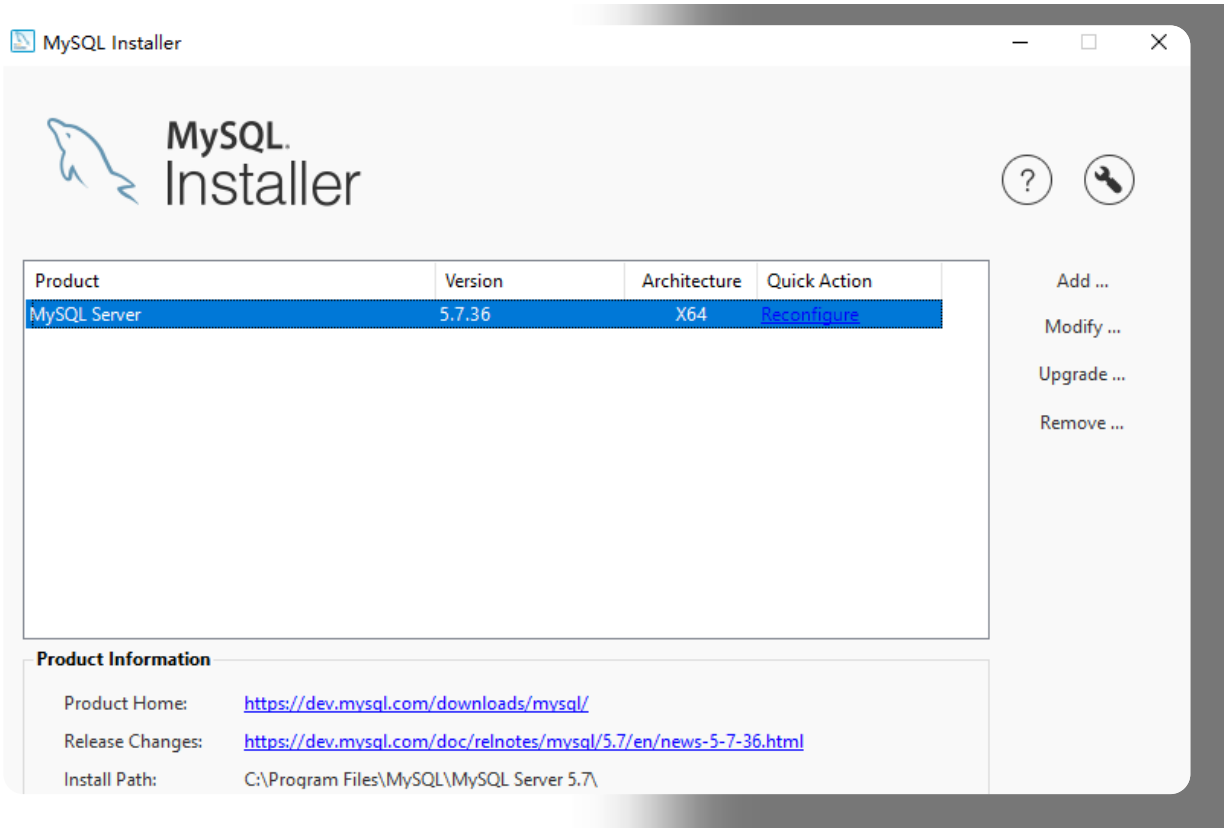

MySQL Installer
MySQL. Installer
MySQL Server 5.7.36
Type and Networking
Accounts and Roles
Windows Service
Apply Configuration
Apply Configuration
Click [Execute] to apply the changes
Configuration Steps
Log
Writing configuration file
Updating Windows Firewall rules
Adjusting Windows service
Initializing database (may take a long time)
Starting the server
Applying security settings
Updating the Start menu link
1
< Back
Execute
Cancel

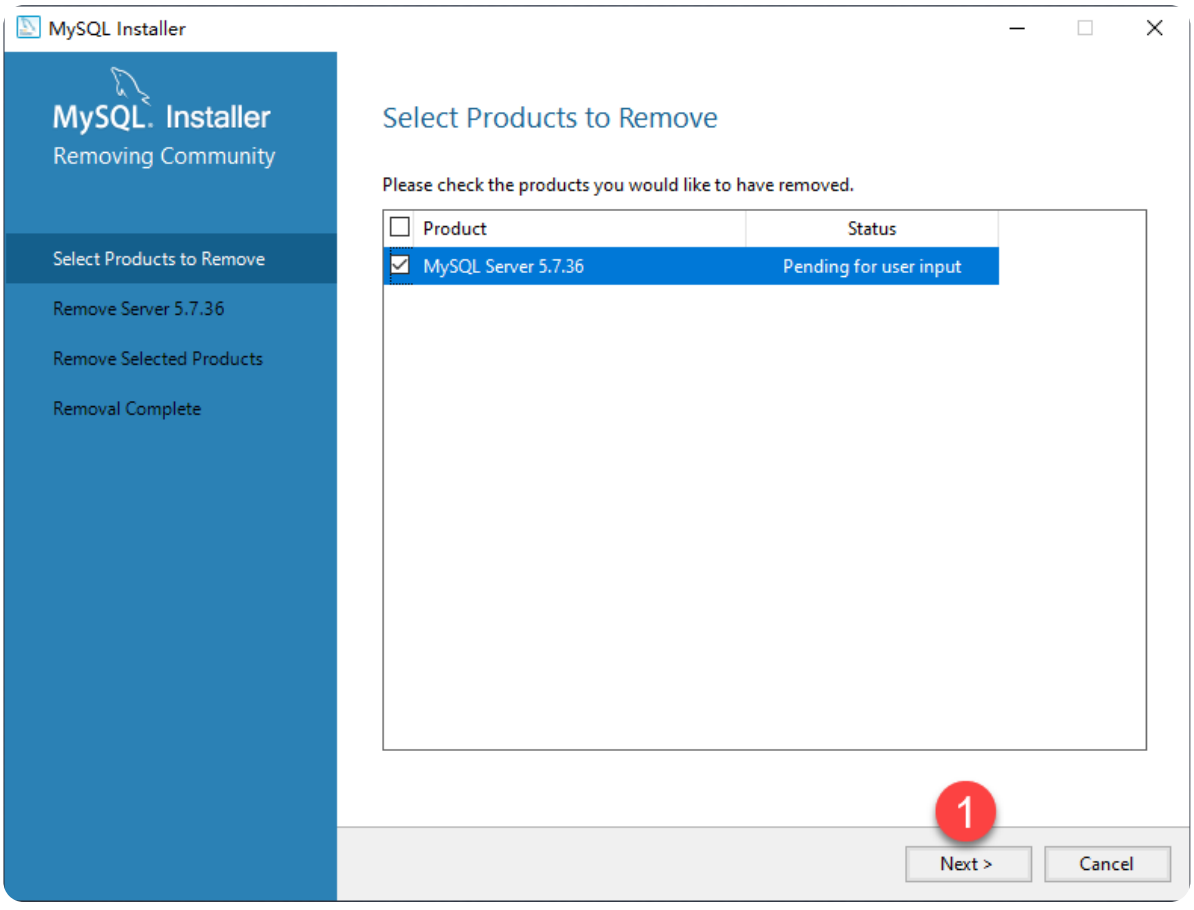

MySQL Installer
MySQL. Installer
MySQL Server 5.7.36
Type and Networking
Accounts and Roles
Windows Service
Apply Configuration
Apply Configuration
The configuration operation has completed.
Configuration Steps
Log
Writing configuration file
Updating Windows Firewall rules
Adjusting Windows service
Initializing database (may take a long time)
Starting the server
Applying security settings
Updating the Start menu link
The configuration for MySQL Server 5.7.36 was successful.
Click Finish to continue.
1
Finish

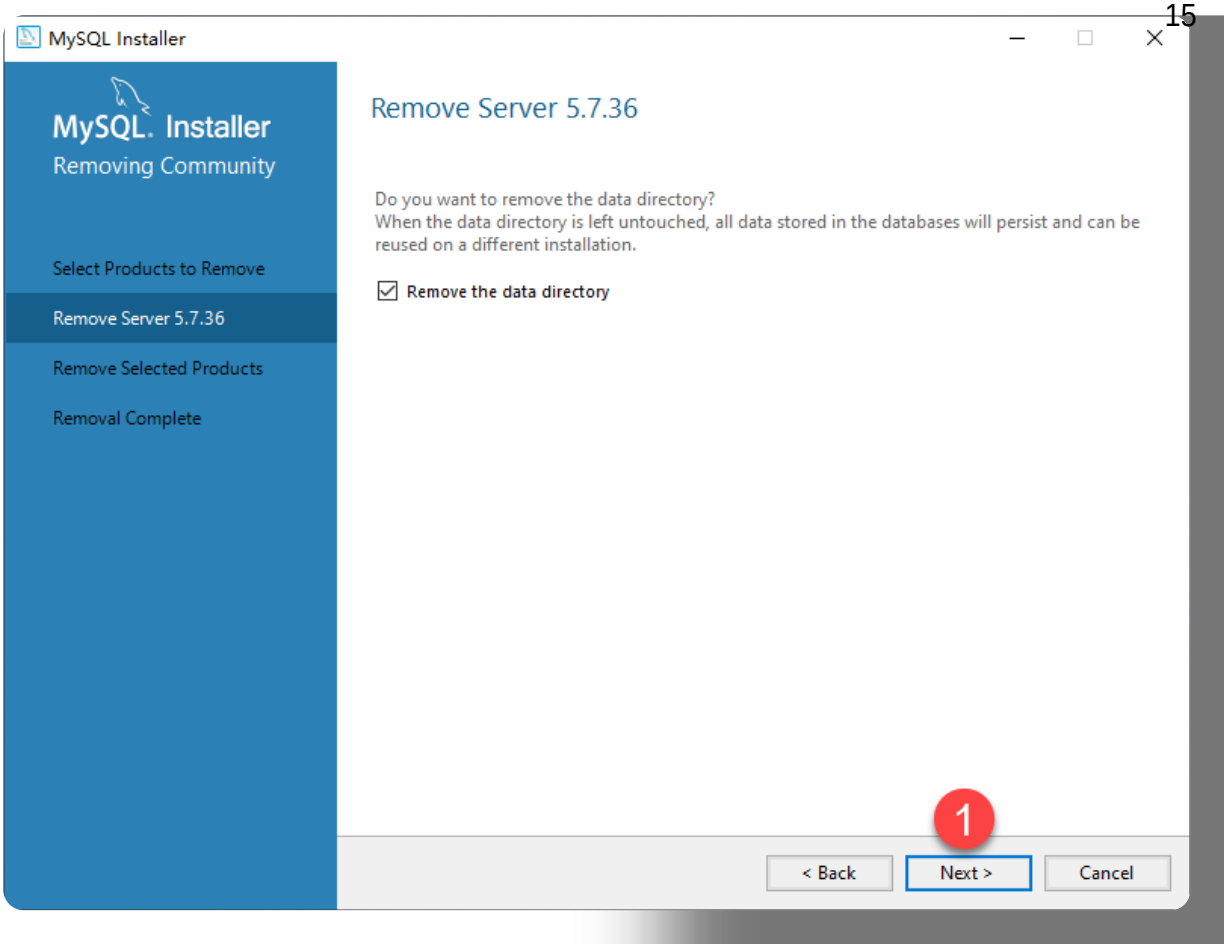

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Product Configuration
We'll now walk through a configuration wizard for each of the following products.
You can cancel at any point if you wish to leave this wizard without configuring all the products.
Product
Status
MySQL Server 5.7.36
Configuration complete.
1
Next >
Cancel

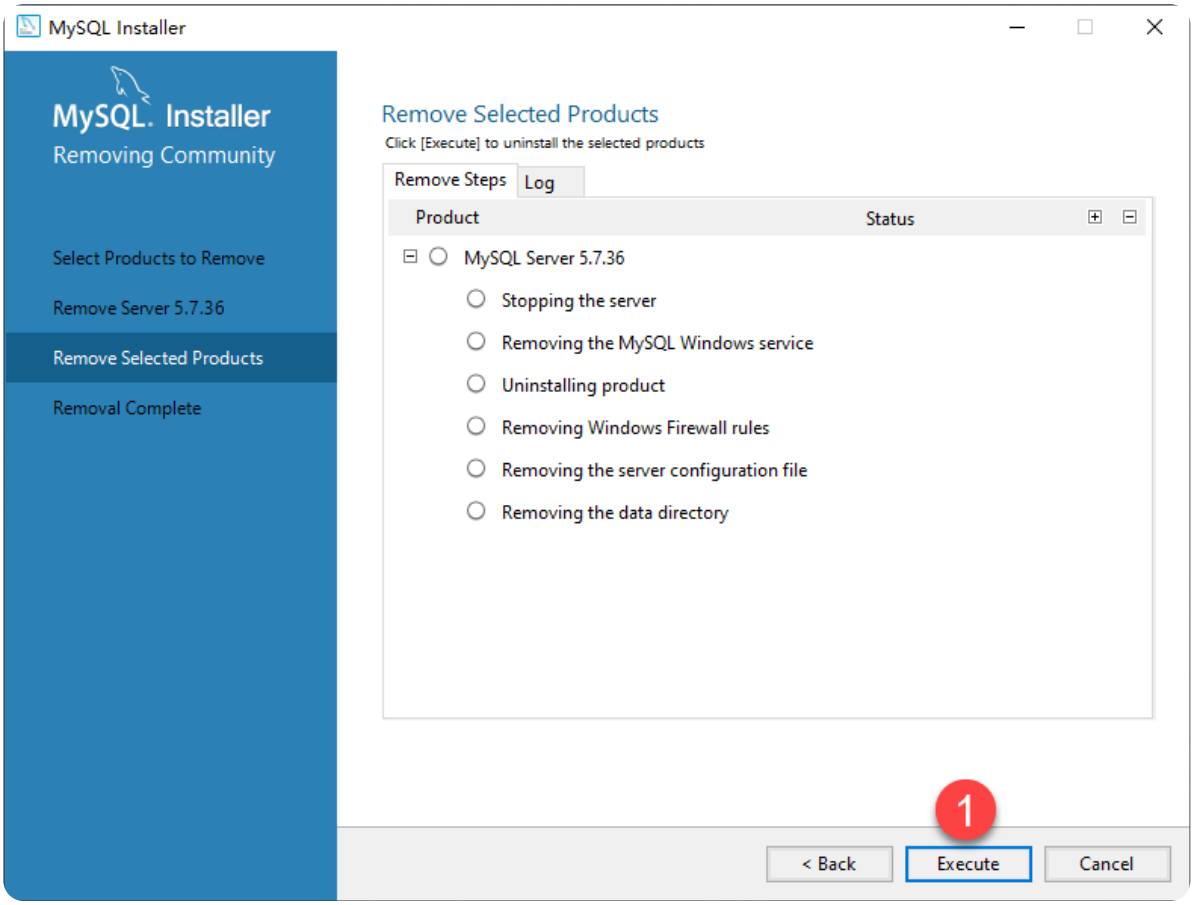

MySQL Installer
MySQL. Installer
Adding Community
Choosing a Setup Type
Select Products
Installation
Product Configuration
Installation Complete
Installation Complete
The installation procedure has been completed.
Copy Log to Clipboard
1
Finish

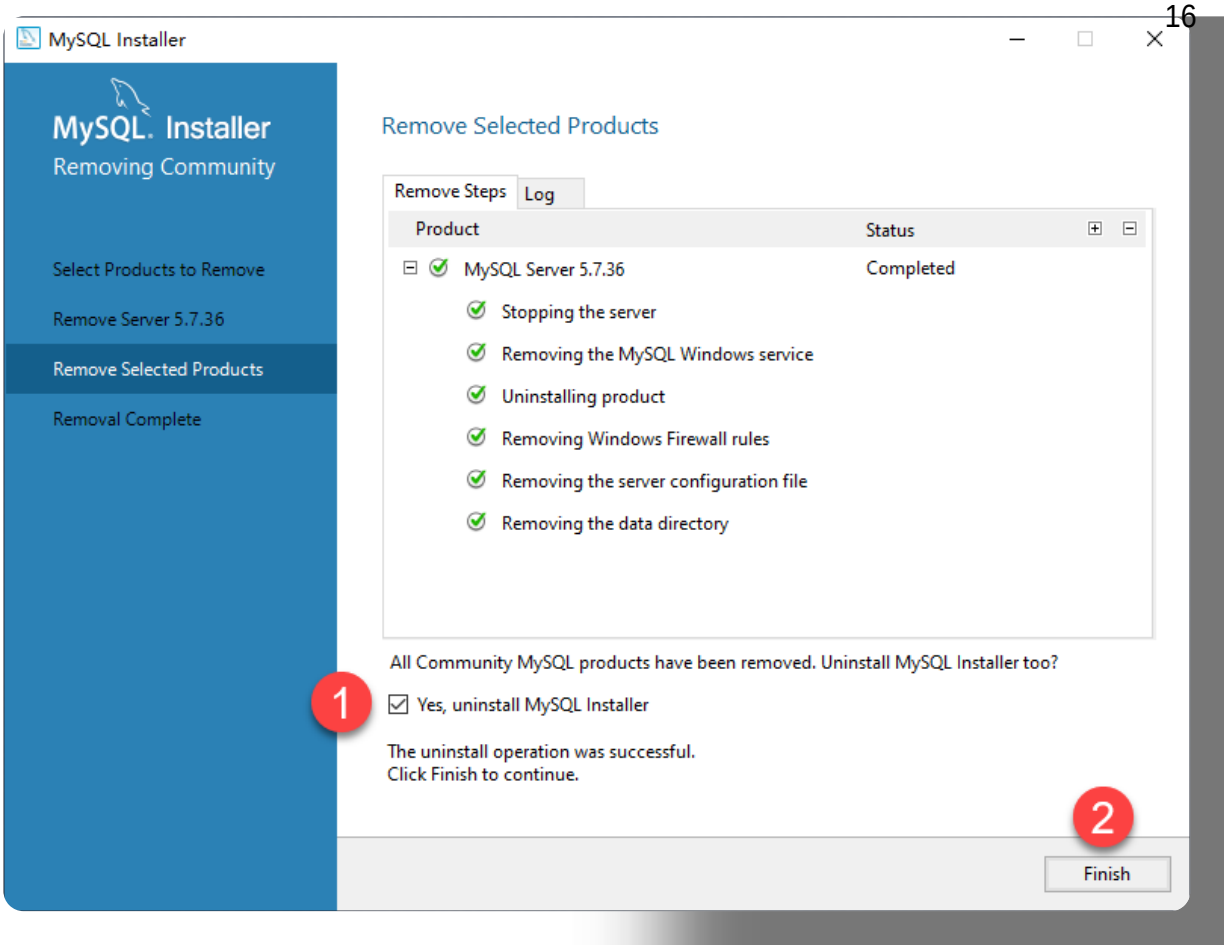
MSQL卸载

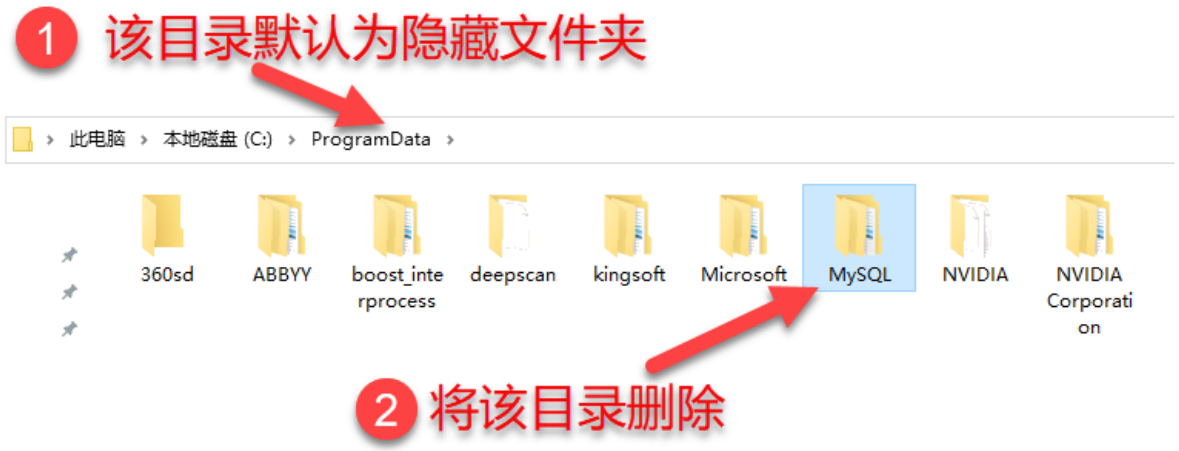

MySQL Installer
MySQL. Installer
Product
Version
Architecture
Quick Action
MySQL Server
5.7.36
X64
Reconfigure
Add ...
Modify ...
Upgrade ...
Remove ...
Product Information
Product Home:
https://dev.mysql.com/downloads/mysql/
Release Changes:
https://dev.mysql.com/doc/relnotes/mysql/5.7/en/news-5-7-36.html
Install Path:
C:\Program Files\MySQL\MySQL Server 5.7\


MySQL Installer
MySQL. Installer
Removing Community
Select Products to Remove
Remove Server 5.7.36
Remove Selected Products
Removal Complete
Select Products to Remove
Please check the products you would like to have removed.
Product
Status
MySQL Server 5.7.36
Pending for user input
1
Next >
Cancel

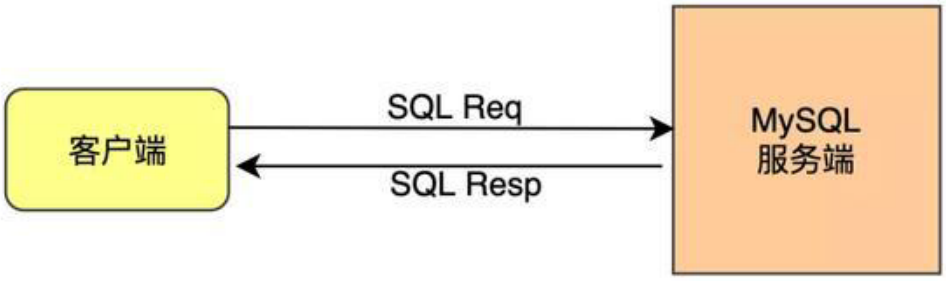

MySQL Installer
MySQL. Installer
Removing Community
Select Products to Remove
Remove Server 5.7.36
Remove Selected Products
Removal Complete
Remove Server 5.7.36
Do you want to remove the data directory?
When the data directory is left untouched, all data stored in the databases will persist and can be reused on a different installation.
Remove the data directory
1
< Back
Next >
Cancel


MySQL Installer
MySQL. Installer
Removing Community
Select Products to Remove
Remove Server 5.7.36
Remove Selected Products
Removal Complete
Remove Selected Products
Click [Execute] to uninstall the selected products
Remove Steps
Log
Product
Status
MySQL Server 5.7.36
Stopping the server
Removing the MySQL Windows service
Uninstalling product
Removing Windows Firewall rules
Removing the server configuration file
Removing the data directory
1
< Back
Execute
Cancel


MySQL Installer
MySQL. Installer
Removing Community
Select Products to Remove
Remove Server 5.7.36
Remove Selected Products
Removal Complete
Remove Selected Products
Remove Steps
Log
Product
Status
MySQL Server 5.7.36
Completed
Stopping the server
Removing the MySQL Windows service
Uninstalling product
Removing Windows Firewall rules
Removing the server configuration file
Removing the data directory
All Community MySQL products have been removed. Uninstall MySQL Installer too?
1
Yes, uninstall MySQL Installer
The uninstall operation was successful.
Click Finish to continue.
2
Finish

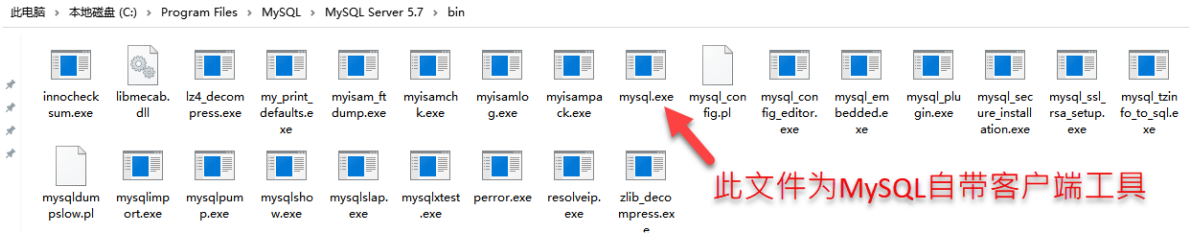

1 该目录默认为隐藏文件夹
此电脑 › 本地磁盘 (C:) › ProgramData ›
360sd
ABBYY
boost_interprocess
deepscan
kingsoft
Microsoft
MySQL
NVIDIA
NVIDIA Corporation
2 将该目录删除

连接MySQL

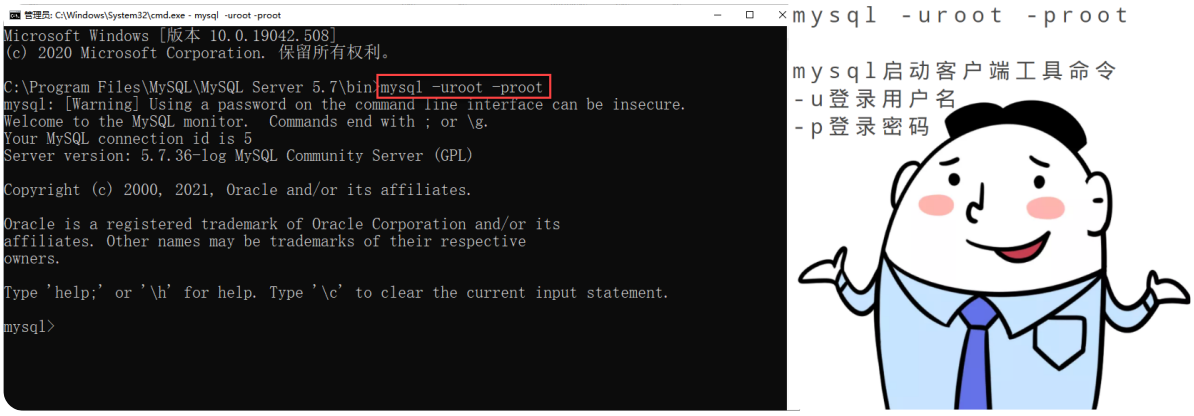


MySQL客户端与服务端通过SQL进行交互

通过MySQL自带客户端工具



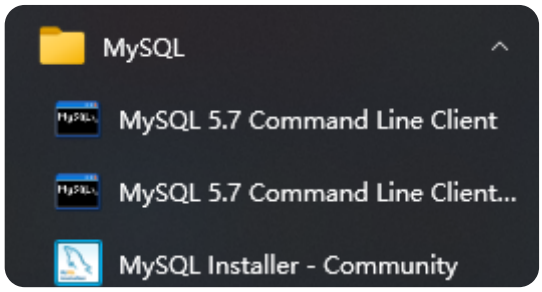
可通过MySQL自带的客户端工具链接MySQL。



通过客户端工具可对MySQL进行操作。



通过MySQL客户端工具的快捷方式连接MySQL





实时效果反馈

1.链接MySQL的命令是?

A sql

B ms

C plugin

D mysql

2.查看MySQL中所有数据库命令是?

A show databases;

B show database;

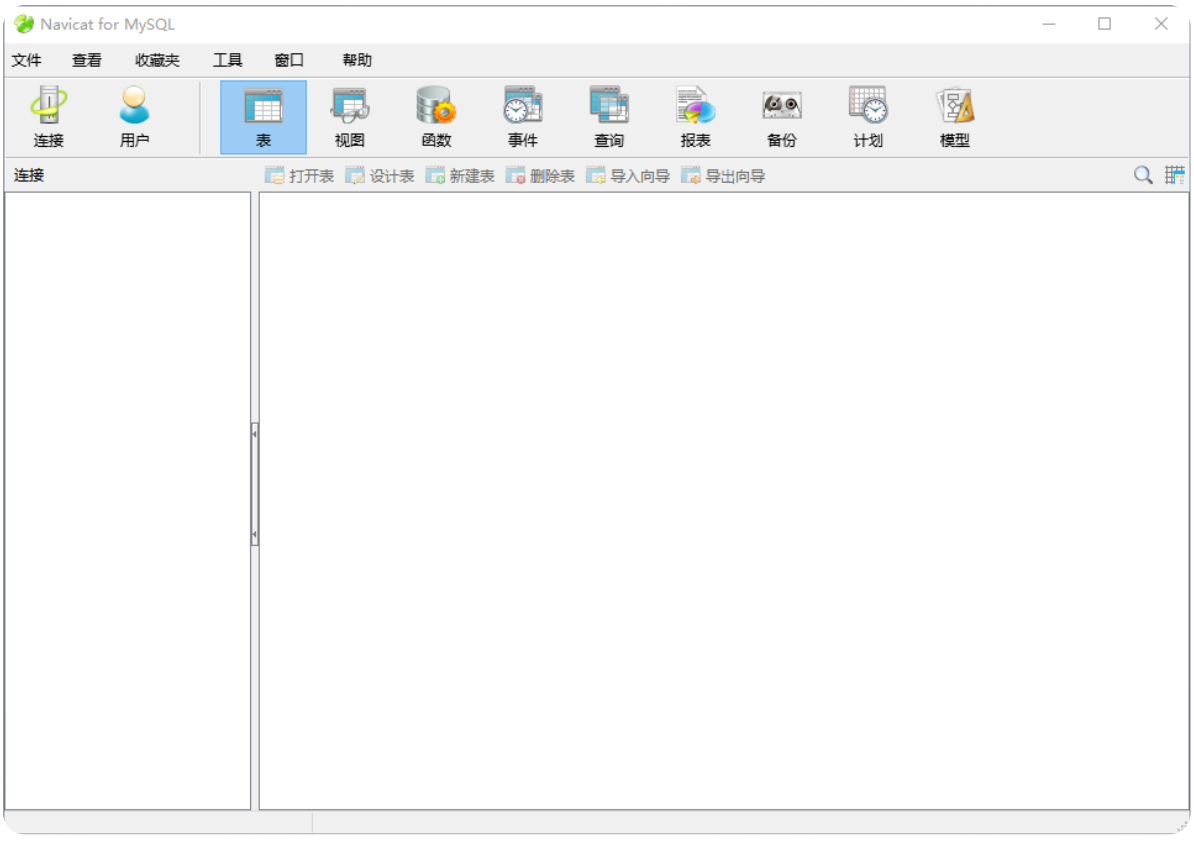
C show dbase;

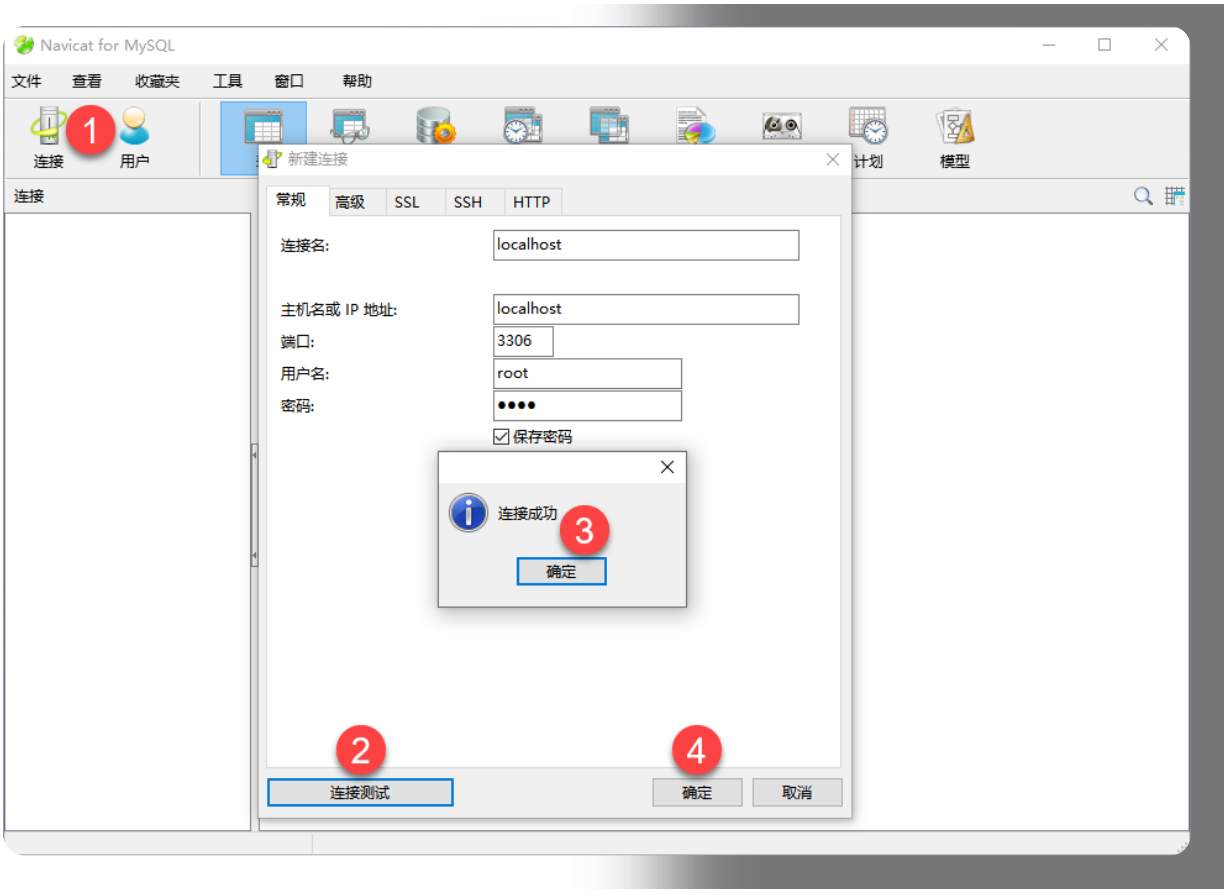
D show data;

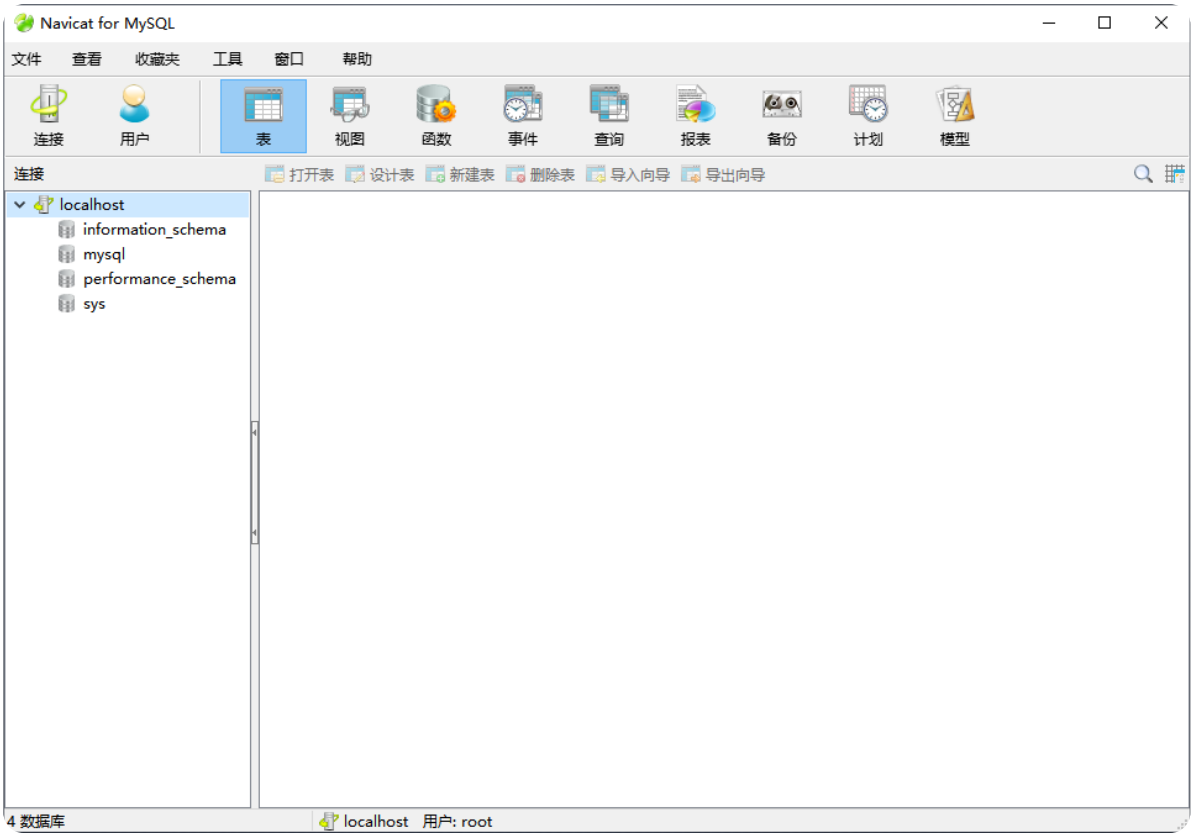
答案

1=>D 2=>A

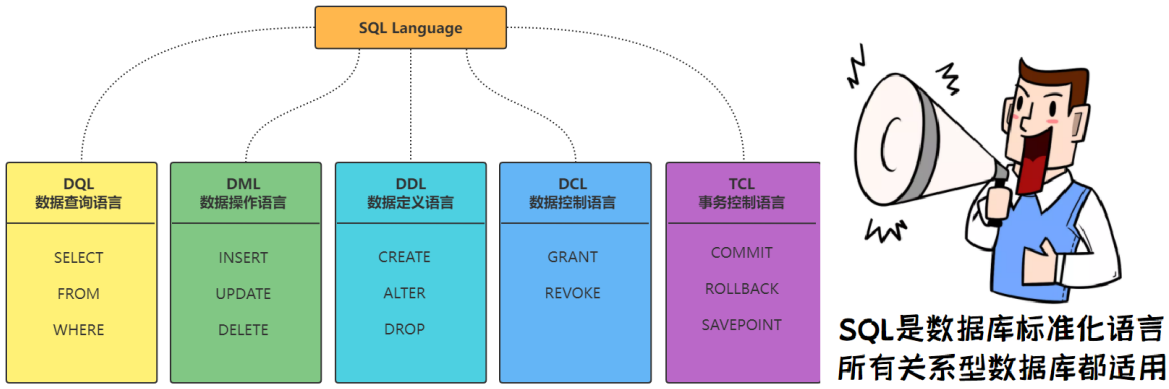
Navicat工具的使用




Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户
计划 模型
连接
新建连接
常规 高级 SSL SSH HTTP
连接名: localhost
主机名或 IP 地址: localhost
端口: 3306
用户名: root
密码: ••••
保存密码
连接成功
确定
1
2
3
4
连接测试
确定 取消


Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
information_schema
mysql
performance_schema
sys
4 数据库
localhost 用户: root

SQL语言



SQL语言简介

结构化查询语言(Structured Query Language)简称 SQL(发音:sequal['si:kwəl])，是一种数据库查询和程序设计语言，用于存取数据以及查询、更新和管理关系数据库系统。

SQL 能做什么?

- SQL 面向数据库执行查询
- SQL 可在数据库中插入新的记录
- SQL 可更新数据库中的数据
- SQL 可从数据库删除记录
- SQL 可创建新数据库
- SQL 可在数据库中创建新表
- SQL 可在数据库中创建存储过程
- SQL 可在数据库中创建视图
- SQL 可以设置表、存储过程和视图的权限

SQL 标准

SQL 是 1986 年 10 月由美国国家标准局（ANSI）通过的数据库语言美国标准，接着，国际标准化组织（ISO）颁布了 SQL 正式国际标准。1989 年 4 月，ISO 提出了具有完整性特征的 SQL89 标准，1992 年 11 月又公布了 SQL92 标准，在此标准中，把数据库分为

三个级别：基本集、标准集和完全集。在 1999 年推出 99 版标准。最新版本为 SQL2016 版。比较有代表性的几个版本：SQL86、SQL92、SQL99。

SQL语言分类

1. 数据查询语言（DQL：Data Query Language）其语句，也称为“数据检索语句”，用以从表中获得数据，确定数据怎样在应用程序给出。关键字 SELECT 是 DQL（也是所有 SQL）用得最多的动词。
   - SELECT
   - FROM
   - WHERE
   - ORDER BY
   - HAVING
2. 数据操作语言（DML：Data Manipulation Language）其语句包括动词 INSERT，UPDATE 和 DELETE。它们分别用于添加，修改和删除表中的行。
   - INSERT：添加数据
   - UPDATE：更新数据
   - DELETE：删除数据
3. 数据定义语言（DDL：Data Definition Language）定义数据库对象语言，其语句包括动词 CREATE 和 DROP 等。
   - CREATE：创建数据库对象
   - ALTER：修改数据库对象
   - DROP：删除数据库对象
4. 数据控制语言（DCL：Data Control Language）它的语句通过 GRANT 或 REVOKE 获得许可，确定用户对数据库对象的访问。
   - GRANT：授予用户某种权限
   - REVOKE：回收授予的某种权限
5. 事务控制语言（TCL ： Transaction Control Language）它的语句能确保被 DML 语句影响的表的所有行及时得以更新。
   - COMMIT：提交事务

- ROLLBACK：回滚事务
- SAVEPOINT：设置回滚点

> 注意：
>
> 数据操纵语言DML（insert、update、delete）针对表中的数据；
>
> 而数据定义语言DDL（create、alter、drop）针对数据库对象，比如数据库database、表table、索引index、视图view、存储过程procedure、触发器trigger；

SQL语言语法

1. SQL语句不区分大小写，关键字建议大写。
2. SQL语句可以单行或多行书写，以分号结尾。

实时效果反馈

1.SQL语言语法中要求关键字。

A 不区分大小写

B 区分大小写

C 可简写

D 可省略

2.如下说法正确的是。

A SQL语句必须多行书写，以分号结尾；

B SQL语句必须单行书写，以分号结尾；

C SQL语句可以单行或多行书写，以冒号结尾;

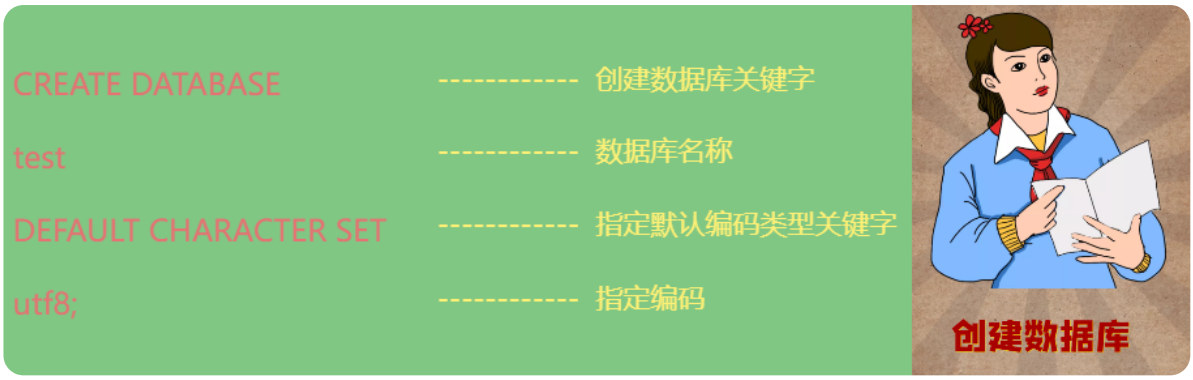
D SQL语句可以单行或多行书写，以分号结尾;

答案

1=>A　　2=>D

创建与删除数据库

创建数据库



1. 使用DDL语句创建数据库

```
CREATE DATABASE 数据库名 DEFAULT CHARACTER SET 字符编码;
```

示例：
创建一个test 的数据库，并查看该数据库，以及该数据库的编码。
创建数据库：

```
create database test default character set utf8;
```

查看数据库：

```
show databases;
```

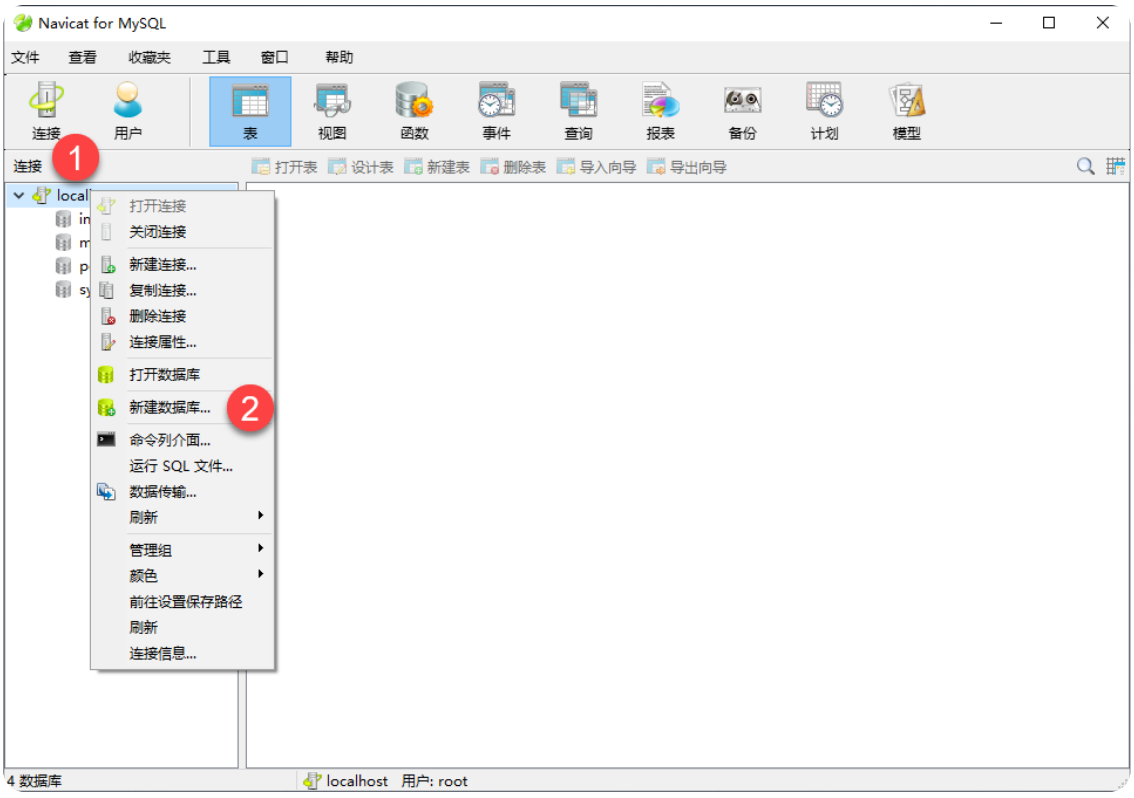
查看数据库编码：

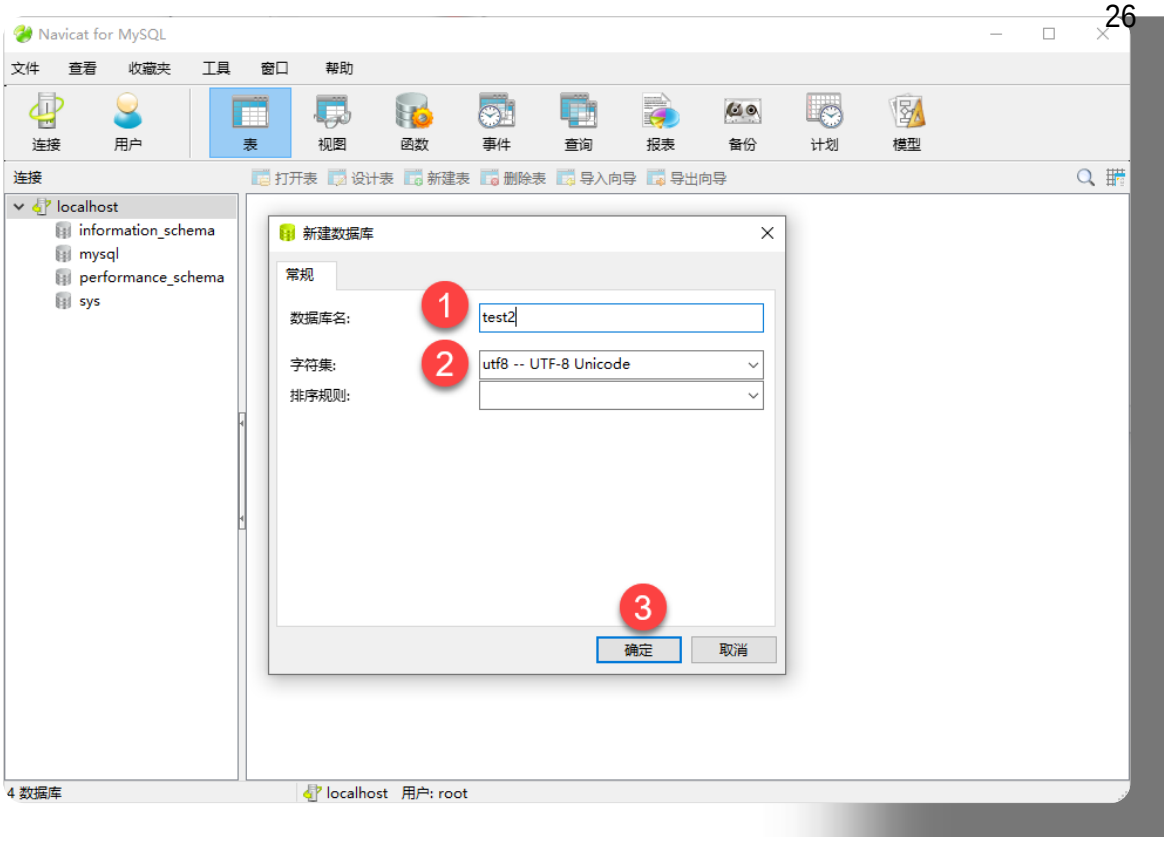
```
select
schema_name,default_character_set_name
from information_schema.schemata
where schema_name = 'test';
```

2. 使用Navicat创建数据库

示例：

创建一个test2 的数据库。





删除数据库



1. 使用DDL语言删除数据库

```
DROP DATABASE 数据库名称;
```

示例：

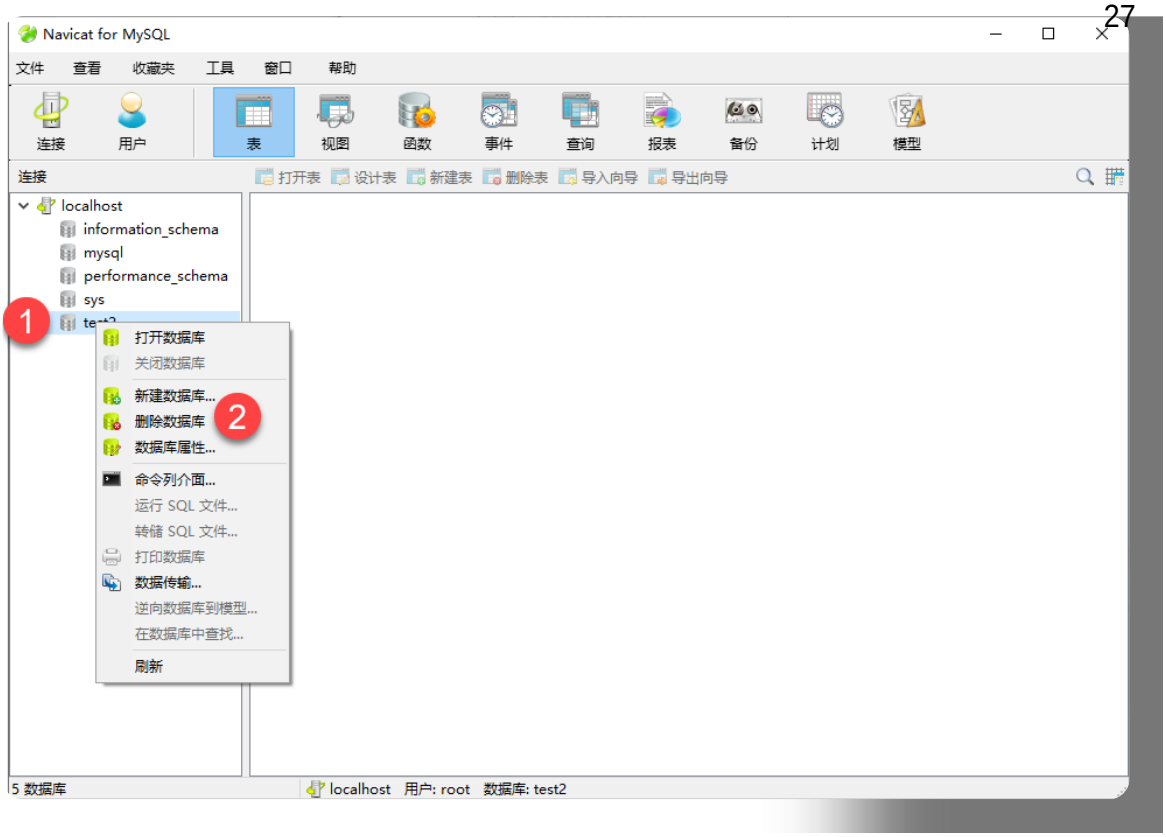
删除 test 数据库

```
drop database test;
```

2. 使用Navicat删除数据库

示例：

删除test2数据库



选择数据库

在创建表时，需要先选择数据库。

```
USE 数据库名;
```

示例：

创建一个名称为 bjsxt 的数据库，编码为 utf8。

```
create database bjsxt default character set utf8;
```

选择该数据库。

```
USE bjsxt;
```

实时效果反馈

1.创建数据库语句: CREATE　数据库名 DEFAULT CHARACTER SET 字符编码;

A DATABASE

B DATABASES

C DATA

D BASE

2.删除数据库语句:　DATABASE　数据库名称

A REMOVE

B DELETE

C DROP

D DROPS

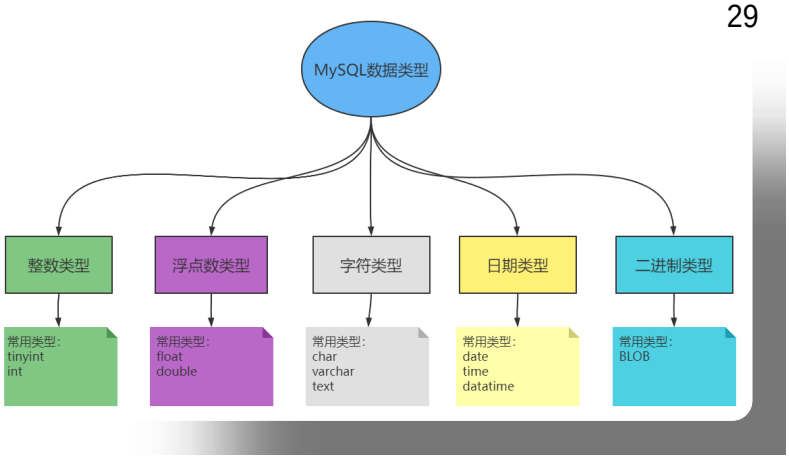
答案

1=>A　　2=>C

MySQL中的数据类型

MySQL数据库数据类型分类





整数类型

MySQL数据类型	含义（有符号）
tinyint(m)	1个字节 范围(-128~127)
smallint(m)	2个字节 范围(-32768~32767)
mediumint(m)	3个字节 范围(-8388608~8388607)
int(m)	4个字节 范围(-2147483648~2147483647)
bigint(m)	8个字节 范围(+-9.22*10的18次方)

数值类型中的长度 m 是指显示长度，并不表示存储长度，只有字段指定 zerofill 时有用

例如： int(3)，如果实际值是 2，如果列指定了 zerofill，查询结果就是 002，左边用 0 来 填充

浮点类型

MySQL数据类型	含义
float(m,d)	单精度浮点型 8位精度(4字节) m总个数，d小数位
double(m,d)	双精度浮点型 16位精度(8字节) m总个数，d小数位

字符类型

MySQL数据类型	含义
char(n)	固定长度，最多255个字符
tinytext	可变长度，最多255个字符
varchar(n)	可变长度，最多65535个字符
text	可变长度，最多65535个字符
mediumtext	可变长度，最多2的24次方-1个字符
longtext	可变长度，最多2的32次方-1个字符

char和varchar：

1. char长度固定， 即每条数据占用等长字节空间；适合用在身份证号码、手机号码等定长。
2. varchar可变长度，可以设置最大长度；适合用在长度可变的属性。
3. text不设置长度， 当不知道属性的最大长度时，适合用text。

按照查询速度： char最快， varchar次之，text最慢。

字符串型使用建议：

1. 经常变化的字段用varchar
2. 知道固定长度的用char
3. 尽量用varchar
4. 超过255字符的只能用varchar或者text
5. 能用varchar的地方不用text

日期类型

MySQL数据类型	含义
date	日期 YYYY-MM-DD
time	时间 HH:MM:SS
datetime	日期时间 YYYY-MM-DD HH:MM:SS
timestamp	时间戳YYYYMMDD HHMMSS

二进制数据(BLOB)

1. BLOB和TEXT存储方式不同，TEXT以文本方式存储，英文存储区分大小写，而Blob是以二进制方式存储，不分大小写。
2. BLOB存储的数据只能整体读出。
3. TEXT可以指定字符集，BLOB不用指定字符集。

实时效果反馈

1.数值类型中的 int(m)其中m是指，只有字段指定 zerofill 时有用。

A 存储长度

B 显示长度

C 固定长度

D 可变长度

2.浮点类型中的double(m,d)其中m是指。

A 整数长度，

B 小数长度

C 总长度；

D 整数最大值;

答案

1=>B　2=>C

创建表与删除表

创建表

CREATE TABLE ---------- 创建表关键字

表名(列名 类型,列名 类型.....); ---------- 表名、列名与类型



1. 使用DDL语句创建表

```
CREATE TABLE 表名(列名 类型,列名 类型.....);
```

示例：

创建一个 employees 表包含雇员 ID ， 雇员名字，雇员薪水。

```
create table employees(employee_id
int,employee_name varchar(10),salary
float(8,2));
```

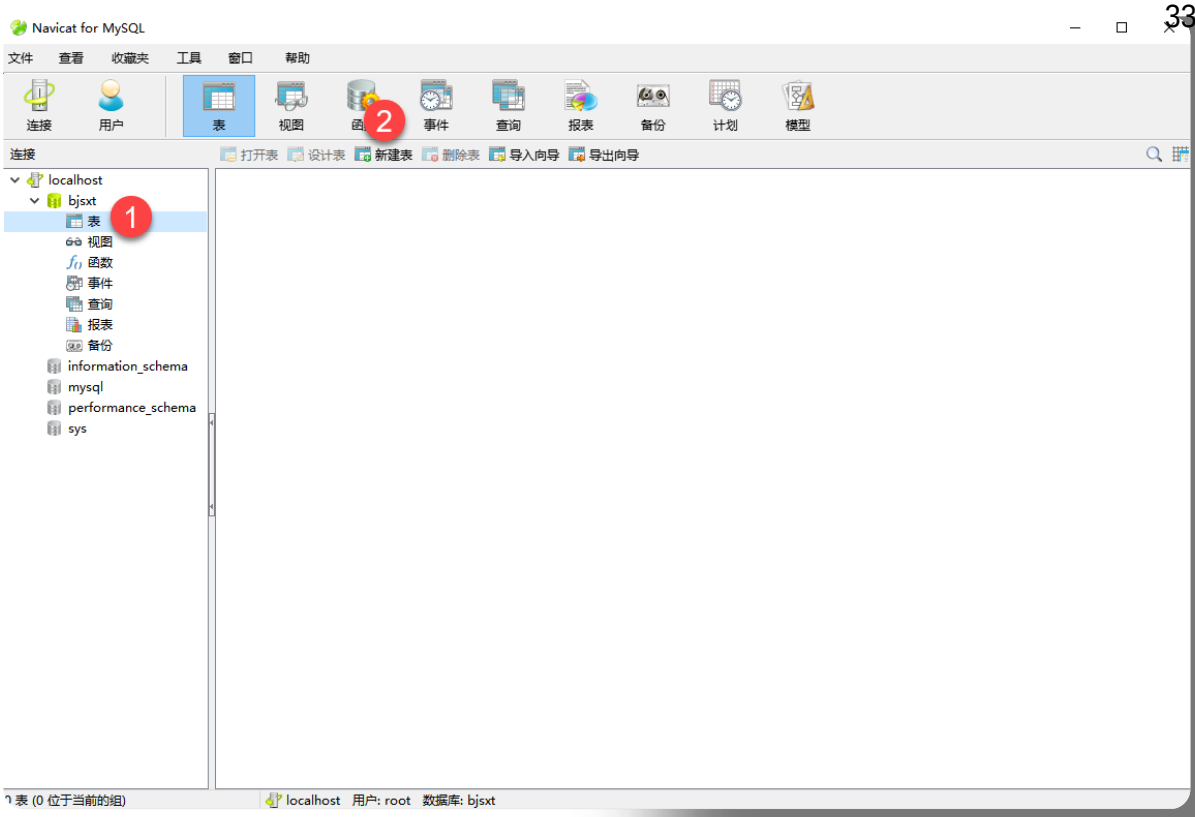
查看已创建的表。

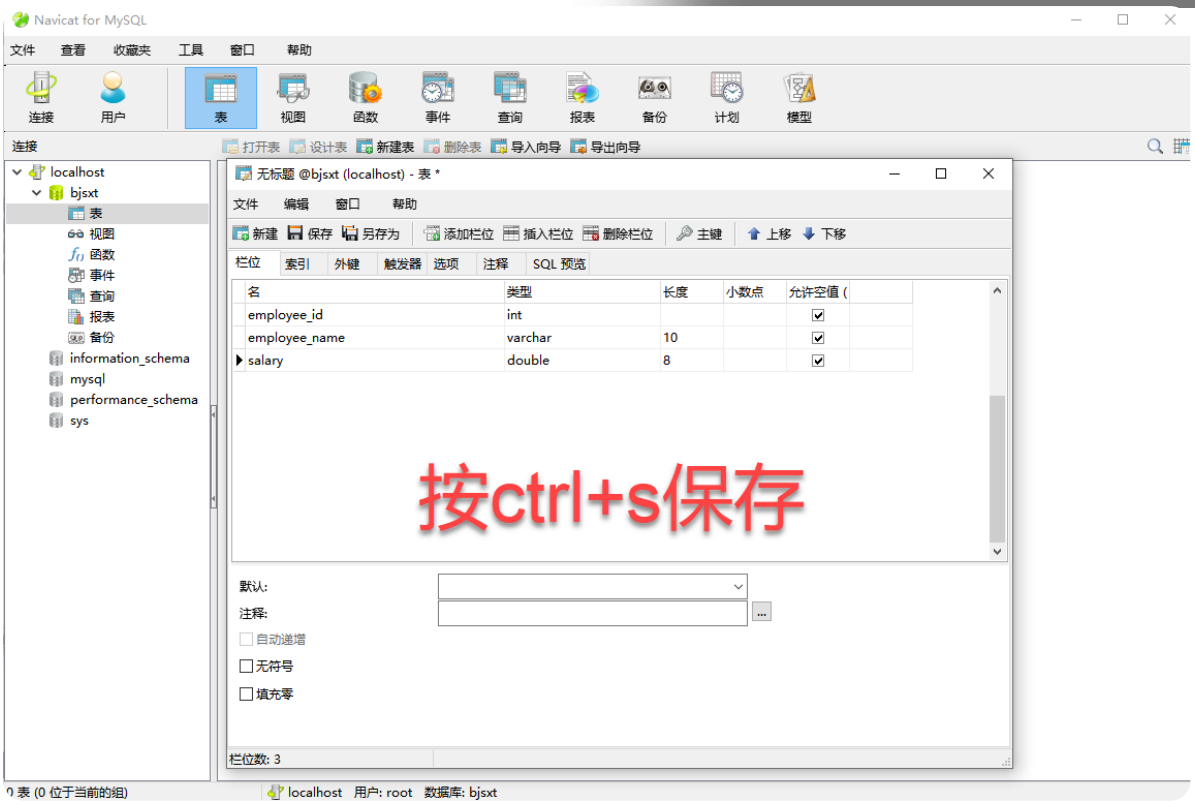
```
show tables;
```

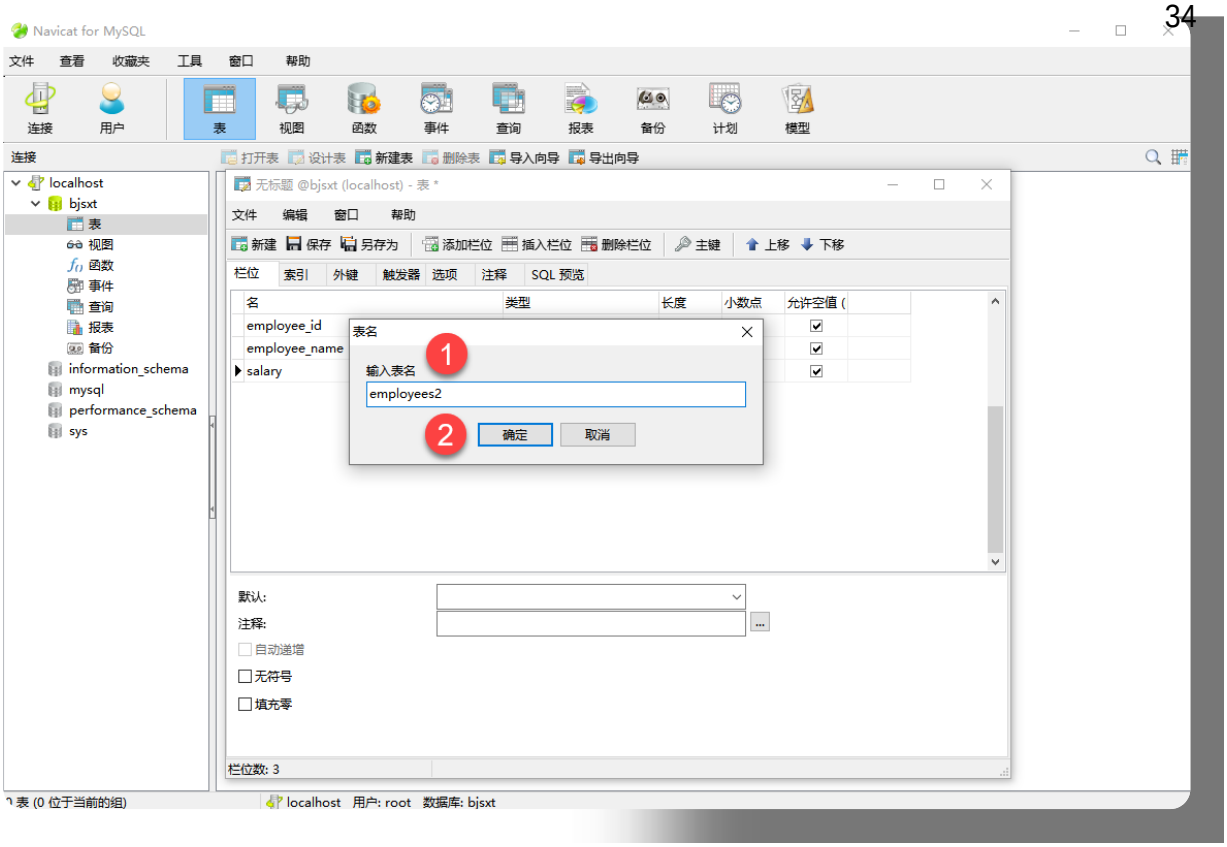
2. 使用Navicat创建表

示例：

创建employees2表。


Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接 打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
bjsxt
表
视图
函数
事件
查询
报表
备份
information_schema
mysql
performance_schema
sys
1
2
表 (0 位于当前的组)
localhost 用户: root 数据库: bjsxt


Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接 打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
bjsxt
表
视图
函数
事件
查询
报表
备份
information_schema
mysql
performance_schema
sys
无标题 @bjsxt (localhost) - 表 *
文件 编辑 窗口 帮助
新建 保存 另存为 添加栏位 插入栏位 删除栏位 主键 上移 下移
栏位 索引 外键 触发器 选项 注释 SQL 预览
名 类型 长度 小数点 允许空值 (
employee_id int
employee_name varchar 10
salary double 8
按ctrl+s保存
默认:
注释:
自动递增
无符号
填充零
栏位数: 3
表 (0 位于当前的组)
localhost 用户: root 数据库: bjsxt



删除表

DROP TABLE ---------- 删除表关键字

表名; ---------- 表名



1. 使用DDL语句删除表

```
DROP TABLE 表名;
```

示例：

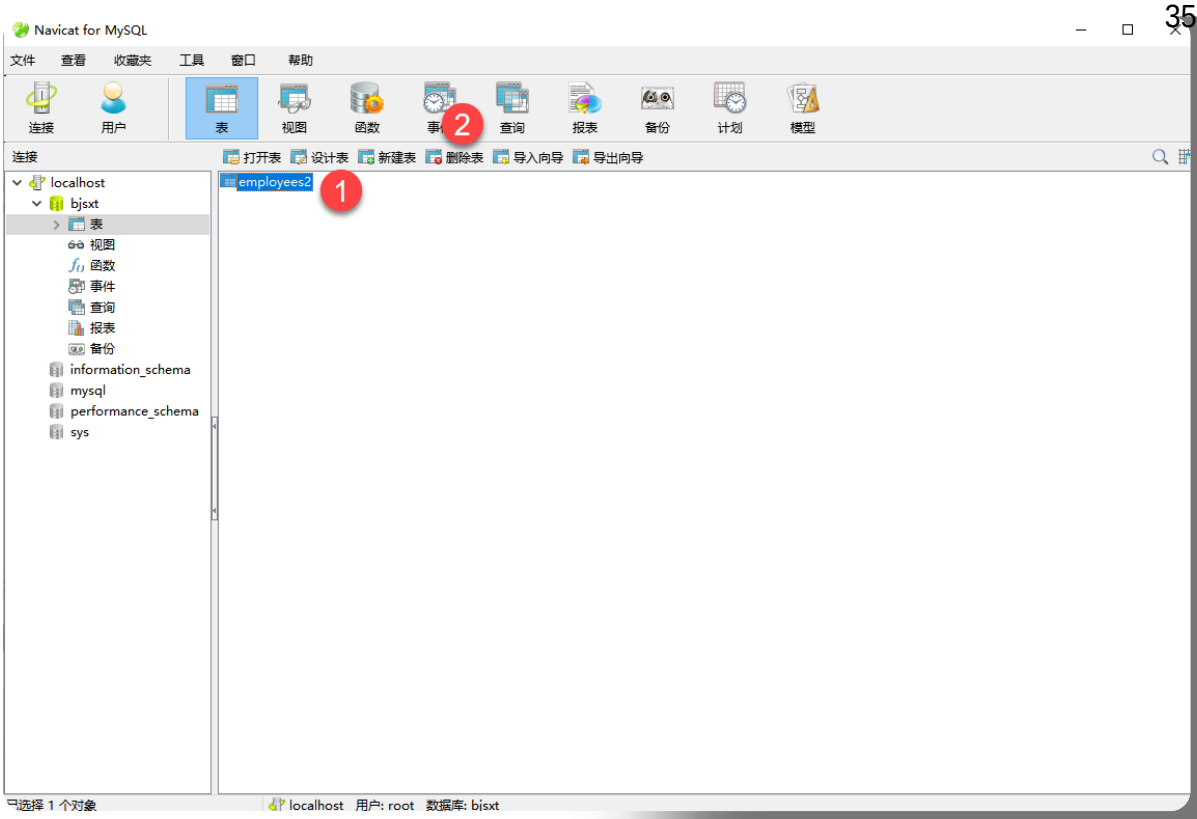
删除 employees 表。

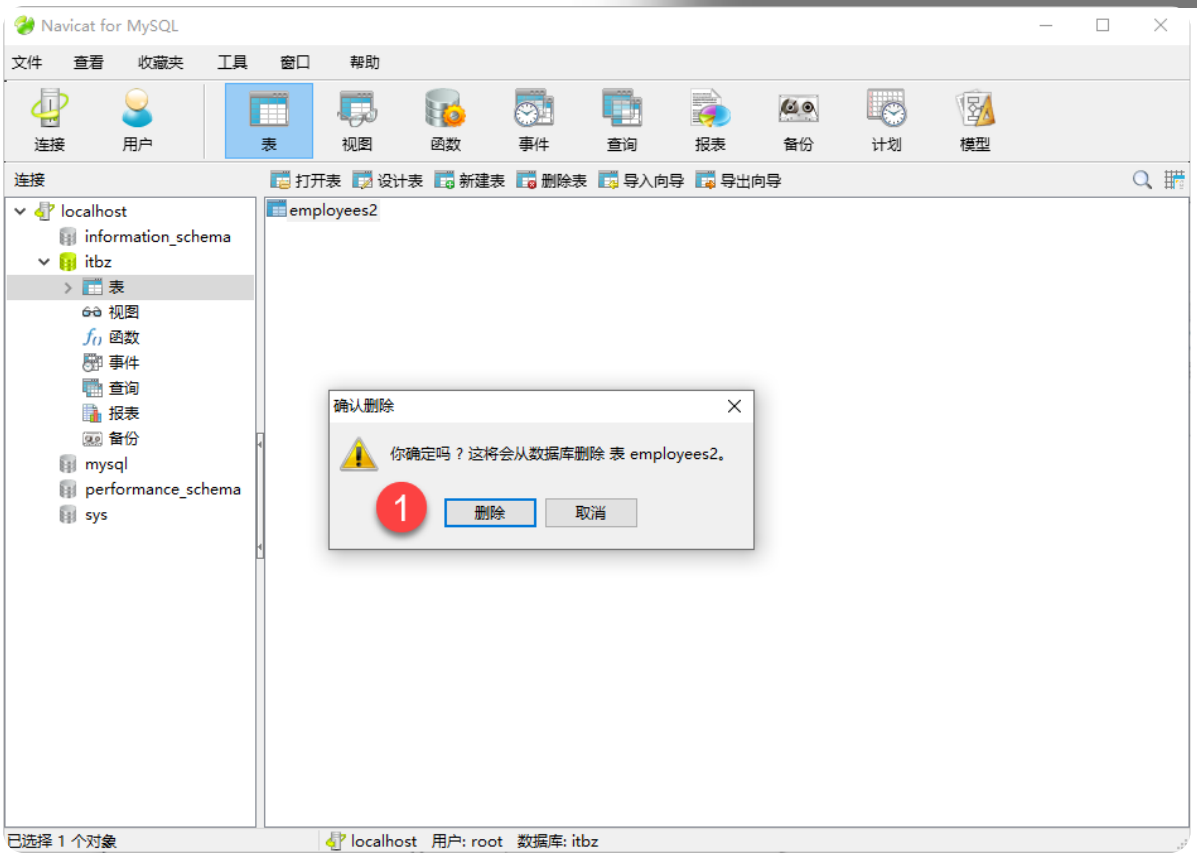
```
drop table employees;
```

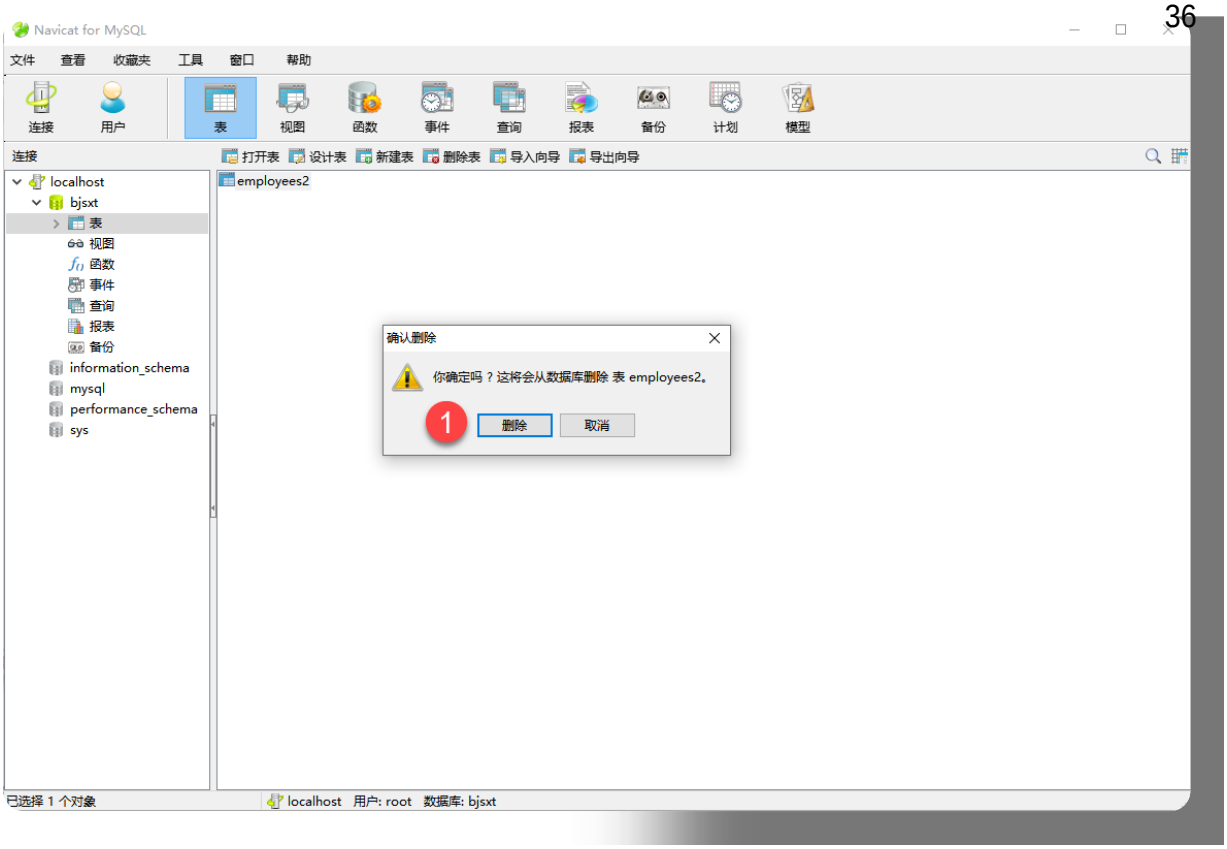
2. 使用Navicat删除表

示例：

删除employees2表


Navigate for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
bjsxt
表
视图
函数
事件
查询
报表
备份
information_schema
mysql
performance_schema
sys
employees2
1
2
已选择 1 个对象
localhost 用户: root 数据库: bjsxt


Navigate for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
information_schema
itbz
表
视图
函数
事件
查询
报表
备份
mysql
performance_schema
sys
employees2
确认删除
你确定吗？这将会从数据库删除 表 employees2。
1
删除
取消
已选择 1 个对象
localhost 用户: root 数据库: itbz



实时效果反馈

1.创建表语句: CREATE (列名 类型,列名 类型.....);

A 表名 TABLES

B TABLES 表名

C 表名 TABLE

D TABLE 表名

2.删除表语句： 表名;

A REMOVE TABLE

B DELETE TABLE

C DROP TABLE

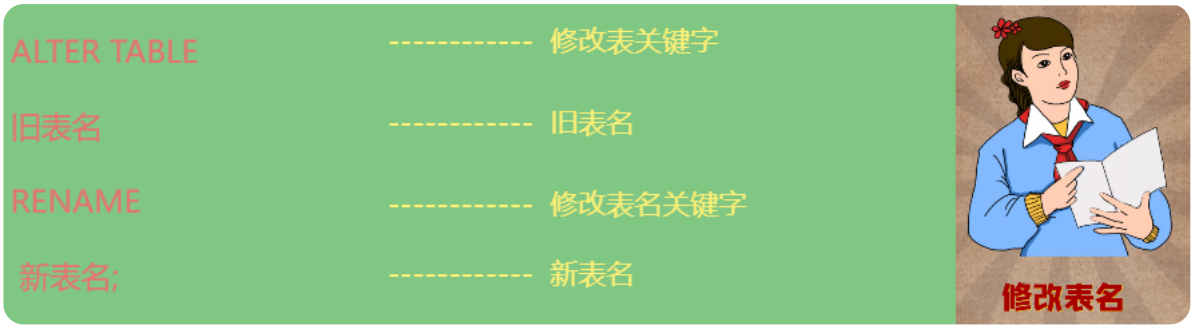
D TABLE DROP

答案

1=>D　2=>C

修改表

修改表名



1. 使用DDL语句修改表

```
ALTER TABLE  旧表名 RENAME  新表名;
```

示例一：

创建一个 employees 表包含雇员 ID，雇员名字，雇员薪水。

```
create table employees(employee_id
int,employee_name varchar(10),salary
float(8,2));
```

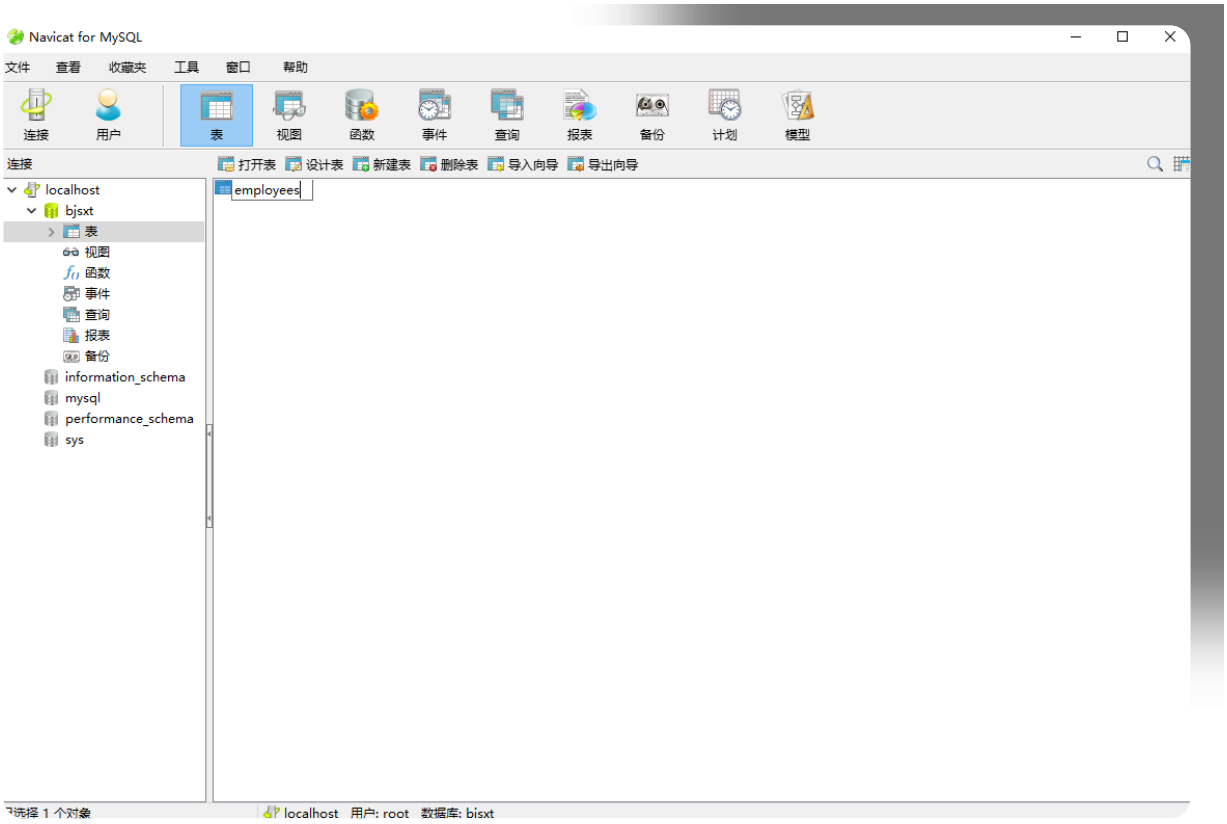
示例二：

将 employees 表名修改为 emp。

```
alter table employees rename emp;
```

2 使用Navicat修改表名

选择表按F2。



实时效果反馈

1.修改表名语句：ALTER TABLE 旧表名 新表名;

A RENAME

B NAME

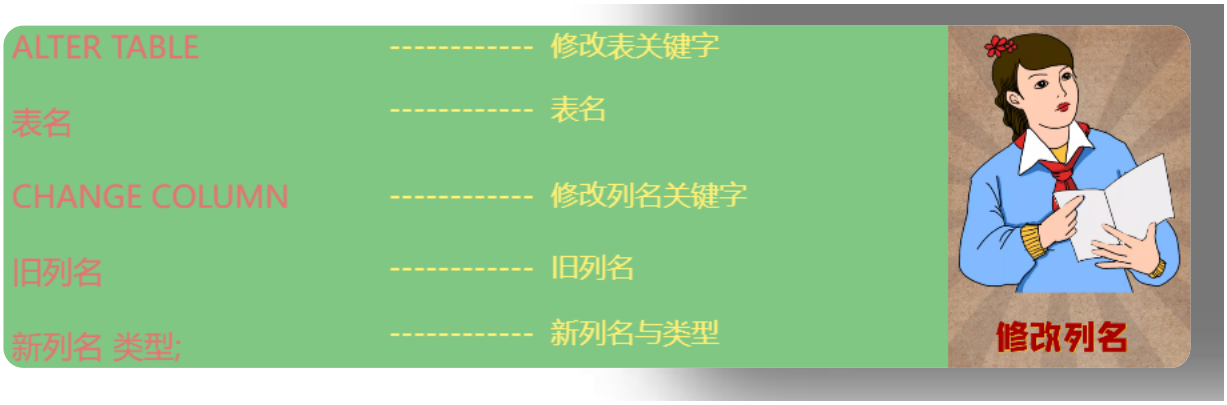
C UPDATE NAME

D MODIFY NAME

答案

1=>A

修改列名



1. 使用DDL语句修改列名

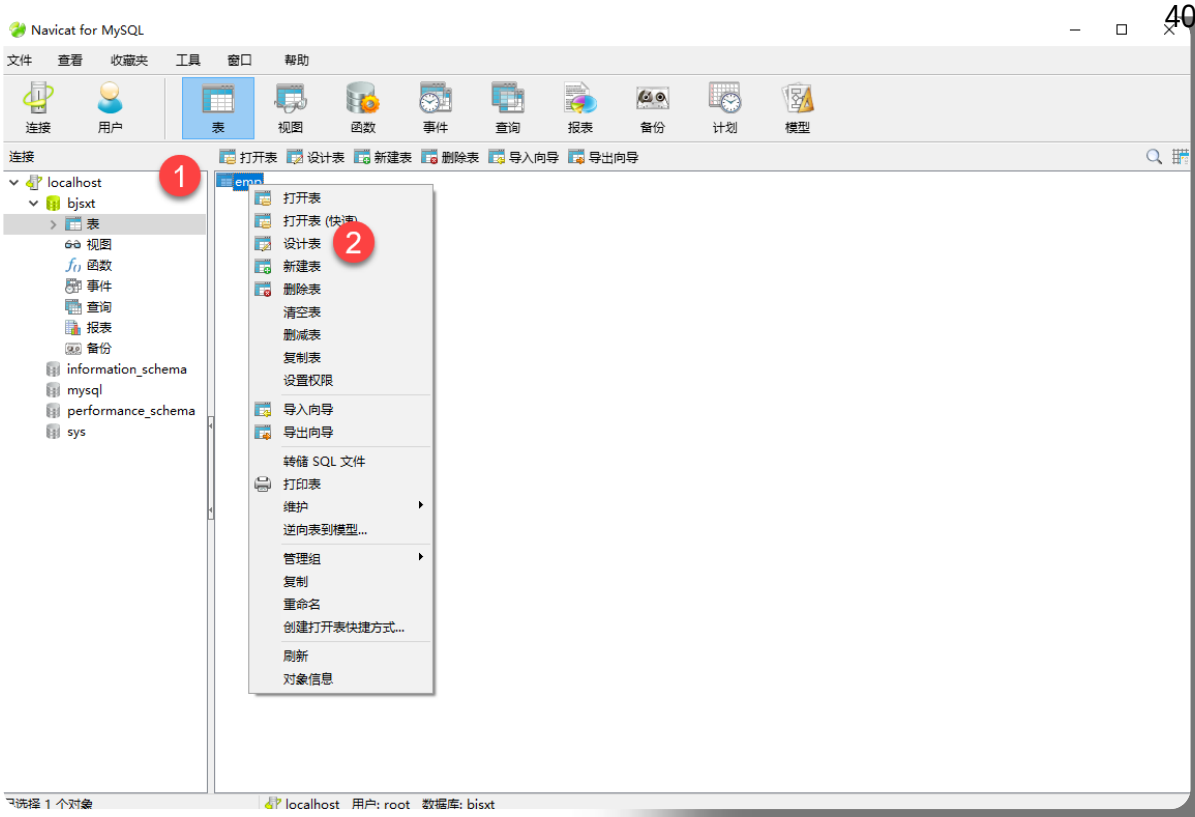
```
ALTER TABLE 表名 CHANGE COLUMN 旧列名 新列名 类型;
```

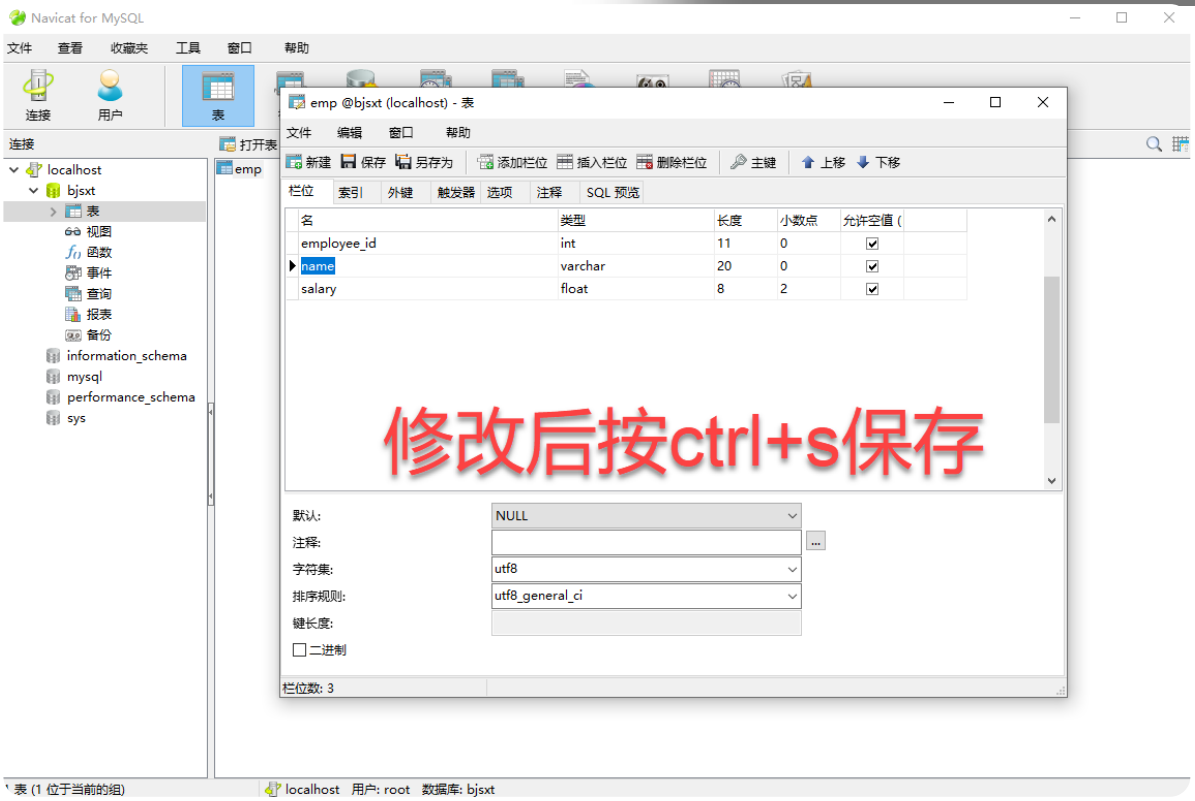
示例：

将 emp 表中的 employee_name 修改为 name。

```
alter table emp change column employee_name name varchar(20);
```

2. 使用Navicat修改列名





实时效果反馈

1.修改列名语句：ALTER TABLE 表名 旧列名 新列名 类型;

A MODIFY COLUMN

B UPDATE COLUMN

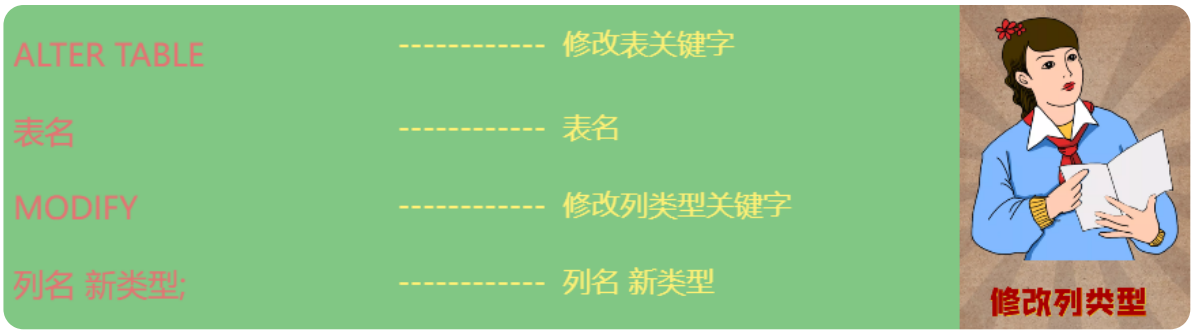
C CHANGE COLUMN

D COLUMN

答案

1=>C

修改列类型



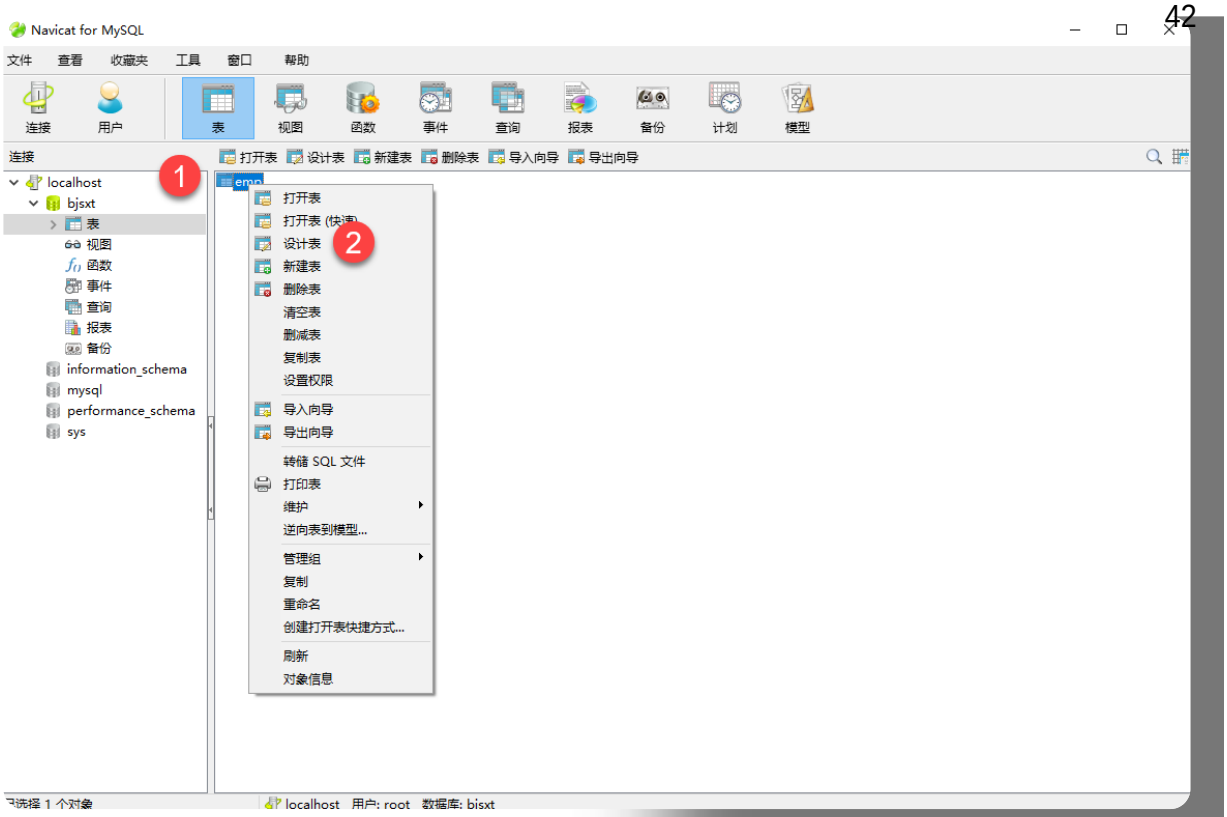
1. 使用DDL语句修改列类型

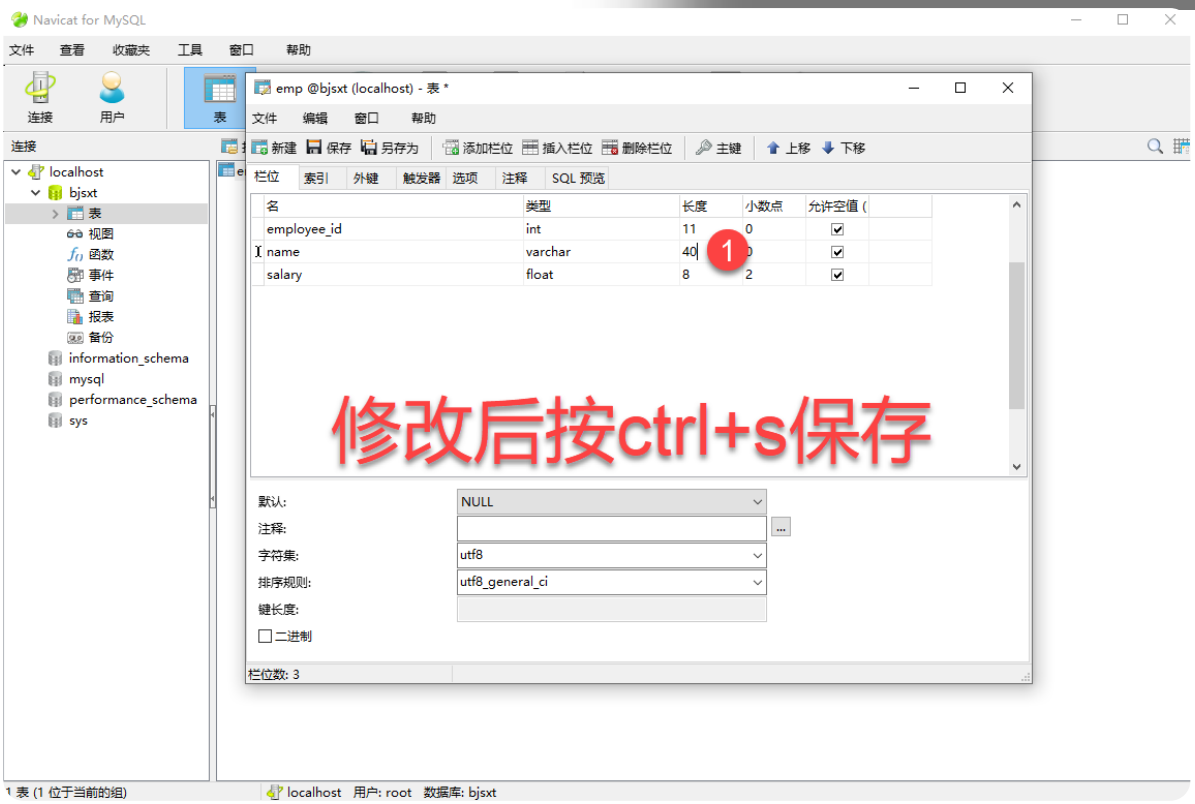
```
ALTER TABLE  表名 MODIFY  列名 新类型;
```

示例：
将 emp 表中的 name 的长度指定为 40。

```
alter table emp modify name varchar(40);
```

2. 使用Navicat修改列类型





实时效果反馈

1.修改列类型语句：ALTER TABLE 表名 ；

A UPDATE 列名

B UPDATE 列名 新类型

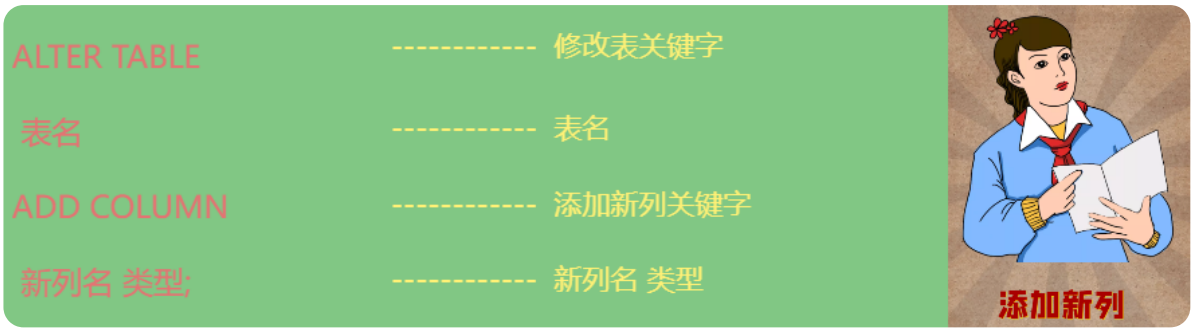
C MODIFY 列名

D MODIFY 列名 新类型

答案

1=>D

添加新列



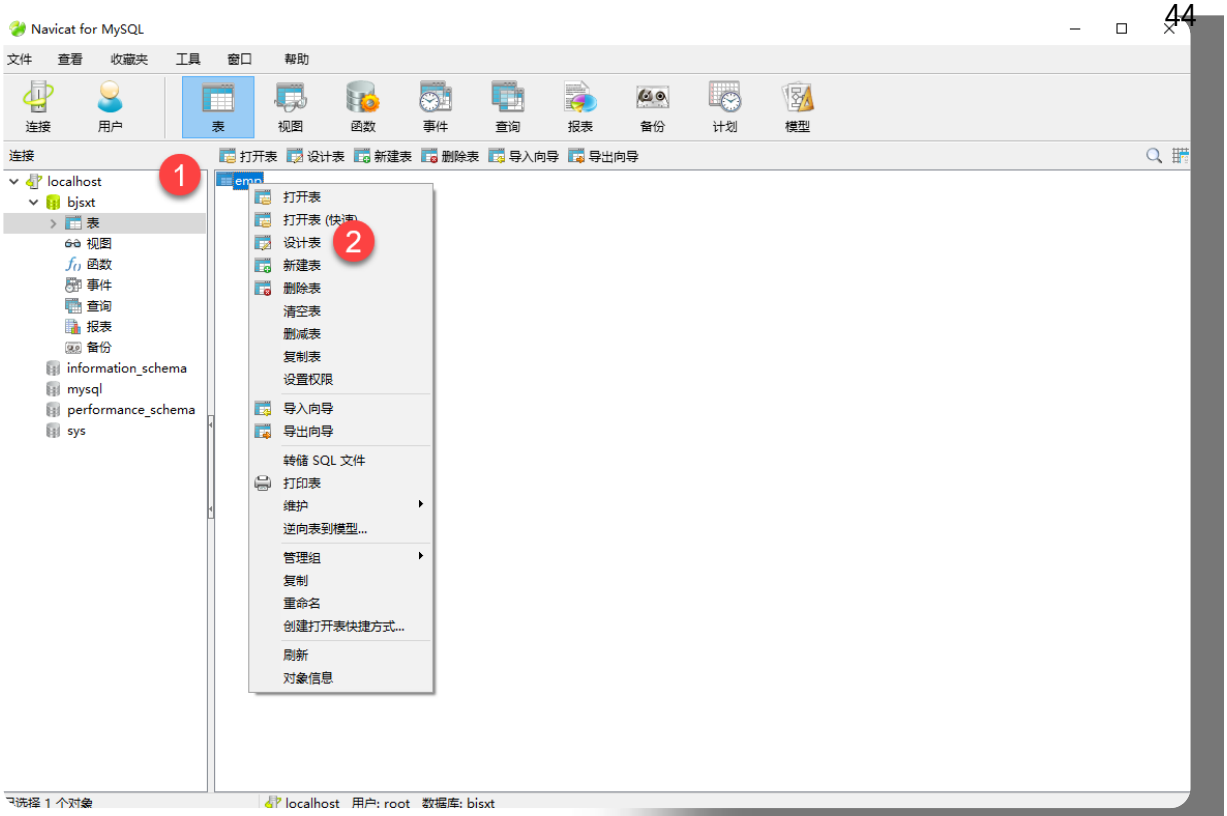
1. 使用DDL语句添加新列

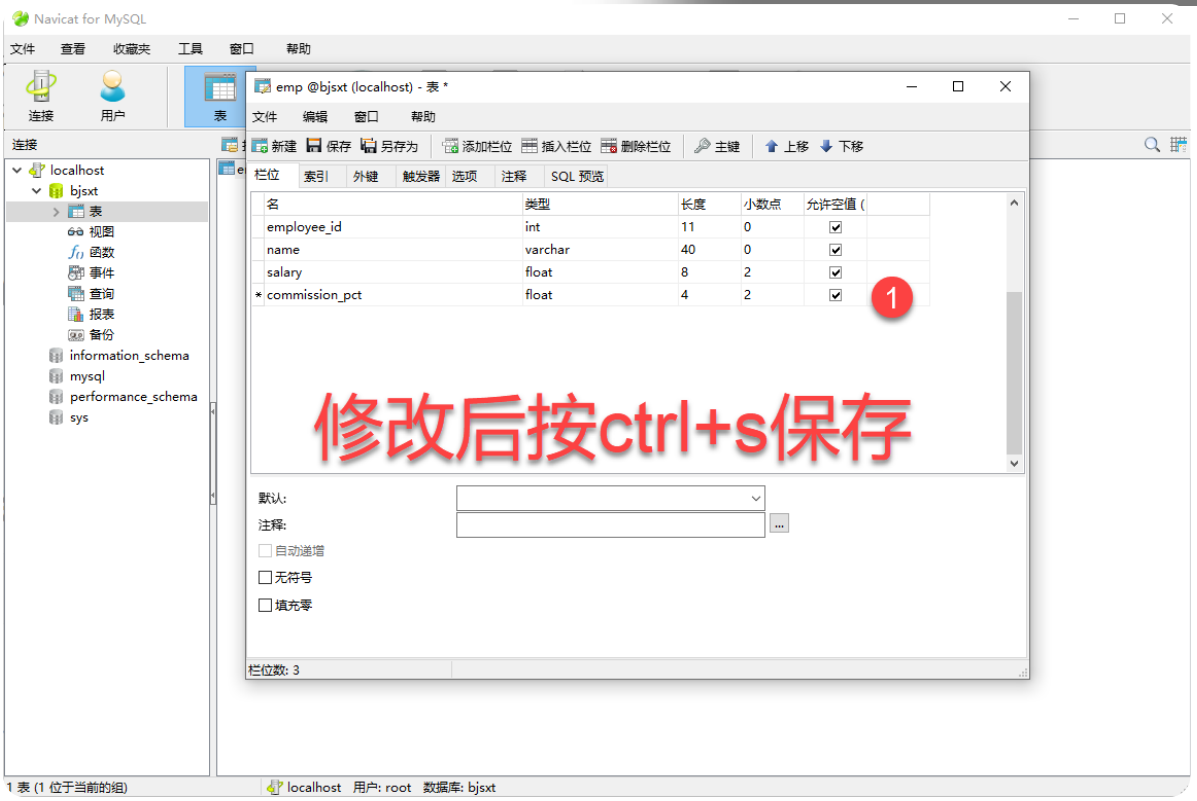
```
ALTER TABLE  表名 ADD COLUMN  新列名 类型;
```

示例：
在 emp 表中添加佣金列，列名为 commission_pct。

```
alter table emp add column commission_pct float(4,2);
```

2. 使用Navicat添加新列





实时效果反馈

1.添加新列语句：ALTER TABLE 表名；

A ADD COLUMN 新列名 类型

B ADD 新列名 类型

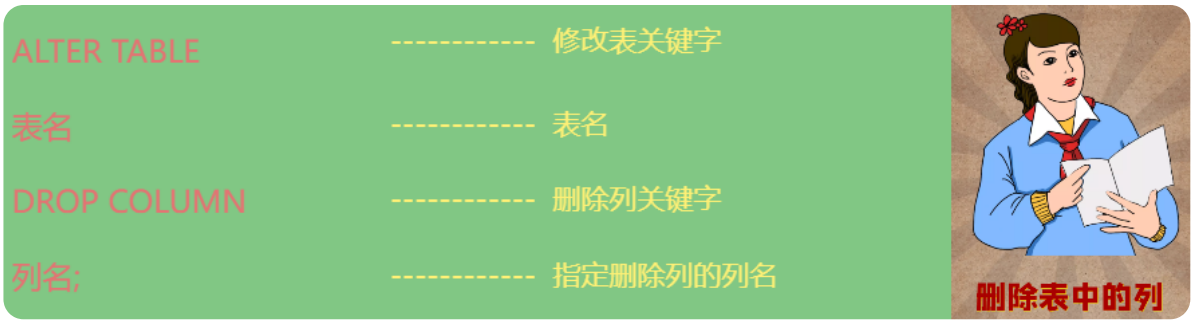
C ADD COLUMN 新列名

D COLUMN 新列名 类型

答案

1=>A

删除指定列



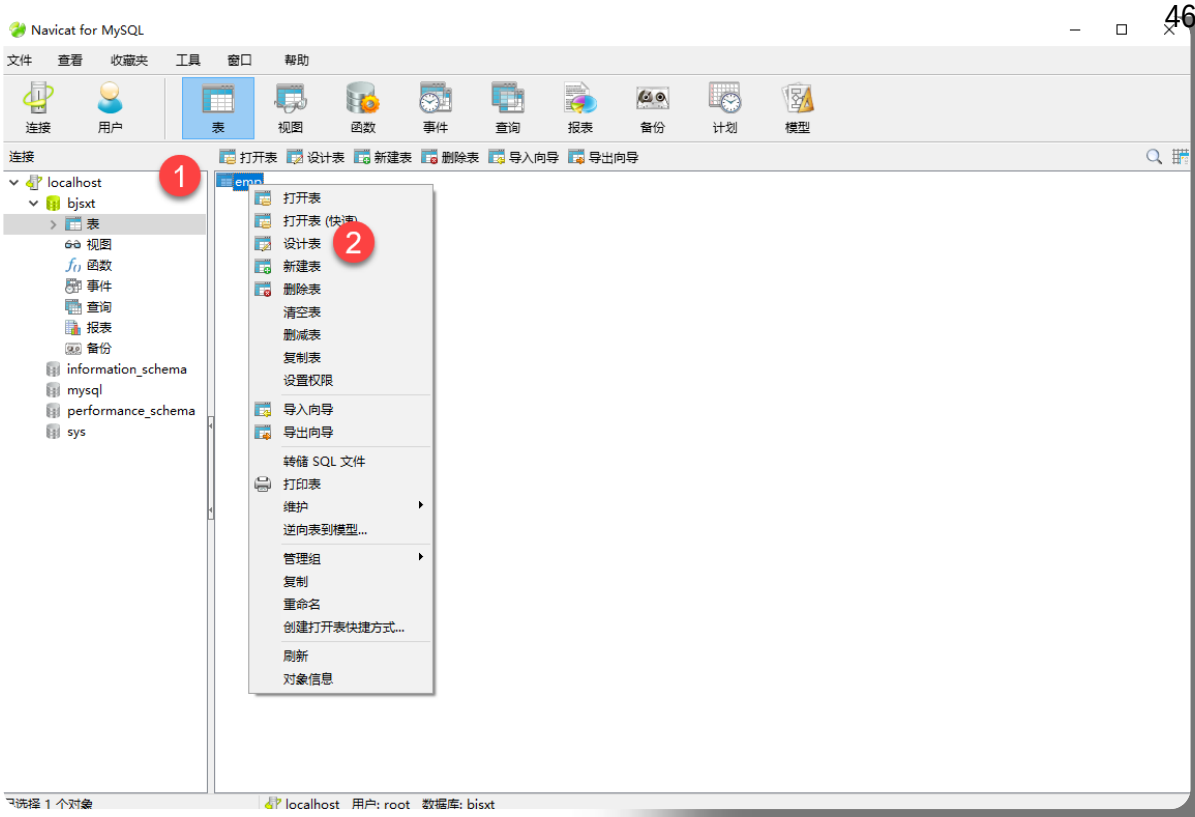
1. 使用DDL语句删除指定的列

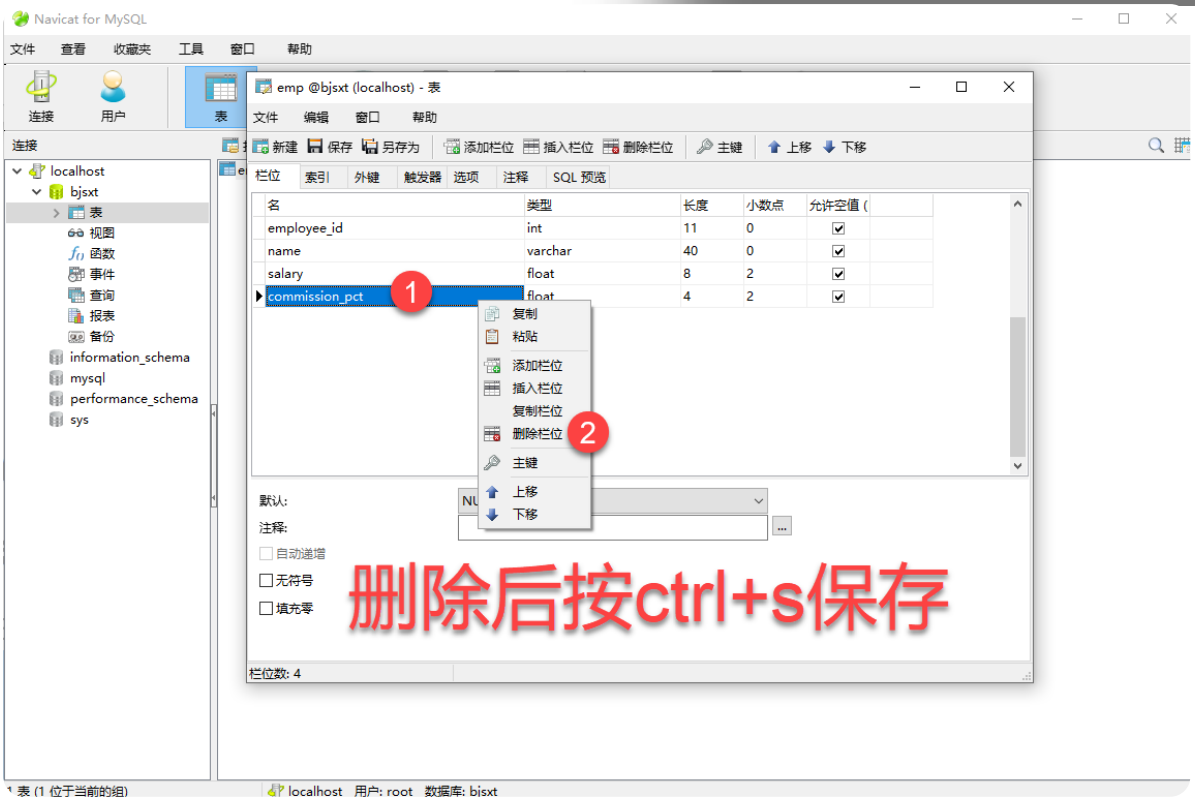
```
ALTER TABLE  表名 DROP COLUMN  列名;
```

示例：
删除 emp 表中的 commission_pct。

```
alter table emp drop column
commission_pct;
```

2. 使用Navicat删除指定的列





实时效果反馈

1.删除指定列语句：ALTER TABLE 表名；

A REMOVE 列名

B REMOVE COLUMN 列名

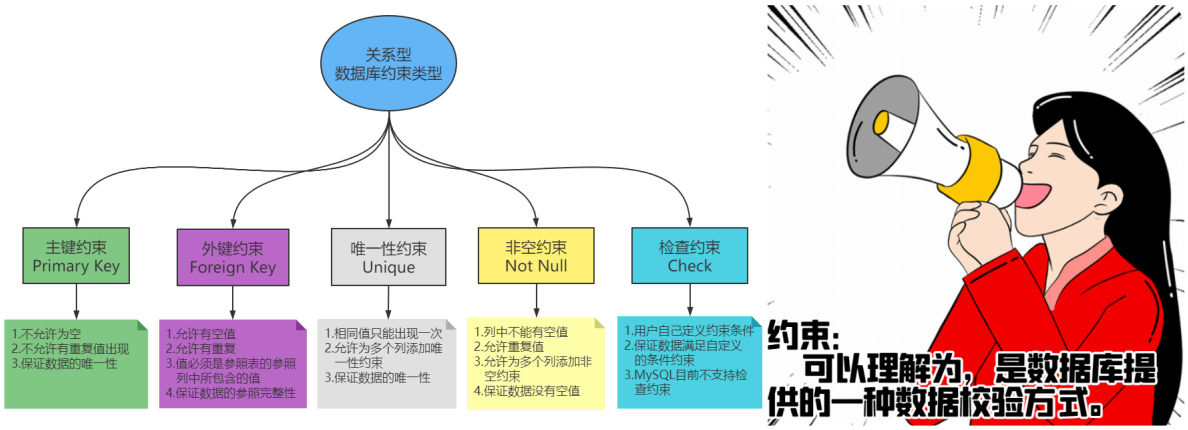
C DROP 列名

D DROP COLUMN 列名

答案

1=>D

MySQL中的约束



约束概述

数据库约束是对表中的数据进行进一步的限制，保证数据的正确性、有效性和完整性。

1. 主键约束(Primary Key) PK

> 主键约束是使用最频繁的约束。在设计数据表时，一般情况下，都会要求表中设置一个主键。
> 主键是表的一个特殊字段，该字段能唯一标识该表中的每条信息。例如，学生信息表中的学号是唯一的。

2. 外键约束(Foreign Key) FK

> 外键约束经常和主键约束一起使用，用来确保数据的一致性。

3. 唯一性约束(Unique)

> 唯一约束与主键约束有一个相似的地方，就是它们都能够确保列的唯一性。与主键约束不同的是，唯一约束在一个表中可以有多个，并且设置唯一约束的列是允许有空值的。

4. 非空约束(Not Null)

> 非空约束用来约束表中的字段不能为空。

5. 检查约束(Check)

> 检查约束也叫用户自定义约束，是用来检查数据表中，字段值是否有效的一个手段，但目前 MySQL 数据库不支持检查约束。

实时效果反馈

1.如下对主键约束描述正确的是。

A 允许有重复出现，不能有空值

B 不允许有重复出现，但可以有空值

C 不允许有重复出现，不能有空值

D 允许有重复，允许有空值

2.如下对外键约束描述正确的是。

A 不允许重复，不允许有空值

B 不允许重复，允许有空值

C 允许有重复，不允许有空值

D 允许有重复，允许有空值

答案

1=>C　2=>D

添加主键约束(Primary Key)

1. 单一主键
   使用一个列作为主键列，当该列的值有重复时，则违反唯一约束。

2. 联合主键
   使用多个列作为主键列，当多个列的值都相同时，则违反唯一约束。

修改表添加主键约束

1. 使用DDL语句添加主键约束

```
ALTER TABLE  表名 ADD PRIMARY KEY(列名)
```

示例：
将 emp 表中的 employee_id 修改为主键。

```
alter table emp add primary
key(employee_id);
```

主键自增长

MySQL 中的自动增长类型要求：

- 一个表中只能有一个列为自动增长。
- 自动增长的列的类型必须是整数类型。
- 自动增长只能添加到具备主键约束与唯一性约束的列上。
- 删除主键约束或唯一性约束，如果该列拥有自动增长能力，则需要先去掉自动增长然 后在删除约束。

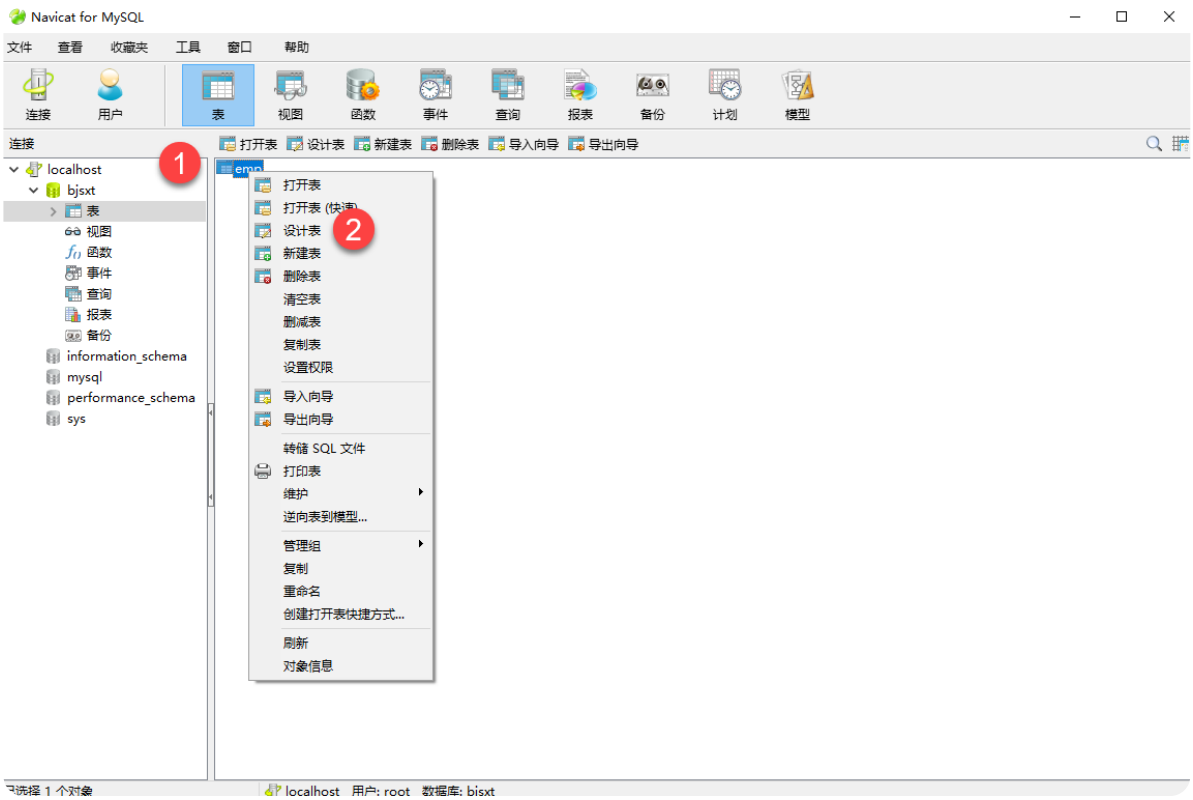
```
alter table 表名 modify 主键 类型 auto_increment;
```

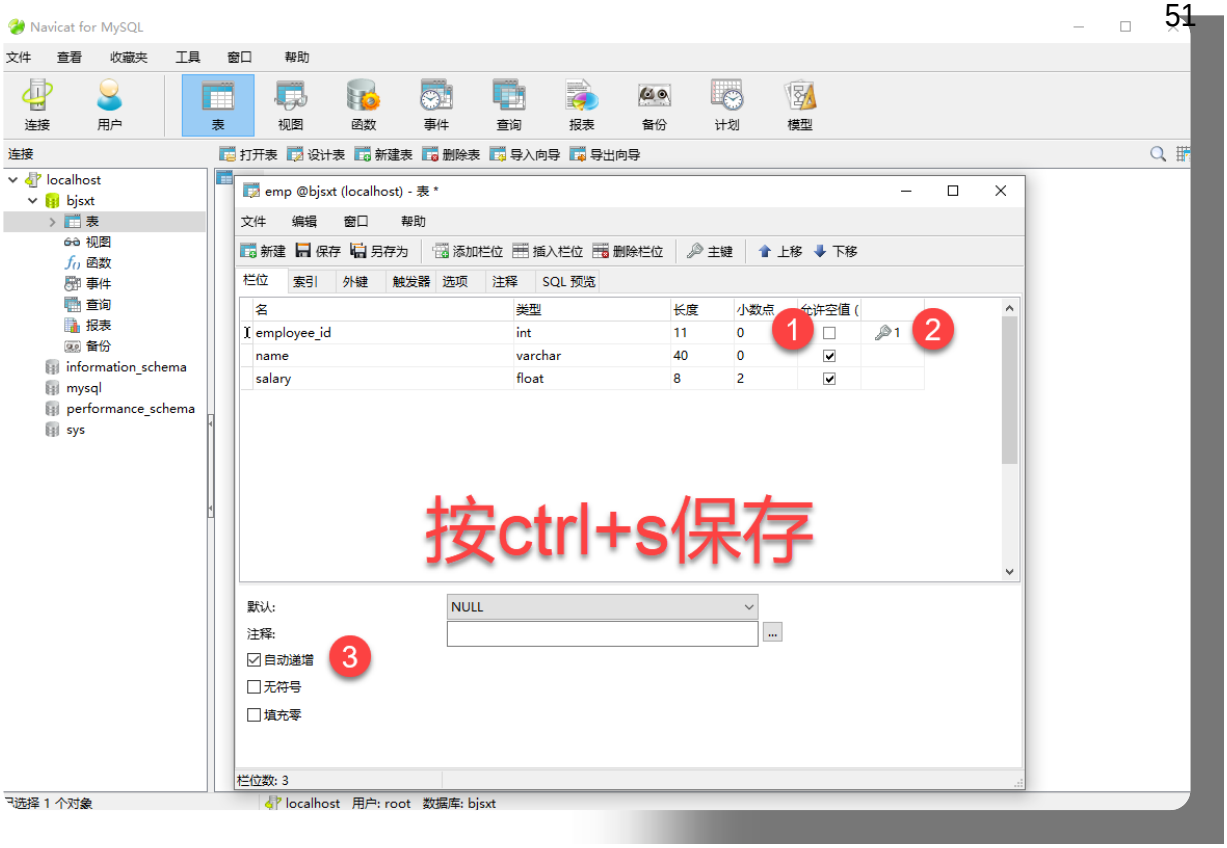
示例：

将 emp 表中的 employee_id 主键修改为自增。

```
alter table emp modify employee_id int auto_increment;
```

2. 使用Navicat添加主键约束





实时效果反馈

1.添加主键约束语句：ALTER TABLE 表名；

A ADD PRIMARY KEY(列名)

B ADD KEY(列名)

C ADD PRIMARY(列名)

D ADD PRIMARY KEY

答案

1=>A

删除主键

1. 使用DDL语句删除主键

```
ALTER TABLE  表名 DROP PRIMARY KEY;
```

> 注意：
> 删除主键时，如果主键列具备自动增长能力，需要先去掉自动增长，然后在删除 主键。

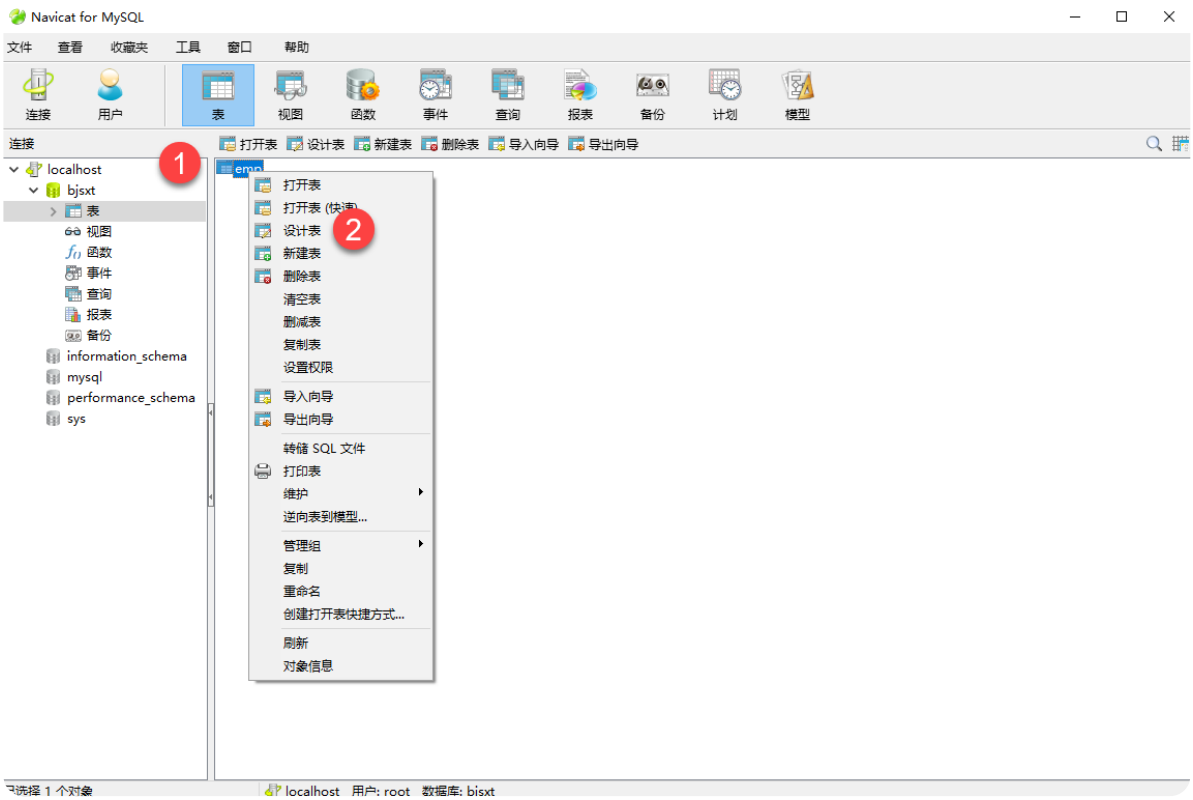
示例：
删除emp表中的 employee_id 主键约束。
去掉自动增长：

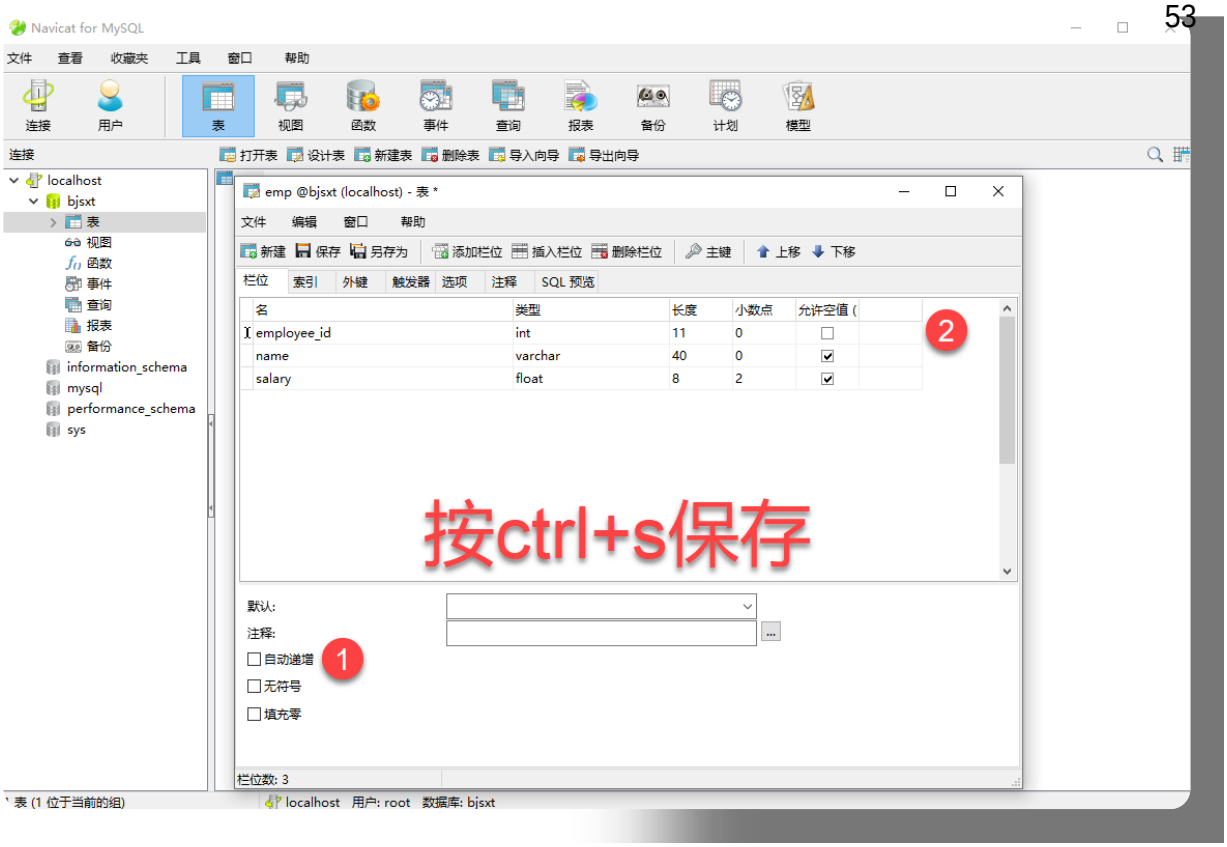
```
alter table emp modify employee_id int;
```

删除主键：

```
alter table emp drop primary key;
```

2. 使用Navicat删除主键





实时效果反馈

1.删除主键约束语句：ALTER TABLE 表名；

A REMOVE PRIMARY

B REMOVE PRIMARY KEY

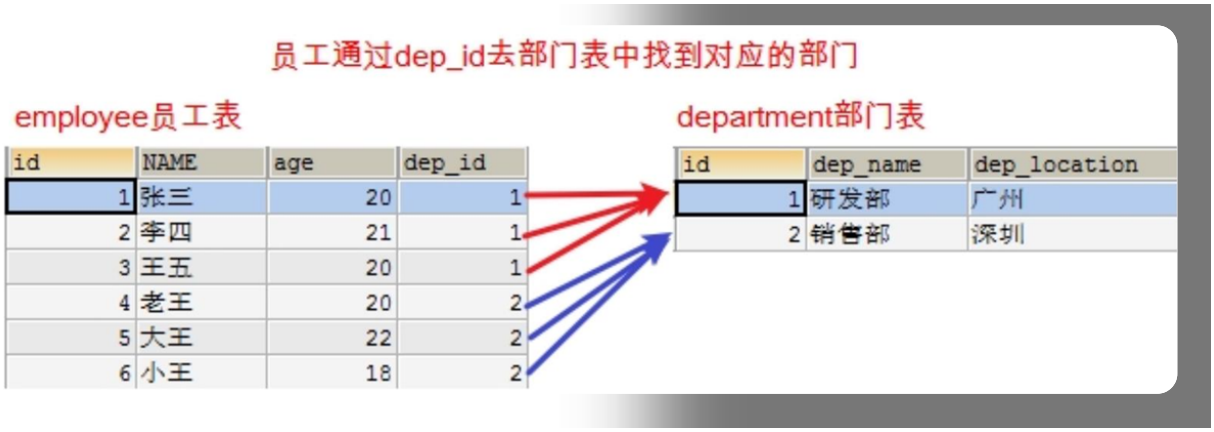
C DROP PRIMARY KEY

D DROP PRIMARY

答案

1=>C

添加外键约束(Foreign Key)



id	NAME	age	dep_id
1	张三	20	1
2	李四	21	1
3	王五	20	1
4	老王	20	2
5	大王	22	2
6	小王	18	2

id	dep_name	dep_location
1	研发部	广州
2	销售部	深圳

修改表添加外键约束

1. 使用DDL语句添加外键约束

```
ALTER TABLE 表名 ADD CONSTRAINT 约束名 FOREIGN KEY( 列 名 ) REFERENCES 参照的表名(参照的列名);
```

示例一：

创建 departments 表包含 department_id 、department_name ， location_id。

```
create table departments(department_id int,department_name varchar(30),location_id int);
```

示例二：

修改departments表，向department_id列添加主键约束与自动递增。

```
alter table departments add primary key(department_id);
alter table departments modify department_id int auto_increment;
```

示例三：

修改 emp 表，添加 dept_id 列。

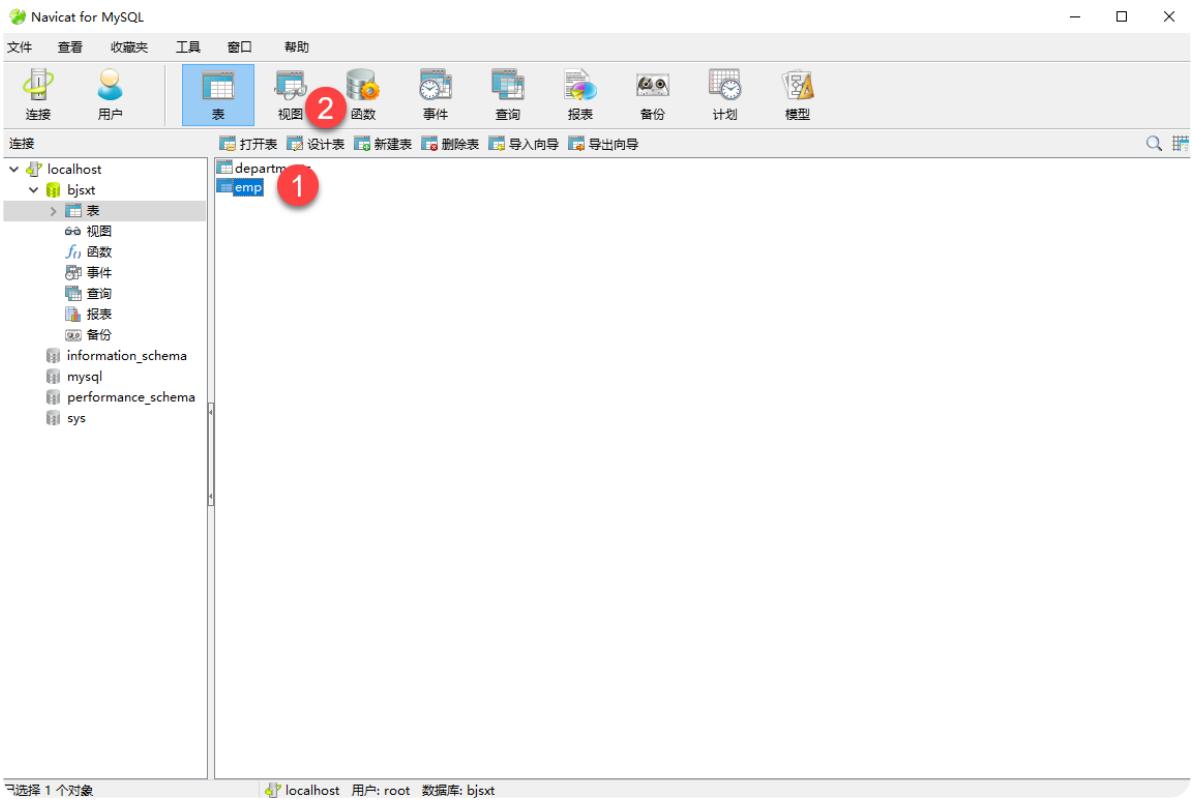
```
alter table emp add column dept_id int;
```

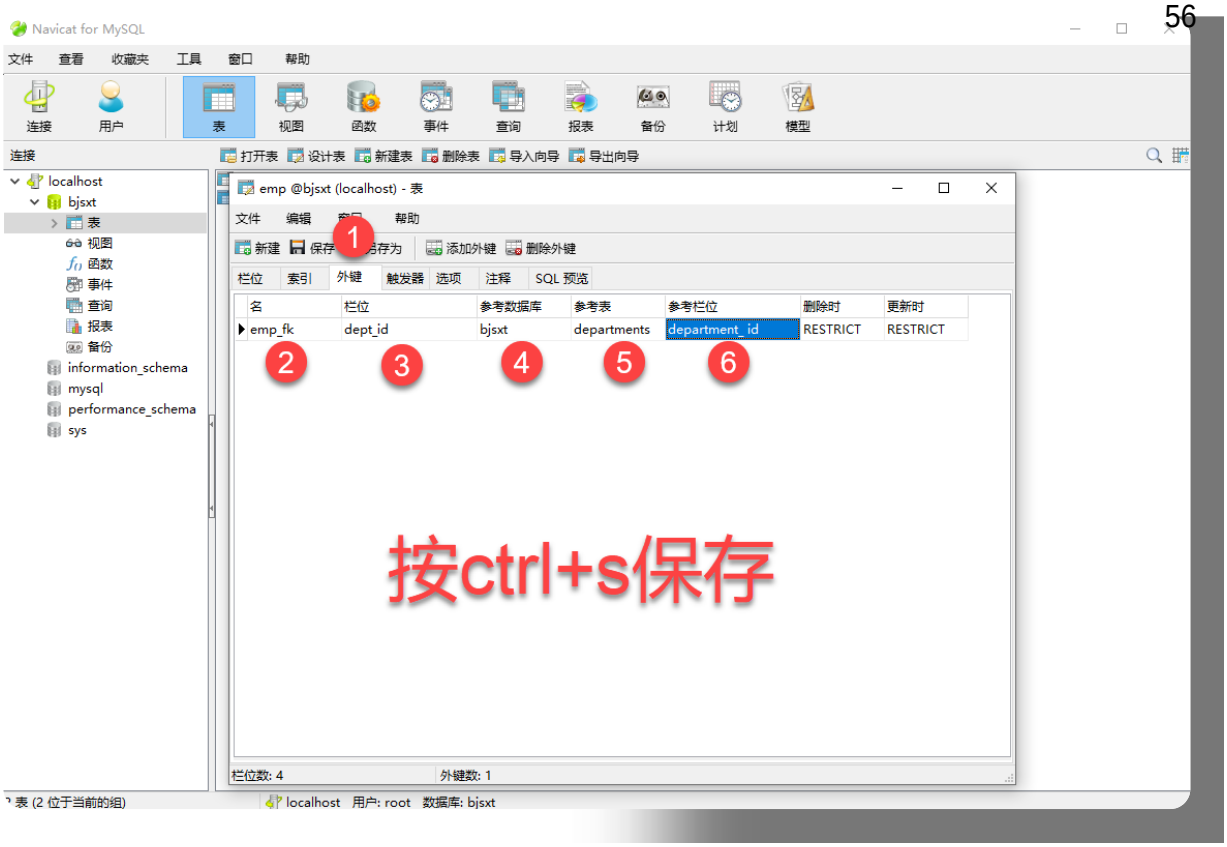
示例四：

向 emp 表中的 dept_id 列添加外键约束。

```
alter table emp add constraint emp_fk
foreign key(dept_id) references
departments(department_id);
```

2. 使用Navicat添加外键约束





实时效果反馈

1.添加外键约束语句：ALTER TABLE 表名 ADD CONSTRAINT ;

A FOREIGN KEY(列 名) REFERENCES 参照的表名(参照的列名)

B 约束名 FOREIGN KEY(列 名)

C 约束名 (列 名) REFERENCES 参照的表名(参照的列名)

D 约束名 FOREIGN KEY(列 名) REFERENCES 参照的表名(参照的列名)

答案

1=>D

删除外键约束

1. 使用DDL语句删除外键约束。

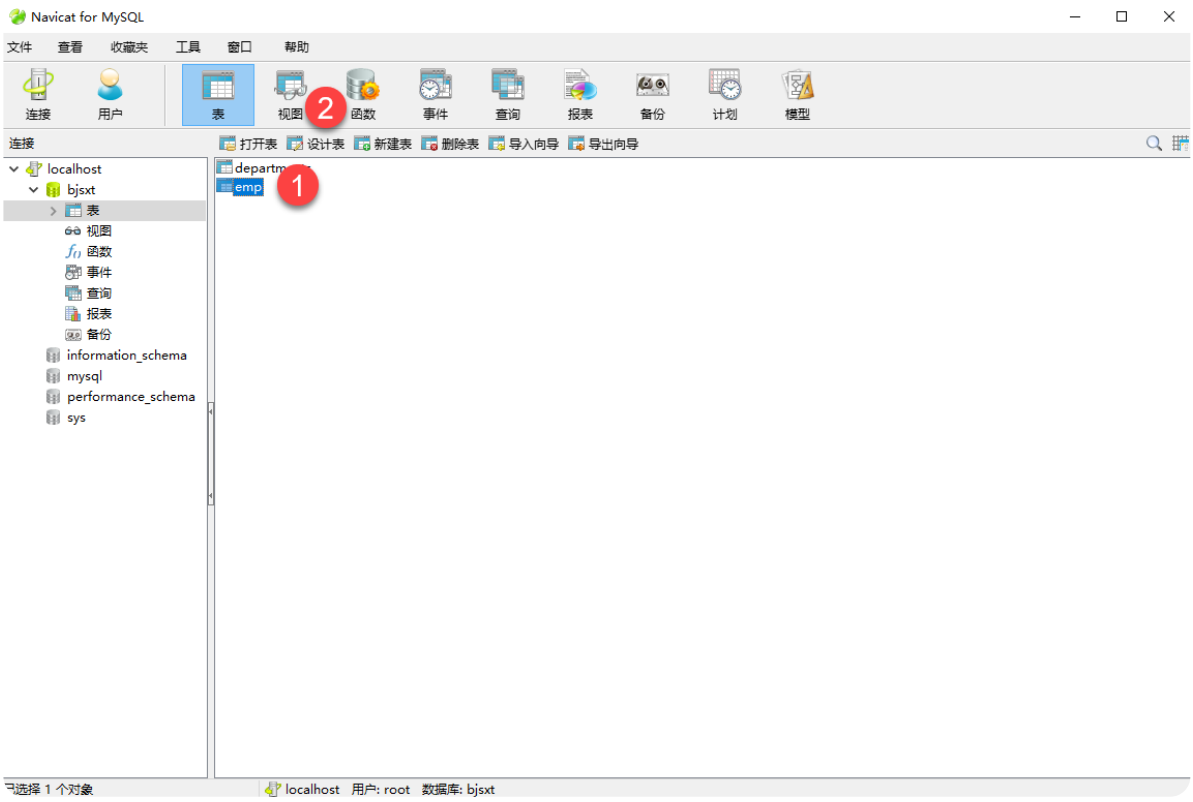
```
ALTER TABLE  表名 DROP FOREIGN KEY  约束名;
```

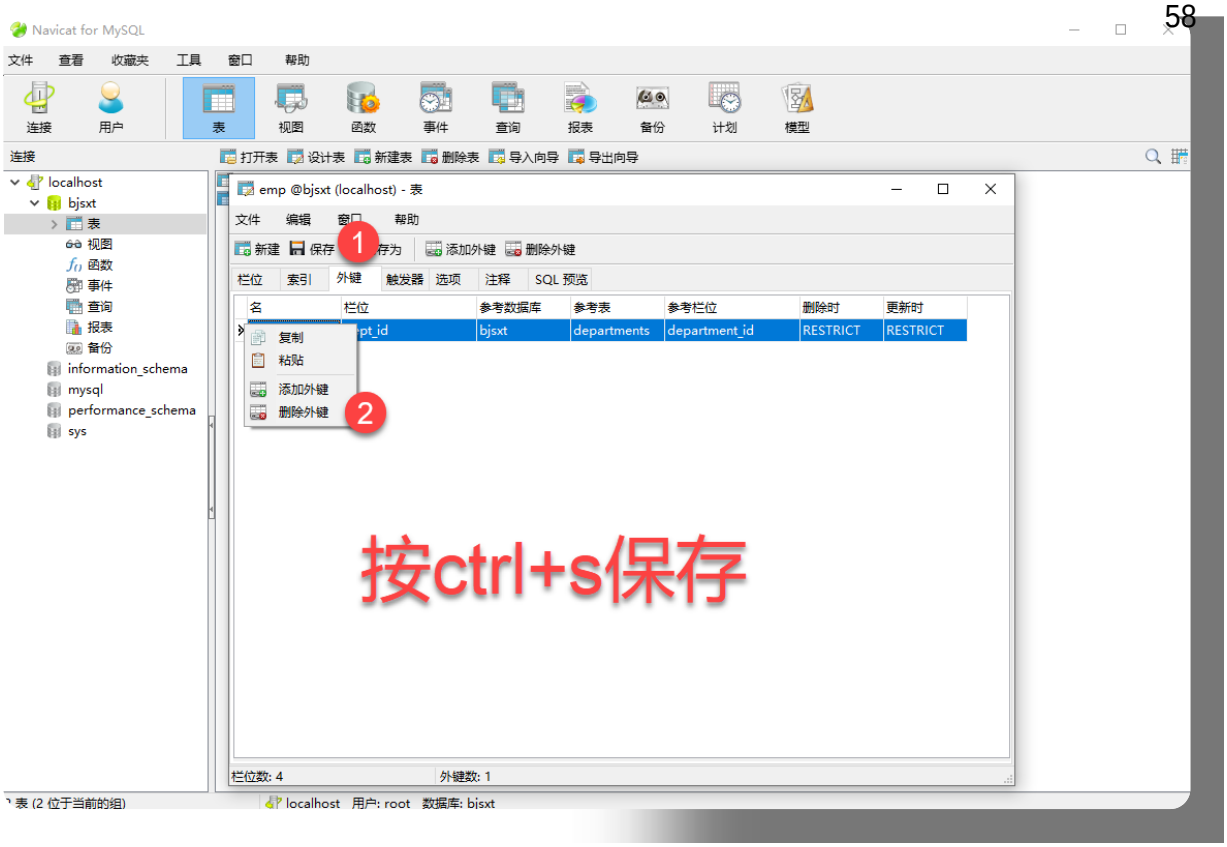
示例：

删除 dept_id 的外键约束。

```
alter table emp drop foreign key emp_fk;
```

2. 使用Navicat删除外键约束





实时效果反馈

1.删除外键语句：ALTER TABLE 表名；

A REMOVE FOREIGN KEY

B REMOVE FOREIGN KEY 约束名

C DROP FOREIGN KEY 约束名

D DROP FOREIGN KEY

答案

1=>C

添加唯一性约束(Unique)

修改表添加唯一性约束

1. 使用DDL语句添加唯一性约束。

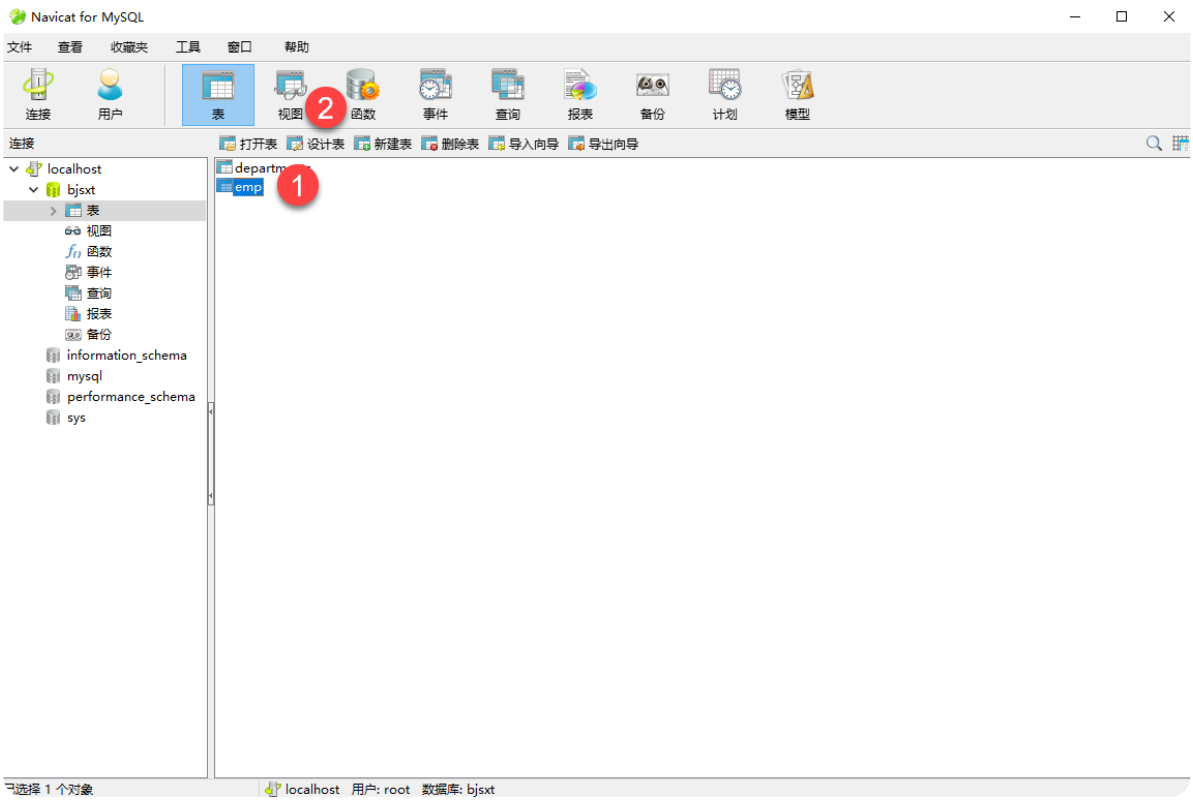
```
ALTER TABLE  表名 ADD CONSTRAINT  约束名
UNIQUE(列名);
```

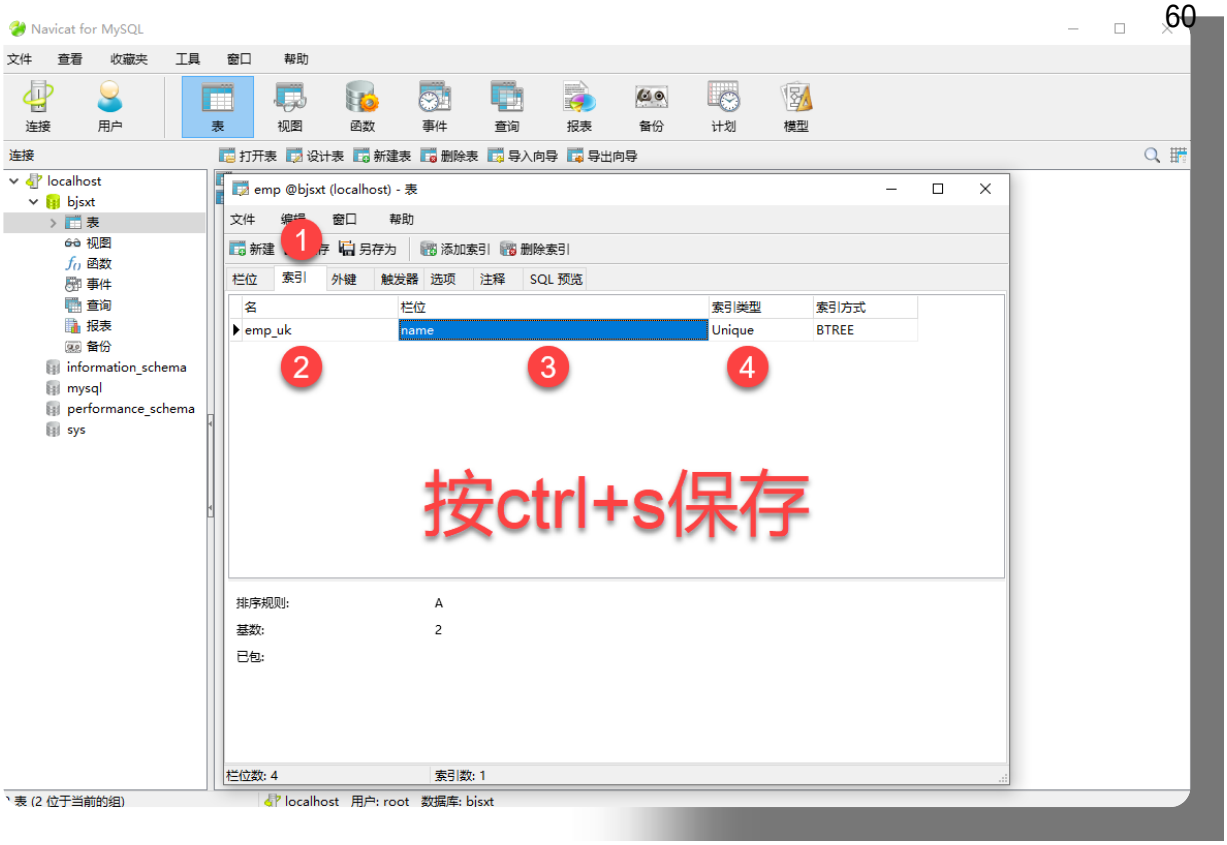
示例：

向 emp 表中的 name 添加唯一约束。

```
alter table emp add constraint emp_uk
unique(name);
```

2. 使用Navicat添加唯一性约束





实时效果反馈

1.添加唯一性约束语句：ALTER TABLE 表名 ADD CONSTRAINT ;

A UNIQUE(列名)

B 约束名 UNIQUE(列名)

C 约束名 UNIQUE

D 约束名(列名)

答案

1=>B

删除唯一性约束

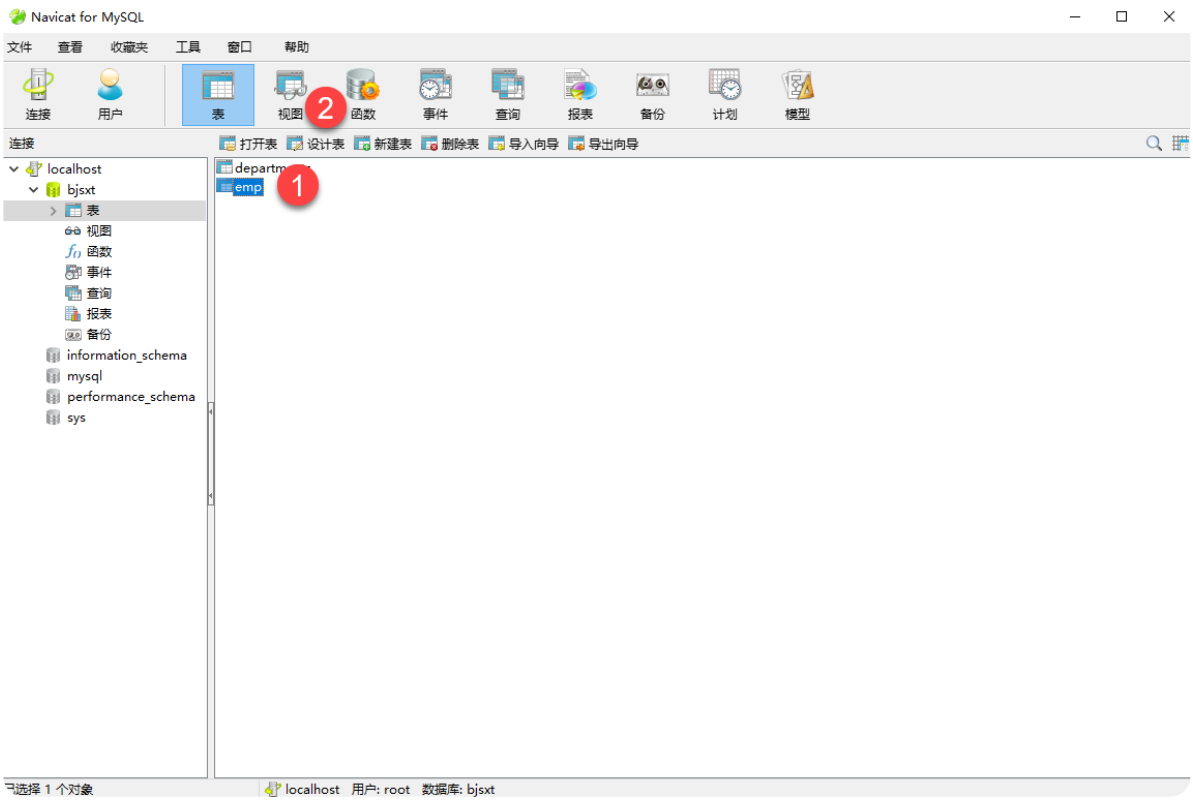
1. 使用DDL语句删除唯一性约束。

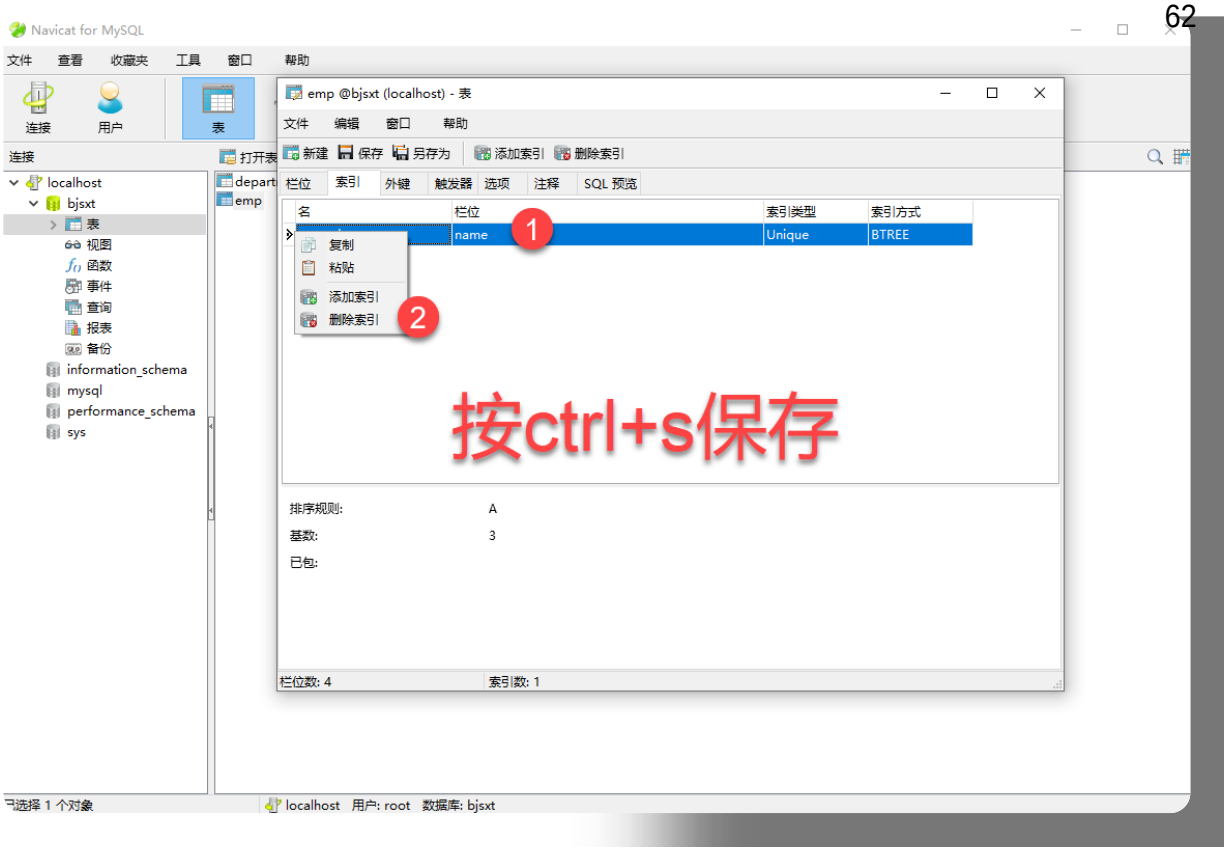
```
ALTER TABLE  表名 DROP KEY  约束名;
```

示例：
删除 name 的唯一约束。

```
alter table emp drop key emp_uk;
```

2. 使用Navicat删除唯一性约束。





实时效果反馈

1.删除唯一性约束语句：ALTER TABLE 表名；

A DROP KEY 约束名

B DROP KEY

C REMOVE KEY 约束名

D REMOVE KEY

答案

1=>A

非空约束(Not Null)

修改表添加非空约束

1. 使用DDL语句添加非空约束。

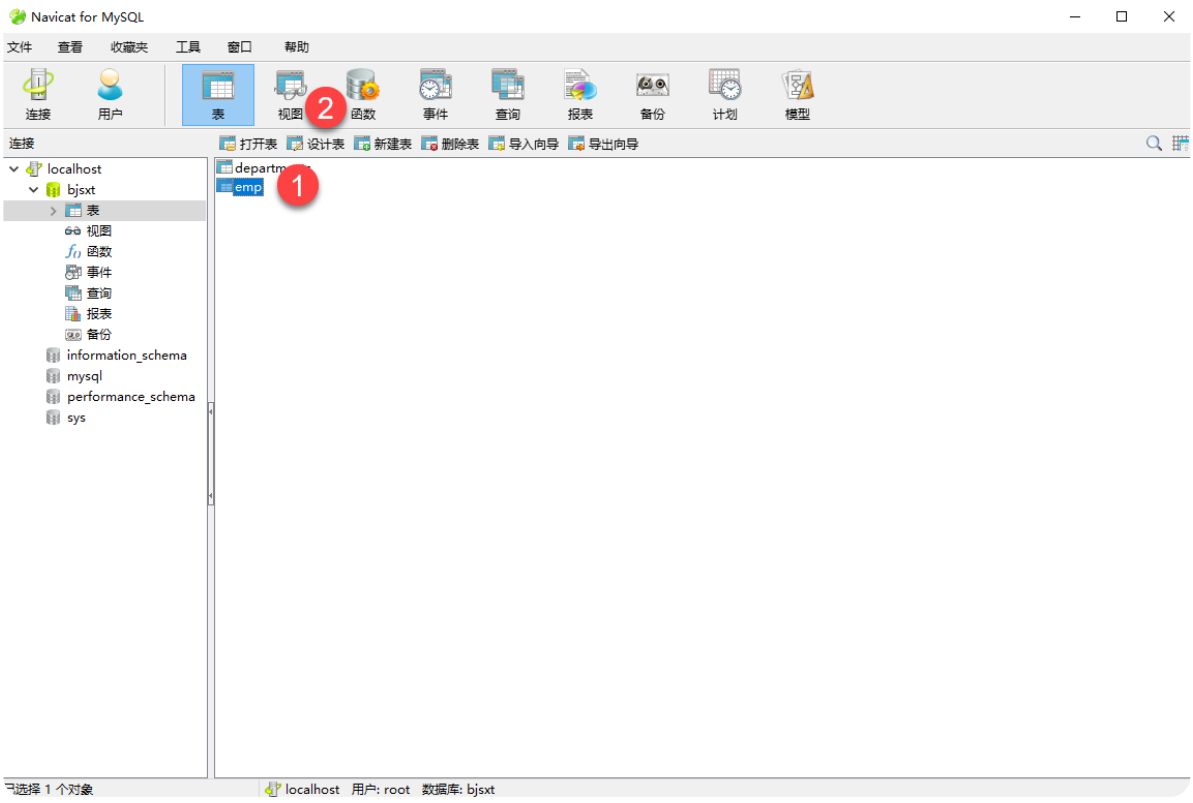
```
ALTER TABLE  表名 MODIFY  列名 类型 NOT NULL;
```

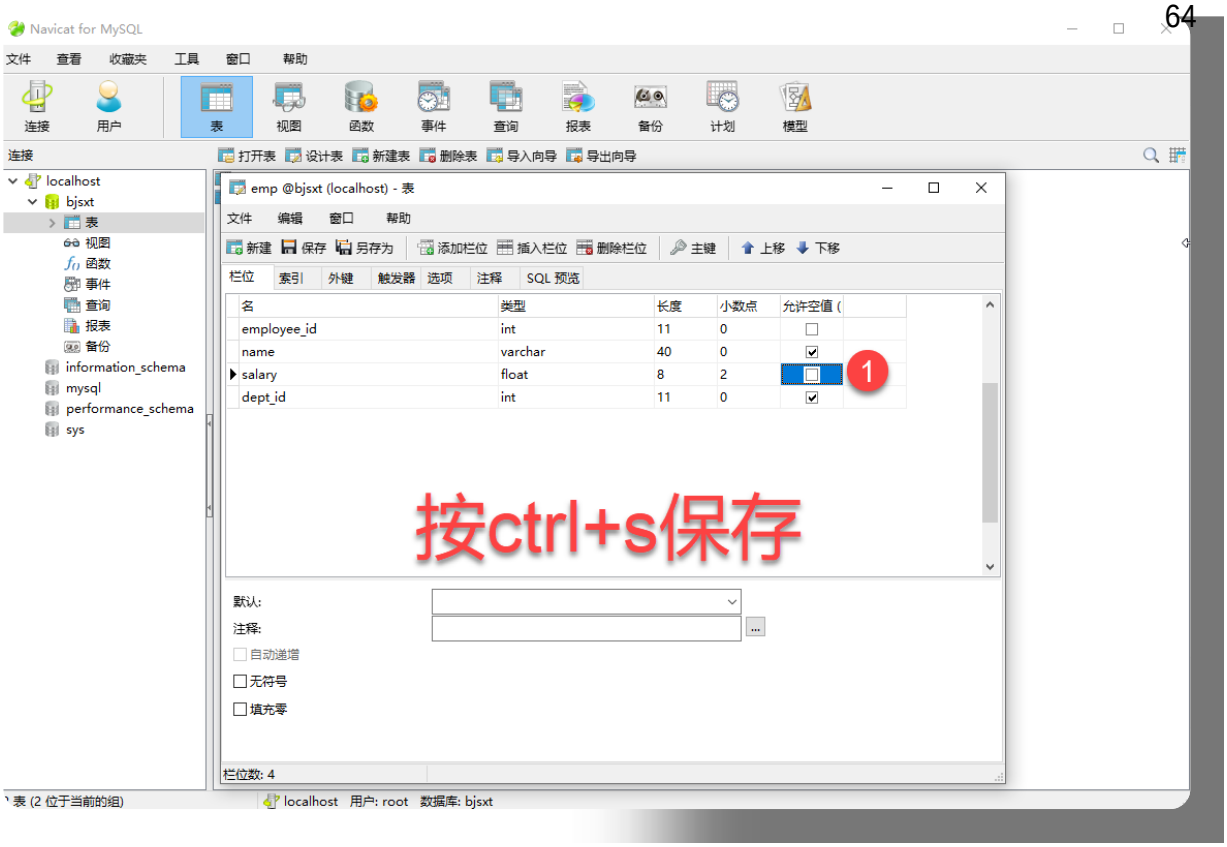
示例：

向 emp 表中的 salary 添加非空约束。

```
alter table emp modify salary float(8,2) not NULL;
```

2. 使用Navicat添加非空约束。





实时效果反馈

1.添加非空约束语句：ALTER TABLE 表名；

A UPDATE 列名 NOT NULL

B UPDATE 列名 类型 NOT NULL

C MODIFY 列名 NOT NULL

D MODIFY 列名 类型 NOT NULL

答案

1=>D

删除非空约束

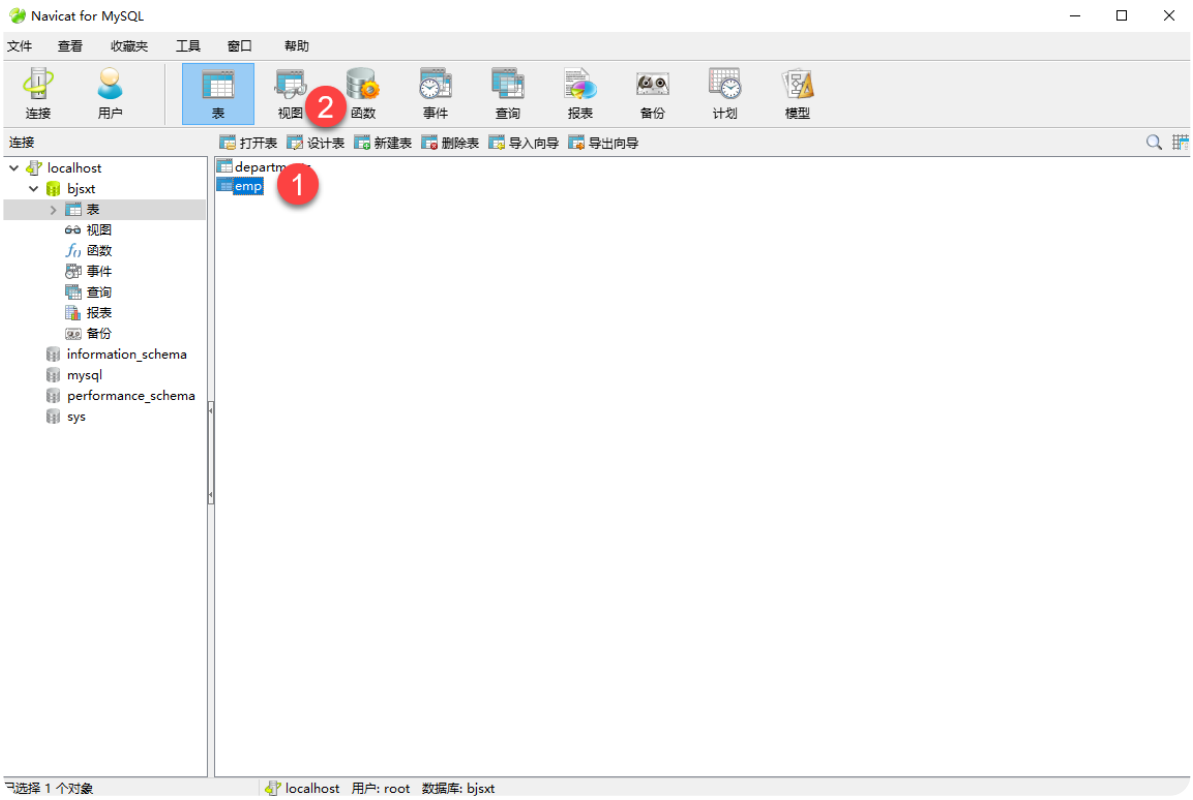
1. 使用DDL语句删除非空约束。

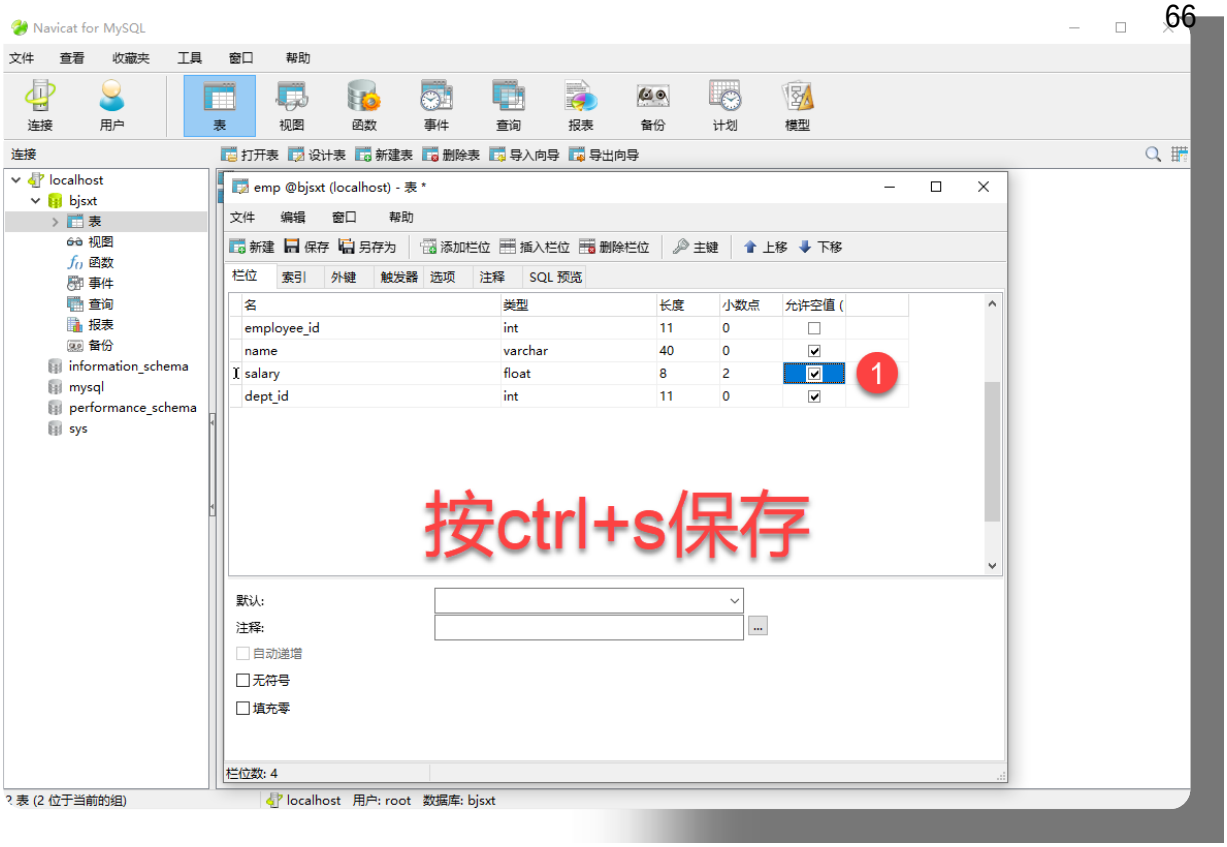
```
ALTER TABLE  表名 MODIFY  列名 类型 NULL;
```

示例：
删除emp表中salary 的非空约束。

```
alter table emp modify salary float(8,2)
NULL;
```

2. 使用Navicat删除非空约束。





实时效果反馈

1.删除非空约束语句：ALTER TABLE 表名；

A UPDATE 列名 类型 NULL

B UPDATE 类型 NULL

C MODIFY 列名 NULL

D MODIFY 列名 类型 NULL

答案

1=>D

创建表时添加约束

查询表中的约束信息：

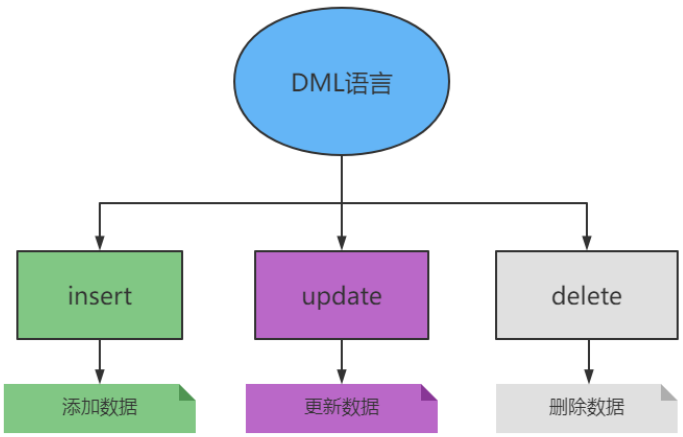
```
SHOW KEYS FROM  表名;
```

示例：

创建 depts 表包含 department_id 该列为主键且自动增长，department_name 列不 允许重复，location_id 列不允含有空值。

```
create table depts(department_id int primary key auto_increment,department_name varchar(30) unique,location_id int not null);
```

MySQL中DML操作





添加数据(INSERT)

1. 选择插入

```
INSERT INTO  表名(列名 1 , 列名 2 , 列名 3.....) VALUES(值 1 , 值 2 , 值 3......);
```

示例:
向 departments 表中添加一条数据，部门名称为 market ， 工作地点 ID 为 1。

```
insert into departments(department_name,location_id) values("market", 1);
```

2. 完全插入

```
INSERT INTO  表名 VALUES(值 1 , 值 2 , 值 3......);
```

> 注意:
>
> 如果主键是自动增长，需要使用 default 或者 null 或者 0 占位。

示例一:

向 departments 表中添加一条数据，部门名称为 development ，工作地点 ID 为 2 。使用 default 占位。

```
insert into departments values(default,"development",2);
```

示例二：

向 departments 表中添加一条数据，部门名称为human ， 工作地点 ID 为 3 。使用 null 占 位。

```
insert into departments
values(null,"human",3);
```

示例三：

向 departments 表中添加一条数据，部门名称为 teaching ， 工作地点 ID 为 4 。使用 0 占 位。

```
insert into departments
values(0,"teaching",4);
```

实时效果反馈

1.插入数据时，对于自增长主键占位方式不正确的是;

A insert into departments values(default,"development",2);

B insert into departments values(null,"development",2);

C insert into departments values(0,"development",2);

D insert into departments values("","development",2);

答案

1=>D

默认值处理(DEFAULT)

在 MySQL 中可以使用 DEFAULT 为列设定一个默认值。如果在插入数据时并未指定该列的值，那么 MySQL 会将默认值添加到该列中。

创建表时指定列的默认值

```
CREATE TABLE 表名(列名 类型 default 默认值,.....);
```

示例：

创建 emp3 表，该表包含 emp_id 主键且自动增长，包含 name，包含 address 该列默认 值为"未知"。

```
create table emp3(emp_id int primary key auto_increment,name varchar(10),address varchar(50) default 'Unknown');
```

修改表添加新列并指定默认值

```
ALTER TABLE 表名 ADD COLUMN 列名 类型 DEFAULT 默认值;
```

示例：

修改 emp3 表，添加job_id 该列默认值为 0。

```
alter table emp3 add column job_id int
default 0;
```

插入数据时的默认值处理

如果在插入数据时并未指定该列的值，那么MySQL 会将默认值添加到该列中。如果是 完全项插入需要使用 default 来占位。

示例：

向 emp3 表中添加数据，要求 address 列与job_id 列使用默认值作为该列的值。

```
insert into emp3(name) values("admin");
insert into emp3
values(default,"oldlu",default,default);
```

实时效果反馈

1.创建表时指定默认值：CREATE TABLE 表名();

A 列名 default 默认值

B 列名 类型 默认值

C 列名 类型 default 默认值

D default 列名 类型 默认值

答案

1=>C

更新数据(UPDATE)

```
UPDATE 表名 SET  列名=值，列名=值 WHERE 条件;
```

> 注意：
>
> 更新语句中一定要给定更新条件，否则表中的所有数据都会被更新。

示例：

更新 emp3 表中的 id 为 1 的数据，添加 address 为 BeiJing。

```
update emp3 set address = "BeiJing" where emp_id = 1;
```

实时效果反馈

1.更新数据：UPDATE 表名；

A 列名=值，列名=值 WHERE 条件

B SET 列名=值，列名=值 WHERE 条件

C SET 列名=值 列名=值 WHERE 条件

D WHERE 条件 SET 列名=值，列名=值

答案

1=>B

删除数据(DELETE)

DELETE删除数据

```
DELETE FROM  表名 WHERE 条件;
```

> 注意：
>
> 在DELETE语句中，如果没有给定删除条件则会删除表中的所有数据。

示例：

删除 emp3 表中 emp_id 为 1 的雇员信息。

```
delete from emp3 where emp_id = 1;
```

TRUNCATE清空表

```
TRUNCATE TABLE  表名;
```

示例：

删除 emp3 表中的所有数据。

```
truncate table emp3;
```

清空表时DELETE与 TRUNCATE 区别

- truncate 是整体删除(速度较快)， delete 是逐条删除(速度较慢)；
- truncate 不写服务器 log，delete 写服务器 log，也就是 truncate 效率比 delete 高的原因；
- truncate 是会重置自增值，相当于自增列会被置为初始值，又重新从 1 开始记录，而 不是接着原来的值。而 delete 删除以后， 自增值仍然会继续累加。

实时效果反馈

1.删除数据：DELETE ；

A 表名 WHERE 条件

B FROM WHERE 条件

C WHERE 条件 FROM 表名

D FROM 表名 WHERE 条件

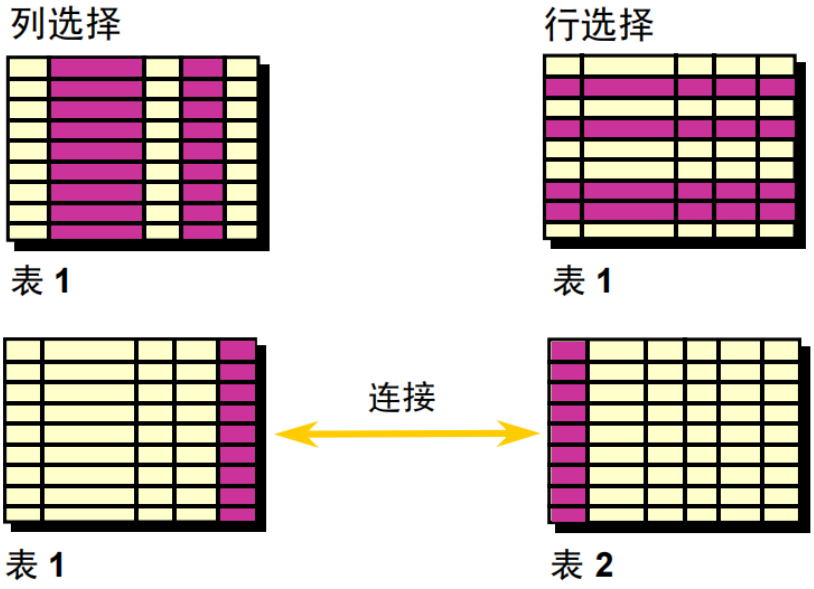
答案

1=>D

MySQL查询数据

SELECT基本查询

SELECT语句的功能



SELECT 语句从数据库中返回信息。使用一个 SELECT 语句，可以做下面的事：

- **列选择：**能够使用 SELECT 语句的列选择功能选择表中的列，这些列是想
  要用查询返回的。当查询时，能够返回列中的数据。
- **行选择：**能够使用 SELECT 语句的行选择功能选择表中的行，这些行是想
  要用查询返回的。能够使用不同的标准限制看见的行。
- **连接：**能够使用 SELECT 语句的连接功能来集合数据，这些数据被存储在不
  同的表中，在它们之间可以创建连接，查询出我们所关心的数据。

SELECT基本语法

基本 SELECT 语句

```
SELECT    *|{[DISTINCT] column|expression [alias],...}
FROM      table;
```

- SELECT 确定哪些列
- FROM 确定哪张表

基本 SELECT 语句

在最简单的形式中，SELECT 语句必须包含下面的内容：

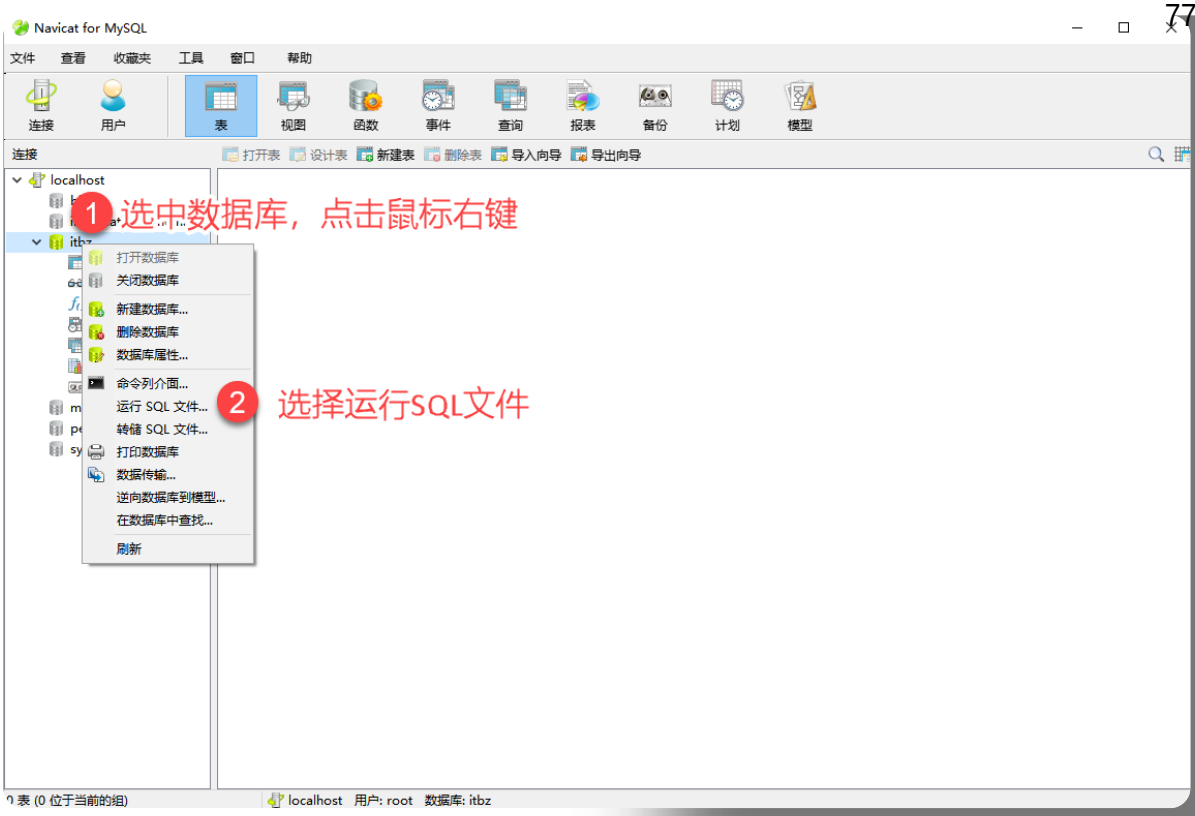
- 一个 SELECT 子句，指定被显示的列
- 一个 FROM 子句，指定表，该表包含 SELECT 子句中的字段列表

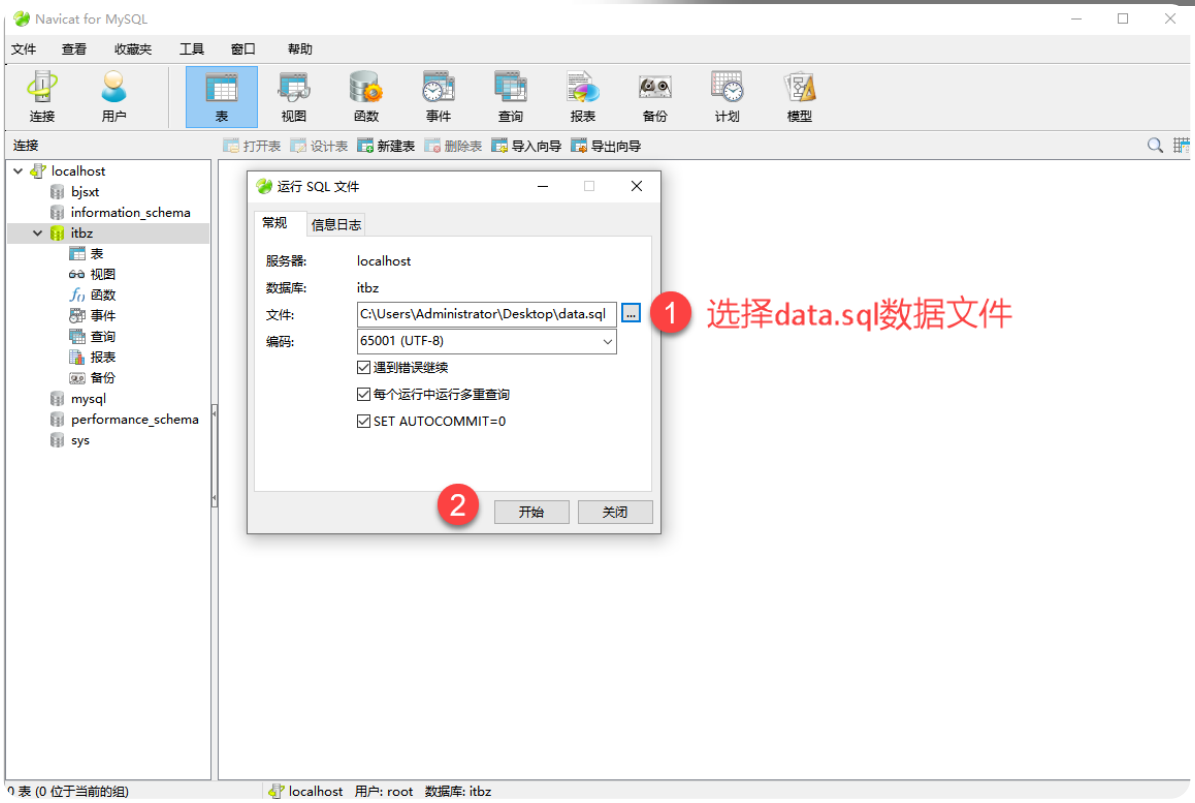
在语法中：

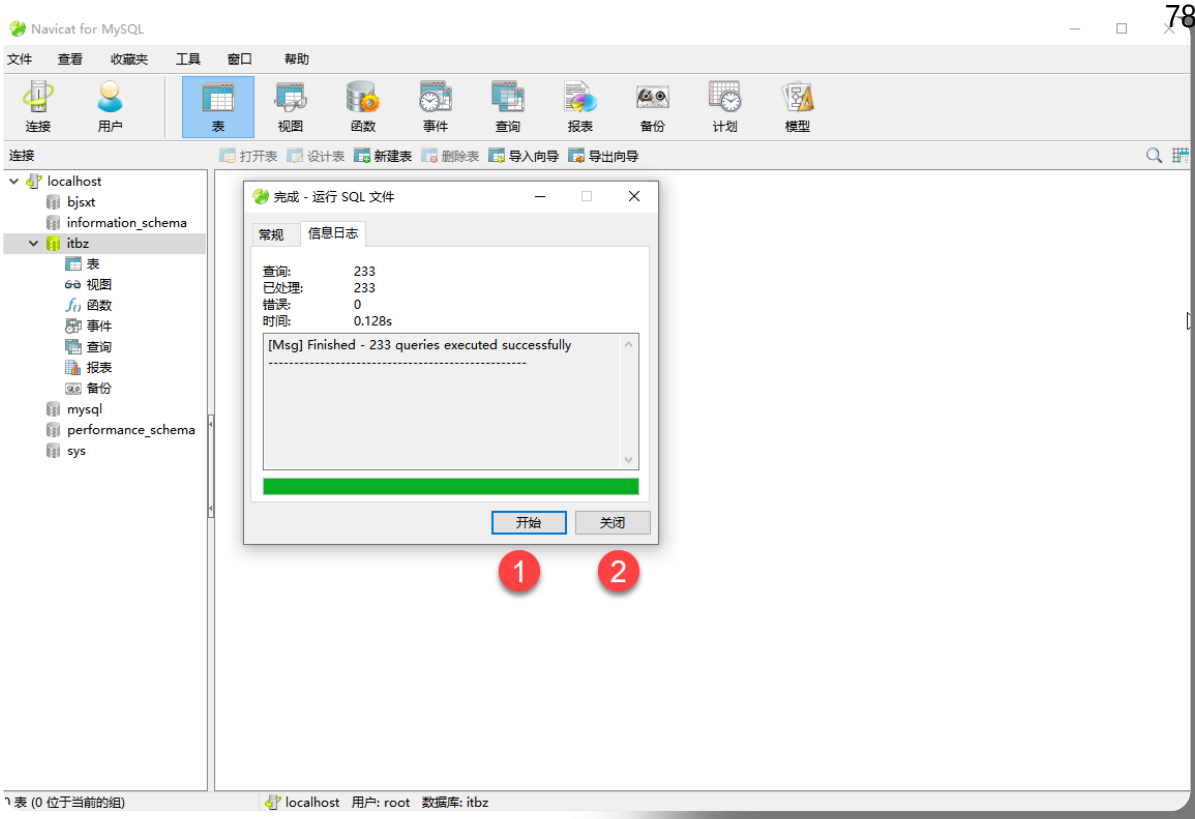
语句	含义
SELECT	是一个或多个字段的列表
*	选择所有的列
DISTINCT	禁止重复
column | expression	选择指定的字段或表达式
alias	给所选择的列不同的标题
FROM table	指定包含列的表

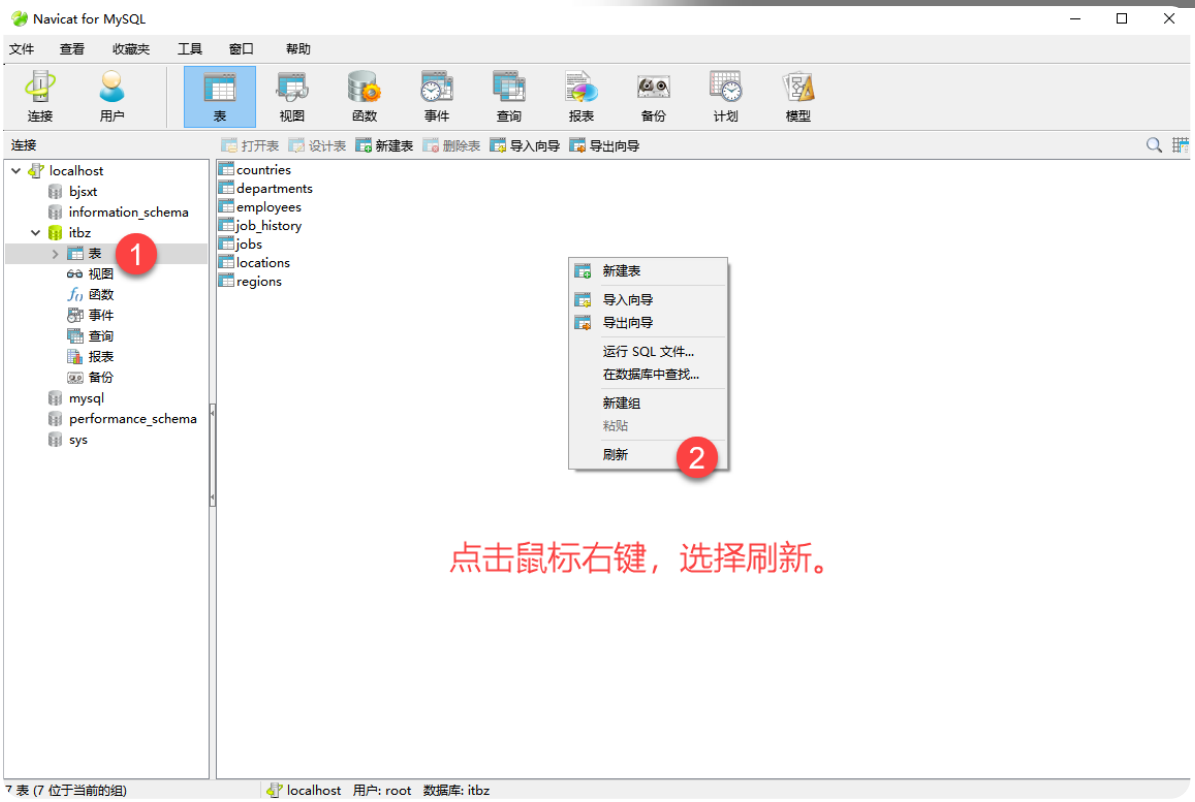
添加测试数据

将data.sql文件通过Navicat导入到MySQL中itbz数据库中。该文件包含了课程中所使用的案例表。


Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
1 选中数据库，点击鼠标右键
打开数据库
关闭数据库
新建数据库...
删除数据库
数据库属性...
命令列介面...
运行 SQL 文件...
2 选择运行SQL文件
转储 SQL 文件...
打印数据库
数据传输...
逆向数据库到模型...
在数据库中查找...
刷新
表 (0 位于当前的组)
localhost 用户: root 数据库: itbz


Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
bjsxt
information_schema
itbz
表
视图
函数
事件
查询
报表
备份
mysql
performance_schema
sys
运行 SQL 文件
常规 信息日志
服务器: localhost
数据库: itbz
文件: C:\Users\Administrator\Desktop\data.sql
编码: 65001 (UTF-8)
遇到错误继续
每个运行中运行多重查询
SET AUTOCOMMIT=0
1 选择data.sql数据文件
2
开始 关闭
表 (0 位于当前的组)
localhost 用户: root 数据库: itbz





点击鼠标右键，选择刷新。

实时效果反馈

1.在最简单的查询形式中，SELECT 语句必须包含。

A 一个SELECT子句，一个FROM子句；

B 一个SELECT子句；

C 一个FROM子句；

D 一个SELECT子句，一个FROM子句，一个INSERT子句；

答案

1=>A

查询中的列选择

选择所有列

选择所有列

```
SELECT *
FROM   departments;
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting		1700

8 rows selected.

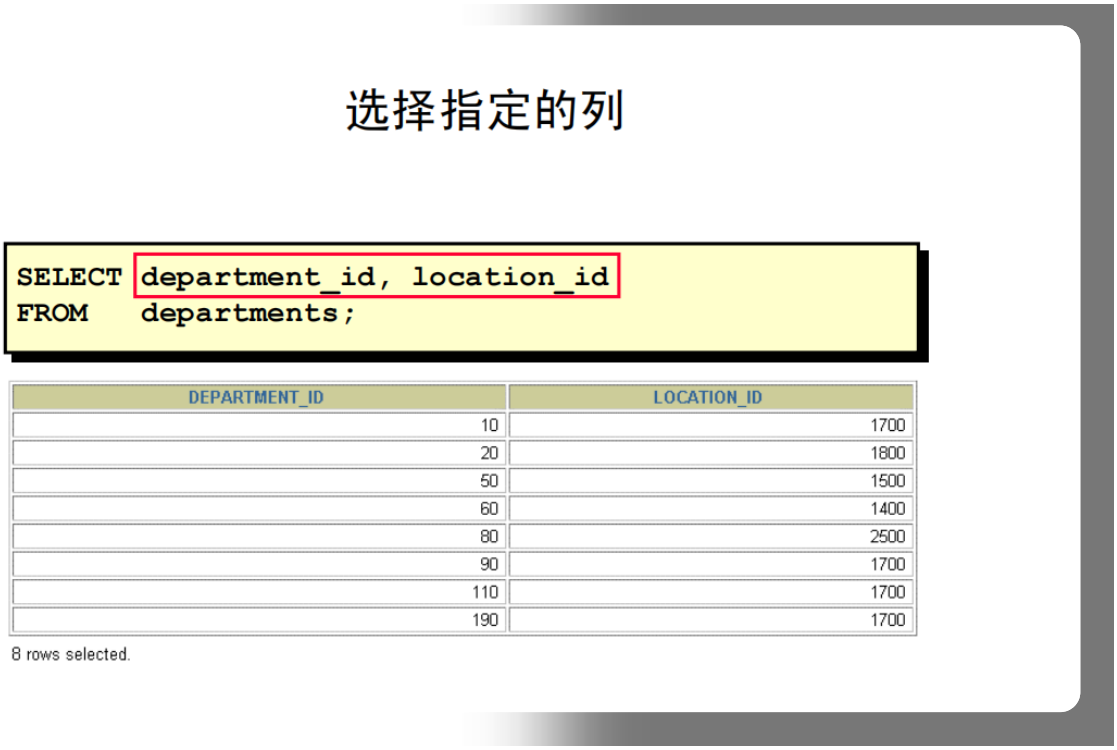
用跟在 SELECT 关键字后面的星号 (*)，你能够显示表中数据的所有列。

示例：

查询 departments 表中的所有数据。

```
1 select * from departments;
```

选择指定列



能够用 SELECT 语句来显示表的指定列，指定列名之间用逗号分隔。

示例：

查询 departments 表中所有部门名称。

```
select department_name from departments;
```

实时效果反馈

1.用 SELECT 语句来显示表的指定列，指定列名之间用分隔

A 空格

B 逗号

C 分号

D 冒号

答案

1=>B

查询中的算术表达式

算术表达式

用算术运算符创建数字和日期数据的表达式

操作	说明
+	加
-	减
*	乘
/	除

需要修改数据显示方式，如执行计算，或者作假定推测，这些都可能用到算

术表达式。一个算术表达式可以包含列名、固定的数字值和算术运算符。

使用算术运算符

使用算术运算符

```
SELECT last_name, salary, salary + 300
FROM   employees;
```

LAST_NAME	SALARY	SALARY+300
King	24000	24300
Kochhar	17000	17300
De Haan	17000	17300
Hunold	9000	9300
Ernst	6000	6300

示例：

查询雇员的年薪，并显示他们的雇员ID，名字。

```
1 select employees_id,last_name, 12*salary from
  employees;
```

运算符的优先级

运算符的优先级



- 乘法和除法比加法和减法的优先级高
- 相同优先级的运算符从左到右计算
- 圆括号用于强制优先计算，并且使语句更清晰

如果算术表达式包含有一个以上的运算，乘法和除法先计算。如果在一个表达式中

的运算符优先级相同，计算从左到右进行。可以用圆括号强制其中的表达式先计算。

示例一：

计算 employees 表中的员工全年薪水加 100 以后的薪水是多少，并显示他们的员工ID与名字。

```
select employees_id,last_name, 12*salary+100
from employees;
```

示例二：

计算 employees 表中的员工薪水加 100 以后的全年薪水是多少，并显示他们的员工ID与名字。

```
select employees_id,last_name, 12*
(salary+100) from employees;
```

实时效果反馈

1.一个算术表达式可以包含

A 列名

B 固定的数字值

C 算术运算符

D 列名、固定的数字值和算术运算符

答案

1=>D

MySQL中定义空值

定义空值

- null 是一个未分配的、未知的，或不适用的值
- null 不是 0，也不是空格

```
SELECT last_name, job_id, salary, commission_pct
FROM   employees;
```

LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT
King	AD_PRES	24000	
Kochhar	AD_VP	17000	
...			
Zlotkey	SA_MAN	10500	.2
Abel	SA_REP	11000	.3
Taylor	SA_REP	8600	.2

如果一行中的某个列缺少数据值，该值被置为 *null*， 或者说包含一个空。

空是一个难以获得的、未分配的、未知的，或不适用的值。空和 0 或者空格不相同。 0 是一个数字，而空格是一个字符。

算术表达式中的空值

算术表达式中的空值

包含空值的算术表达式计算结果为空

```
SELECT last_name, 12*salary*commission_pct
FROM   employees;
```

LAST_NAME	12*SALARY*COMMISSION_PCT
King	
Kochhar	
...	
Zlotkey	25200
Abel	39600
Taylor	20640

示例：

计算年薪包含佣金。

```
1 select 12*salary*commission_pct from
  employees;
```

实时效果反馈

1.下列说法正确的是

A 包含空值的算术表达式结算结果为计算后的值

B 包含空值的算术表达式结算结果为“ ”

C 包含空值的算术表达式结算结果为空

D 包含空值的算术表达式结算结果为0

答案

1=>C

MySQL中的别名

使用列别名

使用列别名

```
SELECT last_name AS name, commission_pct comm
FROM   employees;
```

NAME	COMM
King	
Kochhar	
De Haan	

...

20 rows selected.

```
SELECT   列名 AS   列别名 FROM   表名 WHERE   条件;
```

示例：

查询 employees 表将雇员 last_name 列定义别名为 name。

```
select last_name as name from employees;
select last_name name from employees;
```

使用表别名

```
SELECT   表别名.列名   FROM   表名 as 表别名 WHERE   条件;
```

示例：

查询 employees 表为表定义别名为emp，将雇员 last_name 列定义别名为 name。

```sql
select emp.last_name name from employees emp;
```

实时效果反馈

1.列别名：SELECT　FROM 表名;

A 列名 AS 列别名 | 列别名

B 列名 AS 列别名 | 列名 列别名

C 列别名 | 列名 列别名

D AS 列别名

答案

1=>B

MySQL中去除重复

相同的行

SELECT 默认显示所有的行，包括相同的行

```
SELECT department_id
FROM   employees;
```

DEPARTMENT_ID
90
90
90
60
60
60
50
50
50

除去相同的行

除去相同的行

在 SELECT 子句中用 DISTINCT 关键字除去相同的行

```
SELECT DISTINCT department_id
FROM   employees;
```

DEPARTMENT_ID
10
20
50
60
80
90
110

8 rows selected.

```sql
SELECT DISTINCT 列名 FROM 表名;
```

示例：

查询 employees 表，显示唯一的部门 ID。

```sql
select distinct department_id from employees;
```

实时效果反馈

1.剔除相同行：SELECT 列名 FROM 表名;

A DISTINCT

B DISTINCTS

C DIST

D DT

答案

1=>A

查询中的行选择

用选择限制行

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90
103	Hunold	IT_PROG	60
104	Ernst	IT_PROG	60
107	Lorentz	IT_PROG	60
124	Mourgos	ST_MAN	50

...

20 rows selected.

"返回所有在部门90的雇员"

EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
100	King	AD_PRES	90
101	Kochhar	AD_VP	90
102	De Haan	AD_VP	90

用 WHERE 子句限制从查询返回的行。一个 WHERE 子句包含一个必须满足的条件，WHERE 子句紧跟着 FROM 子句。如果条件是 true，返回满足条件的行。

在语法中：

WHERE	限制查询满足条件的行
condition	由列名、表达式、常数和比较操作组成

```
SELECT * |  投影列 FROM  表名 WHERE  选择条件;
```

示例：

查询 departments 表中部门 ID 为 90 的部门名称与工作地点 ID。

```
select department_name,location_id from
departments where department_id =4;
```

实时效果反馈

1.用 子句限制从查询返回的行。

A DISTINCT

B SELECT

C FROM

D WHERE

答案

1=>D

MySQL中的比较条件

比较条件

运算	含义
=	等于
>	大于
>=	大于等于
<	小于
<=	小于等于
<>	不等于

符号 != 也能够表示 不等于条件。

示例一：

查询 employees 表中员工薪水大于等于 3000 的员工的姓名与薪水。

```
select last_name,salary from employees where salary >= 3000;
```

示例二：

查询 employees 表中员工薪水不等于 5000 的员工的姓名与薪水。

```sql
select last_name,salary from employees where salary<>5000;
```

实时效果反馈

1.在MySQL数据中用表示不等于。

A <> | ^=

B <> | !=

C <> | 不等于

D ^= | 不等于

答案

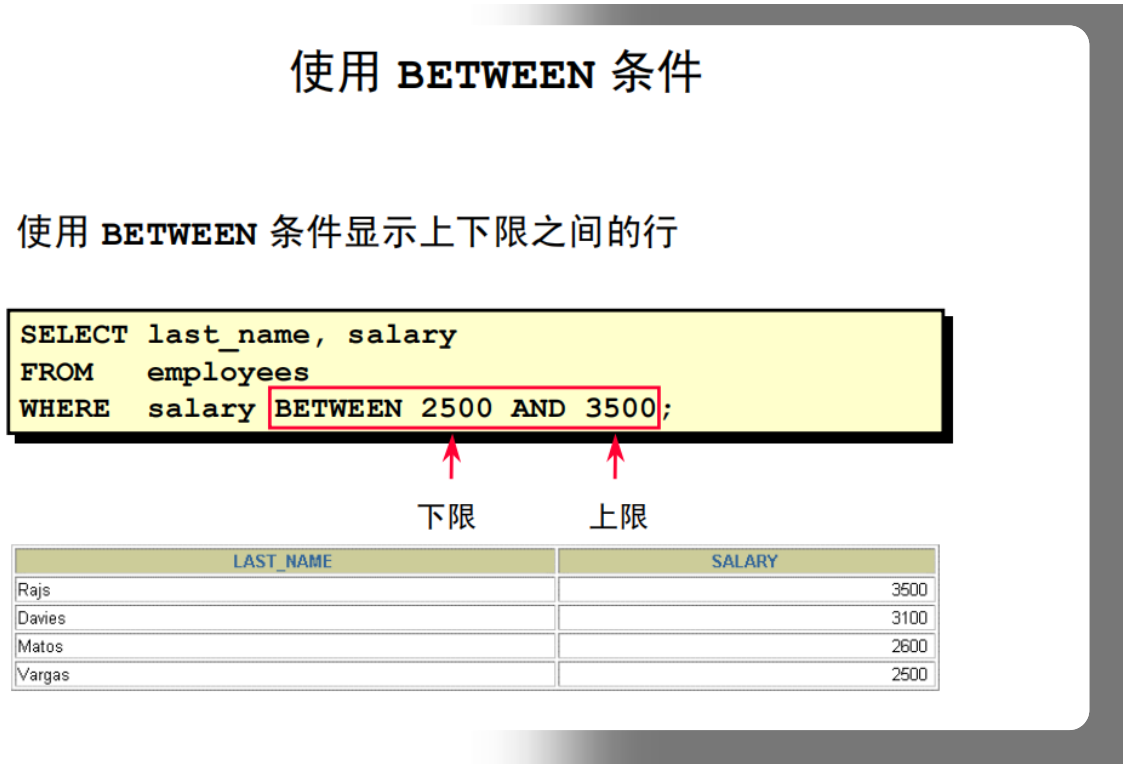
1=>B

其他比较条件

其它比较条件

操作	含义
BETWEEN ...AND...	在两个值之间 (包含)
IN(set)	匹配一个任意值列表
LIKE	匹配一个字符模板
IS NULL	是一个空值

使用BETWEEN条件



可以用 BETWEEN 范围条件显示基于一个值范围的行。指定的范围包含一个下限和一个上限。

示例：

查询 employees 表，薪水在 3000-8000 之间的雇员ID、名字与薪水。

```
select employee_id,last_name,salary from
employees where salary between 3000 and 8000;
```

使用IN条件

使用 IN 条件

使用 IN 成员条件测试在列表中的值

```
SELECT employee_id, last_name, salary, manager_id
FROM   employees
WHERE  manager_id IN (100, 101, 201);
```

EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
202	Fay	6000	201
200	Whalen	4400	101
205	Higgins	12000	101
101	Kochhar	17000	100
102	De Haan	17000	100
124	Mourgos	5800	100
149	Zlotkey	10500	100
201	Hartstein	13000	100

8 rows selected.

示例：

查询 employees 表，找出薪水是 5000,6000,8000 的雇员ID、名字与薪水。

```
select employee_id,last_name,salary from
employees where salary in(5000,6000,8000);
```

使用LIKE条件

使用 LIKE 条件

- 使用 LIKE 条件执行有效搜索串值的通配符搜索
- 搜索条件既可以包含文字也可以包含数字：
  - % 表示零个或多个字符
  - _ 表示一个字符

```
SELECT   first_name
FROM     employees
WHERE    first_name LIKE 'S%';
```

示例：

查询 employees 中雇员名字第二个字母是 e 的雇员名字。

```
1 select last_name from employees where
  last_name like '_e%';
```

使用NULL条件

使用 NULL 条件

用 IS NULL 操作来测试空值

```
SELECT last_name, manager_id
FROM   employees
WHERE  manager_id IS NULL;
```

LAST_NAME	MANAGER_ID
King	

NULL 条件，包括 IS NULL 条件和 IS NOT NULL 条件。

IS NULL 条件用于空值测试。空值的意思是难以获得的、未指定的、未知的或者不适用的。因此，你不能用 = ，因为 null 不能等于或不等于任何值。

示例一：

找出 emloyees 表中那些没有佣金的雇员雇员ID、名字与佣金。

```
select employee_id,last_name,commission_pct
from employees where commission_pct is null;
```

示例二：

找出 employees 表中那些有佣金的雇员ID、名字与佣金。

```
select employee_id,last_name,commission_pct
from employees where commission_pct is not
null;
```

实时效果反馈

1.如下说法正确的是。

A 在LIKE语句中，%表示零个或多个字符，_表示一个字符

B 在LIKE语句中，%表示一个字符，_表示零个或多个字符

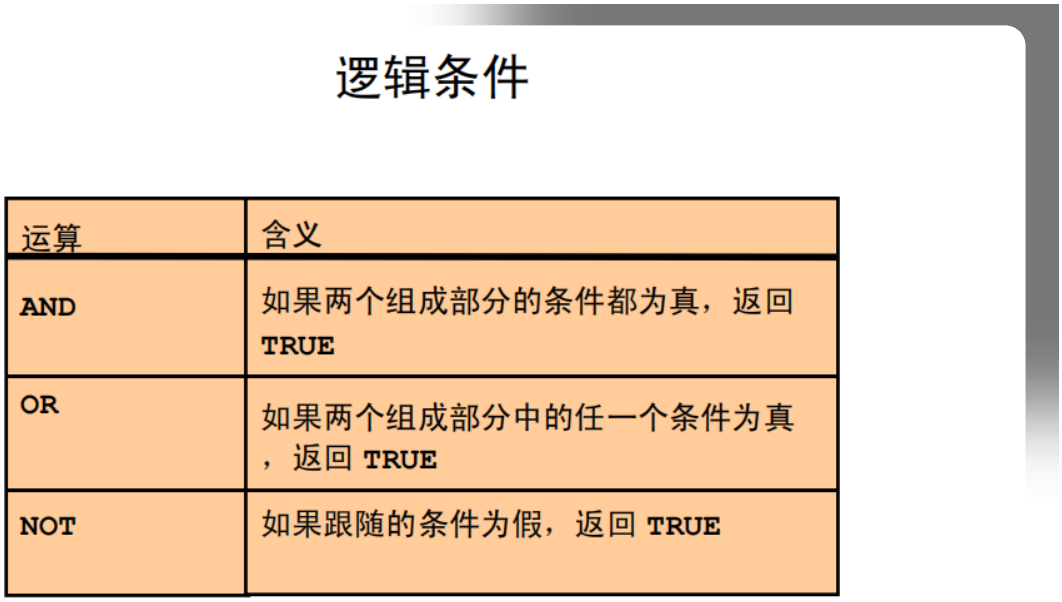
C 在LIKE语句中，%表示零个，_表示一个字符或多个字符

D 在LIKE语句中，%表示多个字符，_表示一个或零个字符

答案

1=>A

逻辑条件



逻辑条件组合两个比较条件的结果来产生一个基于这些条件的单个的结果，或者逆

转一个单个条件的结果。当所有条件的结果为真时，返回行。

SQL 的三个逻辑运算符是：

- AND
- OR
- NOT

可以在 WHERE 子句中用 AND 和 OR 运算符使用多个条件。

示例一：

查询 employees 表中雇员薪水是 8000 的并且名字中含有e 的雇员名字与薪水。

```
select last_name,salary from employees where salary = 8000 and last_name like '%e%';
```

示例二：

查询 employees 表中雇员薪水是 8000 的或者名字中含有e 的雇员名字与薪水。

```
select last_name,salary from employees where
salary = 8000 or last_name like '%e%';
```

示例三：

查询 employees 表中雇员名字中不包含 u 的雇员的名字。

```
select last_name from employees where
last_name not like '%u%';
```

实时效果反馈

1.SQL 的三个逻辑运算符是。

A AND、NULL、NOT

B BETWEEN、OR、NOT

C AND、OR、IN

D AND、OR、NOT

答案

1=>D

优先规则

优先规则

求值顺序	
1	算术运算
2	连字操作
3	比较操作
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	NOT 逻辑条件
7	AND 逻辑条件
8	OR 逻辑条件

使用圆括号改变优先规则

优先规则

```
SELECT last_name, job_id, salary
FROM   employees
WHERE  job_id = 'SA_REP'
OR     job_id = 'AD_PRES'
AND    salary > 15000;
```

LAST_NAME	JOB_ID	SALARY
King	AD_PRES	24000
Abel	SA_REP	11000
Taylor	SA_REP	8600
Grant	SA_REP	7000

在图片的例子中，有两个条件：

- 第一个条件是 job_id 是 AD_PRES 并且薪水高于 15,000。
- 第二个条件是 job_id 是 SA_REP。

优先规则

使用圆括号强制优先权

```
SELECT last_name, job_id, salary
FROM   employees
WHERE  (job_id = 'SA_REP'
OR     job_id = 'AD_PRES')
AND    salary > 15000;
```

LAST_NAME	JOB_ID	SALARY
King	AD_PRES	24000

在图片中的例子有两个条件：

- 第一个条件是 job_id 是 AD_PRES 或者 SA_REP 。
- 第二个条件是薪水高于$15,000

实时效果反馈

1.对于优先规则，下列说法正确的是。

A NOT、AND、OR、算术运算

B 算术运算、OR、NOT、AND

C AND、算术运算、NOT、OR

D 算术运算、NOT、AND、OR

答案

1=>D

使用 ORDER BY 排序

ORDER BY 子句

- 用 ORDER BY 子句排序行
  - ASC: 升序排序，默认
  - DESC: 降序排序
- ORDER BY 子句在 SELECT 语句的最后

```
SELECT   last_name, job_id, department_id, hire_date
FROM     employees
ORDER BY hire_date ;
```

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
King	AD_PRES	90	17-JUN-87
Whalen	AD_ASST	10	17-SEP-87
Kochhar	AD_VP	90	21-SEP-89
Hunold	IT_PROG	60	03-JAN-90
Ernst	IT_PROG	60	21-MAY-91

...

20 rows selected.

在一个不明确的查询结果中排序返回的行。ORDER BY 子句用于排序。如果使用了 ORDER BY 子句，它必须位于 SQL 语句的最后。

SELECT 语句的执行顺序如下：

- FROM 子句
- WHERE 子句
- SELECT 子句
- ORDER BY 子句

示例一：

查询 employees 表中的所有雇员，显示他们的ID、名字与薪水，并按薪水升序排序。

```
select employee_id,last_name,salary from employees order by salary;
select employee_id,last_name,salary from employees order by salary asc;
```

示例二：

查询 employees 表中的所有雇员，显示他们的ID与名字，并按雇员名字降序排序。

```
select employee_id,last_name from employees order by last_name desc;
```

使用别名排序

用列别名排序

```
SELECT employee_id, last_name, salary*12 annsal
FROM   employees
ORDER BY annsal;
```

EMPLOYEE_ID	LAST_NAME	ANNSAL
144	Vargas	30000
143	Matos	31200
142	Davies	37200
141	Rajs	42000
107	Lorentz	50400
200	Whalen	52800
124	Mourgos	69600
104	Ernst	72000
202	Fay	72000
178	Grant	84000

...

20 rows selected.

示例：

显示雇员ID，名字。计算雇员的年薪，年薪列别名为annsal，并对该列进行升序排序，

```
select employee_id,last_name ,12*salary
annsal from employees order by annsal;
```

多列排序

多列排序

- ORDER BY 列表的顺序就是排序的顺序

```
SELECT last_name, department_id, salary
FROM   employees
ORDER BY department_id, salary DESC;
```

LAST_NAME	DEPARTMENT_ID	SALARY
Whalen	10	4400
Hartstein	20	13000
Fay	20	6000
Mourgos	50	5800
Rajs	50	3500
Davies	50	3100
Matos	50	2600
Vargas	50	2500

...

20 rows selected.

- 你可以排序一个不在 SELECT 列表中的列

示例：

以升叙排序显示 DEPARTMENT_ID 列，同时以降序排序显示 SALARY 列。

```
select department_id,salary from employees
order by department_id asc ,salary desc;
```

实时效果反馈

1.对SELECT 语句的执行顺序描述正确的是。

A SELECT 子句、FROM 子句、WHERE 子句、ORDER BY 子句

B ORDER BY 子句、FROM 子句、WHERE 子句、SELECT 子句

C WHERE 子句 、FROM 子句、SELECT 子句、ORDER BY 子句

D FROM 子句 、WHERE 子句、SELECT 子句、ORDER BY 子句

答案

1=>D

练习

1.创建一个查询，显示收入超过 12,000 的雇员的名字和薪水。

```
select
LAST_NAME,SALARY
from employees
WHERE SALARY > 12000;
```

2.创建一个查询，显示雇员号为 176 的雇员的名字和部门号。

```
SELECT
LAST_NAME,DEPARTMENT_ID
from employees
where EMPLOYEE_ID = 176;
```

3.显示所有薪水不在 5000 和 12000 之间的雇员的名字和薪水。

```
select
LAST_NAME,SALARY
from employees
where salary not BETWEEN 5000 and 12000;
```

4.显示所有在部门 20 和 50 中的雇员的名字和部门号，并以名字按字母顺序排序。

```
LAST_NAME,DEPARTMENT_ID
FROM employees
WHERE DEPARTMENT_ID IN (20,50)
ORDER BY LAST_NAME asc;
```

5.列出收入在 5,000 和 12,000 之间，并且在部门 20 或50 工作的雇员的名字和薪水。将列标题分别显示为 Employee 和 Monthly Salary

```
SELECT
LAST_NAME Employee,SALARY 'Monthly Salary'
FROM employees
WHERE SALARY BETWEEN 5000 and 12000
AND
DEPARTMENT_ID in(20,50);
```

6.显示所有没有主管经理的雇员的名字和工作岗位。

```
SELECT
LAST_NAME,JOB_ID
FROM employees
WHERE MANAGER_ID is null;
```

7.显示所有有佣金的雇员的名字、薪水和佣金。以薪水和佣金的降序排序数据。

```
SELECT
LAST_NAME,SALARY,COMMISSION_PCT
from employees
where COMMISSION_PCT is not NULL
ORDER BY SALARY DESC , COMMISSION_PCT desc;
```

8.显示所有名字中有一个 a 和一个 e 的雇员的名字。

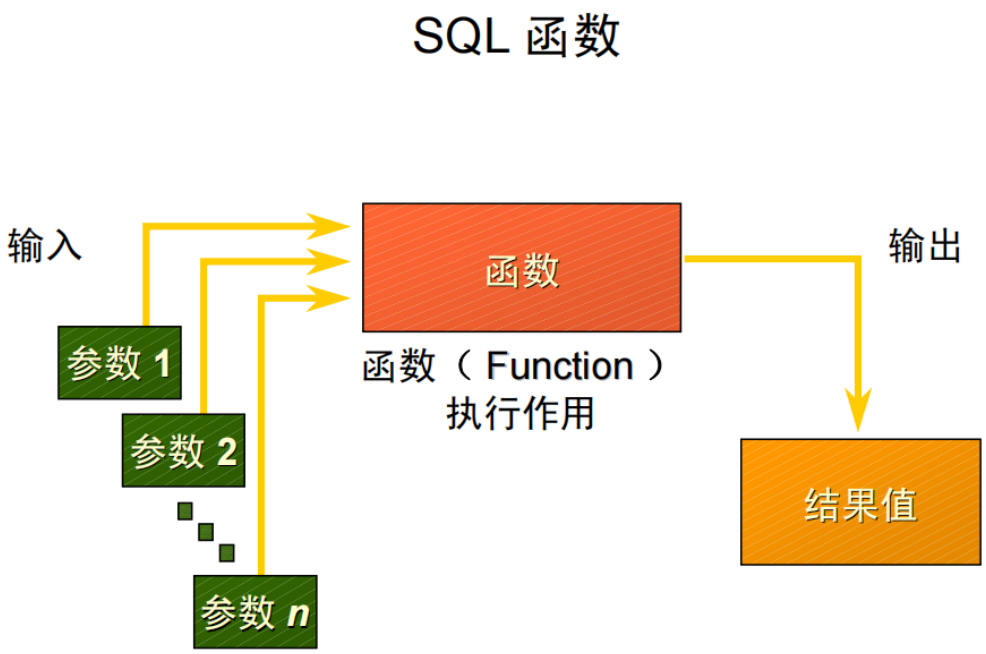
```
SELECT
LAST_NAME
from employees
where LAST_NAME LIKE '%a%'
AND
LAST_NAME LIKE '%e%';
```

9.显示所有工作岗位是销售代表（SA_REP）或者普通职员(ST_CLERK)，并且薪水不等于 2,500、3,500 或 7,000 的雇员的名字、工作岗位和薪水。

```sql
SELECT
LAST_NAME,JOB_ID,SALARY
from employees
WHERE
JOB_ID in('SA_REP','ST_CLIERK')
AND
SALARY not IN(2500,3500,7000)
```

SQL函数

函数介绍

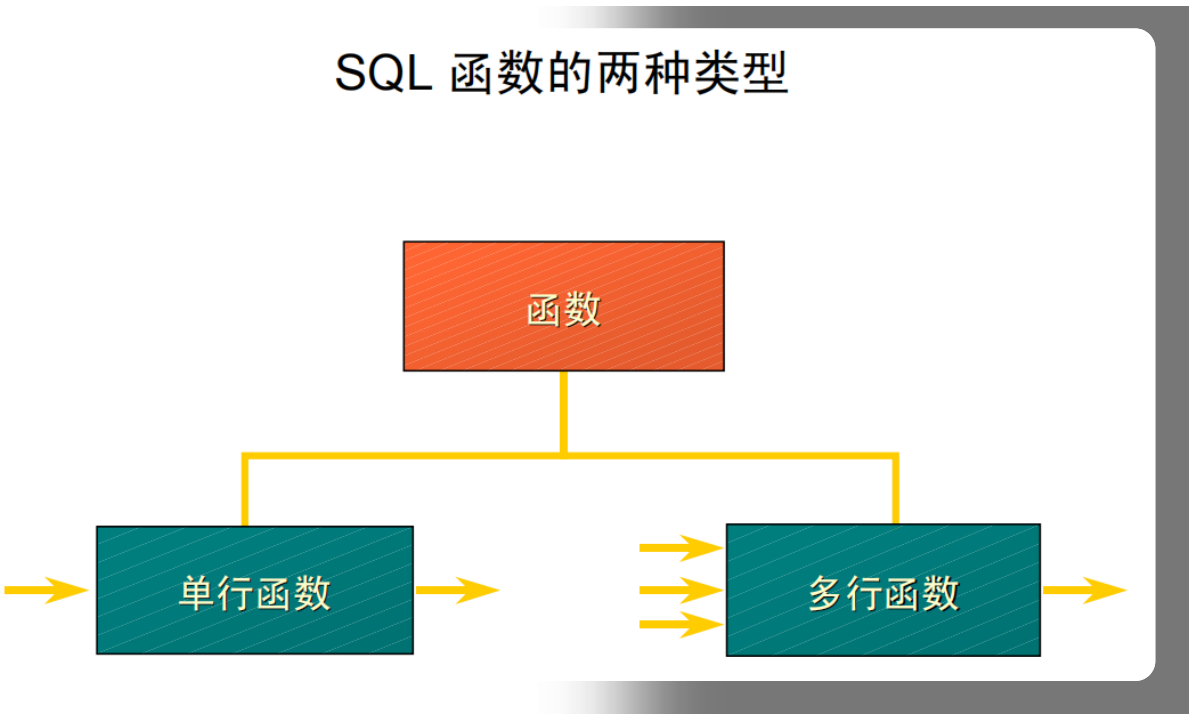


函数是 SQL 的一个非常强有力的特性，函数能够用于下面的目的：

- 执行数据计算
- 修改单个数据项
- 操纵输出进行行分组
- 格式化显示的日期和数字
- 转换列数据类型

SQL 函数有输入参数，并且总有一个返回值。

函数分类



1. 单行函数

   单行函数仅对单个行进行运算，并且每行返回一个结果。

   常见的函数类型：

   - 字符
   - 数字
   - 日期
   - 转换

2. 多行函数

多行函数能够操纵成组的行，每个行组给出一个结果，这些函数也被称为组函数。

实时效果反馈

1.下列对单行函数描述正确的是。

A 单行函数对多个行进行运算，并且每行返回一个结果

B 单行函数仅对单个行进行运算，并且每行返回多个结果

C 单行函数仅对单个行进行运算，并且每行返回一个结果

D 单行函数仅对单个行进行运算，并且每行不会返回结果

答案

1=>C

单行函数

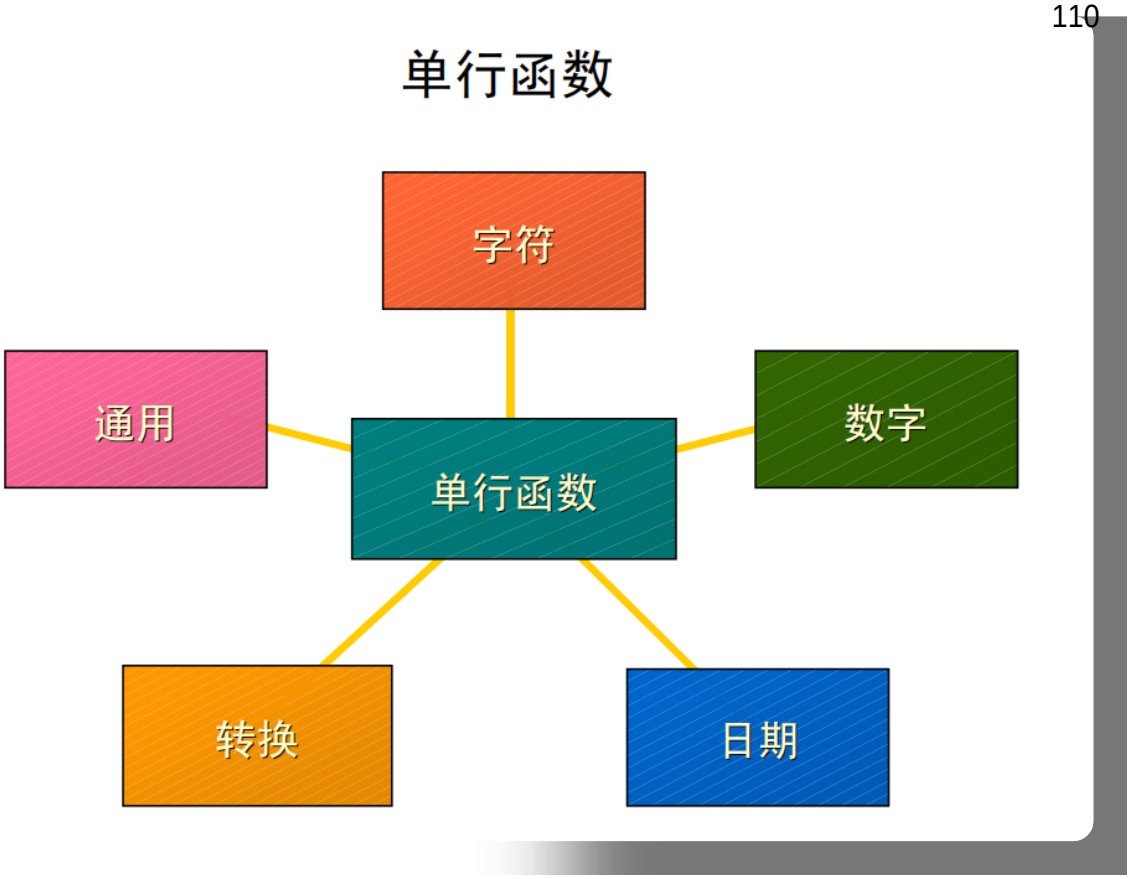
单行函数

单行函数：

- 操纵数据项
- 接受多个参数并返回一个值
- 作用于每一个返回行
- 每行返回一个结果
- 可以修改数据类型
- 可以嵌套
- 接受多个参数，参数可以是一个列或者一个表达式

```
function_name [(arg1, arg2,...)]
```

单行函数分类



实时效果反馈

1.单行函数包含了。

A 字符函数、运算函数、日期函数、转换函数、通用函数

B 字符函数、数字函数、日期函数、转换函数、通用函数

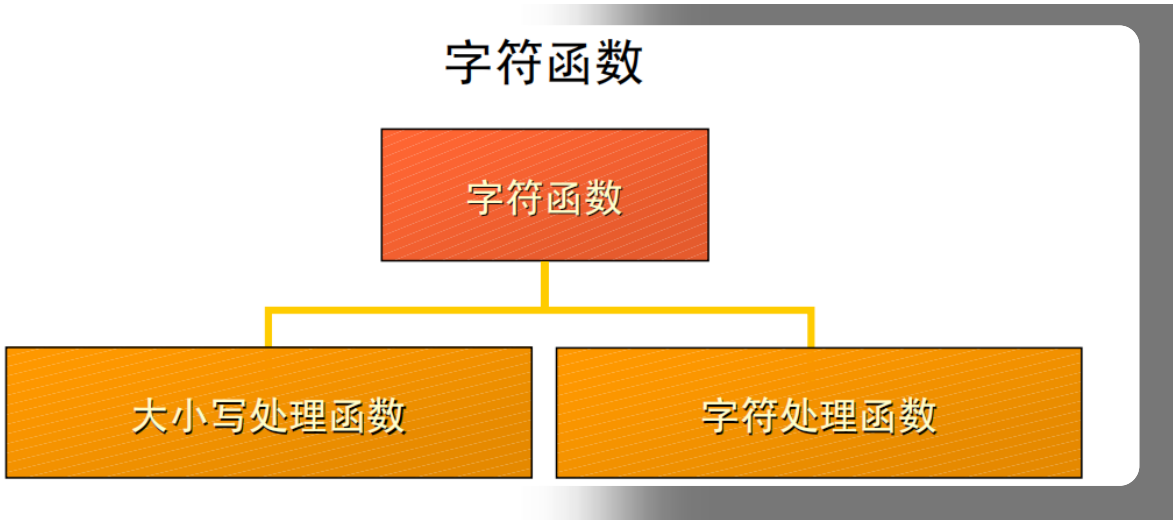
C 字符函数、数字函数、处理函数、转换函数、通用函数

D 字符函数、数字函数、日期函数、定义函数、通用函数

答案

1=>B

字符函数



大小写处理函数

函数	描述	实例
LOWER(s)|LCASE(s)	将字符串 s 转换为小写	将字符串 OLDLU转换为小写：`SELECT LOWER("OLDLU"); -- oldlu`
UPPER(s)|UCASE(s)	将字符串s转换为大写	将字符串 oldlu转换为大写：`SELECT UPPER("oldlu"); -- OLDLU`

示例：

显示雇员 Davies 的雇员号、姓名和部门号，将姓名转换为大写。

```
select
employee_id,UPPER(last_name),department_id
from employees where last_name = 'davies';
```

字符处理函数

函数	描述	实例
LENGTH(s)	返回字符串 s 的长度	返回字符串oldlu的字符数 `SELECT LENGTH("oldlu"); --5;`
CONCAT(s1,s2...sn)	字符串 s1,s2 等多个字符串合并为一个字符串	合并多个字符串 `SELECT CONCAT("sxt ", "teacher ", "oldlu"); --sxt teacher oldlu;`
LPAD(s1,len,s2)	在字符串 s1 的开始处填充字符串 s2，使字符串长度达到 len	将字符串 x 填充到 oldlu字符串的开始处：`SELECT LPAD('oldlu',8,'x'); -- xxxoldlu`
LTRIM(s)	去掉字符串 s 开始处的空格	去掉字符串 oldlu开始处的空格：`SELECT LTRIM(" oldlu") ;-- oldlu`
REPLACE(s,s1,s2)	将字符串 s2 替代字符串 s 中的字符串 s1	将字符串 oldlu 中的字符 o 替换为字符 O：`SELECT REPLACE('oldlu','o','O'); --Oldlu`
REVERSE(s)	将字符串s的顺序反过来	将字符串 abc 的顺序反过来：`SELECT REVERSE('abc'); -- cba`
RPAD(s1,len,s2)	在字符串 s1 的结尾处添加字符串 s2，使字符串的长度达到 len	将字符串 xx填充到 oldlu字符串的结尾处：`SELECT RPAD('oldlu',8,'x'); -- oldluxxx`
RTRIM(s)	去掉字符串 s 结尾处的空格	去掉字符串 oldlu 的末尾空格：`SELECT RTRIM("oldlu "); -- oldlu`
SUBSTR(s, start, length)	从字符串 s 的 start 位置截取长度为 length 的子字符串	从字符串 OLDLU中的第 2 个位置截取 3个字符：`SELECT SUBSTR("OLDLU", 2, 3); --LDL`
SUBSTRING(s, start, length)	从字符串 s 的 start 位置截取长度为 length 的子字符串	从字符串 OLDLU中的第 2 个位置截取 3个字符：`SELECT SUBSTRING("OLDLU", 2, 3); --LDL`
TRIM(s)	去掉字符串 s 开始和结尾处的空格	去掉字符串 oldlu 的首尾空格：`SELECT TRIM(' oldlu ');--oldlu`

示例：

显示所有工作岗位名称从第 4 个字符位置开始，包含字符串 REP的雇员的ID信息，将雇员的姓和名连接显示在一起，还显示雇员名的的长度，以及名字中字母 a 的位置。

```
1 SELECT employee_id, CONCAT(last_name,
  first_name) NAME,
2 job_id, LENGTH(last_name),INSTR(last_name,
  'a') "Contains 'a'?" FROM employees WHERE
  SUBSTR(job_id, 4) = 'REP';
```

数字函数

函数名	描述	实例
ABS(x)	返回 x 的绝对值	返回 -1 的绝对值：SELECT ABS(-1) -- 返回1
ACOS(x)	求 x 的反余弦值(参数是弧度)	SELECT ACOS(0.25);
ASIN(x)	求反正弦值(参数是弧度)	SELECT ASIN(0.25);
ATAN(x)	求反正切值(参数是弧度)	SELECT ATAN(2.5);
ATAN2(n, m)	求反正切值(参数是弧度)	SELECT ATAN2(-0.8, 2);
AVG(expression)	返回一个表达式的平均值，expression 是一个字段	返回 Products 表中Price 字段的平均值：SELECT AVG(Price) AS AveragePrice FROM Products;
CEIL(x)	返回大于或等于 x 的最小整数	SELECT CEIL(1.5) -- 返回2
CEILING(x)	返回大于或等于 x 的最小整数	SELECT CEILING(1.5); -- 返回2
COS(x)	求余弦值(参数是弧度)	SELECT COS(2);
COT(x)	求余切值(参数是弧度)	SELECT COT(6);
COUNT(expression)	返回查询的记录总数，expression 参数是一个字段或者 * 号	返回 Products 表中 products 字段总共有多少条记录：SELECT COUNT(ProductID) AS NumberOfProducts FROM Products;
DEGREES(x)	将弧度转换为角度	SELECT DEGREES(3.1415926535898) -- 180
n DIV m	整除，n 为被除数，m 为除数	计算 10 除于 5：SELECT 10 DIV 5; -- 2
EXP(x)	返回 e 的 x 次方	计算 e 的三次方：SELECT EXP(3) -- 20.085536923188
FLOOR(x)	返回小于或等于 x 的最大整数	小于或等于 1.5 的整数：SELECT FLOOR(1.5) -- 返回1
GREATEST(expr1, expr2, expr3, ...)	返回列表中的最大值	返回以下数字列表中的最大值：SELECT GREATEST(3, 12, 34, 8, 25); -- 34 返回以下字符串列表中的最大值：SELECT GREATEST("Google", "Runoob", "Apple"); -- Runoob
LEAST(expr1, expr2, expr3, ...)	返回列表中的最小值	返回以下数字列表中的最小值：SELECT LEAST(3, 12, 34, 8, 25); -- 3 返回以下字符串列表中的最小值：SELECT LEAST("Google", "Runoob", "Apple"); -- Apple
LN	返回数字的自然对数，以 e 为底。	返回 2 的自然对数：SELECT LN(2); -- 0.6931471805599453
LOG(x) 或 LOG(base, x)	返回自然对数(以 e 为底的对数)，如果带有 base 参数，则 base 为指定带底数。	SELECT LOG(20.085536923188) -- 3 SELECT LOG(2, 4); -- 2
LOG10(x)	返回以 10 为底的对数	SELECT LOG10(100) -- 2
LOG2(x)	返回以 2 为底的对数	返回以 2 为底 6 的对数：SELECT LOG2(6); -- 2.584962500721156
MAX(expression)	返回字段 expression 中的最大值	返回数据表 Products 中字段 Price 的最大值：SELECT MAX(Price) AS LargestPrice FROM Products;
MIN(expression)	返回字段 expression 中的最小值	返回数据表 Products 中字段 Price 的最小值：SELECT MIN(Price) AS MinPrice FROM Products;
MOD(x,y)	返回 x 除以 y 以后的余数	5 除于 2 的余数：SELECT MOD(5,2) -- 1
PI()	返回圆周率(3.141593)	SELECT PI() --3.141593
POW(x,y)	返回 x 的 y 次方	2 的 3 次方：SELECT POW(2,3) -- 8
POWER(x,y)	返回 x 的 y 次方	2 的 3 次方：SELECT POWER(2,3) -- 8
RADIANS(x)	将角度转换为弧度	180 度转换为弧度：SELECT RADIANS(180) -- 3.1415926535898
RAND()	返回 0 到 1 的随机数	SELECT RAND() --0.93099315644334
ROUND(x)	返回离 x 最近的整数	SELECT ROUND(1.23456) --1
SIGN(x)	返回 x 的符号，x 是负数、0、正数分别返回 -1、0 和 1	SELECT SIGN(-10) -- (-1)

函数名	描述	实例
SIN(x)	求正弦值(参数是弧度)	SELECT SIN(RADIANS(30)) -- 0.5
SQRT(x)	返回x的平方根	25 的平方根：SELECT SQRT(25) -- 5
SUM(expression)	返回指定字段的总和	计算 OrderDetails 表中字段 Quantity 的总和：SELECT SUM(Quantity) AS TotalItemsOrdered FROM OrderDetails;
TAN(x)	求正切值(参数是弧度)	SELECT TAN(1.75); -- -5.52037992250933
TRUNCATE(x,y)	返回数值 x 保留到小数点后 y 位的值（与 ROUND 最大的区别是不会进行四舍五入）	SELECT TRUNCATE(1.23456,3) -- 1.234

ROUND(*column*|*expression*, *n*) 函数

ROUND 函数四舍五入列、表达式或者 n 位小数的值。如果第二个参数是 0 或者缺少，值被四舍五入为整数。如果第二个参数是 2值被四舍五入为两位小数。如果第二个参数是-2，值被四舍五入到小数点左边两位。

```
SELECT ROUND(45.923,2),
ROUND(45.923,0),ROUND(45.923,-1);
```

TRUNCATE(*column*|*expression*,*n*) 函数

TRUNCATE函数的作用类似于 ROUND 函数。如果第二个参数是 0 或者缺少，值被截断为整数。如果第二个参数是 2，值被截断为两位小数。如果第二个参数是-2，值被截断到小数点左边两位。与 ROUND 最大的区别是不会进行四舍五入。

```
SELECT TRUNCATE(45.923,2);
```

使用MOD(*m*,*n*) 函数

MOD 函数找出m 除以n的余数。

示例：

所有job_id是SA_REP的雇员的名字，薪水以及薪水被5000除后的余数。

```
SELECT last_name, salary, MOD(salary, 5000)
FROM employees
WHERE job_id = 'SA_REP';
```

日期函数

在MySQL中允许直接使用字符串表示日期，但是要求字符串的日期格式必须为：‘YYYY-MM-DD HH:MI:SS’ 或者‘YYYY/MM/DD HH:MI:SS’;

函数名	描述	实例
CURDATE()	返回当前日期	SELECT CURDATE(); -> 2018-09-19
CURTIME()	返回当前时间	SELECT CURTIME(); -> 19:59:02
CURRENT_DATE()	返回当前日期	SELECT CURRENT_DATE(); -> 2018-09-19
CURRENT_TIME()	返回当前时间	SELECT CURRENT_TIME(); -> 19:59:02
DATE()	从日期或日期时间表达式中提取日期值	SELECT DATE("2017-06-15"); -> 2017-06-15
DATEDIFF(d1,d2)	计算日期 d1->d2 之间相隔的天数	SELECT DATEDIFF('2001-01-01','2001-02-02') -> -32
DAY(d)	返回日期值 d 的日期部分	SELECT DAY("2017-06-15"); -> 15
DAYNAME(d)	返回日期 d 是星期几，如 Monday,Tuesday	SELECT DAYNAME('2011-11-11 11:11:11') ->Friday
DAYOFMONTH(d)	计算日期 d 是本月的第几天	SELECT DAYOFMONTH('2011-11-11 11:11:11') ->11
DAYOFWEEK(d)	日期 d 今天是星期几，1 星期日，2 星期一，以此类推	SELECT DAYOFWEEK('2011-11-11 11:11:11') ->6
DAYOFYEAR(d)	计算日期 d 是本年的第几天	SELECT DAYOFYEAR('2011-11-11 11:11:11') ->315
HOUR(t)	返回 t 中的小时值	SELECT HOUR('1:2:3') -> 1
LAST_DAY(d)	返回给给定日期的那一月份的最后一天	SELECT LAST_DAY("2017-06-20"); -> 2017-06-30
MONTHNAME(d)	返回日期当中的月份名称，如 November	SELECT MONTHNAME('2011-11-11 11:11:11') -> November
MONTH(d)	返回日期d中的月份值，1 到 12	SELECT MONTH('2011-11-11 11:11:11') ->11
NOW()	返回当前日期和时间	SELECT NOW() -> 2018-09-19 20:57:43
SECOND(t)	返回 t 中的秒钟值	SELECT SECOND('1:2:3') -> 3
SYSDATE()	返回当前日期和时间	SELECT SYSDATE() -> 2018-09-19 20:57:43
TIMEDIFF(time1, time2)	计算时间差值	SELECT TIMEDIFF("13:10:11", "13:10:10"); -> 00:00:01
TO_DAYS(d)	计算日期 d 距离 0000 年 1 月 1 日的天数	SELECT TO_DAYS('0001-01-01 01:01:01') -> 366
WEEK(d)	计算日期 d 是本年的第几个星期，范围是 0 到 53	SELECT WEEK('2011-11-11 11:11:11') -> 45
WEEKDAY(d)	日期 d 是星期几，0 表示星期一，1 表示星期二	SELECT WEEKDAY("2017-06-15"); -> 3
WEEKOFYEAR(d)	计算日期 d 是本年的第几个星期，范围是 0 到 53	SELECT WEEKOFYEAR('2011-11-11 11:11:11') -> 45
YEAR(d)	返回年份	SELECT YEAR("2017-06-15"); -> 2017

示例一：

向 employees 表中添加一条数据，雇员ID：300，名字：kevin ，email：kevin@sxt.cn ，入职时间：2049-5-1 8:30:30，工作部门：‘IT_PROG’。

```
insert  into employees(EMPLOYEE_ID,last_name,email,HIRE_DATE,JOB_ID)
 values(300,'kevin','kevin@sxt.cn','2049-5-18:30:30','IT_PROG');
```

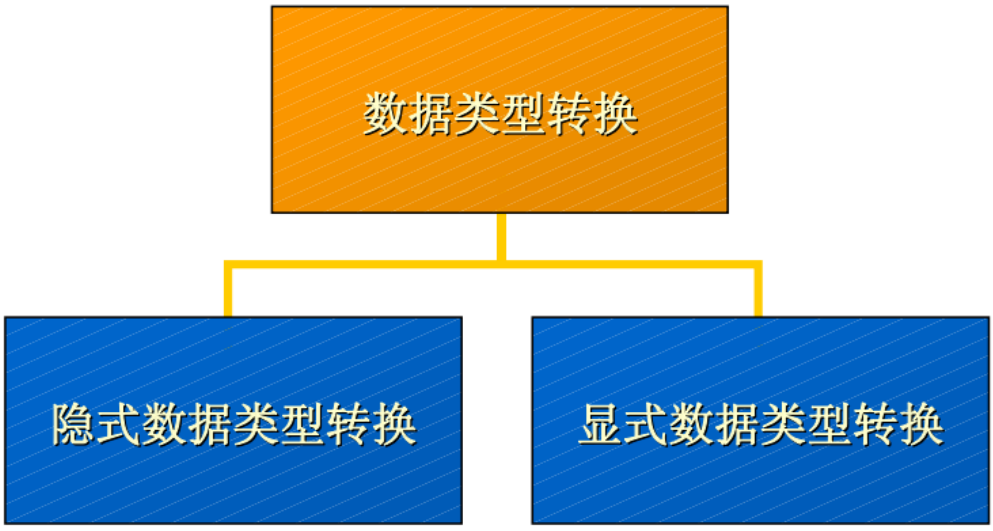
示例二：

显示所有在部门 90 中的雇员的名字和从业的周数。雇员的总工作时间以周计算，用当前日期 (SYSDATE) 减去雇员的受顾日期，再除以 7。

```
SELECT last_name, (SYSDATE()-hire_date)/7 AS WEEKS FROM employees WHERE department_id = 90;
```

转换函数

转换函数



隐式数据类型转换

隐式数据类型转换是指MySQL服务器能够自动地进行类型转换。
如：可以将标准格式的字串日期自动转换为日期类型。

MySQL字符串日期格式为：‘YYYY-MM-DD HH:MI:SS’ 或
‘YYYY/MM/DD HH:MI:SS’;

显示数据类型转换

显示数据类型转换是指需要依赖转换函数来完成相关类型的转换。

如：

- DATE_FORMAT(date,format)　将日期转换成字符串;
- STR_TO_DATE(str,format)　将字符串转换成日期;

格式	描述
%a	缩写星期名
%b	缩写月名
%c	月，数值
%D	带有英文前缀的月中的天
%d	月的天，数值(00-31)
%e	月的天，数值(0-31)
%f	微秒
%H	小时 (00-23)
%h	小时 (01-12)
%I	小时 (01-12)
%i	分钟，数值(00-59)
%j	年的天 (001-366)
%k	小时 (0-23)
%l	小时 (1-12)
%M	月名
%m	月，数值(00-12)
%p	AM 或 PM
%r	时间，12-小时（hh:mm:ss AM 或 PM）
%S	秒(00-59)
%s	秒(00-59)
%T	时间, 24-小时 (hh:mm:ss)
%U	周 (00-53) 星期日是一周的第一天
%u	周 (00-53) 星期一是一周的第一天
%V	周 (01-53) 星期日是一周的第一天，与 %X 使用
%v	周 (01-53) 星期一是一周的第一天，与 %x 使用
%W	星期名
%w	周的天（0=星期日, 6=星期六）
%X	年，其中的星期日是周的第一天，4 位，与 %V 使用
%x	年，其中的星期一是周的第一天，4 位，与 %v 使用
%Y	年，4 位
%y	年，2 位

示例一：

向 employees 表中添加一条数据，雇员ID：400，名字：oldlu ，email：oldlu@sxt.cn ，入职时间：2049 年 5 月 5 日，工作部门：‘IT_PROG’。

```
insert  into
employees(EMPLOYEE_ID,last_name,email,HIRE_DATE,JOB_ID)
 values(400,'oldlu','oldlu@sxt.cn',
STR_TO_DATE('2049 年 5 月 5 日','%Y 年%m 月%d 日'),'IT_PROG');
```

示例二：

查询 employees 表中雇员名字为 King 的雇员的入职日期，要求显示格式为 yyyy 年 MM 月 dd 日。

```
select DATE_FORMAT(hire_date,'%Y 年%m 月%d 日') from employees where last_name = 'King';
```

通用函数

函数名	描述	实例
IF(expr,v1,v2)	如果表达式 expr 成立，返回结果 v1；否则，返回结果 v2。	SELECT IF(1 > 0,'正确','错误') ->正确
IFNULL(v1,v2)	如果 v1 的值不为 NULL，则返回 v1，否则返回 v2。	SELECT IFNULL(null,'Hello Word') ->Hello Word
ISNULL(expression)	判断表达式是否为 NULL	SELECT ISNULL(NULL); ->1
NULLIF(expr1, expr2)	比较两个参数是否相同，如果参数 expr1 与 expr2 相等 返回 NULL，否则返回 expr1	SELECT NULLIF(25, 25); ->
COALESCE(expr1, expr2, ..., expr_n)	返回参数中的第一个非空表达式（从左向右）	SELECT COALESCE(NULL, NULL, NULL, 'bjsxt.com', NULL, 'google.com'); -> bjsxt.com
CASE expression WHEN condition1 THEN result1 WHEN condition2 THEN result2 ... WHEN conditionN THEN resultN ELSE result END	CASE 表示函数开始，END 表示函数结束。如果 condition1 成立，则返回 result1, 如果 condition2 成立，则返回 result2，当全部不成立则返回 result，而当有一个成立之后，后面的就不执行了。	SELECT CASE 'oldlu' WHEN 'oldlu' THEN 'OLDLU' WHEN 'admin' THEN 'ADMIN' ELSE 'kevin' END;

示例一：

查询部门编号是50或者80的员工信息，包含他们的名字、薪水、佣金。在income列中，如果有佣金则显示‘SAL+COMM’，无佣金则显示'SAL'。

```
SELECT last_name, salary, commission_pct,
    if(ISNULL(commission_pct),
'SAL','SAL+COMM') income
FROM employees
WHERE department_id IN (50, 80);
```

示例二：

计算雇员的年报酬，你需要用 12 乘以月薪，再加上它的佣金 (等于年薪乘以佣金

百分比)。

```
SELECT last_name, salary,
IFNULL(commission_pct, 0), (salary*12) +
(salary*12*IFNULL(commission_pct, 0)) AN_SAL
FROM employees;
```

示例三

查询员工表，显示他们的名字、名字的长度该列名为expr1，姓氏、姓氏的长度该列名为expr2。在result列中，如果名字与姓氏的长度相同则显示空，如果不相同则显示名字长度。

```
SELECT first_name, LENGTH(first_name)
"expr1",
       last_name, LENGTH(last_name) "expr2",
     NULLIF(LENGTH(first_name),
LENGTH(last_name)) result
FROM employees;
```

示例四:

查询员工表，显示他们的名字，如果 COMMISSION_PCT 值是非空，显示它。如果COMMISSION_PCT 值是空，则显示 SALARY 。如果 COMMISSION_PCT 和SALARY 值都是空，那么显示 10。在结果中对佣金列升序排序。

```
SELECT last_name,
COALESCE(commission_pct, salary, 10) comm
FROM employees
ORDER BY commission_pct;
```

示例五：

查询员工表，如果 JOB_ID 是 IT_PROG，薪水增加 10%；如果 JOB_ID 是 ST_CLERK，薪水增加 15%；如果 JOB_ID 是 SA_REP，薪水增加 20%。对于所有其他的工作角色，不增加薪水。

```
SELECT last_name, job_id, salary,
        CASE job_id WHEN 'IT_PROG' THEN 1.10*salary
                    WHEN 'ST_CLERK' THEN 1.15*salary
                    WHEN 'SA_REP' THEN 1.20*salary
        ELSE salary END "REVISED_SALARY"
FROM employees;
```

练习

1.显示受雇日期在 1998 年 2 月 20 日 和 2005 年 5 月 1 日 之间的雇员的名字、岗位

和受雇日期。按受雇日期顺序排序查询结果。

```
SELECT
LAST_NAME,JOB_ID,HIRE_DATE
FROM employees
WHERE HIRE_DATE BETWEEN '1998-2-20' AND
'2005-5-1'
order by HIRE_DATE;
```

2.显示每一个在 2002 年受雇的雇员的名字和受雇日期。

```
select
LAST_NAME,HIRE_DATE
FROM employees
where HIRE_DATE like '2002%'
```

3.对每一个雇员，显示 employee number、last_name、salary 和 salary 增加 15%，

并且表示成整数，列标签显示为 New Salary。

```
SELECT
EMPLOYEE_ID,LAST_NAME,SALARY,
ROUND(SALARY *1.15,0)
FROM employees
```

4.写一个查询，显示名字的长度，对所有名字开始字母是 J、A 或 M 的雇员。用雇员的 last

name排序结果。

```
SELECT
LAST_NAME,
LENGTH(LAST_NAME)
FROM employees
WHERE LAST_NAME LIKE 'J%'
OR
LAST_NAME LIKE 'A%'
OR
LAST_NAME LIKE 'M%'
ORDER BY LAST_NAME;
```

5.创建一个查询显示所有雇员的 last name 和 salary。将薪水格式化为 15 个字符长度，用 $左填充 。

```
SELECT
LAST_NAME,LPAD(SALARY,15,'$')
FROM employees;
```

6.创建一个查询显示雇员的 last names 和 commission (佣金) 比率。如果雇员没有佣金，显示 “No Commission”，列标签 COMM。

```
SELECT
LAST_NAME,IFNULL(COMMISSION_PCT,'No
Commission') COMM
FROM employees
```

7.写一个查询，按照下面的数据显示所有雇员的基于 JOB_ID 列值的级别。

工作	级别
AD_PRES	A
ST_MAN	B
IT_PROG	C
SA_REP	D
ST_CLERK	E
不在上面的	0

```
SELECT JOB_ID,
   CASE JOB_ID WHEN 'AD_PRES' THEN 'A'
               WHEN 'ST_MAN' THEN 'B'
               WHEN 'IT_PROG' THEN 'C'
               WHEN 'SA_REP' THEN 'D'
               WHEN 'ST_CLERK' THEN 'E'
   ELSE 0 END
FROM employees;
```

多表查询

多表查询简介

从多表中获得数据

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
202	Fay	20
205	Higgins	110
206	Gietz	110

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1500
60	IT	1400
80	Sales	2500
90	Executive	1700
110	Accounting	1700
190	Contracting	1700

EMPLOYEE_ID	DEPARTMENT_ID	DEPARTMENT_NAME
200	10	Administration
201	20	Marketing
202	20	Marketing
102	90	Executive
205	110	Accounting
206	110	Accounting

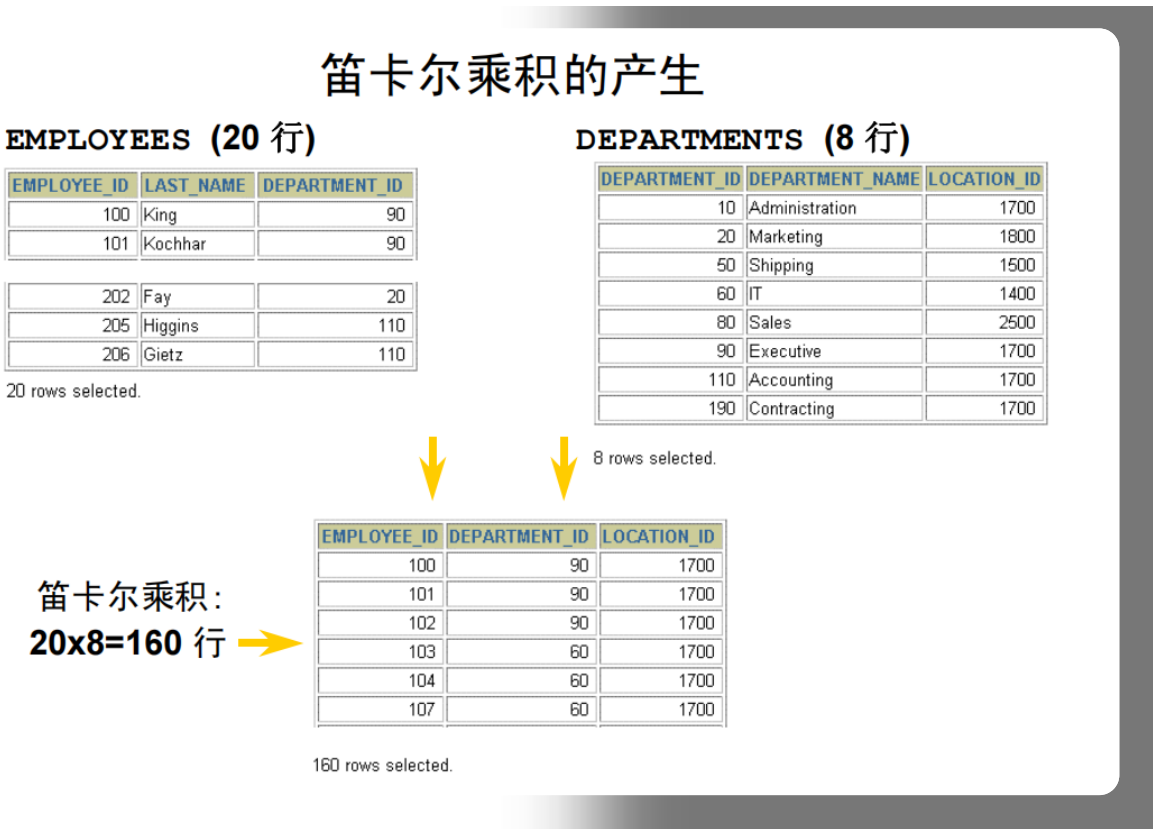
笛卡尔乘积

笛卡尔乘积

- 笛卡尔乘积的形成，当：
  - 一个连接条件被遗漏时
  - 一个连接条件不正确时
  - 在第一个表中的所有行被连接到第二个表的所有行时
- 为了避免笛卡尔乘积的形成，在 WHERE 子句中应当总是包含正确的连接条件

笛卡尔乘积：

当一个连接条件无效或被遗漏时，其结果是一个笛卡尔乘积(*Cartesian product*)，其中所有行的组合都被显示。第一个表中的所有行连接到第二个表中的所有行。一个笛卡尔乘积会产生大量的行，其结果没有什么用。你应该在 WHERE 子句中始终包含一个有效的连接条件，除非你有特殊的需求，需要从所有表中组合所有的行。



多表查询分类

- sql92标准：内连接(等值连接 、非等值连接 、 自连接)。
- sql99标准：内连接、外连接(左外、右外、全外(MySQL不支持全外连接))、交叉连接。

实时效果反馈

1.当一个无效或被遗漏时，其结果是一个笛卡尔乘积。

A 运算符

B 函数运算

C 连接条件

D 排序

答案

1=>C

SQL92标准中的查询

等值连接



EMPLOYEE_ID	DEPARTMENT_ID
200	10
201	20
202	20
124	50
141	50
142	50
143	50
144	50
103	60
104	60
107	60
149	80
174	80
176	80

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
20	Marketing
50	Shipping
50	Shipping
50	Shipping
50	Shipping
50	Shipping
60	IT
60	IT
60	IT
80	Sales
80	Sales
80	Sales

等值连接

为了确定一个雇员的部门名，需要比较 EMPLOYEES 表中的 DEPARTMENT_ID 列与DEPARTMENTS 表中的 DEPARTMENT_ID 列的值。在 EMPLOYEES 和DEPARTMENTS 表之间的关系是一个相等 (*equijoin*) 关系，即，两 个 表 中DEPARTMENT_ID 列的值必须相等。

等值连接特点:

1. 多表等值连接的结果为多表的交集部分;
2. n表连接，至少需要n-1个连接条件;
3. 多表不分主次，没有顺序要求;
4. 一般为表起别名，提高阅读性和性能;
5. 可以搭配排序、分组、筛选....等子句使用;

> 注意：
>
> 等值连接也被称为简单连接 (*simple joins*) 或内连接 (*inner joins*)。

等值连接的使用

用等值连接返回记录

```
SELECT employees.employee_id, employees.last_name,
       employees.department_id, departments.department_id,
       departments.location_id
FROM   employees, departments
WHERE  employees.department_id = departments.department_id;
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID	LOCATION_ID
200	Whalen	10	10	1700
201	Hartstein	20	20	1800
202	Fay	20	20	1800
124	Mourgos	50	50	1500
141	Rajs	50	50	1500
142	Davies	50	50	1500
143	Matos	50	50	1500
144	Vargas	50	50	1500

- SELECT 子句指定要返回的列名：
  – employee last name、employee number 和 department number，这些是

    EMPLOYEES 表中的列

  – department number、department name 和 location ID，这些

    是 DEPARTMENTS 表中的列

- FROM 子句指定数据库必须访问的两个表：

  – EMPLOYEES 表

– DEPARTMENTS 表

- WHERE 子句指定表怎样被连接：EMPLOYEES.DEPARTMENT_ID = DEPARTMENTS.DEPARTMENT_ID，因为 DEPARTMENT_ID 列是两个表的同名列，它必须用表名做前缀以避免混淆。

增加搜索条件

使用 AND 操作符附加搜索条件

EMPLOYEES

LAST_NAME	DEPARTMENT_ID
Whalen	10
Hartstein	20
Fay	20
Mourgos	50
Rajs	50
Davies	50
Matos	50
Vargas	50
Hunold	60
Ernst	60

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
20	Marketing
50	Shipping
50	Shipping
50	Shipping
50	Shipping
50	Shipping
60	IT
60	IT

添加查询条件

除连接之外，可能还要求用 WHERE 子句在连接中限制一个或多个表中的行。

限制不能缺的列

限制不明确的列名

- 在多表中使用表前缀限制修饰列名
- 用表前缀改善性能
- 用列别名区别有相同名称，但在不同表中的列

限制不明确的列名

- 需要在 WHERE 子句中用表的名字限制列的名字以避免含糊不清。没有表前缀，DEPARTMENT_ID 列可能来自 DEPARTMENTS 表，也可能来自 EMPLOYEES 表，这种情况下需要添加表前缀来执行查询。
- 如果列名在两个表之间不相同，就不需要限定列。但是，使用表前缀可以改善性能，因为MySQL服务器可以根据表前缀找到对应的列。
- 必须限定不明确的列名也适用于在其它子句中可能引起混淆的那些列，例如 SELECT子句或 ORDER BY 子句。

使用表别名

使用表别名

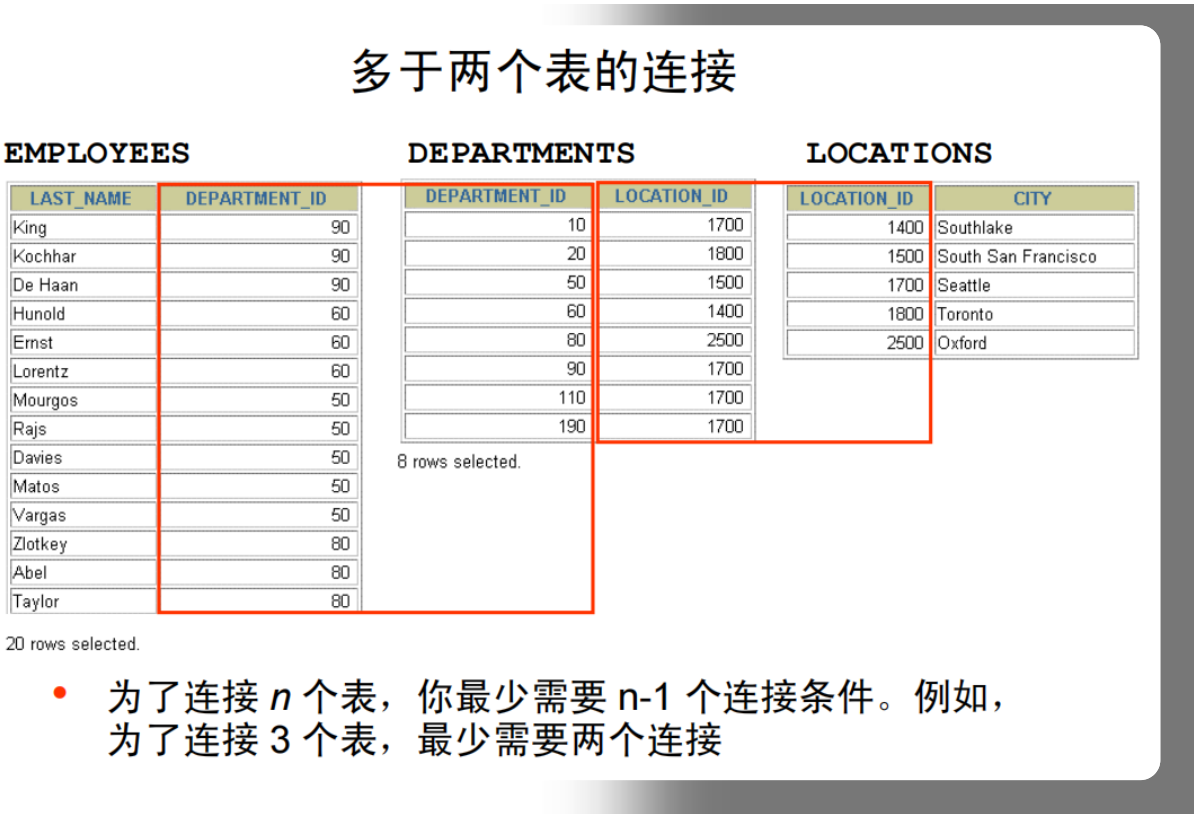
- 使用表别名简化查询
- 使用表别名改善性能

```
SELECT e.employee_id, e.last_name, e.department_id,
       d.department_id, d.location_id
FROM   employees e , departments d
WHERE  e.department_id = d.department_id;
```

表别名定义原则

- 表别名不易过长，短一些更好。
- 表别名应该是有意义的。
- 表别名只对当前的 SELECT 语句有效。

多表连接



示例一：

查询雇员 King 所在的部门名称。

```
select d.department_name from employees
e,departments d   where e.dept_id =
d.department_id and e.last_name = 'King';
```

示例二：

显示每个雇员的 last name、departmentname 和 city。

```
SELECT e.last_name, d.department_name, l.city
FROM employees e, departments d, locations l
WHERE e.department_id = d.department_id
AND d.location_id = l.location_id;
```

实时效果反馈

1.n表连接，至少需要个连接条件。

A n

B 1

C n-1

D n+1

2.等值连接是使用作为连接条件。

A 值的相等性

B 值的相反性

C 值的不同性

D 值的相似性

答案

1=>C　2=>A

非等值连接

非等值连接

EMPLOYEES

LAST_NAME	SALARY
King	24000
Kochhar	17000
De Haan	17000
Hunold	9000
Ernst	6000
Lorentz	4200
Mourgos	5800
Rajs	3500
Davies	3100
Matos	2600
Vargas	2500
Zlotkey	10500
Abel	11000
Taylor	8600

20 rows selected.

JOB_GRADES

GRA	LOWEST_SAL	HIGHEST_SAL
A	1000	2999
B	3000	5999
C	6000	9999
D	10000	14999
E	15000	24999
F	25000	40000

在 EMPLOYEES 表中的工资必须在 JOB_GRADES 表中的最低工资和最高工资之间

非等值连接

一个非等值连接是一种不同于等值操作的连接条件。 EMPLOYEES 表 和JOB_GRADES A 表之间的关系有一个非等值连接例子。在两个表之间的关系是EMPLOYEES 表中的 SALARY 列必须是 JOB_GRADES 表的 LOWEST_SALARY 和HIGHEST_SALARY 列之间的值。使用不同于等于 (=) 的操作符获得关系。

用非等值连接返回记录

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e, job_grades j
WHERE  e.salary
       BETWEEN j.lowest_sal AND j.highest_sal;
```

LAST_NAME	SALARY	GRA
Matos	2600	A
Vargas	2500	A
Lorentz	4200	B
Mourgos	5800	B
Rajs	3500	B
Davies	3100	B
Whalen	4400	B
Hunold	9000	C
Ernst	6000	C

示例一：

创建 job_grades 表，包含 lowest_sal ， highest_sal ，grade_level。

```
create table job_grades(lowest_sal int,highest_sal int ,grade_level varchar(30));
```

示例二：

插入数据

1000 2999 A
2000 4999 B
5000 7999 C
8000 12000 D

```
insert into job_grades values(1000,2999,'A');
insert into job_grades values(2000,4999,'B');
insert into job_grades values(5000,7999,'C');
insert into job_grades values(8000,12000,'D');
```

示例三：

查询所有雇员的薪水级别。

```
select e.last_name,j.grade_level from employees e ,job_grades j where e.salary between j.lowest_sal and j.highest_sal;
```

实时效果反馈

1.如下说法错误的是。

A 一个非等值连接是一种使用等值作为连接条件。

B 一个非等值连接是一种使用大于作为连接条件。

C 一个非等值连接是一种使用小于作为连接条件。

D 一个非等值连接是一种使用不等于作为连接条件。

答案

1=>A

自连接

自连接

EMPLOYEES （工人）

EMPLOYEE_ID	LAST_NAME	MANAGER_ID
100	King	
101	Kochhar	100
102	De Haan	100
103	Hunold	102
104	Ernst	103
107	Lorentz	103
124	Mourgos	100

EMPLOYEES （经理）

EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
107	Lorentz
124	Mourgos



在 WORKER 表中的 MANAGER_ID
等于 MANAGER 表中的 EMPLOYEE_ID

自连接

连接一个表到它自己。有时需要连接一个表到它自己。为了找到每个雇员的经理的名字，则需要连接EMPLOYEES 表到它自己，或执行一个自连接。

连接一个表到它本身

```
SELECT worker.last_name , manager.last_name

FROM   employees worker, employees manager
WHERE  worker.manager_id = manager.employee_id ;
```

图片中的例子连接 EMPLOYEES 表到它自己。为了在 FROM 子句中模拟两个表，对于相同的表 EMPLOYEES，用两个别名，分别为 worker 和 manager。在该例中，WHERE 子句包含的连接意味着“一个工人的经理号匹配该经理的雇员号”。

示例一：

查询每个雇员的经理的名字以及雇员的名字，雇员名字列别名为W，经理列别名为M。

```
SELECT
worker.LAST_NAME W,manager.LAST_NAME M
from employees worker,employees manager
where worker.MANAGER_ID =
manager.EMPLOYEE_ID;
```

示例二：

查询Fox的经理是谁？显示他的名字。

```
SELECT
worker.LAST_NAME,manager.LAST_NAME
from employees worker,employees manager
where worker.MANAGER_ID = manager.EMPLOYEE_ID
AND
worker.LAST_NAME = 'Fox';
```

实时效果反馈

1.如下描述自连接正确的是。

A 一个表不允许自己连接自己

B 连接一个表到它自己就是自连接

C 自连接不能有连接条件

D 自连接中不能使用where子句

答案

1=>B

SQL99标准中的查询

MySQL5.7 支持部分的SQL99 标准。

SQL99中的交叉连接(CROSS JOIN)



示例:

使用交叉连接查询 employees 表与 departments 表。

```
select * from employees cross join
departments;
```

实时效果反馈

1.交叉连接(CROSS JOIN)是SQL的标准。

A 1986

B 1992

C 1999

D 2016

答案

1=>C

SQL99中的自然连接(NATURAL JOIN)

创建自然连接

- **NATURAL JOIN** 子句基于两个表之间有相同名字的所有列
- 它从两个表中选择在所有的匹配列中有相等值的行
- 如果有相同名字的列的数据类型不同，返回一个错误

自然连接

连接只能发生在两个表中有相同名字和数据类型的列上。如果列有相同的名字，但

数据类型不同，NATURAL JOIN 语法会引起错误。

自然连接查询

用自然连接返回记录

```
SELECT department_id, department_name,
       location_id, city
FROM   departments
NATURAL JOIN locations ;
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	CITY
60	IT	1400	Southlake
50	Shipping	1500	South San Francisco
10	Administration	1700	Seattle
90	Executive	1700	Seattle
110	Accounting	1700	Seattle
190	Contracting	1700	Seattle
20	Marketing	1800	Toronto
80	Sales	2500	Oxford

8 rows selected.

在图片例子中，LOCATIONS 表被用 LOCATION_ID 列连接到DEPARTMENT表，这是在两个表中唯一名字相同的列。如果存在其它的同名同类型的列，自然连接会使用等值连接的方式连接他们，连接条件的关系为and。

自然连接也可以被写为等值连接：

```
SELECT  d.department_id, d.department_name,
        d.location_id ,  l.city
FROM
         departments d ,  locations l
WHERE
        d.location_id  =  l.location_id;
```

示例：

使用自然连接查询所有有部门的雇员的名字以及部门名称。

```
select e.last_name,d.department_name from
employees e natural join departments d;
```

实时效果反馈

1.自然连接中，如有多个类型与名称相同的列，在连接时连接条件的关系为。

A OR

B IN

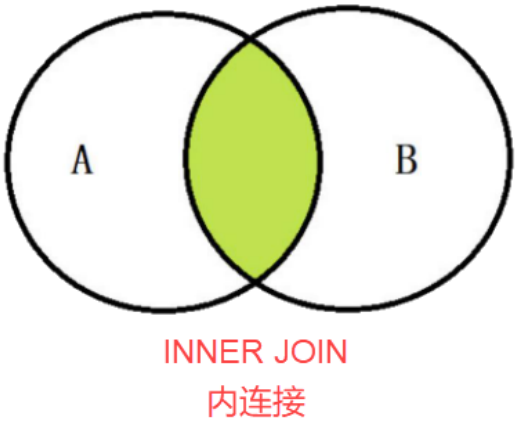
C NOT

D AND

答案

1=>D

SQL99中的内连接(INNER JOIN)





语法：

- SELECT 查询列表;
- FROM 表1 别名;
- INNER JOIN 连接表(INNER关键字可省略);
- ON 连接条件;

用ON子句指定连接条件

用 ON 子句返回记录

```
SELECT e.employee_id, e.last_name, e.department_id,
       d.department_id, d.location_id
FROM   employees e JOIN departments d
ON       e.department_id = d.department_id ;
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID	LOCATION_ID
200	Whalen	10	10	1700
201	Hartstein	20	20	1800
202	Fay	20	20	1800
124	Mourgos	50	50	1500
141	Rajs	50	50	1500
142	Davies	50	50	1500
143	Matos	50	50	1500

用ON子句指定更多的连接条件

用 ON 子句创建三向连接

```
SELECT employee_id, city, department_name
FROM   employees e
JOIN   departments d
ON     d.department_id = e.department_id
JOIN   locations l
ON     d.location_id = l.location_id;
```

EMPLOYEE_ID	CITY	DEPARTMENT_NAME
103	Southlake	IT
104	Southlake	IT
107	Southlake	IT
124	South San Francisco	Shipping
141	South San Francisco	Shipping
142	South San Francisco	Shipping
143	South San Francisco	Shipping
144	South San Francisco	Shipping

示例:

查询雇员名字为 Fox 的雇员 ID， 薪水与部门名称。

```
select
e.employee_id,e.salary,d.department_name from
employees e inner JOIN departments d on
e.department_id = d.department_id where
e.last_name = 'Fox';
```

实时效果反馈

1.在内连接中使用指定连接的表，用指定连接条件。

A INNER JOIN ON

B BETWEEN ON

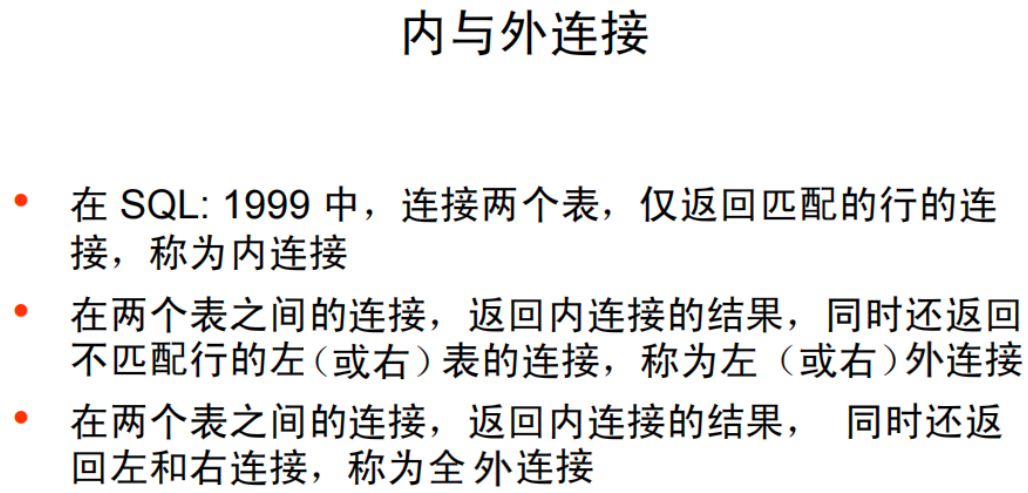
C INNER JOIN IN

D BETWEEN AND

答案

1=>A

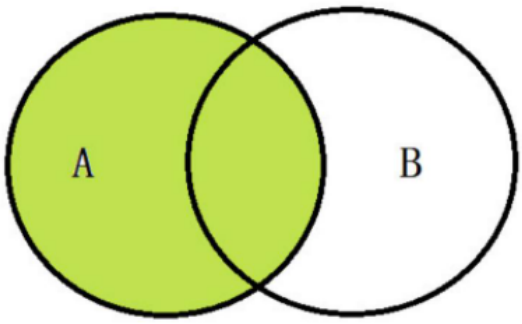
外连接查询(OUTER JOIN)



孤儿数据(Orphan Data)

孤儿数据是指被连接的列的值为空的数据。

左外连接(LEFT OUTER JOIN)



LEFT OUTER JOIN
左外连接



左外连接

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e
LEFT OUTER JOIN departments d
ON    (e.department_id = d.department_id) ;
```

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Whalen	10	Administration
Fay	20	Marketing
Hartstein	20	Marketing
De Haan	90	Executive
Kochhar	90	Executive
King	90	Executive
Gietz	110	Accounting
Higgins	110	Accounting
Grant		

20 rows selected.

左外连接

左边的表 (EMPLOYEES) 中即使没有与 DEPARTMENTS 表中匹配的行，该查询

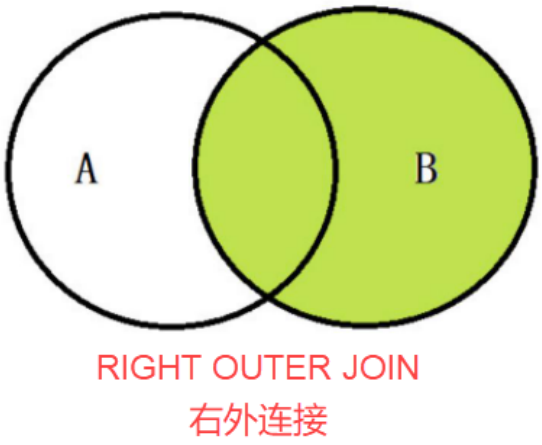
也会取回 EMPLOYEES 表中所有的行。

示例：

查询所有雇员的名字以及他们的部门名称，包含那些没有部门的雇员。

```sql
select e.last_name,d.department_name from
employees e LEFT OUTER JOIN departments d on
e.dept_id = d.department_id;
```

右外连接(RIGTH OUTER JOIN)





右外连接

```sql
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e
RIGHT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
King	90	Executive
Kochhar	90	Executive
Whalen	10	Administration
Hartstein	20	Marketing
Fay	20	Marketing
Higgins	110	Accounting
Gietz	110	Accounting
		Contracting

20 rows selected.

右外连接

右边的表 (DEPARTMENTS) 中即使没有与 EMPLOYEES 表中匹配的行，该查询

也会取回 DEPARTMENTS 表中所有的行。

示例：

查询所有雇员的名字以及他们的部门名称，包含那些没有雇员的部门。

```
select  e.last_name,d.department_name  from employees  e RIGHT  OUTER JOIN  departments d on e.DEPARTMENT_ID = d.department_id;
```

实时效果反馈

1.如下说法正确的是。

A 左外连接会显示左表中的孤儿数据。

B 左外连接会显示右表中的孤儿数据。

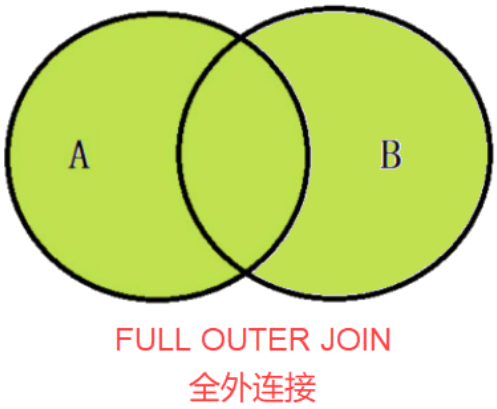
C 左外连接会显示左右表中的孤儿数据。

D 左外连接不会显示表中的孤儿数据。

答案

1=>A

全外连接(FULL OUTER JOIN)



FULL OUTER JOIN
全外连接



注意： MySQL 中不支持 FULL OUTER JOIN 连接

可以使用 union 实现全完连接。

- UNION: 可以将两个查询结果集合并，返回的行都是唯一的，如同对整个结果集合使用了 DISTINCT。
- UNION ALL: 只是简单的将两个结果合并后就返回。这样，如果返回的两个结果集中有重复的数据， 那么返回的结果集就会包含重复的数据了。

语法结构

```
(SELECT  投影列 FROM 表名 LEFT OUTER JOIN  表名 ON  连接条件)
UNION
(SELECT  投影列 FROM 表名 RIGHT OUTER JOIN  表名 ON  连接条件)
```

示例：

查询所有雇员的名字以及他们的部门名称，包含那些没有雇员的部门以及没有部门的雇 员。

```
(select e.last_name,d.department_name from
employees e LEFT OUTER JOIN departments d  on
e.department_id =  d.department_id)
UNION
(select  e1.last_name,d1.department_name
 from employees e1 RIGHT OUTER JOIN
departments d1 on d1.department_id =
e1.department_id)
```

实时效果反馈

1.UNION的作用是：。

A 合并结果集并不剔除重复数据

B 合并结果集并剔除重复数据

C 剔除重复数据

D 指定连接条件

答案

1=>B

练习

1.写一个查询显示所有雇员的 last name、department id、and department name。

```
SELECT
e.LAST_NAME,e.DEPARTMENT_ID,d.DEPARTMENT_NAME
FROM employees e,departments d
WHERE e.DEPARTMENT_ID = d.DEPARTMENT_ID;
```

2.创建一个在部门 80 中的所有工作岗位的唯一列表，在输出中包括部门的地点。

```
SELECT DISTINCT
e.JOB_ID,d.LOCATION_ID
FROM employees e,departments d
WHERE e.DEPARTMENT_ID = d.DEPARTMENT_ID
AND e.DEPARTMENT_ID = 80;
```

3.写一个查询显示所有有佣金的雇员的 last name、department name、location ID 和城

市

```
SELECT
e.LAST_NAME,d.DEPARTMENT_NAME,d.LOCATION_ID,l
.CITY
FROM employees e,departments d,locations l
WHERE e.DEPARTMENT_ID = d.DEPARTMENT_ID
AND d.LOCATION_ID = l.LOCATION_ID
AND e.COMMISSION_PCT is not null;
```

4.显示所有在其 last names 中有一个小写 *a* 的雇员的 last name 和 department

name。

```
select
e.LAST_NAME,d.DEPARTMENT_NAME
FROM employees e,departments d
where e.DEPARTMENT_ID = d.DEPARTMENT_ID
AND e.LAST_NAME like '%a%';
```

5.用sql99的内连接写一个查询显示那些工作在 Toronto 的所有雇员的 last name、job、department number 和 department name。

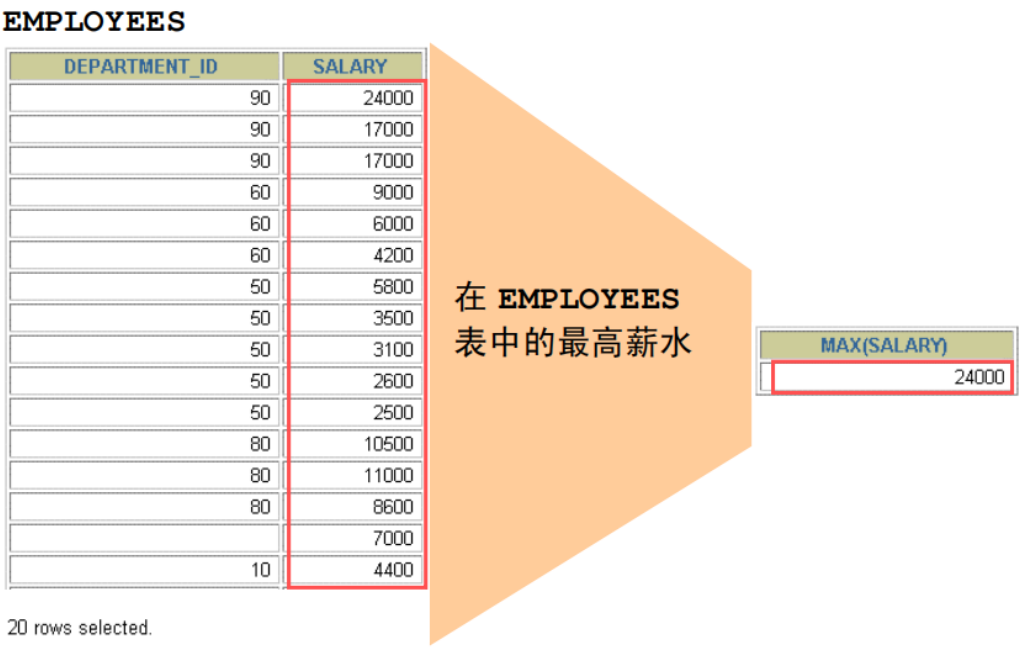
```
SELECT
e.LAST_NAME,e.JOB_ID,e.DEPARTMENT_ID,d.DEPARTMENT_NAME
FROM employees e INNER JOIN departments d
ON e.DEPARTMENT_ID = d.DEPARTMENT_ID INNER JOIN locations l
ON d.LOCATION_ID = l.LOCATION_ID
WHERE l.CITY = 'Toronto';
```

6.显示雇员的 last name 和 employee number 连同他们的经理的 last name 和manager number。列标签分别为 Employee、Emp#、Manager 和 Mgr#

```sql
SELECT
w.LAST_NAME empname,w.DEPARTMENT_ID
empid,m.LAST_NAME mgrname,m.EMPLOYEE_ID mgrid
FROM employees w INNER JOIN employees m
ON w.MANAGER_ID = m.EMPLOYEE_ID;
```

聚合函数

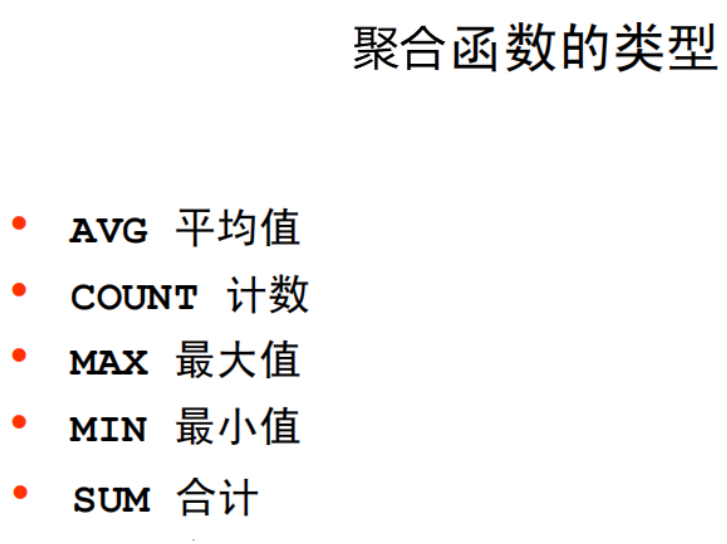
聚合函数介绍



聚合函数

聚合函数也称之为多行函数，组函数或分组函数。聚合函数不象单行函数，聚合函数对行的分组进行操作，对每组给出一个结果。如果在查询中没有指定分组，那么聚合函数则将查询到的结果集视为一组。

聚合函数类型



聚合函数说明：

函数名	描述	实例
AVG(expression)	返回一个表达式的平均值，expression 是一个字段	返回 Products 表中Price 字段的平均值：`SELECT AVG(Price) AS AveragePrice FROM Products;`
COUNT(expression)	返回查询的记录总数，expression 参数是一个字段或者 * 号	返回 Products 表中 products 字段总共有多少条记录：`SELECT COUNT(ProductID) AS NumberOfProducts FROM Products;`
MAX(expression)	返回字段 expression 中的最大值	返回数据表 Products 中字段 Price 的最大值：`SELECT MAX(Price) AS LargestPrice FROM Products;`
MIN(expression)	返回字段 expression 中的最小值	返回数据表 Products 中字段 Price 的最小值：`SELECT MIN(Price) AS MinPrice FROM Products;`
SUM(expression)	返回指定字段的总和	计算 OrderDetails 表中字段 Quantity 的总和：`SELECT SUM(Quantity) AS TotalItemsOrdered FROM OrderDetails;`

聚合函数使用方式

聚合函数的语法

```
SELECT        [column,] group_function(column), ...
FROM          table
[WHERE        condition]
[GROUP BY     column]
[ORDER BY     column];
```

使用聚合函数的原则

- DISTINCT 使得函数只考虑不重复的值;
- 所有聚合函数忽略空值。为了用一个值代替空值，用 IFNULL 或 COALESCE 函数。

实时效果反馈

1.聚合函数对行的进行操作，并给出结果。

A 分组 多个。

B 分组 一个。

C 单条 多个。

D 单条 一个。

答案

1=>B

AVG 和 SUM 函数

AVG(arg)函数

对分组数据做平均值运算。

arg:参数类型只能是数字类型。

SUM(arg)函数

对分组数据求和。

arg:参数类型只能是数字类型。

示例:

计算员工表中工作编号含有REP的工作岗位的平均薪水与薪水总和。

```
SELECT AVG(salary),SUM(salary)
FROM employees
WHERE job_id LIKE '%REP%';
```

实时效果反馈

1.AVG函数是对分组数据求的运算。

A 和

B 差

C 方差

D 平均值

2.SUM函数是对分组数据求的运算。

A 和

B 差

C 方差

D 平均值

答案

1=>D　2=>A

MIN 和 MAX 函数

MIN(arg)函数

求分组中最小数据。

arg:参数类型可以是字符、数字、 日期。

MAX(arg)函数

求分组中最大数据。

arg:参数类型可以是字符、数字、 日期。

示例：

查询员工表中入职时间最短与最长的员工，并显示他们的入职时间。

```
SELECT MIN(hire_date), MAX(hire_date) FROM employees;
```

实时效果反馈

1.MIN函数是对分组数据求的运算。

A 最小值

B 最大值

C 差值

D 平均值

2.MAX函数是对分组数据求的运算。

A 最小值

B 最大值

C 差值

D 平均值

答案

1=>A　2=>B

COUNT 函数

返回分组中的总行数。

COUNT 函数有三种格式：

- COUNT(*)：返回表中满足 SELECT 语句的所有列的行数，包括重复行，包括有空值列
  的行。
- COUNT(expr)：返回在列中的由 *expr* 指定的非空值的数。
- COUNT(DISTINCT expr)：返回在列中的由 *expr* 指定的唯一的非空值的数。

使用 DISTINCT 关键字

- COUNT(DISTINCT expr) 返回对于表达式 *expr* 非空并且值不相同的行数
- 显示 EMPLOYEES 表中不同部门数的值

示例一：

显示员工表中部门编号是80中有佣金的雇员人数。

```
SELECT COUNT(commission_pct) FROM employees
WHERE department_id = 80;
```

示例二：

显示员工表中的部门数。

```sql
SELECT COUNT(DISTINCT department_id) FROM
employees;
```

组函数和 Null 值

在组函数中使用 IFNULL 函数

```sql
SELECT AVG(IFNULL(commission_pct, 0)) FROM
employees;
```

实时效果反馈

1.COUNT函数计算结果集中的。

A 方差

B 标准差

C 均方差

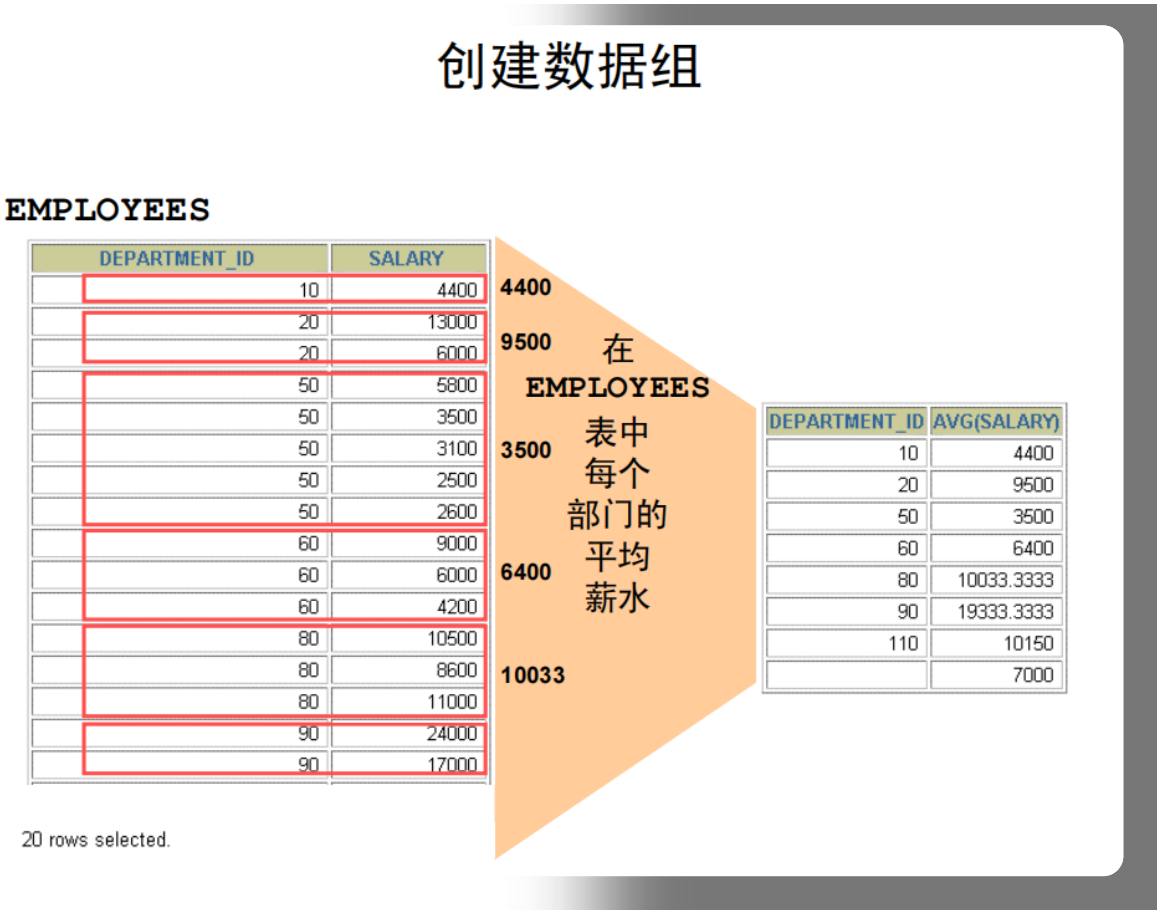
D 总行数

答案

1=>D

数据分组(GROUP BY)

创建数据组



创建数据组

在没有进行数据分组之前，所有聚合函数是将结果集作为一个大的信息组进行处理。但是，有时，则需要将表的信息划分为较小的组，可以用 GROUP BY 子句实现。

GROUP BY 子句语法

创建数据组：GROUP BY 子句语法

```
SELECT      column, group_function(column)
FROM        table
[WHERE      condition]
[GROUP BY   group_by_expression]
[ORDER BY   column];
```

用 GROUP BY 子句划分表中的行到较小的组中

原则

- 使用 WHERE 子句，可以在划分行成组以前过滤行。
- 如果有WHERE子句，那么GROUP BY 子句必须在WHERE的子句后面。
- 在 GROUP BY 子句中必须包含列。

使用 GROUP BY 子句

使用 GROUP BY 子句

在 SELECT 列表中的不在组函数中的所有列必须在 GROUP BY 子句中

```
SELECT    department_id, AVG(salary)
FROM      employees
GROUP BY department_id ;
```

DEPARTMENT_ID	AVG(SALARY)
10	4400
20	9500
50	3500
60	6400
80	10033.3333
90	19333.3333
110	10150
	7000

8 rows selected.

GROUP BY 子句

下面是包含一个 GROUP BY 子句 SELECT 语句的求值过程：

- SELECT 子句指定要返回的列：
- 在 EMPLOYEES 表中的部门号
  – GROUP BY 子句中指定分组的所有薪水的平均值
  – FROM 子句指定数据库必须访问的表：EMPLOYEES 表。
- WHERE 子句指定被返回的行。因为无 WHERE 子句默认情况下所有行被返回。
- GROUP BY 子句指定行怎样被分组。行用部门号分组，所以 AVG 函数被应用于薪水列，以计算每个部门的平均薪水。

示例：

计算每个部门的员工总数。

```
SELECT DEPARTMENT_ID, COUNT(*) FROM employees
GROUP BY DEPARTMENT_ID;
```

实时效果反馈

1.GROUP BY 是创建的子句。

A 数据组。

B 查询。

C 条件。

D 连接。

答案

1=>A

在多列上使用分组

在多列上使用 GROUP BY 子句

```
SELECT   department_id dept_id, job_id, SUM(salary)
FROM     employees
GROUP BY department_id, job_id ;
```

DEPT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
20	MK_MAN	13000
20	MK_REP	6000
50	ST_CLERK	11700
50	ST_MAN	5800
60	IT_PROG	19200
80	SA_MAN	10500
80	SA_REP	19600
90	AD_PRES	24000
90	AD_VP	34000
110	AC_ACCOUNT	8300
110	AC_MGR	12000
	SA_REP	7000

在组中分组

可以列出多个 GROUP BY 列返回组和子组的摘要结果。可以用 GROUP BY子句中的列的顺序确定结果的默认排序顺序。下面是图片中的 SELECT 语句中包含一个 GROUP BY 子句时的求值过程：

- SELECT 子句指定被返回的列：
  - 部门号在 EMPLOYEES 表中
  - Job ID 在 EMPLOYEES 表中
  - 在 GROUP BY 子句中指定的组中所有薪水的合计
- FROM 子句指定数据库必须访问的表：EMPLOYEES 表。
- GROUP BY 子句指定你怎样分组行：

- 首先，用部门号分组行。
- 第二，在部门号的分组中再用 job ID 分组行。

如此 SUM 函数被用于每个部门号分组中的所有 job ID 的 salary 列。

示例：

计算每个部门的不同工作岗位的员工总数。

```
SELECT e.DEPARTMENT_ID, e.JOB_ID,COUNT(*)FROM employees e
GROUP BY e.DEPARTMENT_ID,e.JOB_ID;
```

实时效果反馈

1.如下说法正确的是。

A GROUP BY 子句可以连接多表。

B GROUP BY 子句可以给定多条件。

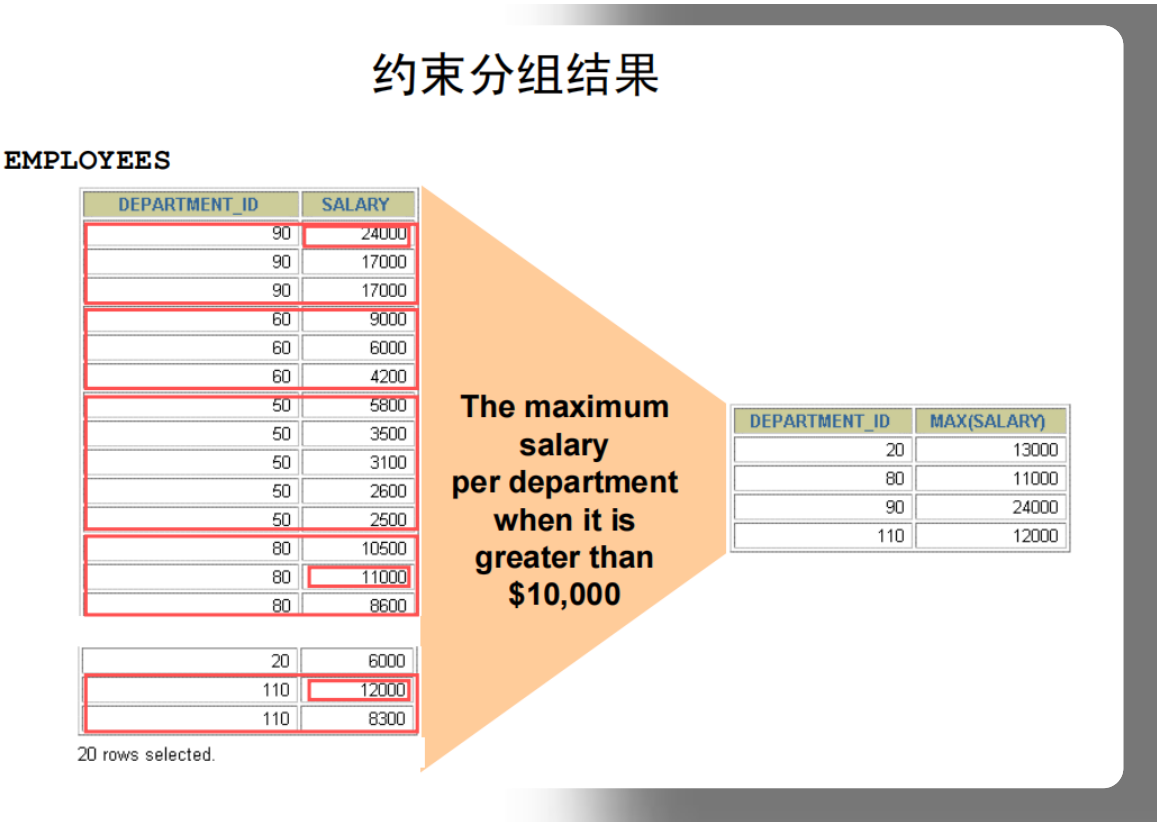
C GROUP BY 子句可以对多列进行分组。

D GROUP BY 子句可以出现在WHERE子句的前面。

答案

1=>C

约束分组结果(HAVING)



HAVING 子句

HAVING 子句是对查询出结果集分组后的结果进行过滤。

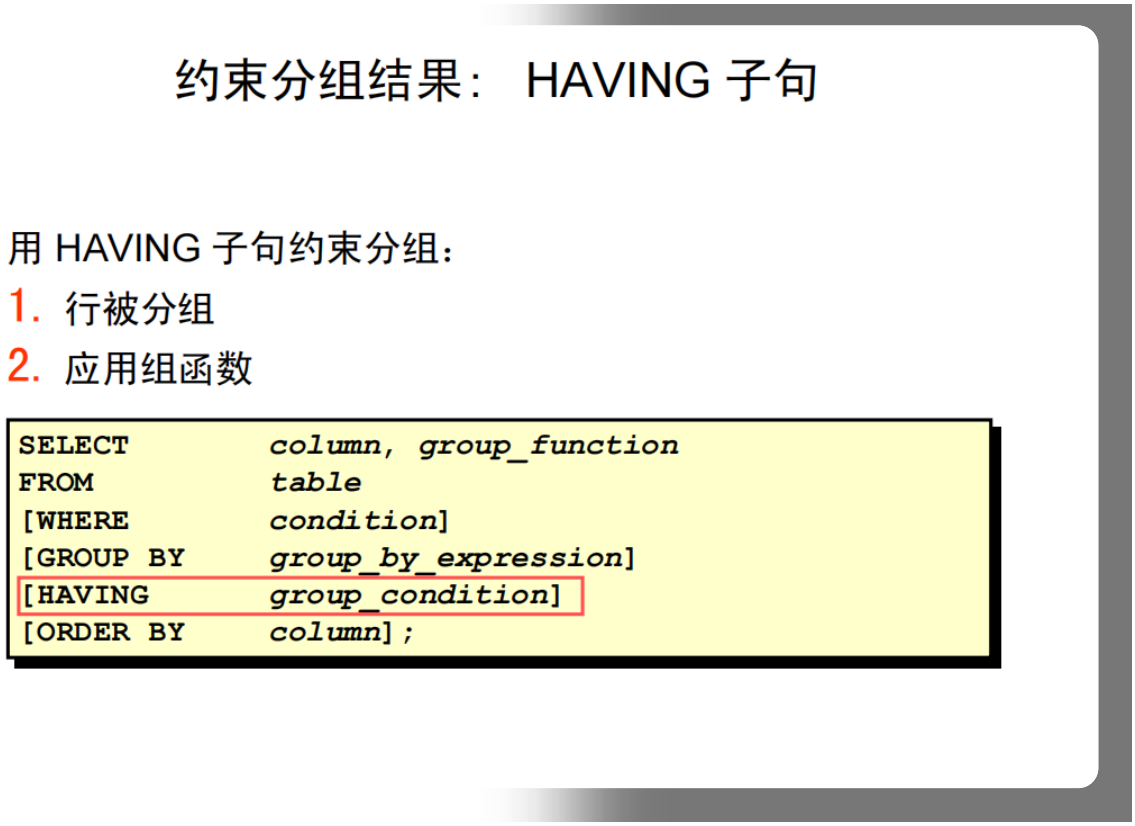
约束分组结果

用 WHERE 子句约束选择的行，用 HAVING 子句约束组。为了找到每个部门中的最高薪水，而且只显示最高薪水大于 $10,000 的那些部门，可以象下面这样做：

1. 用部门号分组，在每个部门中找最大薪水。
2. 返回那些有最高薪水大于 $10,000 的雇员的部门

```
SELECT department_id, MAX(salary) FROM
employees GROUP BY department_id HAVING
MAX(salary)>10000 ;
```

HAVING子句语法



示例：

显示那些合计薪水超过 13,000 的每个工作岗位的合计薪水。排除那些JOB_ID中含有REP的工作岗位，并且用合计月薪排序列表。

```
1 SELECT job_id, SUM(salary) PAYROLL FROM
  employees WHERE job_id NOT LIKE '%REP%'GROUP
  BY job_id HAVING SUM(salary) > 13000 ORDER BY
  SUM(salary);
```

实时效果反馈

1.HAVING 子句是。

A 对数据分组

B 对分组后的结果进行过滤

C 行选择的条件给定

D 给定连接条件

答案

1=>B

练习

1.显示所有雇员的最高、最低、合计和平均薪水，列标签分别为：Max、Min、Sum 和 Avg。四舍五入结果为最近的整数。

```
SELECT
ROUND(MAX(e.SALARY)) max,ROUND(MIN(e.SALARY))
min,ROUND(SUM(e.SALARY)) sum ,
ROUND(AVG(e.SALARY)) avg
FROM employees e;
```

2.写一个查询显示每一工作岗位的人数。

```
SELECT
e.JOB_ID,COUNT(*)
FROM employees e
GROUP BY e.JOB_ID;
```

3.确定经理人数，不需要列出他们，列标签是 Number of Managers。提示：用MANAGER_ID列决定经理号。

```
SELECT
COUNT(DISTINCT e.MANAGER_ID)
FROM employees e;
```

4.写一个查询显示最高和最低薪水之间的差。

```
SELECT
MAX(e.SALARY) - MIN(e.SALARY)
FROM employees e;
```

5.显示经理号和经理付给雇员的最低薪水。排除那些经理未知的人。排除最低薪水小于等于 $6,000 的组。按薪水降序排序输出。

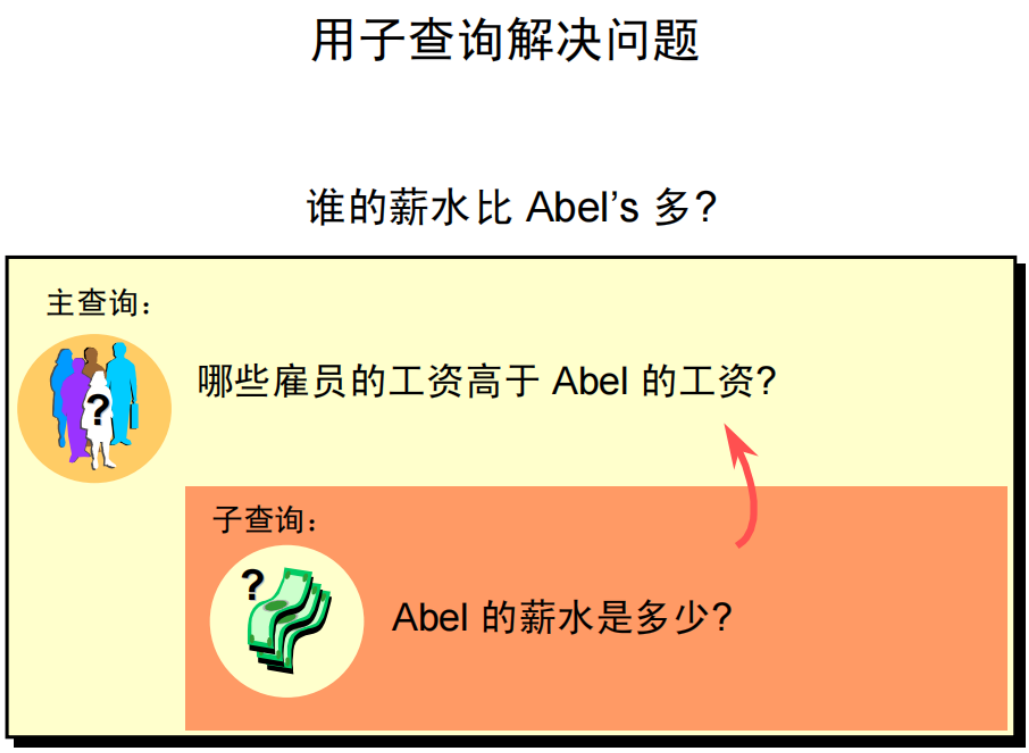
```
SELECT e.MANAGER_ID,MIN(e.SALARY)
FROM employees e
WHERE e.MANAGER_ID is not null
GROUP BY e.MANAGER_ID
HAVING min(e.SALARY) > 6000
ORDER BY min(e.SALARY) desc;
```

6.写一个查询显示每个部门的名字、地点、人数和部门中所有雇员的平均薪水。四舍五入薪水到两位小数。

```
SELECT
d.DEPARTMENT_NAME,d.LOCATION_ID,COUNT(*),ROUND(AVG(e.SALARY))
FROM employees e,departments d
where e.DEPARTMENT_ID = d.DEPARTMENT_ID
GROUP BY d.DEPARTMENT_NAME,d.LOCATION_ID
```

子查询

子查询介绍



用子查询解决问题

假如要写一个查询来找出挣钱比 Abel 的薪水还多的人。为了解决这个问题，需要两个查询：一个找出 Abel 的收入，第二个查询找出收入高于 Abel 的人。可以用组合两个查询的方法解决这个问题。内查询或子查询返回一个值给外查询或主查询。使用一个子查询相当于执行两个连续查询并且用第一个查询的结果作为第二个查询的搜索值。

子查询语法

子查询语法

```
SELECT    select_list
FROM      table
WHERE     expr operator
                    (SELECT       select_list
                    FROM          table);
```

- 子查询（内查询）在主查询之前执行一次
- 子查询的结果被用于主查询（外查询）

子查询

子查询是一个 SELECT 语句，它是嵌在另一个 SELECT 语句中的子句。使用子查询可以用简单的语句构建功能强大的语句。

可以将子查询放在许多的 SQL 子句中，包括：

- WHERE 子句
- HAVING 子句
- FROM 子句

使用子查询

使用子查询

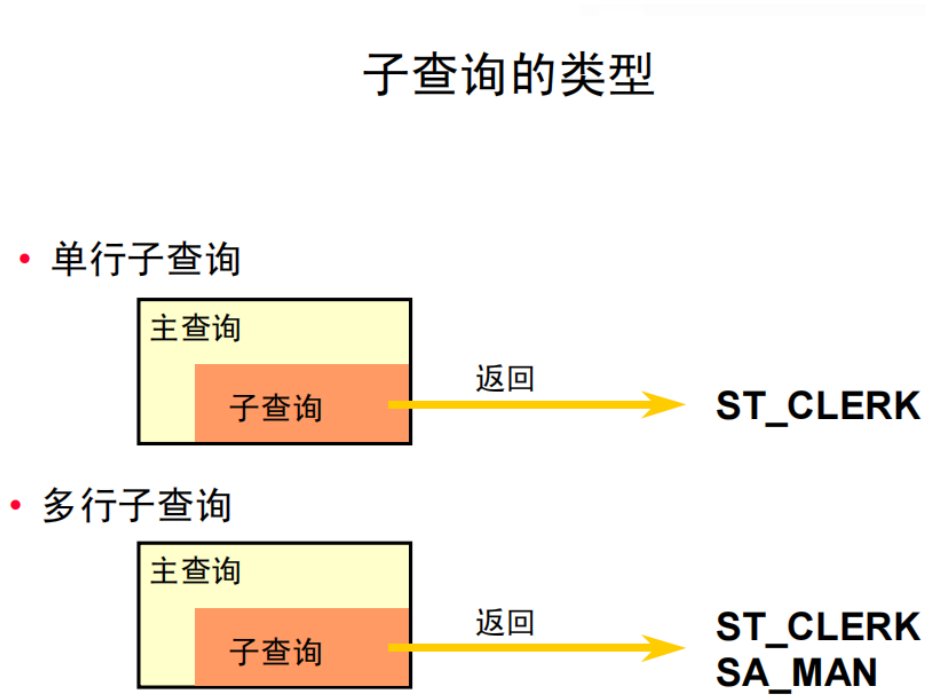
```
SELECT last_name
FROM   employees 11000
WHERE  salary >
               (SELECT salary
                FROM   employees
                WHERE  last_name = 'Abel');
```

LAST_NAME
King
Kochhar
De Haan
Hartstein
Higgins

使用子查询的原则

- 子查询放在圆括号中。
- 将子查询放在比较条件的右边。
- 在单行子查询中用单行运算符，在多行子查询中用多行运算符。

子查询类型



示例：

查询与Fox同一部门的同事，并显示他们的名字与部门ID。

```
select e.LAST_NAME,e.DEPARTMENT_ID FROM
employees e
where e.DEPARTMENT_ID =
(select e1.DEPARTMENT_ID from employees e1
where e1.last_name = 'Fox');
```

实时效果反馈

1.子查询需要放在中。

A 圆括号

B 大括号

C 中括号

D 书名号

答案

1=>A

单行子查询

单行子查询

- 仅返回一行
- 使用单行比较符

运算符	含义
=	等于
>	大于
>=	大于或等于
<	小于
<=	小于或等于
<>	不等于

单行子查询

单行子查询是从内查询返回一行的查询。在该子查询类型中用一个单行操作符。

示例：

查询 Fox的同事，但是不包含他自己。

```
select empl.last_name from employees empl where empl.department_id = (select e.department_id from employees e where e.last_name = 'Fox') and empl.last_name <> 'Fox';
```

实时效果反馈

1.单行子查询是从内查询返回的查询。

A 空数据

B 多行数据

C 多个列数据

D 一行数据

答案

1=>D

多行子查询

多行子查询

- 返回多于一行
- 使用多行比较符

操作	含义
IN	等于列表中的任何成员
ANY	比较子查询返回的每个值
ALL	比较子查询返回的全部值

多行子查询

子查询返回多行被称为多行子查询。对多行子查询要使用多行运算符而不是单行运

算符。

使用ANY运算符

在多行子查询中使用 ANY 运算符

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary < ANY
                (SELECT salary
                 FROM   employees
                 WHERE  job_id = 'IT_PROG')
AND    job_id <> 'IT_PROG';
```

9000, 6000, 4200

ANY 运算符

ANY 运算符比较一个值与一个子查询返回的每一个值。

- < ANY 意思是小于最大值。
- > ANY 意思是大于最小值。
- = ANY 等同于 IN。

使用ALL运算符

在多行子查询中使用 ALL 运算符

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary < ALL
                (SELECT salary
                 FROM   employees
                 WHERE  job_id = 'IT_PROG')
AND    job_id <> 'IT_PROG';
```

9000, 6000, 4200

ALL 运算符比较一个值与子查询返回的每个值。

- < ALL 意思是小于最小值。
- > ALL 意思是大于最大值，

NOT 运算符可以与 IN运算符一起使用。

子查询中的空值

子查询中的空值

```
SELECT emp.last_name
FROM   employees emp
WHERE  emp.employee_id NOT IN
                              (SELECT mgr.manager_id
                               FROM   employees mgr);

no rows selected
```

内查询返回的值含有空值，并因此整个查询无返回行，原因是用大于、小于或不等于比较Null值，都返回null。所以，只要空值可能是子查询结果集的一部分，就不能用 NOT IN 运算符。NOT IN 运算符相当于 <> ALL。

注意，空值作为一个子查询结果集的一部分，如果使用 IN 操作符的话，不是一个问题。IN 操作符相当于 =ANY。

```
1 SELECT emp.last_name FROM employees emp WHERE
  emp.employee_id IN (SELECT mgr.manager_id
  FROM employees mgr);
```

示例:

查找各部门收入为部门最低的那些雇员。显示他们的名字，薪水以及部门 ID。

```
select
em.last_name,em.salary,em.department_id  from
employees  em  where  em.salary in(select
min(e.salary) from employees e group by
e.department_id);
```

实时效果反馈

1.对多行子查询要使用比较符。

A =、>、ALL

B IN、ANY、ALL

C IN、<>、ALL

D IN、ANY、<>

答案

1=>B

练习

1.写一个查询显示与 Zlotkey 在同一部门的雇员的 last name 和 hire date，结果中不包括 Zlotkey。

```
SELECT
e1.LAST_NAME,e1.HIRE_DATE
FROM employees e1
where e1.DEPARTMENT_ID =
(select e.DEPARTMENT_ID FROM employees e
where e.LAST_NAME ='Zlotkey')
AND e1.LAST_NAME <> 'Zlotkey';
```

2.创建一个查询显示所有其薪水高于平均薪水的雇员的雇员号和名字。按薪水的升序排序。

```
SELECT
e1.EMPLOYEE_ID,e1.LAST_NAME
FROM employees e1
WHERE e1.SALARY >
(select AVG(e.SALARY) from employees e)
ORDER BY e1.SALARY;
```

3.写一个查询显示所有工作在有任一雇员的名字中包含一个 *u* 的部门的雇员的雇员号和名字。

```
SELECT
e1.EMPLOYEE_ID,e1.LAST_NAME
FROM employees e1
WHERE e1.DEPARTMENT_ID IN
(SELECT e.DEPARTMENT_ID FROM employees e
WHERE e.LAST_NAME like '%u%')
```

4.显示所有部门地点号 (department location ID) 是 1700 的雇员的 last name、department number 和 job ID。

```
SELECT
e.LAST_NAME,e.DEPARTMENT_ID,e.JOB_ID
FROM employees e
WHERE e.DEPARTMENT_ID IN
(SELECT
d.DEPARTMENT_ID
FROM departments d
WHERE d.LOCATION_ID = 1700)
```

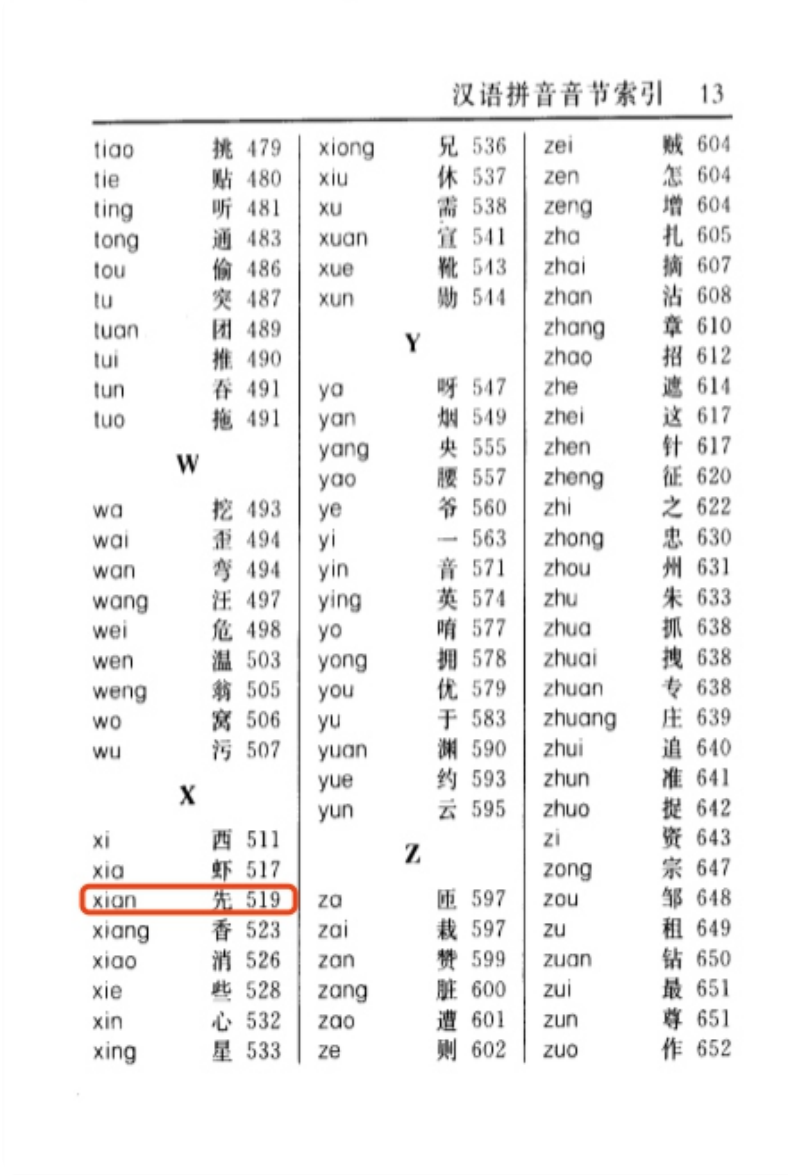
5.显示每个向 King 报告的雇员的名字和薪水。

```
SELECT
e1.LAST_NAME,e1.SALARY
FROM employees e1
where e1.MANAGER_ID IN
(select e.EMPLOYEE_ID from employees e where
e.LAST_NAME ='King')
```

6.显示在 Executive 部门的每个雇员的 department number、last name 和 job ID。

```
SELECT
e.DEPARTMENT_ID,e.LAST_NAME,e.JOB_ID
FROM employees e
WHERE e.DEPARTMENT_ID =
(select d.DEPARTMENT_ID FROM departments d
WHERE d.DEPARTMENT_NAME = 'Executive');
```

MySQL中的索引



汉语拼音音节索引 13

tiao 挑 479
tie 贴 480
ting 听 481
tong 通 483
tou 偷 486
tu 突 487
tuan 团 489
tui 推 490
tun 吞 491
tuo 拖 491

W

wa 挖 493
wai 歪 494
wan 弯 494
wang 汪 497
wei 危 498
wen 温 503
weng 翁 505
wo 窝 506
wu 污 507

X

xi 西 511
xia 虾 517
xian 先 519
xiang 香 523
xiao 消 526
xie 些 528
xin 心 532
xing 星 533
xiong 兄 536
xiu 休 537
xu 需 538
xuan 宣 541
xue 靴 543
xun 勋 544

Y

ya 呀 547
yan 烟 549
yang 央 555
yao 腰 557
ye 爷 560
yi 一 563
yin 音 571
ying 英 574
yo 哟 577
yong 拥 578
you 优 579
yu 于 583
yuan 渊 590
yue 约 593
yun 云 595

Z

za 匝 597
zai 栽 597
zan 赞 599
zang 脏 600
zao 遭 601
ze 则 602
zei 贼 604
zen 怎 604
zeng 增 604
zha 扎 605
zhai 摘 607
zhan 沾 608
zhang 章 610
zhao 招 612
zhe 遮 614
zhei 这 617
zhen 针 617
zheng 征 620
zhi 之 622
zhong 忠 630
zhou 州 631
zhu 朱 633
zhua 抓 638
zhuai 拽 638
zhuan 专 638
zhuang 庄 639
zhui 追 640
zhun 准 641
zhuo 捉 642
zi 资 643
zong 宗 647
zou 邹 648
zu 租 649
zuan 钻 650
zui 最 651
zun 尊 651
zuo 作 652



索引介绍

索引是对数据库表中的一列或多列值进行排序的一种结构，使用索引可以快速访问数据库表中的特定信息。索引是一种特殊的文件，它们包含着对数据表里所有记录的位置信息。更通俗的说，数据库索引好比是一本书前面的目录，能加快数据库的查询速度。MySQL

索引的建立对于MySQL 的高效运行是很重要的，索引可以大大提高MySQL 的检索速度。

索引的作用

索引相当于图书上的目录，可以根据目录上的页码快速找到所需的内容，提高性能（查询速度）。

索引优点：

1. 通过创建唯一性索引，可以保证数据库表中的每一行数据的唯一性；
2. 可以加快数据的检索速度；
3. 可以加速表与表之间的连接；
4. 在使用分组和排序进行检索的时候，可以减少查询中分组和排序的时间；

索引缺点

1. 创建索引和维护索引要耗费时间，这种时间随着数据量的增加而增加；
2. 索引需要占用物理空间，数据量越大，占用空间越大；
3. 会降低表的增删改的效率，因为每次增删改索引都需要进行动态维护；

什么时候需要创建索引

1. 频繁作为查询条件的字段应该创建索引；
2. 查询中排序的字段创建索引将大大提高排序的速度（索引就是排序加快速查找）；
3. 查询中统计或者分组的字段；

什么时候不需要创建索引

1. 频繁更新的字段不适合创建索引，因为每次更新不单单是更新记录，还会更新索引，保存索引文件；
2. where条件里用不到的字段，不创建索引；
3. 表记录太少，不需要创建索引；

4. 经常增删改的表；
5. 数据重复且分布平均的字段，因此为经常查询的和经常排序的字段建立索引。注意某些数据包含大量重复数据，因此他建立索引就没有太大的效果，例如性别字段，只有男女，不适合建立索引；

MySQL中的索引类型

- 普通索引：
  最基本的索引，它没有任何限制。

- 唯一索引：
  索引列的值必须唯一，但允许有空值，如果是组合索引，则列值的组合必须唯一。

- 主键索引：
  特殊的索引，唯一的标识一条记录，不能为空，一般用primary key来约束。

- 联合索引：
  在多个字段上建立索引，能够加速查询到速度。

实时效果反馈

1.索引的作用是为了提高速度。

A 添加数据

B 修改数据

C 更新数据

D 查询数据

答案

1=>D

普通索引

是最基本的索引，它没有任何限制。在创建索引时，可以指定索引长度。length 为可选参数，表示索引的长度，只有字符串类型的字段才能指定索引长度，如果是 BLOB 和 TEXT 类型，必须指定 length。

> 创建索引时需要注意：
>
> 如果指定单列索引长度，length 必须小于这个字段所允许的最大字符个数。

查询索引

```
SHOW INDEX FROM table_name;
```

直接创建索引

```
CREATE INDEX index_name ON
table(column(length));
```

示例：

为 emp3 表中的 name 创建一个索引，索引名为 emp3_name_index;

```
create index emp3_name_index ON emp3(name);
```

修改表添加索引

```
ALTER TABLE table_name ADD INDEX index_name
(column(length));
```

示例:

修改 emp3 表，为 addrees 列添加索引，索引名为 emp3_address_index;

```
alter table emp3 add index
emp3_address_index(address);
```

创建表时指定索引列

```
CREATE TABLE `table` (
COLUMN TYPE ,
PRIMARY KEY (`id`),
INDEX index_name (column(length))
);
```

示例:

创建 emp4 表，包含 emp_id,name,address 列， 同时为 name 列创建索引，索引名为 emp4_name_index。

```
create table emp4(emp_id int primary key auto_increment,name varchar(30),address varchar(50),index emp4_name_index(name));
```

删除索引

```
DROP INDEX indexname ON tablename;
```

示例：

删除 mep3 表中索引名为 emp3_address_index 的索引。

```
drop index emp3_address_index on emp3;
```

实时效果反馈

1.如果指定单列索引长度，length 必须这个字段所允许的最大字符个数。

A 小于

B 大于

C 等于

D 不等于

2.直接创建索引：CREATE index_name ON table(column(length));。

A INDEXS

B TABLE

C COLUMN

D INDEX

答案

1=>A 2=>D

唯一索引

唯一索引与普通索引类似，不同的就是： 索引列的值必须唯一，但允许有空值。

创建唯一索引

```
CREATE UNIQUE INDEX indexName ON table(column(length));
```

示例：

为 emp 表中的 name 创建一个唯一索引，索引名为 emp_name_index。

```
create unique index emp_name_index on
emp(name);
```

修改表添加唯一索引

```
ALTER TABLE table_name ADD UNIQUE indexName
(column(length));
```

示例：

修改 emp 表，为 salary 列添加唯一索引，索引名为 emp_salary_index。

```
alter table emp add unique
emp_salary_index(salary);
```

创建表时指定唯一索引

```
CREATE TABLE `table` (
COLUMN TYPE ,
PRIMARY KEY (`id`),
UNIQUE index_name (column(length))
);
```

示例：

创建 emp5 表，包含 emp_id,name,address 列，同时为 name 列创建唯一索引。索引名为 emp5_name_index。

```
create table emp5(emp_id int primary key
,name varchar(30),address varchar(30),unique
emp5_name_index(name));
```

实时效果反馈

1.唯一索引列的值，有空值。

A 允许不唯一 允许

B 必须唯一 允许

C 必须唯一 不允许

D 允许不唯一 不允许

2.创建唯一索引：CREATEindexName ON table(column(length));

A UNIQUE

B INDEX

C UNIQUE INDEXS

D UNIQUE INDEX

答案

1=>B 2=>D

主键索引

主键索引是一种特殊的唯一索引，一个表只能有一个主键，不允许有空值。一般是在建表的时候同时创建主键索引。

修改表添加主键索引

```
ALTER TABLE  表名 ADD PRIMARY KEY(列名);
```

示例：

修改 emp 表为 employee_id 添加主键索引。

```
alter table emp add primary key(employee_id);
```

创建表时指定主键索引

```
CREATE TABLE `table` (
COLUMN TYPE ,
PRIMARY KEY(column)
);
```

示例：

创建 emp6 表，包含 emp_id,name,address 列，同时为 emp_id 列创建主键索引。

```
create table emp6(employee_id int primary key auto_increment,name varchar(20),address varchar(50));
```

实时效果反馈

1.主键索引列的值，有空值。

A 允许不唯一 允许

B 必须唯一 允许

C 必须唯一 不允许

D 允许不唯一 不允许

答案

1=>C

组合索引

组合索引是指使用多个字段创建的索引，只有在查询条件中使用了创建索引时的第一个字段，索引才会被使用(最左前缀原则)。

最左前缀原则

就是最左优先。

如： 我们使用表中的 name， address， salary 创建组合索引，那么想要组合索引生效， 我们只能使用如下组合：

name/address/salary

name/address

name/

如果使用 addrees/salary 或者是 salary 则索引不会生效。

添加组合索引

```
ALTER TABLE table_name ADD INDEX index_name
(column(length),column(length));
```

示例：

修改 emp6 表，为 name， address 列创建组合索引。

```
alter table emp6 add index
emp6_index_n_a(name,address);
```

创建表时创建组合索引

```
CREATE TABLE `table` (
COLUMN TYPE ,
INDEX index_name
(column(length),column(length))
);
```

示例：

创建 emp7 表，包含 emp_id,name,address 列，同时为 name,address 列创建组合索引。

```
create table emp7(emp_id int primary key auto_increment  ,name varchar(20),address varchar(30),index emp7_index_n_a(name,address));
```

实时效果反馈

1.组合索引是指使用创建的索引。

A 主键列

B 外键列

C 一个列

D 多个列

答案

1=>D

MySQL事务

事务简介

事务是指作为单个逻辑工作单元执行的一系列操作，要么完全地执行，要么完全地不执行。

事务定义(Transaction)

- 事务是一个最小的不可再分的工作单元；通常一个事务对应一个完整的业务(例如银行账户转账业务，该业务就是一个最小的工作单元)
- 一个完整的业务需要批量的DML(insert、update、delete)语句共同联合完成
- 事务只和DML语句有关，或者说DML语句才有事务。这个和业务逻辑有关，业务逻辑不同，DML语句的个数不同

事务四大特征(ACID)

- 原子性(ATOMICITY)
  事务中的操作要么都不做，要么就全做。

- 一致性(CONSISTENCY)
  一个事务应该保护所有定义在数据上的不变的属性(例如完整性约束)。在完成了一个成功的事务时，数据应处于一致的状态。

- 隔离性(ISOLATION)
  一个事务的执行不能被其他事务干扰。

- 持久性(DURABILITY)
  一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。

事务类型

- 显式事务
  需要我们手动的提交或回滚。
  DML 语言中的所有操作都是显示事务操作。

- 隐式事务
  数据库自动提交不需要我们做任何处理，同时也不具备回滚性。
  DDL、DCL 语言都是隐式事务操作

实时效果反馈

1.如下描述事务四大特征正确的是。

A 原子性、可读性、隔离性、持久性

B 可读性、一致性、隔离性、持久性

C 可读性、一致性、可写性、持久性

D 原子性、一致性、隔离性、持久性

答案

1=>D

使用事务

TCL语句	描述
start transaction	事务开启
commit	事物提交
rollback	事物回滚

示例一：

创建account账户表，包含id、卡号、用户名、余额。

```
create table account(
id int primary key auto_increment,
cardnum varchar(20) not null,
username varchar(30) not null,
balance double(10,2)
);
```

示例二：

向account表中插入两条数据。

```
insert into account(cardnum,username,balance)
VALUES('123456789','张三',2000);
insert into account(cardnum,username,balance)
VALUES('987654321','李四',2000);
```

示例三：

在一个事务中完成转账业务。

```
START TRANSACTION
update account set balance = balance-200
where cardnum = '123456789';
update account set balance = balance+200
where cardnum = '987654321';
select * from account;
-- 当我们关闭数据库重新打开后，张三和李四的账户余额并没发生任何变化。
```

```
-- 这是因为当我们使用“START TRANSACTION”开启一个事务后，该事务的提交方式不再是自动的，
-- 而是需要手动提交，而在这里，我们并没有使用事务提交语句COMMIT，
-- 所以对account表中数据的修改并没有永久的保存到数据库中，也就是说我们的转账事务并没有执行成功

-- 提交转账事务
commit;

-- 事务的回滚让数据库恢复到了执行事务操作前的状态。
-- 需要注意的是事务的回滚必须在事务提交之前，因为事务一旦提交就不能再进行回滚操作。
rollback;
```

实时效果反馈

1.事务提交语句。

A commit

B confirm

C transaction commit

D transaction confirm

2.事务回滚语句。

A rollback

B revoke

C transaction rollback

D transaction revoke

答案

1=>A 2=>A

事务的并发问题

脏读（读取未提交数据）

指一个事务读取了另外一个事务未提交的数据。

A事务读取B事务尚未提交的数据，此时如果B事务发生错误并执行回滚操作，那么A事务读取到的数据就是脏数据。

时间顺序	转账事务	取款事务
1		开始事务
2	开始事务	
3		查询账户余额为2000元
4		取款1000元，余额被更改为1000元
5	查询账户余额为1000元（产生脏读）	
6		取款操作发生未知错误，事务回滚，余额变更为2000元
7	转入2000元，余额被更改为3000元（脏读的1000+2000）	
8	提交事务	
备注	按照正确逻辑，此时账户余额应该为4000元	

不可重复读（前后多次读取，数据内容不一致）

在一个事务内读取表中的某一行数据，多次读取结果不同。

事务A在执行读取操作，由整个事务A比较大，前后读取同一条数据需要经历很长的时间 。而在事务A第一次读取数据，比如此时读取了小明的年龄为20岁，事务B执行更改操作，将小明的年龄更改为30岁，此时事务A第二次读取到小明的年龄时，发现其年龄是30岁，和之前的数据不一样了，也就是数据不重复了，系统不可以读取到重复的数据，成为不可重复读。

时间顺序	事务A	事务B
1	开始事务	
2	第一次查询，小明的年龄为20岁	
3		开始事务
4	其他操作	
5		更改小明的年龄为30岁
6		提交事务
7	第二次查询，小明的年龄为30岁	
备注	按照正确逻辑，事务A前后两次读取到的数据应该一致	

幻读（前后多次读取，数据总量不一致）

是指在一个事务内读取到了别的事务插入的数据，导致前后读取数量总量不一致。

事务A在执行读取操作，需要两次统计数据的总量，前一次查询数据总量后，此时事务B执行了新增数据的操作并提交后，这个时候事务A读取的数据总量和之前统计的不一样，就像产生了幻觉一样，平白无故的多了几条数据，成为幻读。

时间顺序	事务A	事务B
1	开始事务	
2	第一次查询，数据总量为100条	
3		开始事务
4	其他操作	
5		新增100条数据
6		提交事务
7	第二次查询，数据总量为200条	
备注	按照正确逻辑，事务A前后两次读取到的数据总量应该一致	

事务的隔离级别

事务的隔离级别用于决定如何控制并发用户读写数据的操作。数据库是允许多用户并发访问的，如果多个用户同时开启事务并对同一数据进行读写操作的话，有可能会出现脏读、不可重复读和幻读问题，所以MySQL中提供了四种隔离级别来解决上述问题。

事务的隔离级别从低到高依次为：

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

隔离级别越低，越能支持高并发的数据库操作。

隔离级别/读异常	脏读	不可重复读	幻读
SERIALIZABLE	Y	Y	Y
REPEATABLE READ	Y	Y	N
READ COMMITTED	Y	N	N
READ UNCOMMITTED	N	N	N

提示：Y表示解决 N表示未解决

查看MySQL默认事务隔离级别

```
SELECT @@transaction_isolation;
```

设置事务隔离级别

对当前session有效。

```
set session transaction isolation level read uncommitted;
set session transaction isolation level read committed;
set session transaction isolation level repeatable read;
set session transaction isolation level serializable;
```

实时效果反馈

1.MySQL默认事务隔离级别是。

A uncommitted

B committed

C repeatable read

D serializable

答案

1=>C

MySQL的用户管理

MySQL 是一个多用户的数据库系统，按权限，用户可以分为两种：root 用户，超级管理员，和由 root 用户创建的普通用户。

用户管理

创建用户

```
CREATE USER username IDENTIFIED BY
'password';
```

查看用户

```
SELECT USER,HOST FROM mysql.user;
```

示例：

创建一个 u_sxt 的用户，并查看创建是否成功。

```
create user u_sxt IDENTIFIED by 'sxt';
select user,host from mysql.user;
```

权限管理

新用户创建完后是无法登陆的，需要分配权限。
GRANT 权限 ON 数据库.表 TO 用户名@登录主机 IDENTIFIED BY "密码"

登陆主机：

字段	含义
%	匹配所有主机
localhost	localhost 不会被解析成 IP 地址，直接通过 UNIXsocket 连接
127.0.0.1	会通过 TCP/IP 协议连接，并且只能在本机访问
:: 1	::1 就是兼容支持 ipv6 的，表示同 ipv4 的 127.0.0. 1

权限列表

权 限	作用范围	作 用
all [privileges]	服务器	所有权限
select	表、列	选择行
insert	表、列	插入行
update	表、列	更新行
delete	表	删除行
create	数据库、表、索引	创建
drop	数据库、表、视图	删除
reload	服务器	允许使用flush语句
shutdown	服务器	关闭服务
process	服务器	查看线程信息
file	服务器	文件操作
grant option	数据库、表、存储过程	授权
references	数据库、表	外键约束的父表
index	表	创建/删除索引
alter	表	修改表结构
show databases	服务器	查看数据库名称
super	服务器	超级权限
create temporary tables	表	创建临时表
lock tables	数据库	锁表
execute	存储过程	执行
replication client	服务器	允许查看主/从/二进制日志状态
replication slave	服务器	主从复制
create view	视图	创建视图
show view	视图	查看视图
create routine	存储过程	创建存储过程
alter routine	存储过程	修改/删除存储过程
create user	服务器	创建用户
event	数据库	创建/更改/删除/查看事件
trigger	表	触发器
create tablespace	服务器	创建/更改/删除表空间/日志文件

权 限	作用范围	作 用
proxy	服务器	代理成为其它用户
usage	服务器	没有权限

```
GRANT ALL PRIVILEGES ON *.* TO
'username'@'localhost' IDENTIFIED BY
'password'
```

示例：

为 u_sxt 用户分配只能查询 bjsxt 库中的 emp 表，并且只能在本机登陆的权限。

```
grant select ON bjsxt.emp to
'u_sxt'@'localhost' IDENTIFIED by 'sxt';
```

刷新权限

每当调整权限后，通常需要执行以下语句刷新权限。

```
FLUSH PRIVILEGES;
```

删除用户

```
DROP USER username@localhost;
```

示例：

删除 u_sxt 用户。

```
drop user 'u_sxt'@'localhost';
```

实时效果反馈

1.创建用户语句：CREATE username 'password';

A USERS IDENTIFIED BY

B USER IDENTIFIED

C USER IDENTIFIED BY

D USER BY

2.删除用户语句： username@localhost;

A DELETE USER

B DELETE USERS

C DROP USERS

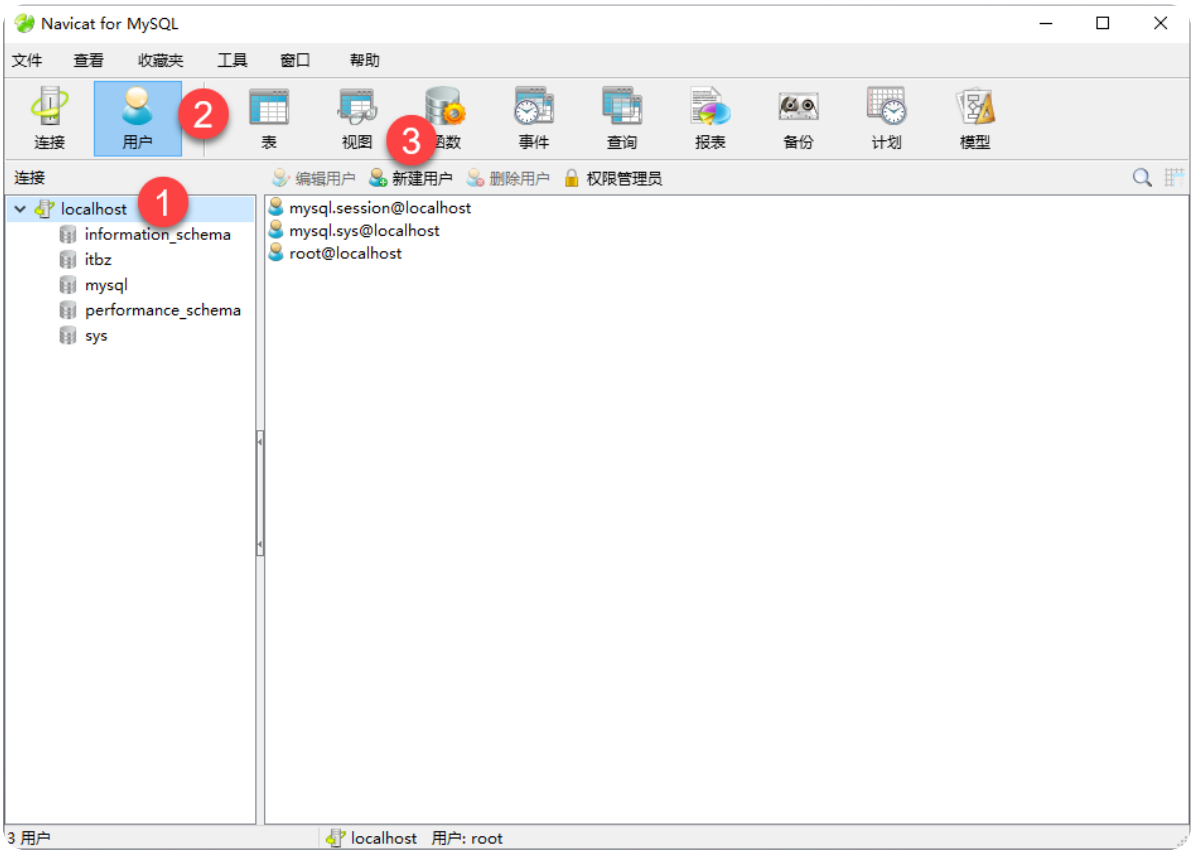
D DROP USER

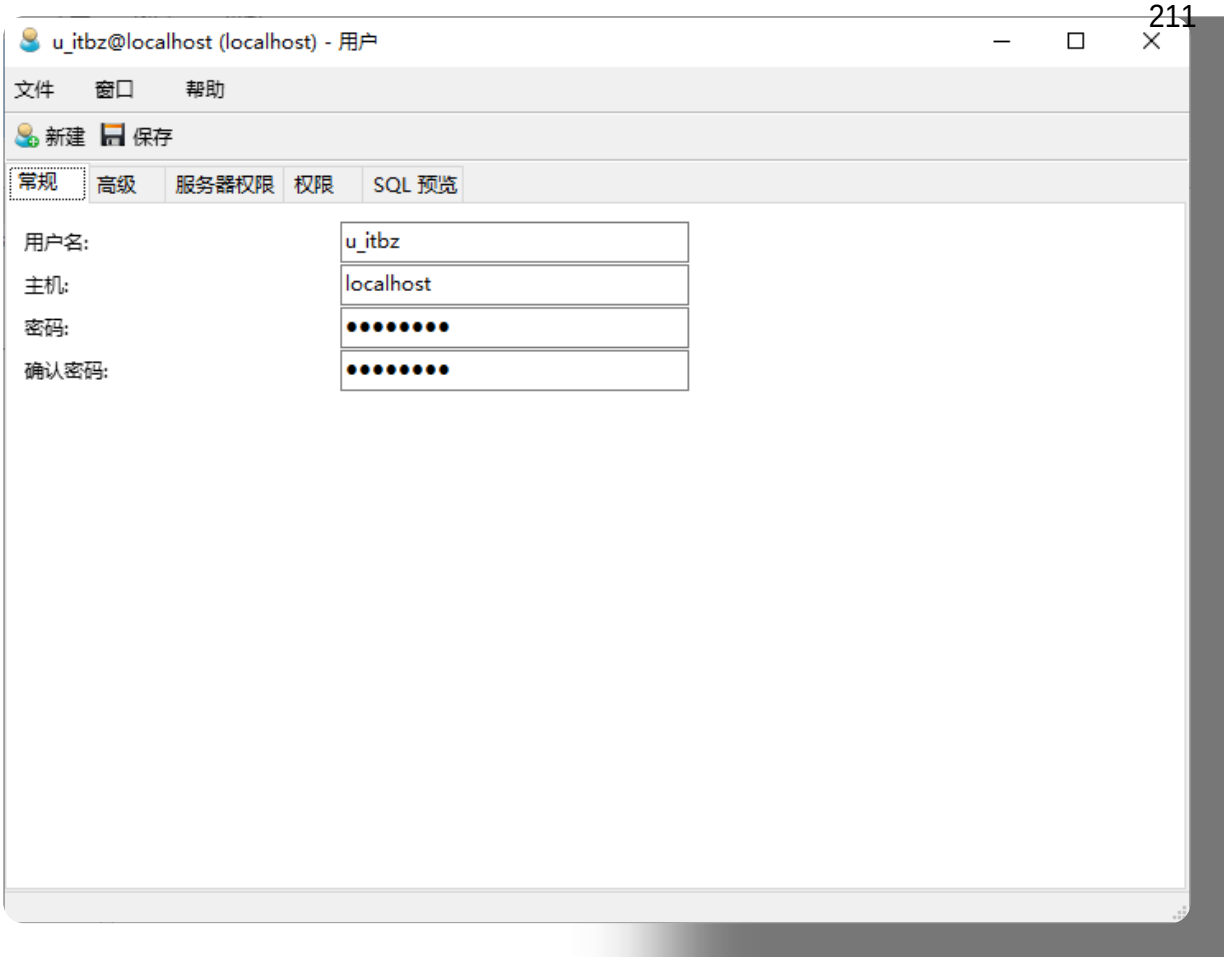
答案

1=>C 2=>D

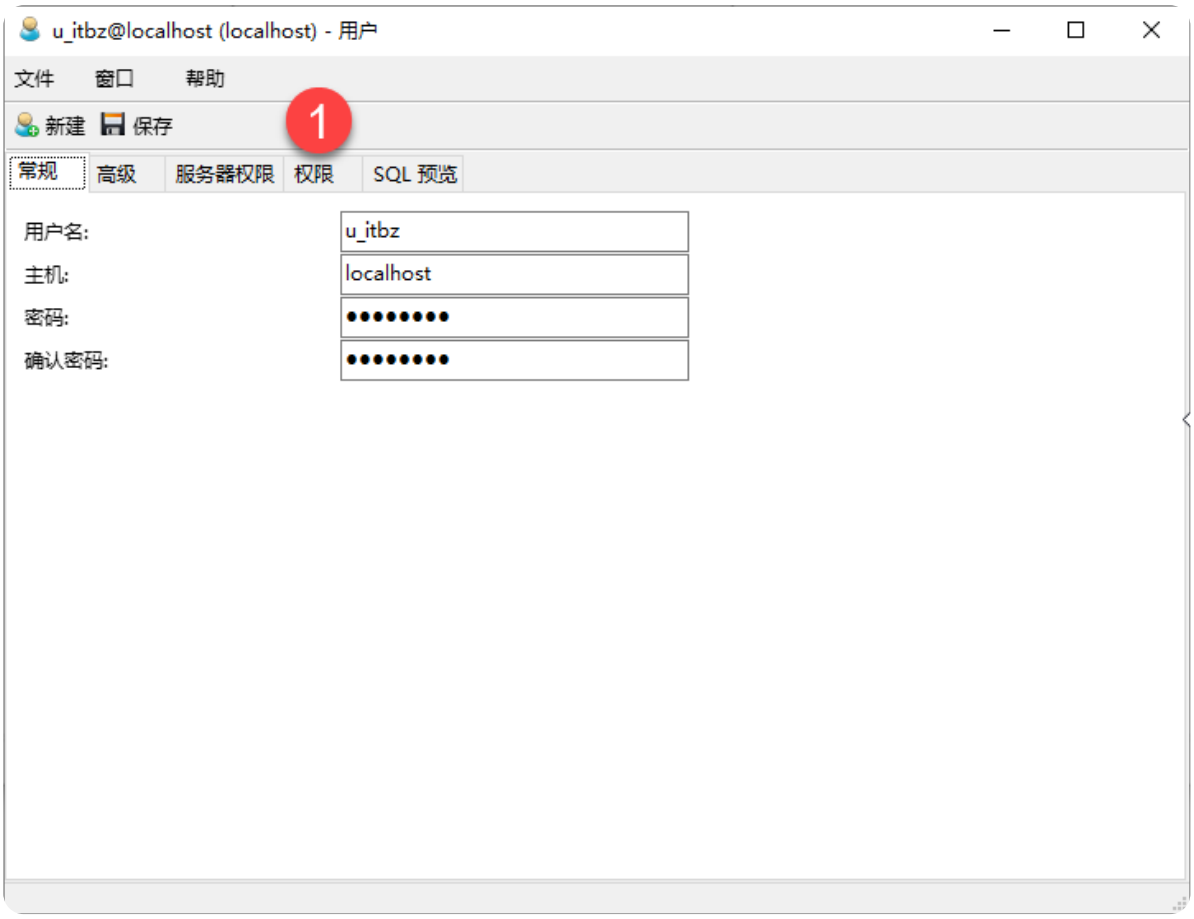
Navicat工具

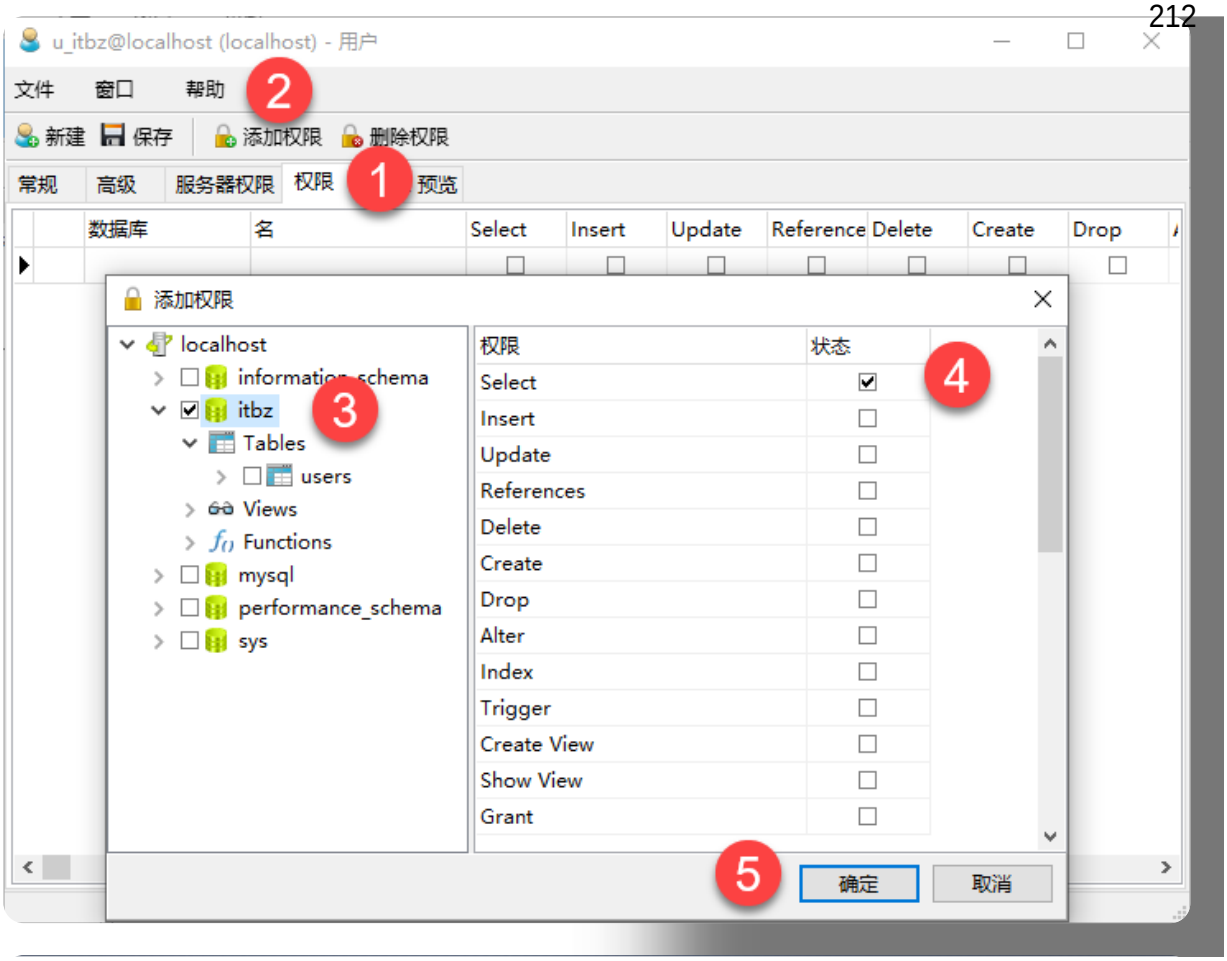
创建用户

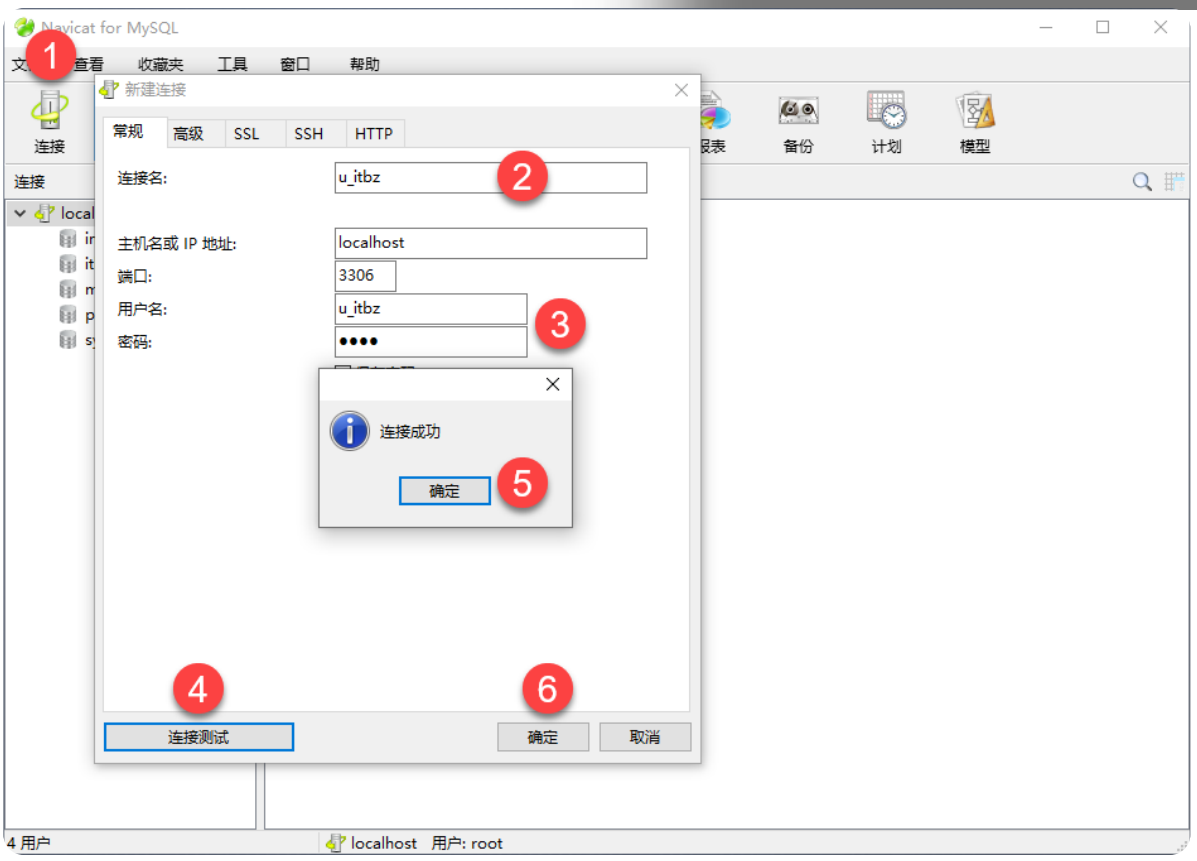


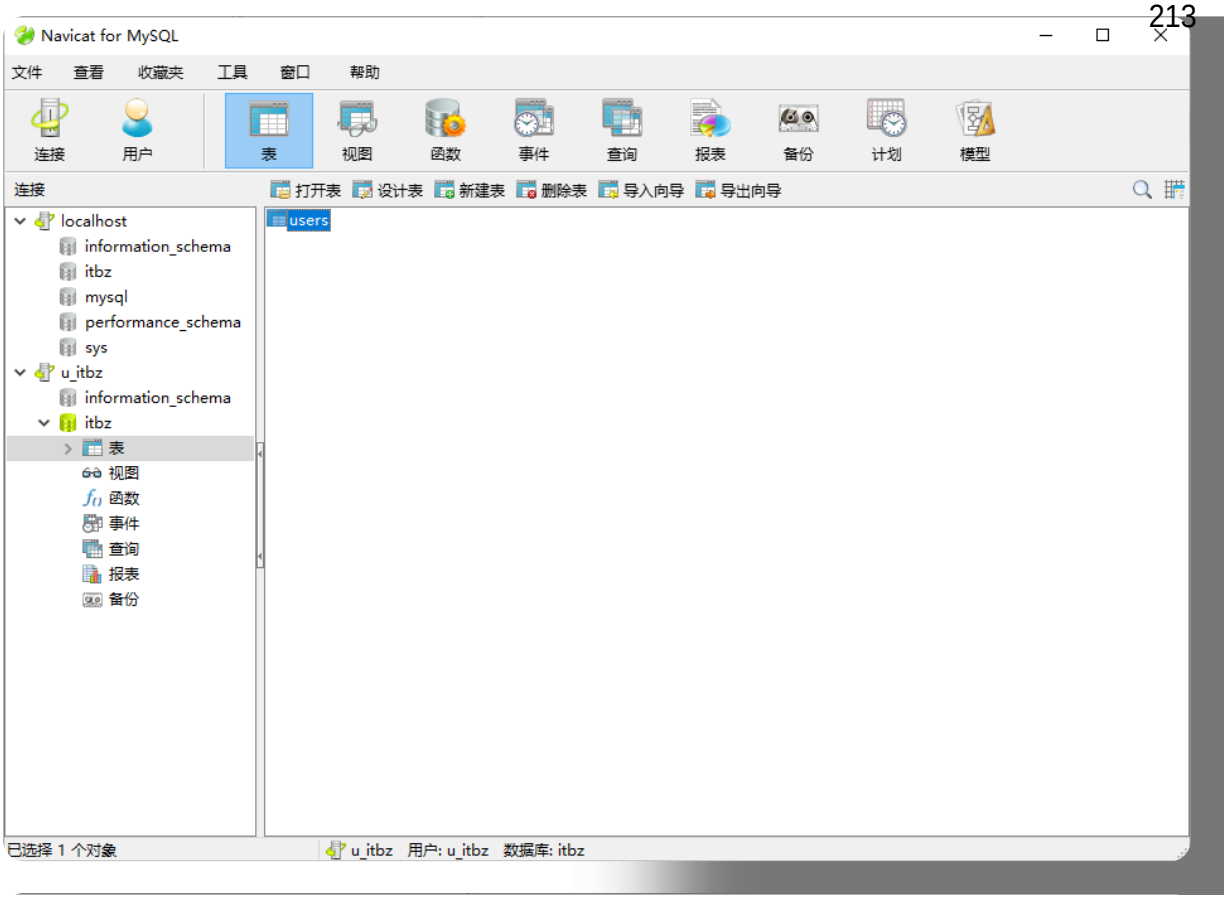

u_itbz@localhost (localhost) - 用户
文件
窗口
帮助
新建
保存
常规
高级
服务器权限
权限
SQL 预览
用户名:
u_itbz
主机:
localhost
密码:
确认密码:

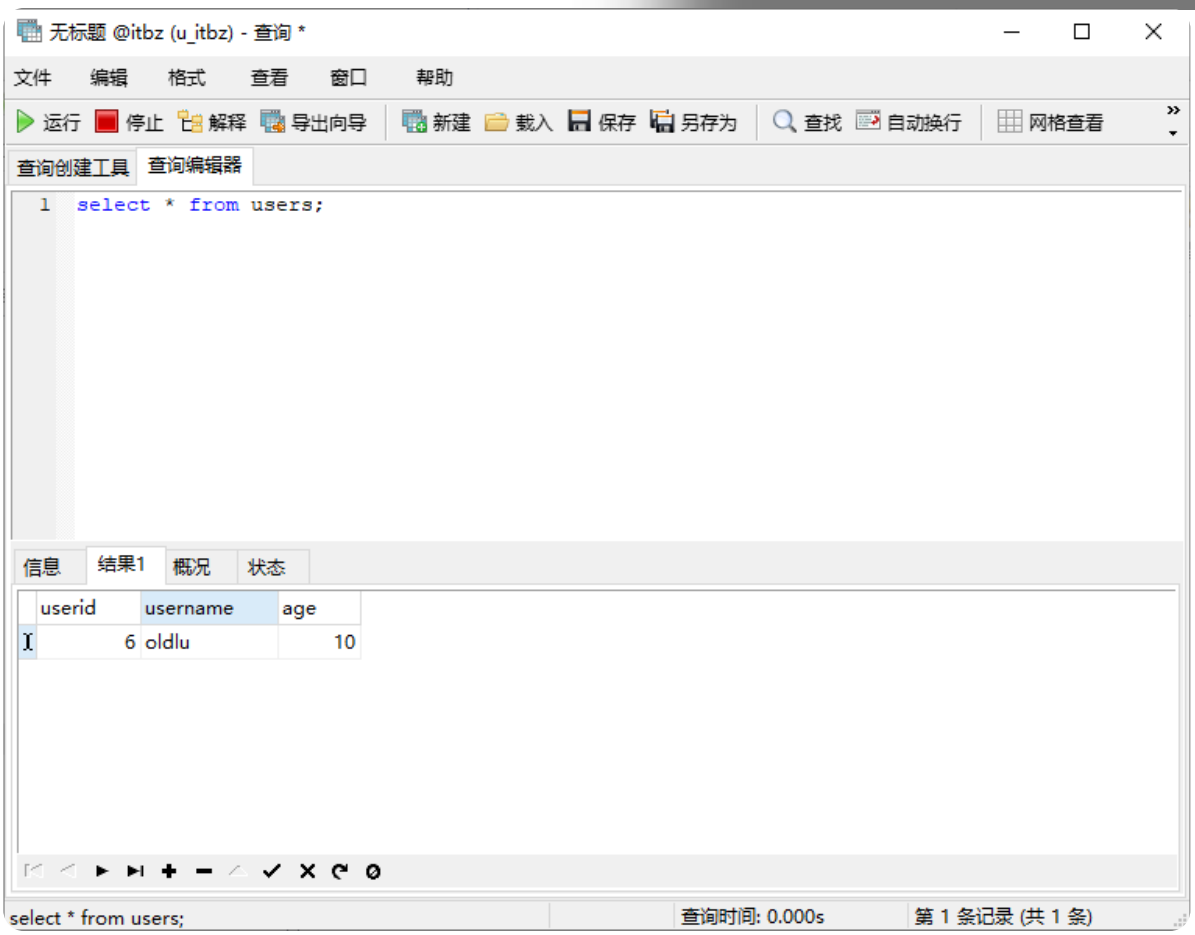
分配权限

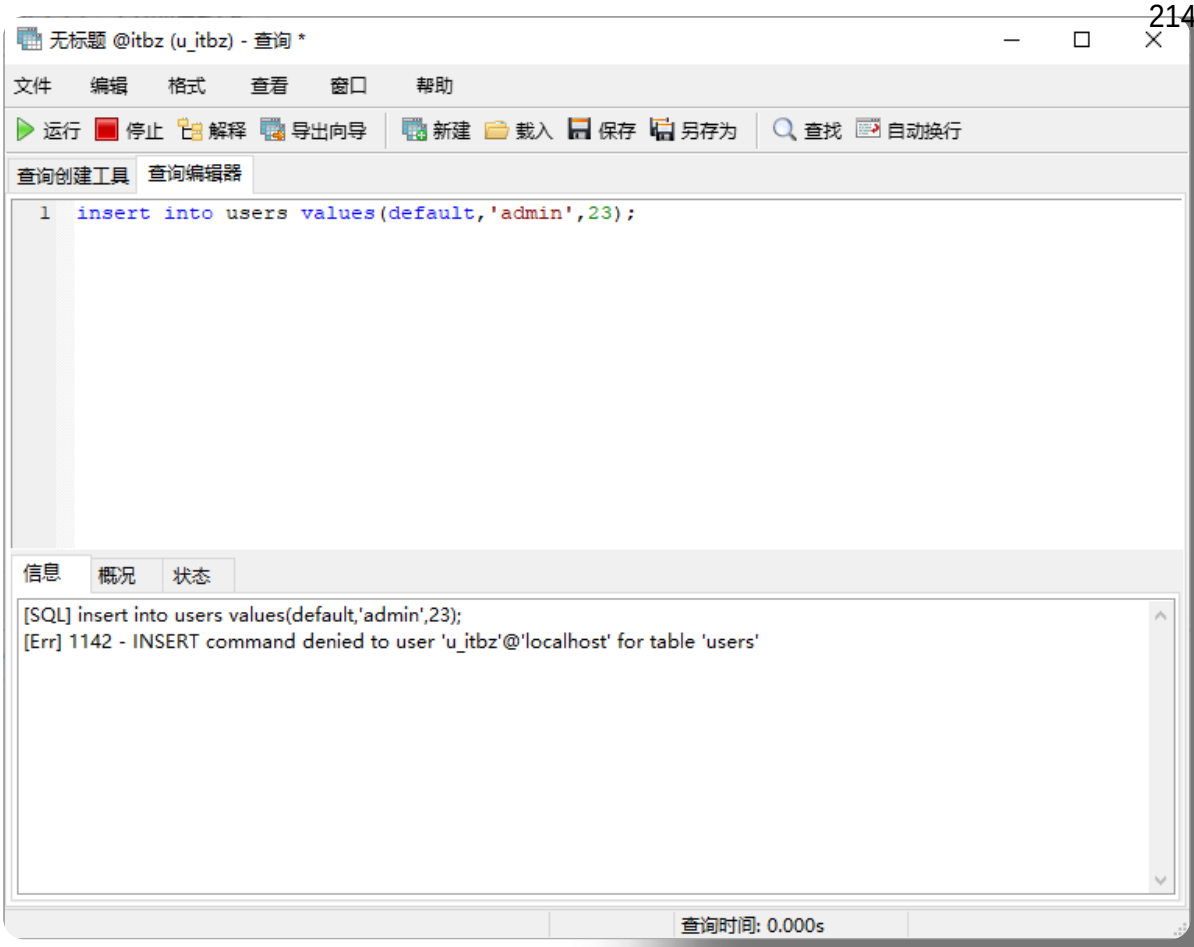

u_itbz@localhost (localhost) - 用户
文件
窗口
帮助
新建
保存
1
常规
高级
服务器权限
权限
SQL 预览
用户名:
u_itbz
主机:
localhost
密码:
确认密码:

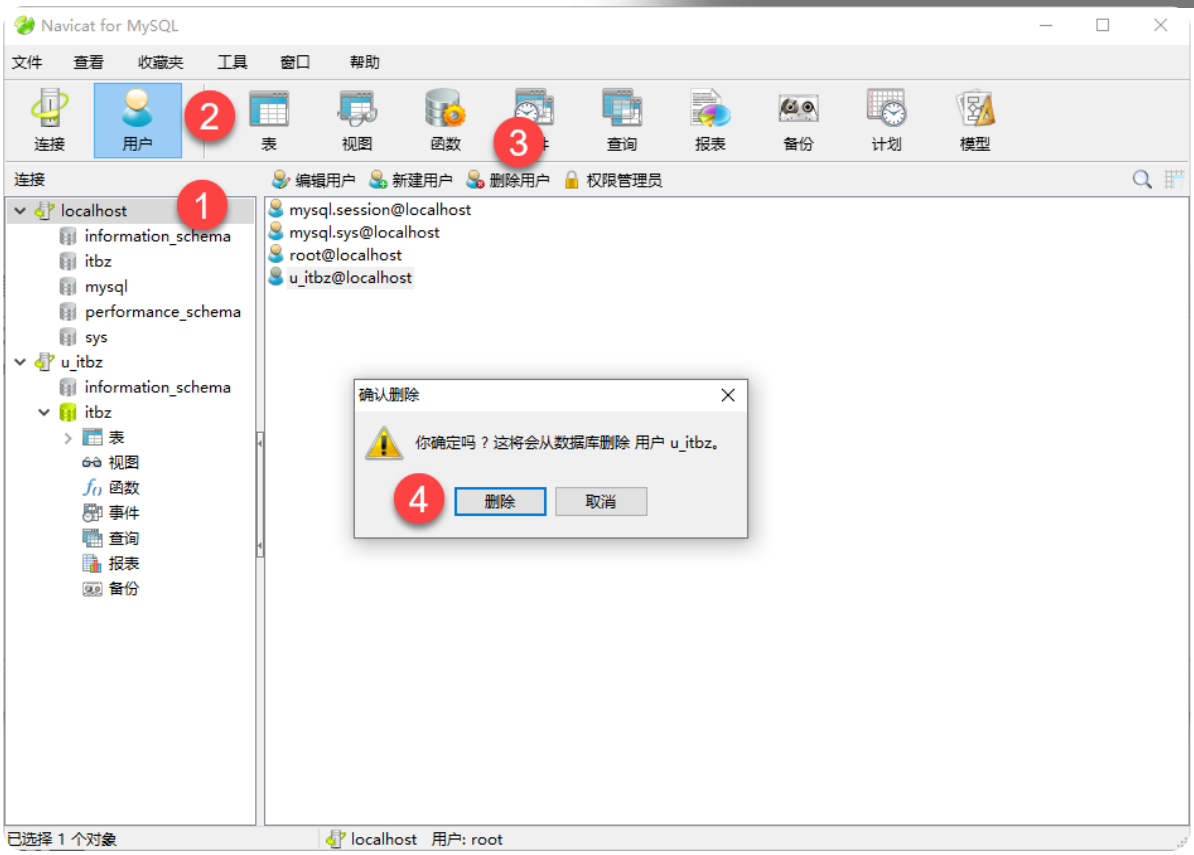

u_itbz@localhost (localhost) - 用户
文件
窗口
帮助
2
新建
保存
添加权限
删除权限
常规
高级
服务器权限
权限
1
预览
数据库
名
Select
Insert
Update
Reference
Delete
Create
Drop
添加权限
localhost
information_schema
itbz
3
Tables
users
Views
Functions
mysql
performance_schema
sys
权限
状态
4
Select
Insert
Update
References
Delete
Create
Drop
Alter
Index
Trigger
Create View
Show View
Grant
5
确定
取消


Navicat for MySQL
1
文
查看
收藏夹
工具
窗口
帮助
连接
报表
备份
计划
模型
连接
新建连接
常规
高级
SSL
SSH
HTTP
连接名:
u_itbz
2
主机名或 IP 地址:
localhost
端口:
3306
用户名:
u_itbz
密码:
3
连接成功
确定
5
4
连接测试
6
确定
取消
4 用户
localhost 用户: root

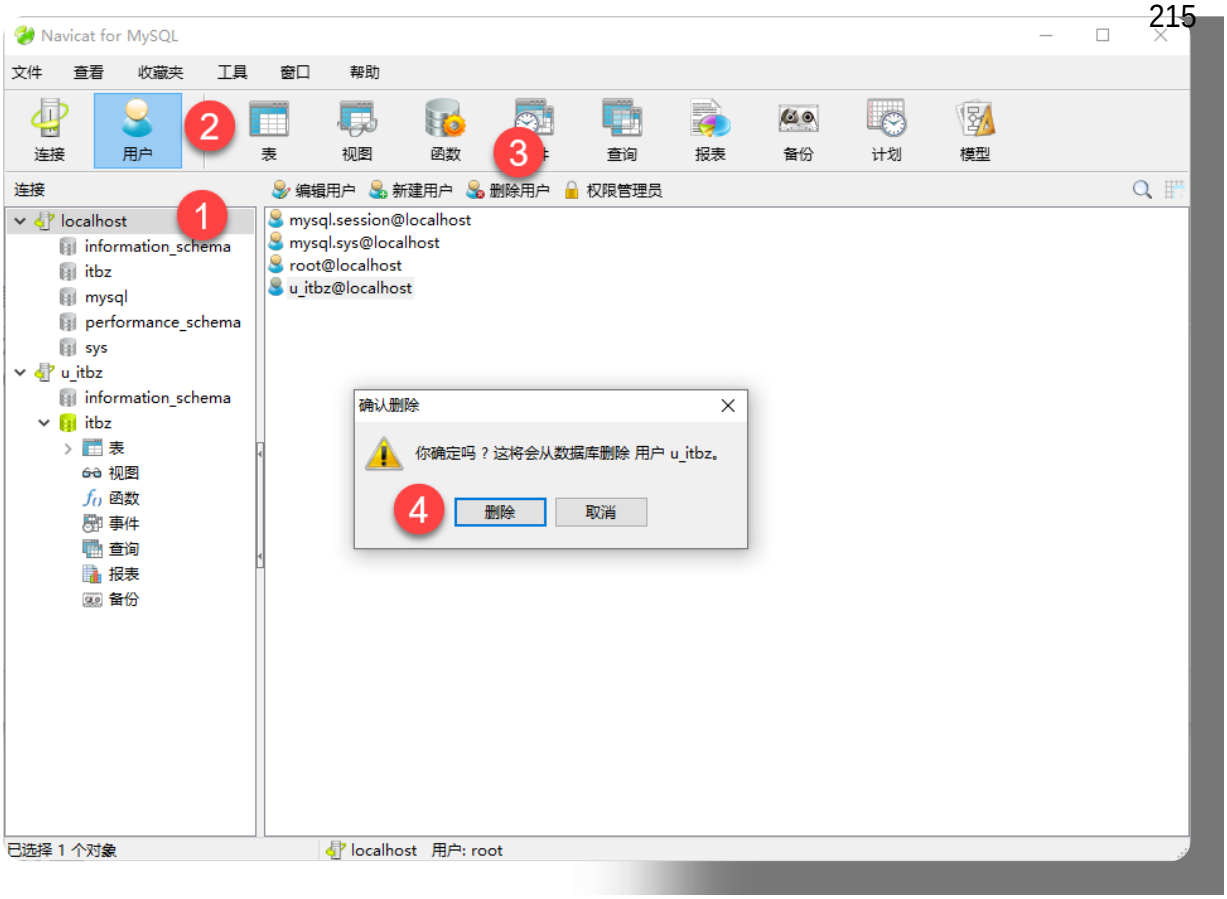

Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 事件 查询 报表 备份 计划 模型
连接
打开表 设计表 新建表 删除表 导入向导 导出向导
localhost
information_schema
itbz
mysql
performance_schema
sys
u_itbz
information_schema
itbz
表
视图
函数
事件
查询
报表
备份
users
已选择 1 个对象
u_itbz 用户: u_itbz 数据库: itbz


无标题 @itbz (u_itbz) - 查询 *
文件 编辑 格式 查看 窗口 帮助
运行 停止 解释 导出向导 新建 载入 保存 另存为 查找 自动换行 网格查看
查询创建工具 查询编辑器
1 select * from users;
信息 结果1 概况 状态
userid username age
6 oldlu 10
select * from users;
查询时间: 0.000s
第 1 条记录 (共 1 条)


无标题 @itbz (u_itbz) - 查询 *
文件 编辑 格式 查看 窗口 帮助
运行 停止 解释 导出向导 新建 载入 保存 另存为 查找 自动换行
查询创建工具 查询编辑器
1 insert into users values(default,'admin',23);
信息 概况 状态
[SQL] insert into users values(default,'admin',23);
[Err] 1142 - INSERT command denied to user 'u_itbz'@'localhost' for table 'users'
查询时间: 0.000s

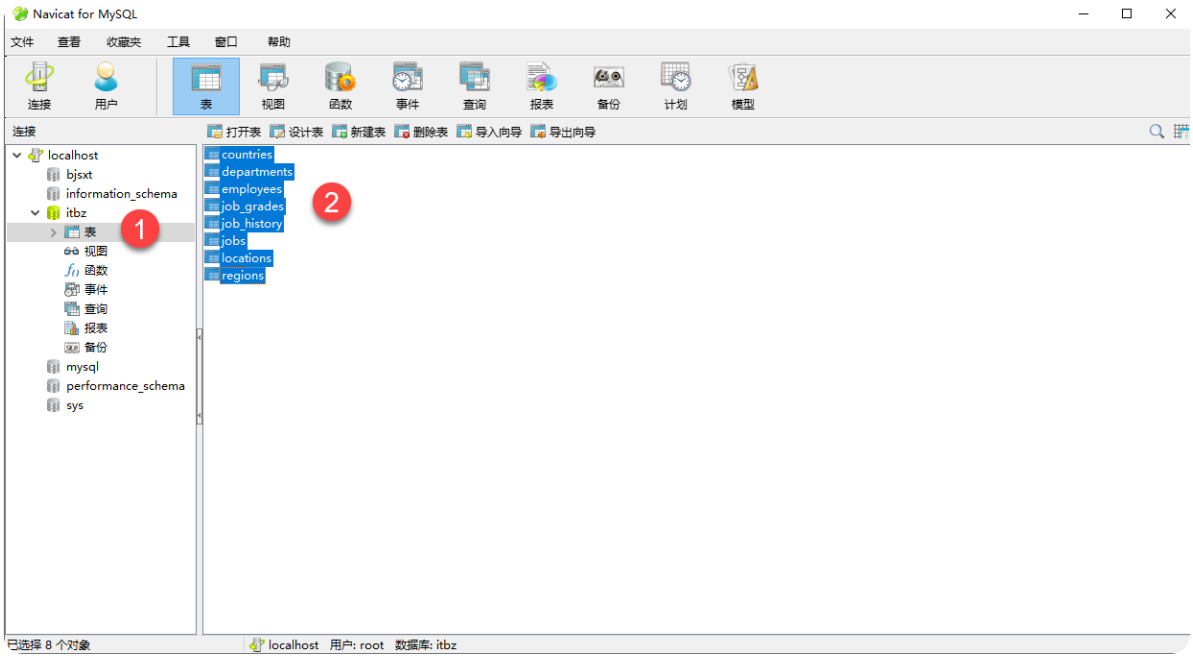

Navicat for MySQL
文件 查看 收藏夹 工具 窗口 帮助
连接 用户 表 视图 函数 查询 报表 备份 计划 模型
2
3
连接
编辑用户 新建用户 删除用户 权限管理员
1
localhost
information_schema
itbz
mysql
performance_schema
sys
u_itbz
information_schema
itbz
表
视图
函数
事件
查询
报表
备份
mysql.session@localhost
mysql.sys@localhost
root@localhost
u_itbz@localhost
确认删除
你确定吗？这将会从数据库删除 用户 u_itbz。
4
删除
取消
已选择 1 个对象
localhost 用户: root

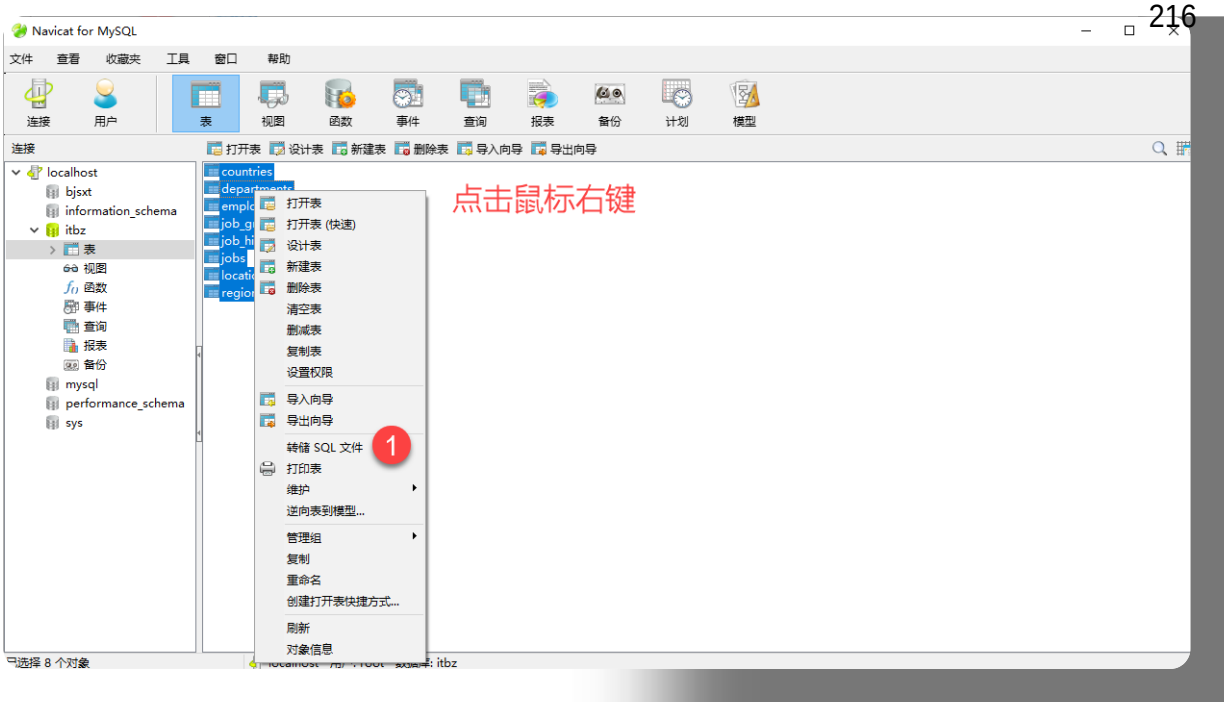
删除用户



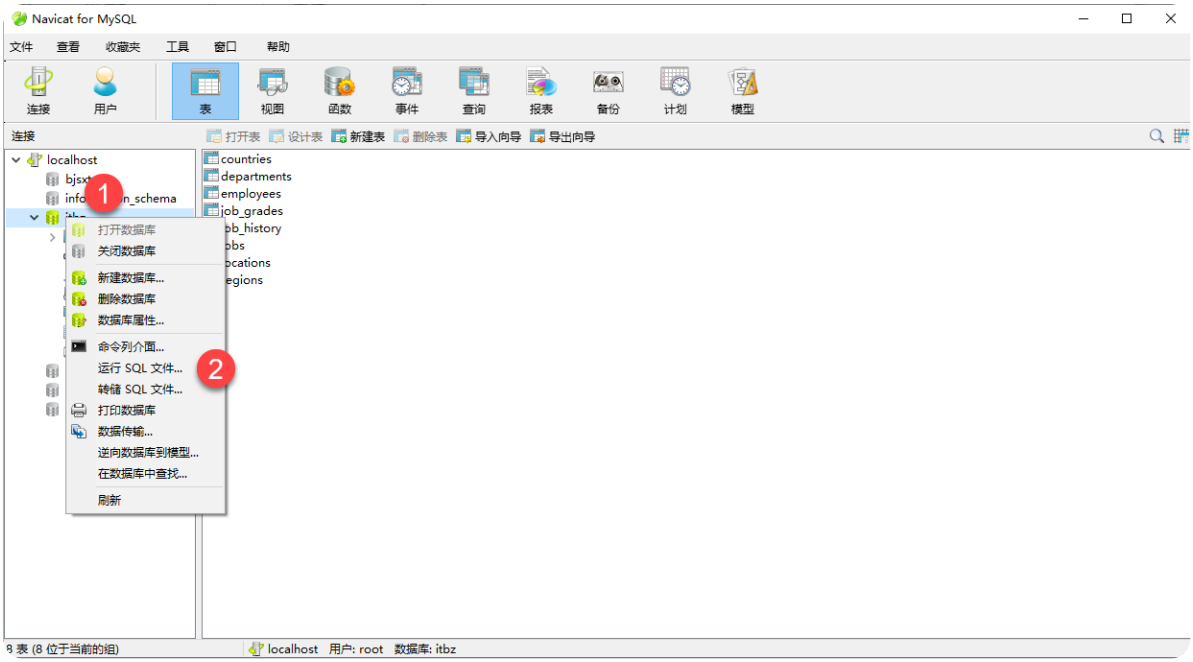
数据的导入导出

导出sql脚本文件

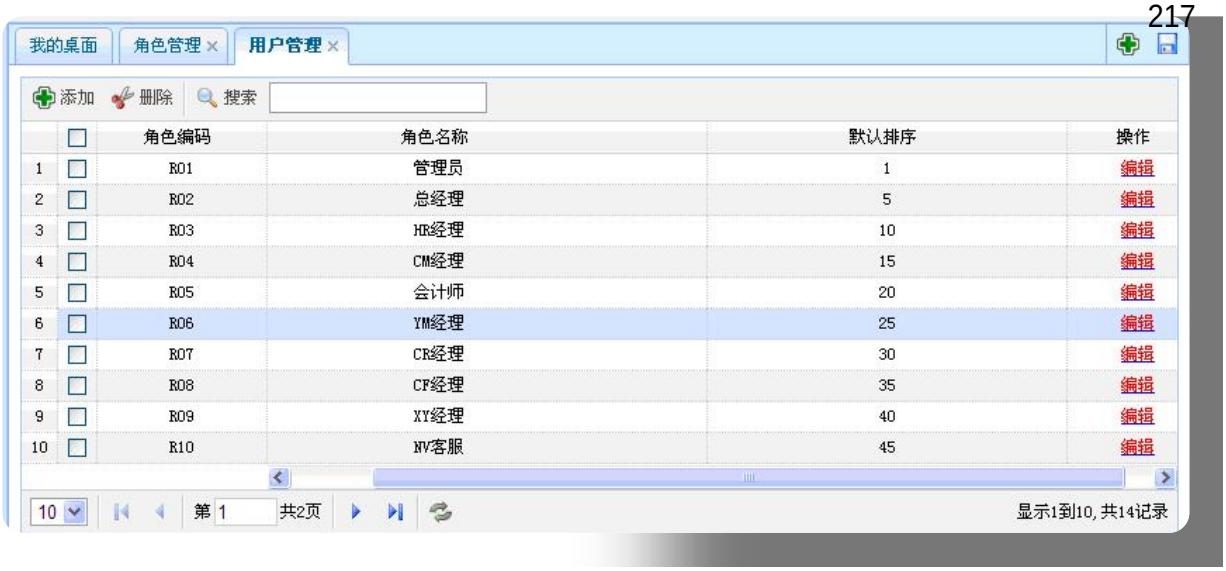




导入sql脚本文件



MySQL分页查询



MySQL 分页查询原则：

- 在 MySQL 数据库中使用 LIMIT 子句进行分页查询。
- MySQL 分页中开始位置为 0。
- 分页子句在查询语句的最后侧。

LIMIT子句

```
SELECT  投影列 FROM  表名 WHERE  条件 ORDER BY
LIMIT  开始位置，查询数量;
```

示例：

查询雇员表中所有数据按 id 排序，实现分页查询，每次返回两条结果。

```
select * from employees order by employees_id
limit 0,2;
```

LIMIT OFFSET子句

```
SELECT  投影列 FROM  表名 WHERE  条件 ORDER BY
LIMIT  查询数量 OFFSET 开始位置;
```

示例：

查询雇员表中所有数据按 id 排序，使用 LIMIT OFFSET 实现分页查询，每次返回两条结果。

```
select * from employees order by employees_id
limit 2 offset 4;
```

实时效果反馈

1.MySQL 分页中开始位置为。

A -1

B 0

C 1

D 10

2.分页子句在查询语句的。

A 最前侧

B 任意位置

C 最后侧

D select子句后侧

答案

1=>B 2=>C